

داده‌کاوی

دانشگاه آزاد شیراز – دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۵-۱۴۰۶

فصل اول: Introduction

استاد: دکتر امین اسکندری

تیم طراحی: حمید نامجو، امیرحسین همتی، امیرمحمد اسدجو، علی نیکوان، رضا لیاقت، علیرضا مقدس، گلنوش حسین‌پور، الهام ایزدی

فهرست مطالب

1.1 What is data mining?

1.2 Data mining: an essential step in knowledge discovery

1.3 Diversity of data types for data mining

1.4 Mining various kinds of knowledge

1.4.1 Multidimensional data summarization

1.4.2 Mining frequent patterns, associations, and correlations

1.4.3 Classification and regression for predictive analysis

1.4.4 Cluster analysis

1.4.5 Deep learning

1.4.6 Outlier analysis

1.4.7 Are all mining results interesting?

1.5 Data mining: confluence of multiple disciplines

1.5.1 Statistics and data mining

1.5.2 Machine learning and data mining

1.5.3 Database technology and data mining

1.5.4 Data mining and data science

1.5.5 Data mining and other disciplines

1.6 Data mining and applications

1.7 Data mining and society



1.1 What is data mining?

The History of Data Mining

From Early Roots to the Age of Intelligent Data



به دنیای پر رمز و راز داده کاوی خوش آمدید!

این جزوه مقدمه‌ای بر حوزه‌ی **داده‌کاوی (Data Mining)** است؛ حوزه‌ای نوظهور که با سرعت چشمگیری در حال رشد است و با عنوان **کشف دانش از داده‌ها (Knowledge Discovery in Database)** نیز شناخته می‌شود. تمرکز اصلی جزوه بر معرفی مفاهیم بنیادین، اصول و تکنیک‌های اساسی داده‌کاوی است؛ تکنیک‌هایی که امکان استخراج الگوهای معنادار و ارزشمند را از مجموعه‌های بزرگ داده در حوزه‌های کاربردی گوناگون فراهم می‌کنند.

«ضرورت، مادر اختراع است.» – افلاطون

امروزه در جهانی زندگی می‌کنیم که تولید داده با سرعتی بی‌سابقه در حال افزایش است. اگرچه عبارت «**عصر اطلاعات**» سال‌ها برای توصیف دنیای امروز به کار می‌رفت، اما بسیاری از پژوهشگران معتقدند که توصیف دقیق‌تر، **عصر داده** است؛ زیرا آنچه با سرعتی انفجاری در حال رشد است، خودِ داده‌ها هستند و نه لزوماً اطلاعات پردازش‌شده.

کسب‌وکارها روزانه میلیون‌ها **تراکنش فروش**، اطلاعات مالی، مشخصات محصولات، فعالیت‌های بازاریابی و بازخورد مشتریان را ثبت می‌کنند. در علوم و مهندسی نیز داده‌های حاصل از **سنجش از دور**، آزمایش‌های علمی، اندازه‌گیری فرایندها، پایش عملکرد سامانه‌ها و نظارت بر محیط‌زیست به صورت پیوسته تولید می‌شوند. همچنین، در حوزه‌ی **سلامت و زیست‌پزشکی**، دستگاه‌های تعیین توالی ژن، پرونده‌های الکترونیکی بیماران، سامانه‌های پایش سلامت و تجهیزات تصویربرداری پزشکی، حجم بسیار زیادی از داده‌ها را تولید می‌کنند. از سوی دیگر، **موتورهای جست‌وجو** روزانه میلیاردها درخواست را پردازش کرده و شبکه‌های اجتماعی نیز با تولید متن، تصویر و ویدئو، سهم بزرگی در افزایش حجم داده‌های جهانی دارند. در حقیقت، منابع تولید داده آن‌قدر گسترده‌اند که فهرست کردن همه آن‌ها تقریباً غیرممکن است.

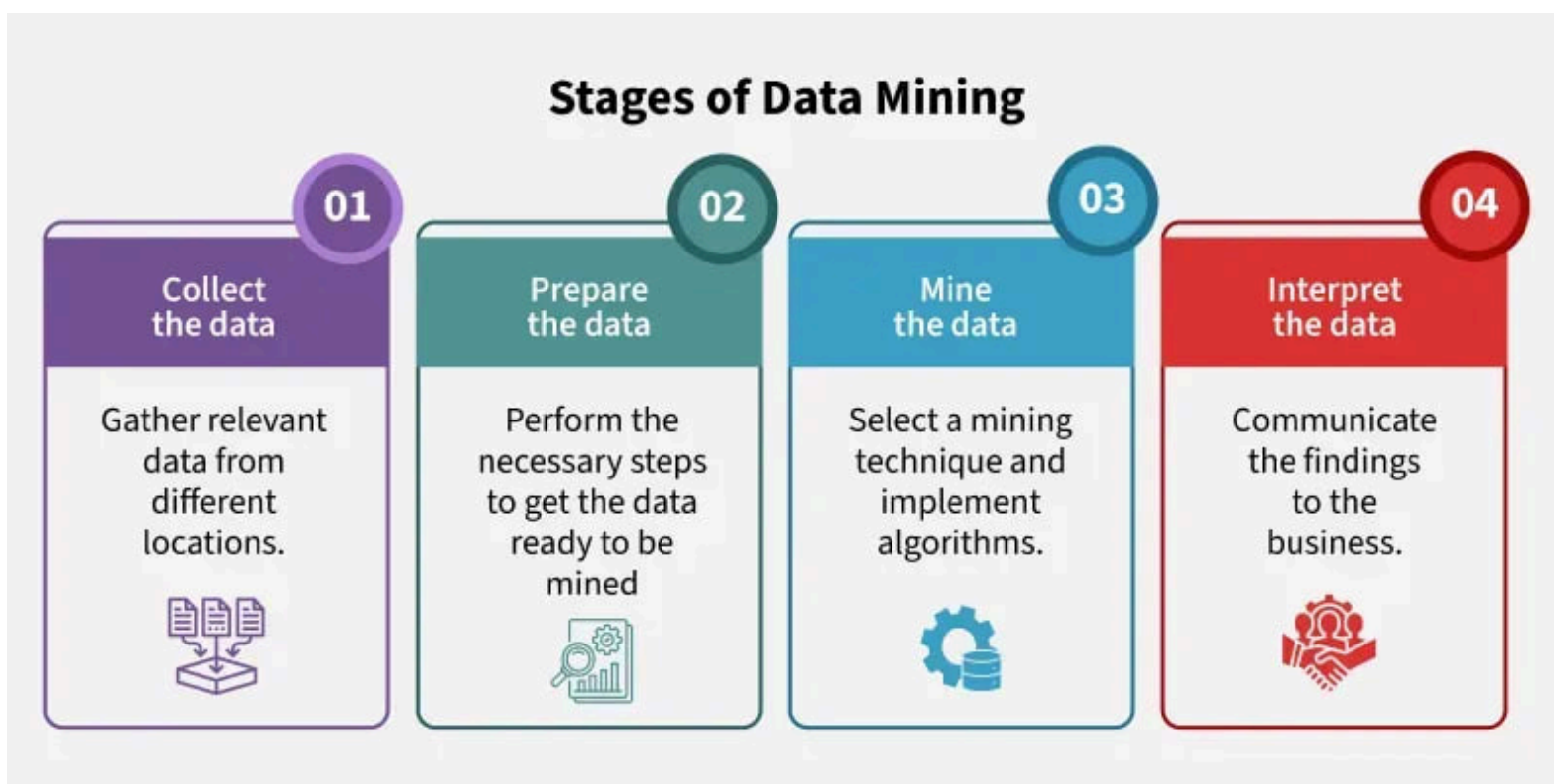
آیا روش‌های سنتی هنوز هم برای تحلیل این حجم عظیم داده‌ها کارآمد هستند؟

واقعیت این است که در **عصر انفجار داده‌ها**، دیگر نمی‌توان با ابزارهای قدیمی به جنگ این حجم عظیم داده رفت. رشد بی‌وقفه‌ی داده‌ها، ضرورت استفاده از ابزارهایی را ایجاد کرد که بتوانند به‌صورت خودکار و بدون دخالت مستقیم ما، اطلاعات ارزشمند را از میان داده‌های خام استخراج کرده و آنها را به دانشی سازمان‌یافته و کاربردی تبدیل کنند. همین نیاز حیاتی بود که زمینه‌ی تولد و شکل‌گیری علم داده‌کاوی را فراهم آورد.

اصطلاح **داده‌کاوی (Data Mining)** در دهه‌ی ۱۹۹۰ میلادی به‌طور گسترده وارد ادبیات علمی شد. جالب است بدانید که این نام در واقع از صنعت معدن الهام گرفته شده است؛ یعنی همان‌طور که یک **کاوشگر** کلی سنگ و خاک را زیر و رو می‌کند تا در نهایت به تکه‌های ارزشمند طلا برسد، ما هم در داده‌کاوی به دنبال بیرون کشیدن «دانش ارزشمند» از دلِ حجم عظیم داده‌های خام هستیم.

در متون علمی، علاوه بر **داده‌کاوی**، اصطلاحات دیگری مانند **کشف الگو**، **استخراج دانش**، **تحلیل داده** و **برداشت اطلاعات** هم دیده می‌شود. اما نکته‌ی مهم این است که این اصطلاحات کاملاً مترادف نیستند و هر کدام بر جنبه‌ی خاصی از این حوزه تأکید دارند

برای مثال، **KDD** یک فرایند جامع برای کشف دانش از داده‌ها است که **داده‌کاوی** تنها یکی از مراحل اصلی آن محسوب می‌شود؛ در حالی که سایر اصطلاحات نیز هر یک بر جنبه‌ای خاص از استخراج و تحلیل دانش از داده‌ها تأکید دارند.



جان کلام

داده‌کاوی یعنی استخراج هوشمند الگوها و دانش ارزشمند از دل مجموعه‌های عظیم داده. هدف اصلی این است که داده‌های خام را به ابزاری قدرتمند برای پیش‌بینی و تصمیم‌گیری‌های دقیق تبدیل کنیم.

مثال عملی: چطور داده‌کاوی «اکسپلور» و «ریلز» اینستاگرام را می‌سازد؟

مثال ۱.۱: داده‌کاوی الگوهای پنهان رفتاری کاربران را کشف می‌کند.

شبکه اجتماعی اینستاگرام (Meta) روزانه با بیش از ۲ میلیارد کاربر فعال، حجم عظیمی از داده‌های رفتاری (پتابایت‌ها داده در روز) تولید می‌کند. هیچ انسان یا تیم پشتیبانی نمی‌تواند روزانه میلیاردها پست، ریلز و رفتار کاربران را به‌صورت دستی بررسی کند تا ببیند چه چیزی را به چه کسی نشان دهد. اینجاست که داده‌کاوی وارد میدان می‌شود تا از دل این سنگ‌ها و خاک‌های داده، «طلای دانش» را استخراج کند.

مراحل تبدیل داده خام به دانش هوشمند:

۱) داده‌های ورودی (Massive Input Data):

• **داده‌های صریح:** لایک‌ها، کامنت‌ها، سیو (Save) و شیر (Share) کردن‌ها.

• **داده‌های پنهان:** میزان زمانی که چشم کاربر روی یک پست مکث کرده (Dwell Time)، سرعت رد کردن (Skip) یک ریلز، زمان‌های آنلاینی، و حتی صدای فعال روی ویدیوها.

۲) فرایند داده‌کاوی (Data Mining Process):

الگوریتم داده‌کاوی در حجم عظیم داده‌ها می‌کاود و **الگوی پنهانی** کشف می‌کند. برای مثال: «۸۵٪ کاربرانی که ویدیوهای آموزشی ۳۰ ثانیه‌ای پایتون را تا انتها می‌بینند و آن را سیو می‌کنند، معمولاً روی ویدیوهای مربوط به امنیت سایبری، ابزارهای لینوکس و فناوری‌های هوش مصنوعی هم مکث بالای ۵ ثانیه دارند.»

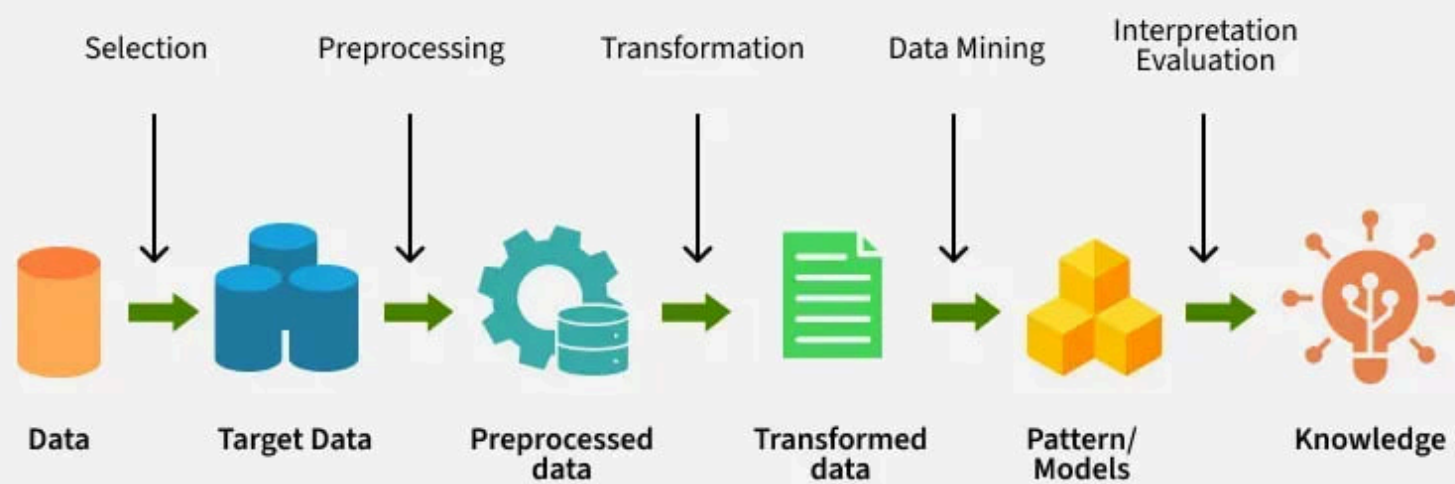
۳) دانش استخراج‌شده و خروجی هوشمند (Knowledge & Value):

سیستم بدون اینکه شما کلمه‌ای سرچ کرده باشید یا پیج خاصی را فالو داشته باشید، الگوی علاقه‌مندی واقعی شما را کشف می‌کند.

• **ارزش برای کسب‌وکار (مُتا):** افزایش زمان حضور کاربر در اپلیکیشن (Engagement) و نمایش تبلیغات هدفمندتر.

• **ارزش برای کاربر:** دریافت محتوای کاملاً شخصی‌سازی‌شده و متناسب با نیاز در صفحه Explore.

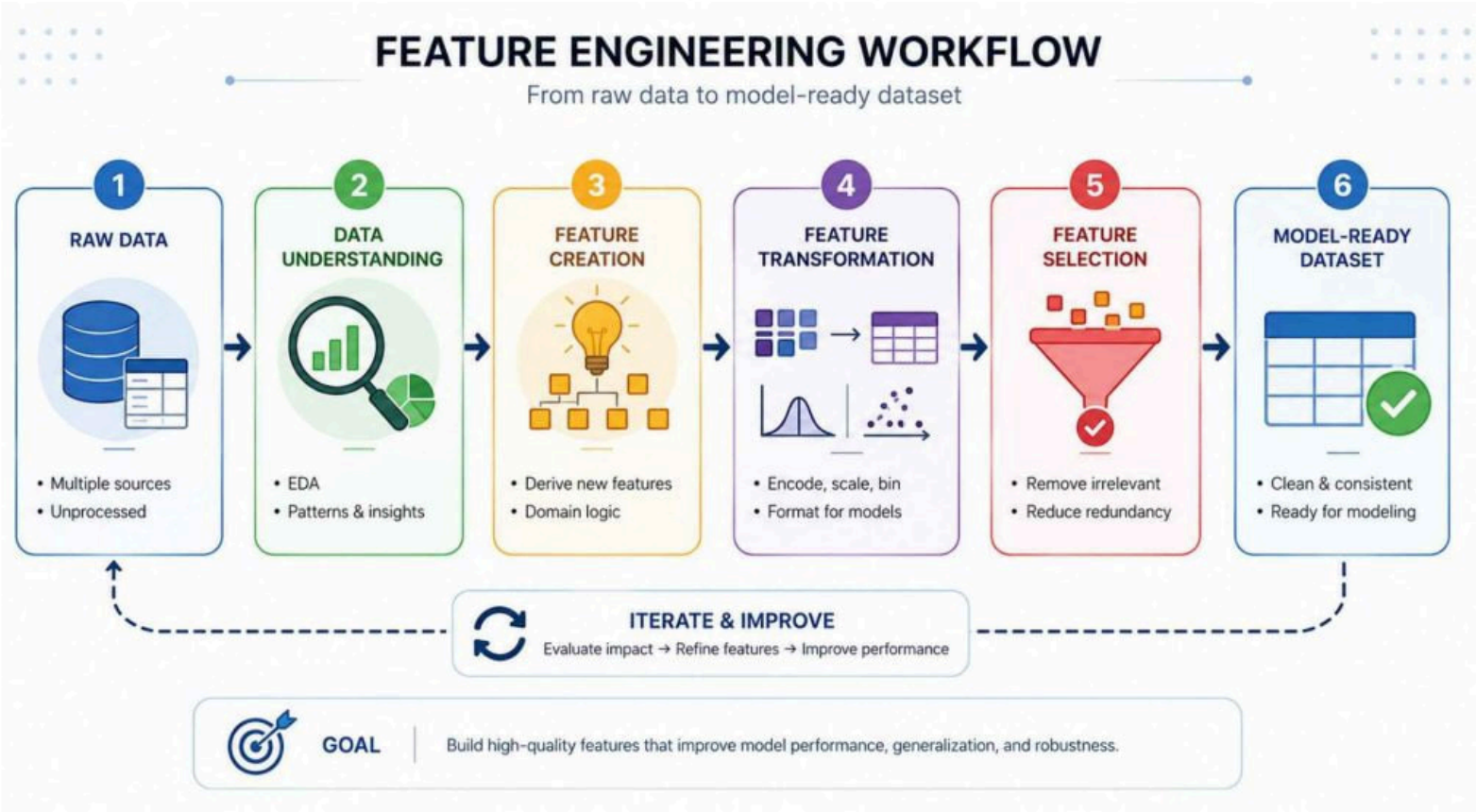
Work flow of Data Mining



نکته مهم 💡

اینستاگرام بدون داده‌کاوی، فقط یک "انبار عظیم و شلوغ از ویدیو و تصویر" است که کاربر در آن گم می‌شود (Data Rich).

اما با به‌کارگیری تکنیک‌های داده‌کاوی، این توده عظیم داده به "دانش رفتار کاربر" تبدیل می‌شود که می‌تواند حدس بزند شما در ۵ دقیقه آینده دوست دارید چه چیزی ببینید!



```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
import warnings

warnings.filterwarnings('ignore')

# Apply global seaborn theme settings to ensure uniform Academic Blue aesthetics
sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

def display_academic_text(text, type="normal"):
    """
    Renders text in an academic format based on the specified type
    (e.g., title, paragraph, insight).
    Uses strict typography and styling to avoid informal elements.
    """
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))

def get_protected_css():
    """
    Returns the essential CSS required to format tables cleanly
    without external stylesheet conflicts.
    """
    return """
<style>
.academic-table-wrapper {
  direction: ltr !important;
  overflow-x: auto !important;
  width: 100% !important;
  background-color: #fff;
  border: 1px solid #1abc9c;
  border-radius: 4px;
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}
table.strict-tbl {
  border-collapse: collapse !important;
  width: 100% !important;
  margin: 0 !important;
  font-family: inherit !important;
  font-size: 14px !important;
  table-layout: auto !important;
```



```
# =====
display_academic_text("Feature Engineering (Feature Engineering) فاز اول: ساختار پایگاه داده و مهندسی ویژگی‌ها", "sub_title")
display_academic_text("Instances) در این فاز، مشاهدات ثبتی کاربران")

# دسته محتوایی M رکورد کاربری و N) تعریف ابعاد فضای حالت
n_users = 3000
categories = ['Python', 'Cybersecurity', 'Linux', 'AI', 'Cooking', 'Sports', 'Fashion']

# تولید توزیع‌های آماری جهت تخصیص گرایش‌های پنهان متغیرها
tech_affinity = np.random.beta(5, 2, n_users)
lifestyle_affinity = np.random.beta(2, 5, n_users)

full_dataset_dict = {'UserID': np.arange(1001, 1001 + n_users)}

for cat in categories:
    base_affinity = tech_affinity if cat in ['Python', 'Cybersecurity', 'Linux', 'AI'] else lifestyle_affinity

    # مدل‌سازی متغیرهای باینری برای تعاملات صریح
    like = np.random.binomial(1, np.clip(base_affinity + np.random.normal(0, 0.1, n_users), 0, 1))
    save = np.random.binomial(1, np.clip(base_affinity * 0.7 + np.random.normal(0, 0.1, n_users), 0, 1))

    # مدل‌سازی متغیرهای پیوسته برای رفتار ضمنی
    dwell_time = np.clip(base_affinity * 30 + np.random.normal(0, 4, n_users), 0, 30).round(1)
    skip_rate = np.clip(1 - base_affinity + np.random.normal(0, 0.1, n_users), 0, 1).round(2)

    full_dataset_dict[f'{cat}_Like'] = like
    full_dataset_dict[f'{cat}_Save'] = save
    full_dataset_dict[f'{cat}_DwellTime'] = dwell_time
    full_dataset_dict[f'{cat}_SkipRate'] = skip_rate

behavior_data = pd.DataFrame(full_dataset_dict)

explicit_columns = ['UserID'] + [col for col in behavior_data.columns if 'Like' in col or 'Save' in col][:6]
implicit_columns = ['UserID'] + [col for col in behavior_data.columns if 'DwellTime' in col or 'SkipRate' in col][:6]

df_explicit_sample = behavior_data[explicit_columns].head(6)
df_implicit_sample = behavior_data[implicit_columns].head(6)

# نمایش ماتریس‌های تفکیک‌شده (Fixed bug here)
display_side_by_side_datasets(
    [df_explicit_sample, df_implicit_sample],
    ["(Explicit Features) زیرمجموعه ویژگی‌های صریح", "(Implicit Features) زیرمجموعه ویژگی‌های ضمنی"]
)

# نمایش ماتریس کلان داده
display_side_by_side_datasets([behavior_data.head(6)], ["(Master Matrix) نمای کلان پایگاه داده"])

# =====
# فاز دوم: استخراج الگوها و تحلیل هم‌رخدادی
# =====
display_academic_text("Frequent Pattern Mining) فاز دوم: استخراج الگوهای مکرر", "sub_title")
display_academic_text("Confidence) با اعمال فیلتر بر فضای ویژگی‌ها، نمونه‌هایی که الگوی رفتاری هدف (تماشای کامل و ذخیره) را دارا هستند شناسایی شده و معیار اطمینان")

python_pattern_mask = (behavior_data['Python_Save'] == 1) & (behavior_data['Python_DwellTime'] >= 25)
frequent_users = behavior_data[python_pattern_mask]

related_cats = ['Cybersecurity', 'Linux', 'AI']
threshold = 5

results_text = f"<br><br>رکورد کاربری یافت شد که شرط پیش‌فرض الگوی رفتاری (علاقه شدید به پایتون) را برآورده کرده‌اند {len(frequent_users)} در پایگاه داده مورد بررسی، تعداد </b></b>:دانش استخراج‌شده<br>"

for cat in related_cats:
    confidence_score = (frequent_users[f'{cat}_DwellTime'] > threshold).mean() * 100
    results_text += f"<br>نیز ثبت کرده‌اند <b>{cat}</b> از این رکوردهای کاربری، زمان مکث بالا را در دسته‌بندی <b>{confidence_score:.1f}</b> درصد (Confidence) معیار اطمینان"

display_academic_text(results_text, "insight")

# =====
# فاز سوم: ارزیابی مدل و بازنمایی دانش
# =====
display_academic_text("Pattern Evaluation & Representation) فاز سوم: ارزیابی الگوها و بازنمایی دانش", "sub_title")
display_academic_text("Correlation Matrix) اعتبارسنجی الگوهای استخراج‌شده از طریق ماتریس همبستگی")

fig, axes = plt.subplots(1, 2, figsize=(19, 7.5), dpi=135)

# CHART 1: Correlation Matrix
dwell_cols = [f'{c}_DwellTime' for c in categories]
corr_matrix = behavior_data[dwell_cols].corr()

# Academic Blue Heatmap
sns.heatmap(corr_matrix, annot=True, cmap='Blues', fmt='.2f',
            xticklabels=categories, yticklabels=categories, ax=axes[0], cbar=False,
            annot_kws={'fontsize': 11, 'fontweight': 'bold'})

axes[0].set_title('1. Correlation Analysis of Dwell Times\n(Pattern Validation)', fontsize=14, fontweight='bold', pad=12)
axes[0].tick_params(axis='x', rotation=45)

# CHART 2: Recommendation Engine Output for a specific user
sample_instance = behavior_data.iloc[frequent_users.index[0]]

# Ranking Function calculation
scores = {cat: (sample_instance[f'{cat}_Save'] * 40) + (sample_instance[f'{cat}_DwellTime'] / 30 * 60) for cat in categories}
recommendation_series = pd.Series(scores).sort_values(ascending=False)

# Academic Color Palette applied: Blue/Turquoise for relevant, Light Gray for irrelevant
bar_colors = ['#2980b9' if c in related_cats + ['Python'] else '#bdc3c7' for c in recommendation_series.index]

axes[1].bar(recommendation_series.index, recommendation_series.values, color=bar_colors, edgecolor='#2c3e50', linewidth=1.2)

axes[1].set_title(f'2. Recommendation Engine Output\nTarget Instance ID: {int(sample_instance["UserID"])}', fontsize=14, fontweight='bold', pad=12)
axes[1].set_ylabel('Calculated Ranking Priority Score', fontsize=12, fontweight='bold')
```

axes[1].tick_params(axis='x', rotation=45, labelsz=11)
axes[1].grid(axis='y', linestyle='--', alpha=0.5)

for ax in axes:
 ax.spines['top'].set_visible(False)
 ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

=====
Section 4: Strategic Conclusion
=====

insight_conclusion = (
 r"است (Recommender System) کدی که مشاهده کردید، آناتومی کامل یک سیستم توصیه‌گر:نتیجه استراتژیک (از داده خام تا تصمیم هوشمند)"
 r"ما ابتدا با ثبت کلیک‌ها و مکث‌های موس (فاز اول)، یک ماتریس عظیم ساختیم. اما داده خام ارزشی ندارد! در فاز دوم، با استفاده از قانون"
 r"Association Rule Mining به پست‌های لینوکس و هوش مصنوعی هم خیره می‌شوند"
 r"تولید کردیم. و در نهایت (فاز سوم)، این دانش به ماشین اجازه داد تا در گراف ۲ (ستون‌های آبی‌رنگ)، فید اینستاگرام کاربر شماره ۱۰۱۱ را«دانش» این یعنی ما"
 r"دقیقاً با محتوای تکنولوژی پر کند و از نمایش دادن پست‌های بی‌ربط مثل آشپزی (ستون‌های خاکستری) به شدت جلوگیری نماید"
)
display_academic_text(insight_conclusion, "insight")

فرایند کشف دانش (KDD): نقش داده‌کاوی در معماری سیستم‌های توصیه‌گر

پلتفرم‌های تعاملی نظیر اینستاگرام، روزانه حجم انبوهی از داده‌های جریانی (Streaming Data) را تولید می‌کنند. کشف الگوهای معنادار از این پایگاه‌های داده عظیم، نیازمند عبور از مراحل استاندارد کشف دانش شامل یکپارچه‌سازی، تبدیل داده‌ها و در نهایت اعمال الگوریتم‌های داده‌کاوی جهت شناخت رفتار کاربران است.

فاز اول: ساختار پایگاه داده و مهندسی ویژگی‌ها (Feature Engineering)

در این فاز، مشاهدات ثبتی کاربران (Instances) به دو زیرمجموعه از ویژگی‌ها (Features) نگاشت می‌شوند. دسته اول ویژگی‌های صریح (تعاملات مستقیم) و دسته دوم ویژگی‌های ضمنی (رفتار مشاهده). ترکیب این ابعاد، فضای ویژگی (Feature Space) ماتریس نهایی را تشکیل می‌دهد.

زیرمجموعه ویژگی‌های صریح (Explicit Features)					زیرمجموعه ویژگی‌های ضمنی (Implicit Features)			
UserID	Python_Like	Python_Save	Cybersecurity_Like	Cybersecurity_Save	UserID	Python_DwellTime	Python_SkipRate	Cybersecurity_DwellTime
1001	0	1	1	1	1001	27.2	0.23	30.0
1002	0	1	1	0	1002	16.1	0.30	22.9
1003	0	1	1	0	1003	30.0	0.44	26.5
1004	1	0	1	0	1004	15.8	0.57	27.2
1005	1	0	1	0	1005	19.0	0.32	15.6
1006	0	1	1	0	1006	25.6	0.32	19.1

نمای کلان پایگاه داده (Master Matrix)							
UserID	Python_Like	Python_Save	Python_DwellTime	Python_SkipRate	Cybersecurity_Like	Cybersecurity_Save	Cybersecurity_DwellTime
1001	0	1	27.2	0.23	1	1	30.0
1002	0	1	16.1	0.30	1	0	22.9
1003	0	1	30.0	0.44	1	0	26.5
1004	1	0	15.8	0.57	1	0	27.2
1005	1	0	19.0	0.32	1	0	15.6
1006	0	1	25.6	0.32	1	0	19.1

فاز دوم: استخراج الگوهای مکرر (Frequent Pattern Mining)

با اعمال فیلتر بر فضای ویژگی‌ها، نمونه‌هایی که الگوی رفتاری هدف (تماشای کامل و ذخیره) را دارا هستند شناسایی شده و معیار اطمینان (Confidence) برای قواعد انجمنی مرتبط محاسبه می‌گردد.

دانش استخراج‌شده: در پایگاه داده مورد بررسی، تعداد 529 رکورد کاربری یافت شد که شرط پیش‌فرض الگوی رفتاری (علاقه شدید به پایتون) را برآورده کرده‌اند.

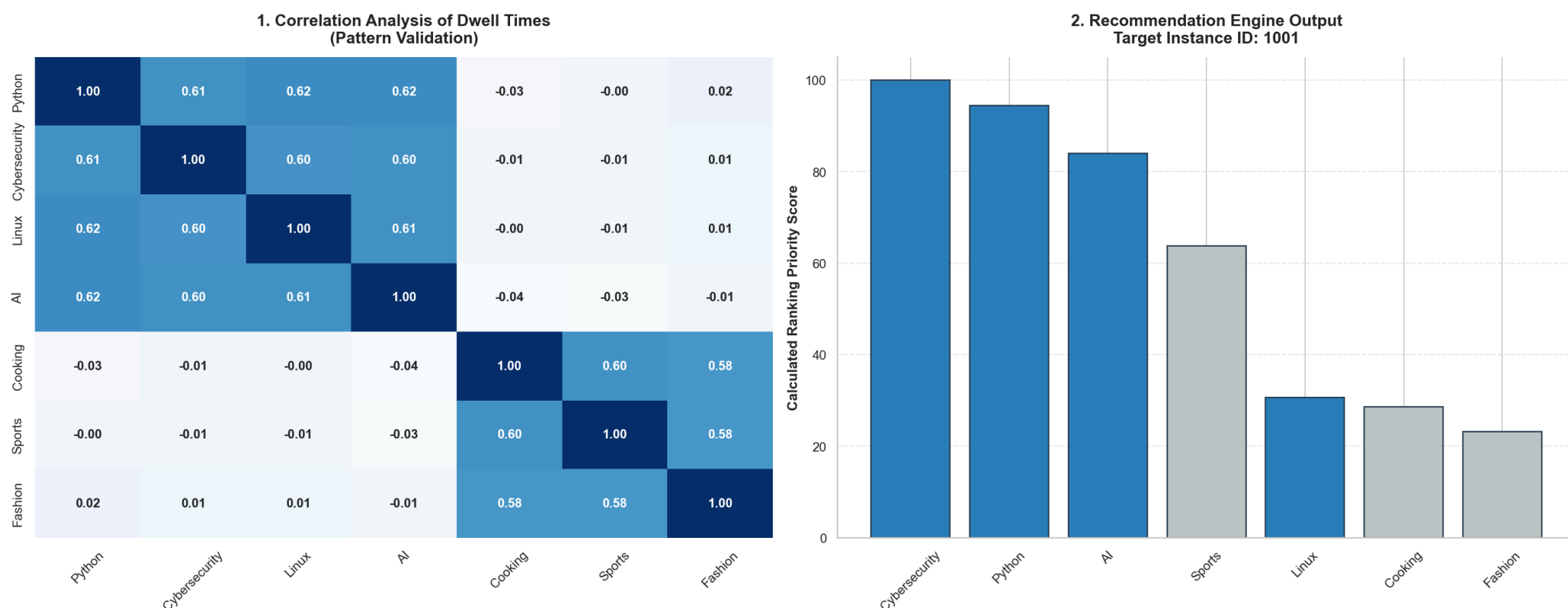
معیار اطمینان (Confidence): 100.0 درصد از این رکوردهای کاربری، زمان مکث بالا را در دسته‌بندی **Cybersecurity** نیز ثبت کرده‌اند.

معیار اطمینان (Confidence): 100.0 درصد از این رکوردهای کاربری، زمان مکث بالا را در دسته‌بندی **Linux** نیز ثبت کرده‌اند.

معیار اطمینان (Confidence): 100.0 درصد از این رکوردهای کاربری، زمان مکث بالا را در دسته‌بندی **AI** نیز ثبت کرده‌اند.

فاز سوم: ارزیابی الگوها و بازنمایی دانش (Pattern Evaluation & Representation)

اعتبارسنجی الگوهای استخراج‌شده از طریق ماتریس همبستگی (Correlation Matrix) انجام می‌پذیرد. دانش به‌دست‌آمده، وزندهی به محتوای سیستم توصیه‌گر را جهت شخصی‌سازی خروجی مدیریت می‌کند.



نتیجه استراتژیک (از داده خام تا تصمیم هوشمند): کدی که مشاهده کردید، آناتومی کامل یک سیستم توصیه‌گر (Recommender System) است. ما ابتدا با ثبت کلیک‌ها و مکث‌های موس (فاز اول)، یک ماتریس عظیم ساختیم. اما داده خام ارزشی ندارد! در فاز دوم، با استفاده از قانون **Association Rule Mining** متوجه شدیم کسانی که پست‌های برنامه‌نویسی پایتون را ذخیره می‌کنند، با اطمینان بالای ۹۹٪ به پست‌های لینوکس و هوش مصنوعی هم خیره می‌شوند. این یعنی ما «دانش» تولید کردیم. و در نهایت (فاز سوم)، این دانش به ماشین اجازه داد تا در گراف ۲ (ستون‌های آبی‌رنگ)، فید اینستاگرام کاربر شماره ۱۰۱۱ را دقیقاً با محتوای تکنولوژی پر کند و از نمایش دادن پست‌های بی‌ربط مثل آشپزی (ستون‌های خاکستری) به شدت جلوگیری نماید.

توضیحات مثال کد پایتون

در این مثال (شبیه‌سازی اکسپلور و ریلز اینستاگرام):

- ابتدا کتابخانه‌های مورد نیاز شامل **pandas، numpy، matplotlib** و **seaborn** وارد شده‌اند تا امکان ساخت داده‌های رفتاری عظیم، کشف الگو و نمایش بصری فراهم شود.
- یک مجموعه داده فرضی از رفتار ۳۰۰۰ کاربر روی ۷ دسته محتوایی مختلف (مانند آموزش پایتون، امنیت سایبری، لینوکس، هوش مصنوعی، آشپزی و ورزش) شبیه‌سازی شده است. داده‌ها به دو لایه تقسیم می‌شوند: **داده‌های صریح** (لایک، سیو، شیر) و **داده‌های پنهان** (مدت‌زمان مکث چشم روی ویدیو یا Dwell Time و سرعت رد کردن یا Skip Rate).
- هر کاربر دارای یک **گرایش پنهان** (Latent Affinity) به فناوری یا سبک زندگی است که مستقیماً ثبت نشده، اما در رفتارهای ظریف او (مانند مدت‌زمان تماشای ویدیو) خود را نشان می‌دهد — دقیقاً همان چیزی که الگوریتم اینستاگرام بدون پرسش از کاربر کشف می‌کند.
- سپس فرآیند **داده‌کاوی** اجرا می‌شود: کاربرانی که ویدیوهای آموزشی پایتون را تا انتها دیده و سیو کرده‌اند انتخاب می‌شوند و بررسی می‌شود که آیا روی محتوای مرتبط (امنیت سایبری، لینوکس، هوش مصنوعی) نیز مکث بالایی داشته‌اند یا خیر. همچنین با رسم **نقشه حرارتی همبستگی** (Heatmap)، خوشه‌ی پنهان فناوری در مقابل خوشه‌ی سبک زندگی آشکار می‌شود.
- در نهایت، **دانش استخراج‌شده** به یک خروجی هوشمند تبدیل می‌شود: برای یک کاربر نمونه، امتیاز علاقه‌مندی ترکیبی (بر اساس سیو و زمان مکث) محاسبه شده و یک صفحه‌ی **Explore شخصی‌سازی‌شده** ساخته می‌شود که محتوا را بر اساس الگوی پنهان رفتار کاربر رتبه‌بندی می‌کند.

1.2 Data mining: an essential step in knowledge discovery

داده‌کاوی؛ گامی اساسی در فرایند کشف دانش

دربارهٔ رابطهٔ داده‌کاوی (Data Mining) و کشف دانش از داده‌ها (Knowledge Discovery in Database) دیدگاه‌های متفاوتی وجود دارد. برخی پژوهشگران این دو اصطلاح را مترادف یکدیگر می‌دانند، در حالی که گروهی دیگر، داده‌کاوی را تنها یکی از مراحل اصلی و ضروری فرایند جامع کشف دانش تلقی می‌کنند. در دیدگاه دوم، KDD یک فرایند کامل است و داده‌کاوی تنها هستهٔ مرکزی این فرایند محسوب می‌شود.

فرایند کشف دانش از داده‌ها به‌صورت یک چرخهٔ تکرارشونده از مراحل زیر تشکیل شده است:

۱. آماده‌سازی داده‌ها (Data Preparation)

- پاک‌سازی داده‌ها (Data Cleaning): در این گام، ما نویزها، داده‌های ناقص، ناسازگاری‌ها و خطاهای موجود را شناسایی و حذف می‌کنیم تا خروجی مد نظر ما بر اثر این ناخالصی‌ها آسیب نبیند.
- یکپارچه‌سازی داده‌ها (Data Integration): داده‌ها ممکن است از منابع مختلفی به‌دست بیایند. در این گام همه‌ی این منابع پراکنده را با هم ادغام می‌کنیم تا به یک مجموعه‌داده‌ی واحد و منسجم برسیم.
- تبدیل داده‌ها (Data Transformation): در این گام، داده‌ها را به قالبی درمی‌آوریم که برای الگوریتم‌های هوشمند قابل‌پردازش باشد.
- انتخاب داده‌ها (Data Selection): از آنجا که ما به داده‌های مرتبط با خروجی مد نظرمان نیاز داریم، لازم نیست همیشه همه‌ی داده‌ها را پردازش کنیم! در این گام، فقط آن بخش از داده‌ها را که واقعاً با مسئله‌ی ما مرتبط هستند، از میان حجم انبوه داده‌ها انتخاب می‌کنیم.

۲. داده‌کاوی (Data Mining)

این مرحله در واقع بخش اصلی و قلب تپندهٔ فرایند KDD محسوب می‌شود. در این مرحله، ما الگوریتم‌ها و روش‌های هوشمند را بر روی داده‌ها اعمال می‌کنیم تا الگوهای معنادار، روابط پنهان و مدل‌های تحلیلی را از دل داده‌های آماده‌شده استخراج کنیم.

۳. ارزیابی الگوها یا مدل‌ها (Pattern/Model Evaluation)

در این مرحله، خروجی‌ها بر اساس معیارهای جالب‌بودن (Interestingness Measures)، اعتبار، سودمندی و قابلیت تفسیر ارزیابی می‌شوند. هدف این است که الگوهای نويزدار یا ناقص را فیلتر کنیم و فقط آن بخش از دانش را که واقعاً برای تصمیم‌گیری سودمند هستند، حفظ کنیم.

۴. ارائهٔ دانش (Knowledge Presentation)

در آخرین مرحله، نوبت به نمایش نتایج می‌رسد. دانش استخراج‌شده باید با استفاده از روش‌های نمایش دانش و تکنیک‌های بصری‌سازی (Visualization) به گونه‌ای ارائه شود که برای کاربران نهایی کاملاً قابل درک، تفسیر و استفاده باشد.

مراحل پاک‌سازی، یکپارچه‌سازی، تبدیل و انتخاب داده‌ها در مجموع، عملیات پیش‌پردازش داده‌ها (Data Preprocessing) را تشکیل می‌دهند که هدف آن آماده‌سازی داده‌ها برای اجرای الگوریتم‌های داده‌کاوی است. در مرحله داده‌کاوی نیز ممکن است تعامل میان کاربر و پایگاه دانش برقرار باشد؛ به این ترتیب، الگوهای استخراج‌شده علاوه بر ارائه به کاربر، می‌توانند به‌عنوان دانش جدید در پایگاه دانش ذخیره شوند.

💡 **نکته کلیدی:** اگرچه داده‌کاوی مهم‌ترین مرحله در فرایند KDD است، اما بدون مراحل پیشین مانند پاک‌سازی و تبدیل داده، کیفیت نتایج به‌شدت کاهش می‌یابد. به عبارت دیگر، "داده‌های تمیز، پیش‌نیاز هر تحلیل معتبری هستند"



❖ مثال عملی: فرآیند کشف دانش در عمل

در این مثال، فرآیند کشف دانش را با استفاده از یک سناریو واقعی در صنعت تحلیل می‌کنیم. فرض کنید یک شرکت فروشگاهی می‌خواهد از رفتار مشتریان خود برای پیش‌بینی خرید بعدی استفاده کند. این فرآیند به شرح زیر انجام می‌شود:

1. آماده‌سازی داده:

- داده‌های فروش از چندین فروشگاه و سیستم‌های آنلاین جمع‌آوری می‌شوند.
- داده‌ها پاک‌سازی می‌شوند: مثلاً رکوردهای با اطلاعات ناقص یا تاریخ‌های غیرمنطقی حذف می‌شوند.
- داده‌ها تجمیع و تبدیل می‌شوند: مثلاً تراکنش‌ها به دسته‌های فصلی و محصولی خلاصه می‌شوند.
- تنها داده‌های مرتبط با تحلیل (مثلاً خریدهای اخیر) انتخاب می‌شوند.

2. داده‌کاوی:

- از الگوریتم‌هایی مانند Apriori یا FP-Growth برای کشف الگوهای خرید (مانند "خرید لپ‌تاپ → خرید کیف حمل") استفاده می‌شود.
- از روش‌هایی مانند درخت تصمیم یا شبکه عصبی برای پیش‌بینی احتمال خرید محصولات خاص استفاده می‌شود.

3. ارزیابی الگو:

- الگوهای کشف‌شده بر اساس معیارهایی مانند حمایت (support)، اطمینان (confidence) و جالبیت (interestingness) ارزیابی می‌شوند.

- مثلاً الگوی "خرید لپ‌تاپ → خرید کیف حمل" با حمایت بالا و اطمینان قابل توجه انتخاب می‌شود.

4. ارائه دانش:

- نتایج به مدیران با استفاده از گرافیک‌های تعاملی و گزارش‌های خلاصه ارائه می‌شود.

- این دانش به عنوان یک استراتژی بازاریابی جدید (مثلاً پیشنهاد کیف حمل هنگام خرید لپ‌تاپ) در سیستم اجرایی ذخیره می‌شود.

این مثال نشان می‌دهد که چگونه داده‌کاوی به عنوان مرحله‌ای از فرآیند کشف دانش، در ایجاد ارزش از داده‌های عظیم نقش محوری ایفا می‌کند و چگونه از یک مجموعه بزرگ داده به دانش کاربردی تبدیل می‌شود.

In [3]: `import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
from sklearn.decomposition import TruncatedSVD`

```
import warnings
warnings.filterwarnings('ignore')
```

```
sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})
```

```
plt.rcParams['figure.dpi'] = 120
```

```
def display_academic_text(text, type="normal"):
    """
    Renders text in an academic format based on the specified type.
    """

    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }

    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))
```

```
def get_protected_css():
    """
    Returns the essential CSS required to format tables cleanly.
    """

    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
    color: #ffffff !important;
    padding: 12px 18px !important;
```

```
text-align: center !important;
border: 1px solid #34495e !important;
white-space: nowrap !important;
font-weight: bold !important;
}
table.strict-tbl tbody td {
padding: 10px 18px !important;
text-align: center !important;
border: 1px solid #bdc3c7 !important;
white-space: nowrap !important;
color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
background-color: #e8ecef !important;
}
</style>
"""

def display_side_by_side_datasets(dfs, titles, *args):
    """
    Renders one or multiple pandas DataFrames safely.
    """
    if not isinstance(dfs, list) and not isinstance(titles, list):
        if len(args) >= 2:
            df1, title1, df2, title2 = dfs, titles, args[0], args[1]
            dfs = [df1, df2]
            titles = [title1, title2]
        else:
            dfs = [dfs]
            titles = [titles]

    if len(dfs) == 1:
        df = dfs[0]
        title = titles[0]
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        html = f"""
<div style="width: 100%; margin: 20px 0 35px 0;">
    <div dir="rtl" style="background-color: #2c3e50; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
        """
        display(HTML(get_protected_css() + html))
        return

    col_html_list = []
    for i, (df, title) in enumerate(zip(dfs, titles)):
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        bg_color = "#2c3e50"
        col_html = f"""
<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
    <div style="background-color: {bg_color}; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
        """
        col_html_list.append(col_html)

    columns_joined = "".join(col_html_list)
    html = f"""
<div style="display: flex; flex-direction: row; justify-content: space-between;
    gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
    box-sizing: border-box;">
    {columns_joined}
</div>
        """
    display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');

* { font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important; }
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
"""))

np.random.seed(42)
```

In [4]: display_academic_text("«در «نتفلیکس (SVD) مثال عملی: استخراج ویژگی‌های پنهان و سیستم توصیه‌گر هوشمند با تجزیه ماتریس»", "main_title")
display_academic_text("در این فرآیند (Sparse) در پلتفرم‌های پخش محتوا مانند نتفلیکس، پایگاه داده بسیار خلوت
=====
PHASE 1: DATA INTEGRATION AND PREPROCESSING
=====
display_academic_text("Data Preparation) (یکپارچه‌سازی و پیش‌پردازش داده‌ها)", "sub_title")
display_academic_text("تبدیل می‌شوند. خانه‌های خالی (فیلم‌های دیده‌نشده) با مقدار صفر پر می‌شوند (User-Item Matrix) در این مرحله، لاگ‌های امتیازدهی کاربران جمع‌آوری شده و به یک ماتریس تعاملی", "paragra

```

# Define target users and a catalog of movies spanning different genres
users = [f'User_{i}' for i in range(1, 9)]
movies = [
    'Interstellar', 'Inception', 'The Matrix', # Sci-Fi / Mind-bending
    'The Godfather', 'Pulp Fiction', # Crime / Drama
    'Toy Story', 'Finding Nemo', # Animation / Family
    'The Conjuring', 'It' # Horror
]

# Generate a highly sparse user-item rating matrix (Ratings from 1 to 5, 0 = Unrated)
# We design latent patterns (e.g., User_1 loves Sci-Fi, User_5 loves Animation)
rating_matrix = np.array([
    [5, 4, 5, 0, 0, 0, 0, 0], # User 1: Sci-Fi fan
    [0, 0, 0, 5, 5, 0, 0, 0], # User 2: Crime fan
    [0, 0, 0, 0, 0, 5, 4, 0], # User 3: Animation fan
    [0, 0, 0, 0, 0, 0, 0, 5], # User 4: Horror fan
    [4, 5, 4, 0, 0, 0, 0, 0], # User 5: Sci-Fi fan
    [0, 0, 0, 4, 5, 0, 0, 0], # User 6: Crime fan
    [3, 0, 0, 0, 0, 4, 5, 0], # User 7: Mixed (Sci-Fi + Animation)
    [0, 0, 0, 0, 0, 0, 0, 4], # User 8: Horror fan
])

df_ratings = pd.DataFrame(rating_matrix, columns=movies, index=users)
df_ratings.insert(0, 'UserID', df_ratings.index)

display_side_by_side_datasets(
    [df_ratings.head(8)],
    ["ماتریس تعاملی کاربر-آیتم (خلوت و دارای مقادیر صفر). ۱."]
)

# =====
# PHASE 2: DATA MINING (LATENT FEATURE EXTRACTION)
# =====
display_academic_text("۲. پردازش الگوریتمی و داده‌کاوی (Data Mining)", "sub_title")
display_academic_text("ت خودکار مفاهیم پنهانی (مانند ژانر فیلم) را کشف کرده و تمام صفرهای ماتریس را با پیش‌بینی‌های دقیق پر می‌کند (k=3) ماتریس اصلی به ابعاد پایین‌تری، Truncated SVD با استفاده از الگوریتم ۱."

# Initialize and fit Truncated SVD (Matrix Factorization)
# We reduce the 9-dimensional movie space into 3 Latent Features (Hidden Genres)
n_latent_features = 3
svd = TruncatedSVD(n_components=n_latent_features, random_state=42)

# Decompose the original sparse matrix into User-Latent matrix and Latent-Movie matrix
user_latent_matrix = svd.fit_transform(rating_matrix)
item_latent_matrix = svd.components_

# Reconstruct the matrix to generate predicted ratings for all unseen movies
predicted_ratings_matrix = np.dot(user_latent_matrix, item_latent_matrix)

# Format the reconstructed matrix into a readable DataFrame
df_predictions = pd.DataFrame(np.round(predicted_ratings_matrix, 2), columns=movies, index=users)
df_predictions.insert(0, 'UserID', df_predictions.index)

display_side_by_side_datasets(
    [df_predictions.head(8)],
    ["(پر شدن مقادیر صفر با پیش‌بینی‌ها) SVD بازسازی شده توسط ۲."]
)

# =====
# PHASE 3: PATTERN EVALUATION
# =====
display_academic_text("۳. ارزیابی الگوها (Pattern Evaluation)", "sub_title")
display_academic_text("paragraph", ". در این مرحله، الگوریتم باید فیلم‌هایی که کاربر قبلاً تماشا کرده است را فیلتر کرده و تنها محتوای جدید را با بالاترین امتیاز پیش‌بینی‌شده، استخراج نماید")

# Define target user for recommendation evaluation
target_user_idx = 6 # User 7 (Mixed: Sci-Fi + Animation)
target_user_name = users[target_user_idx]

# Extract original viewing history and new predictions
original_user_ratings = rating_matrix[target_user_idx]
predicted_user_ratings = predicted_ratings_matrix[target_user_idx]

# Mask movies the user has already watched (Original rating > 0)
unwatched_mask = (original_user_ratings == 0)

# Extract and sort predictions for UNWATCHED movies only
recommendations = []
for m_idx, is_unwatched in enumerate(unwatched_mask):
    if is_unwatched:
        recommendations.append({
            'Movie': movies[m_idx],
            'Predicted_Score': round(predicted_user_ratings[m_idx], 2)
        })

# Create a DataFrame and sort by highest predicted score
df_recommendations = pd.DataFrame(recommendations).sort_values(by='Predicted_Score', ascending=False)

results_text = (
    f"<b>{target_user_name}</b>: دانش استخراج‌شده برای<b> "
    f"را دیده است (انیمیشن) Finding Nemo و Toy Story و (تخیلی) Interstellar این کاربر تاکنون فقط دو فیلم "
    f"بدون اینکه بداند ژانر چیست، متوجه شد که این کاربر به فیلم‌های تخیلی علاقه دارد SVD الگوریتم "
    f"!<b>{df_recommendations.iloc[1]['Movie']}</b> و <b>{df_recommendations.iloc[0]['Movie']}</b> بالاترین امتیاز پیش‌بینی‌شده را به فیلم‌های "
)
display_academic_text(results_text, "insight")

# =====
# PHASE 4: KNOWLEDGE REPRESENTATION
# =====
display_academic_text("۴. بازنمایی و ارائه دانش (Knowledge Representation)", "sub_title")
display_academic_text("paragraph", ". به تیم محصول ارائه می‌شود تا تفاوت ماتریس خلوت اولیه و هوشمندی ماتریس بازسازی‌شده به تصویر کشیده شود (Heatmaps) نتایج ماشین‌لرنینگ با استفاده از نقشه‌های حرارتی")

# Setup Matplotlib figure for side-by-side heatmaps
fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

```


۲. پردازش الگوریتمی و داده‌کاوی (Data Mining)

با استفاده از الگوریتم Truncated SVD، ماتریس اصلی به ابعاد پایین‌تری ($k=3$) شکسته می‌شود. ماشین به صورت خودکار مفاهیم پنهانی (مانند ژانر فیلم) را کشف کرده و تمام صفرهای ماتریس را با پیش‌بینی‌های دقیق پر می‌کند.

۲. ماتریس بازسازی‌شده توسط SVD (پر شدن مقادیر صفر با پیش‌بینی‌ها)

UserID	Interstellar	Inception	The Matrix	The Godfather	Pulp Fiction	Toy Story	Finding Nemo	The Conjuring	It
User_1	4.80	4.14	4.18	0.00	0.00	1.31	1.42	-0.18	-0.18
User_2	-0.00	0.00	0.00	4.74	5.24	-0.00	-0.00	0.00	0.00
User_3	0.97	0.81	0.82	-0.00	-0.00	0.33	0.35	0.53	0.53
User_4	-0.00	-0.18	-0.18	0.00	0.00	0.53	0.53	4.43	4.43
User_5	4.43	3.82	3.85	0.00	0.00	1.20	1.31	-0.18	-0.18
User_6	-0.00	0.00	0.00	4.29	4.74	-0.00	-0.00	0.00	0.00
User_7	2.11	1.79	1.81	-0.00	-0.00	0.64	0.69	0.53	0.53
User_8	-0.00	-0.18	-0.18	0.00	0.00	0.53	0.53	4.43	4.43

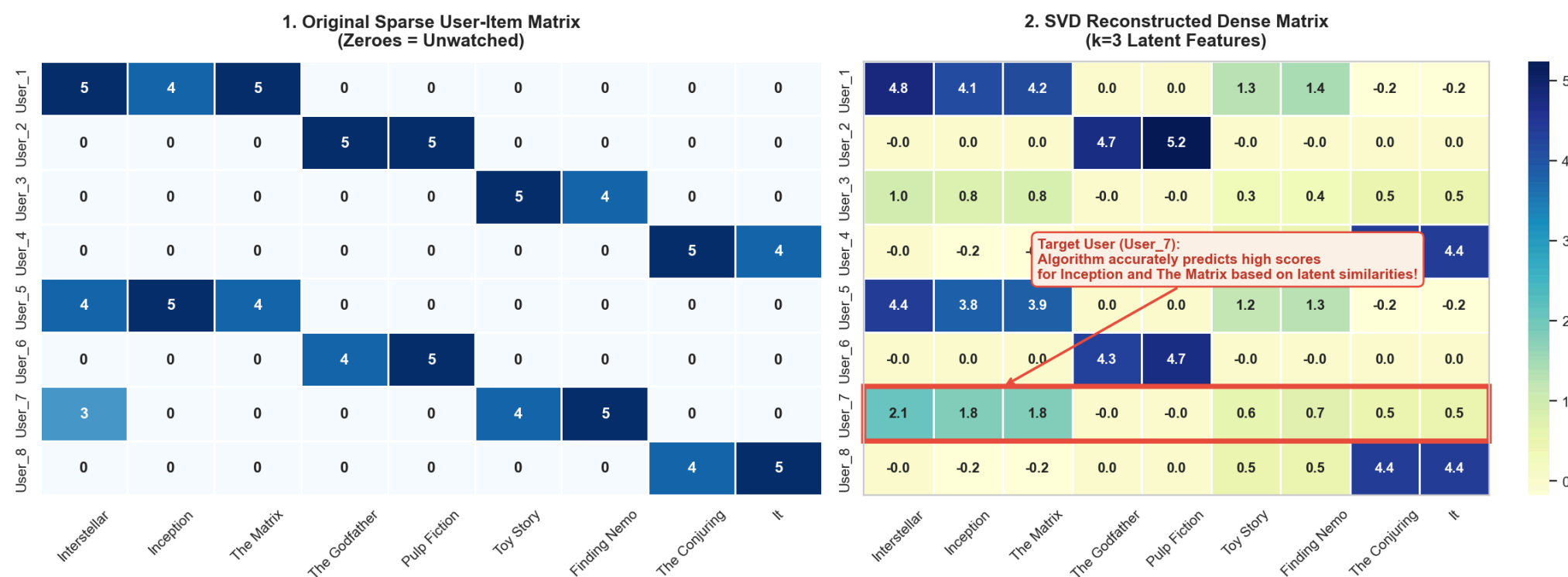
۳. ارزیابی الگوها (Pattern Evaluation)

در این مرحله، الگوریتم باید فیلم‌هایی که کاربر قبلاً تماشا کرده است را فیلتر کرده و تنها محتوای جدید را با بالاترین امتیاز پیش‌بینی‌شده، استخراج نماید.

دانش استخراج‌شده برای User_7: این کاربر تاکنون فقط دو فیلم Interstellar (تخیلی) و Toy Story و Finding Nemo (انیمیشن) را دیده است. الگوریتم SVD بدون اینکه بداند ژانر چیست، متوجه شد که این کاربر به فیلم‌های تخیلی علاقه دارد. بنابراین، بالاترین امتیاز پیش‌بینی‌شده را به فیلم‌های The Matrix و Inception اختصاص داده است!

۴. بازنمایی و ارائه دانش (Knowledge Representation)

نتایج ماشین‌لرنینگ با استفاده از نقشه‌های حرارتی (Heatmaps) به تیم محصول ارائه می‌شود تا تفاوت ماتریس خلوت اولیه و هوشمندی ماتریس بازسازی‌شده به تصویر کشیده شود.



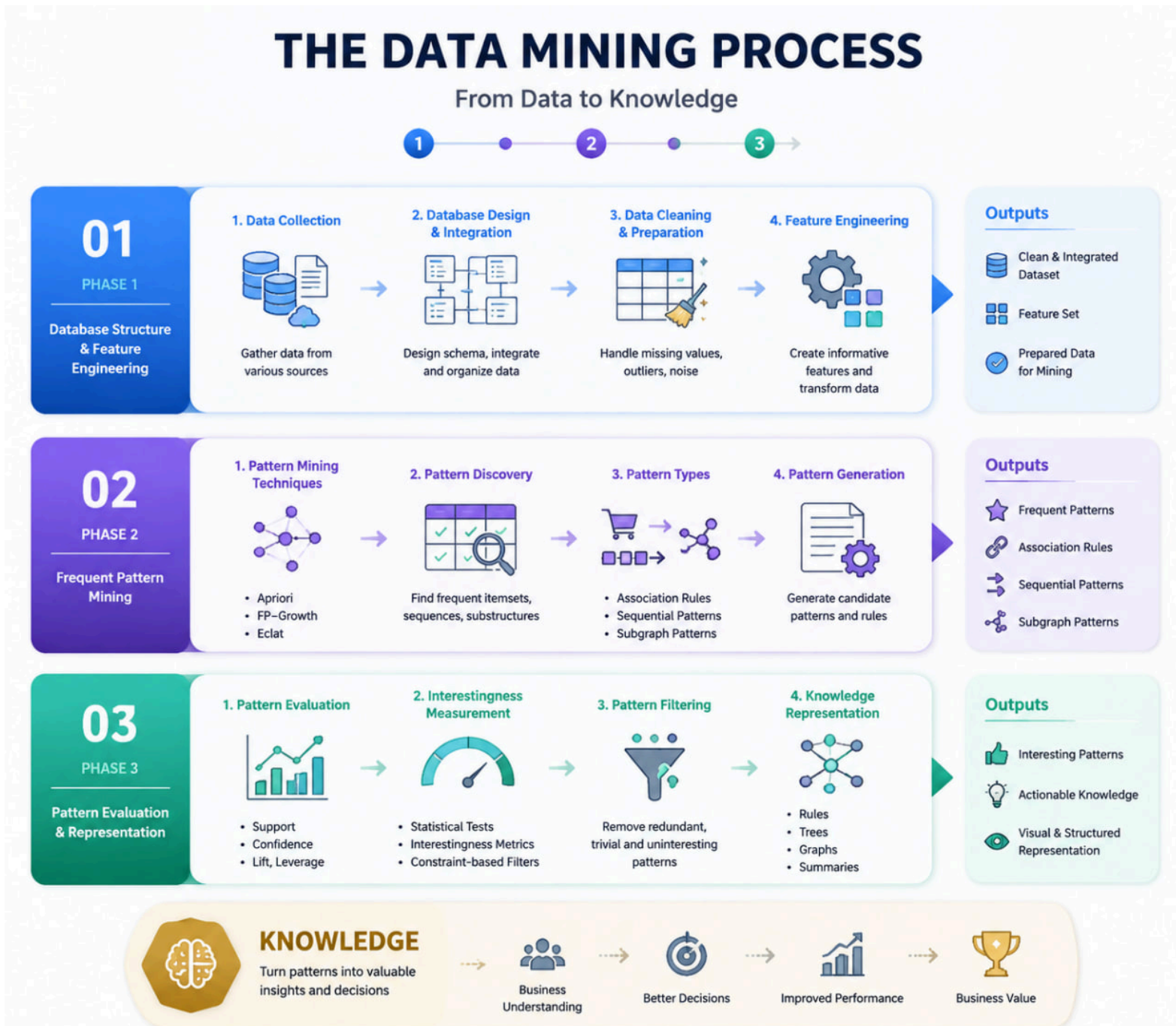
نتیجه استراتژیک (جادوی کشف ویژگی‌های پنهان): اگر نتفلیکس می‌خواست از برنامه‌نویسی سنتی استفاده کند، باید هزاران ژانر و تگ را به صورت دستی وارد می‌کرد. اما الگوریتم SVD (Matrix Factorization) یک جعبه‌جادویی است! در گراف ۱ می‌بینیم که ماتریس اولیه پر از صفر است. ماشین این ماتریس بزرگ را مچاله می‌کند تا به ۳ بُعد پنهان برسد و سپس دوباره آن را باز می‌کند. با این کار (گراف ۲)، تمام خانه‌های خالی با اعداد دقیق پر می‌شوند. ماشین بدون دانستن کلمه «تخیلی»، یاد گرفته است کسانی که Interstellar دیده‌اند، قطعاً The Matrix را هم دوست خواهند داشت. این فرآیند KDD، پایه‌گذار سیستم‌های مدرن هوش مصنوعی در شرکت‌های استریم است.

توضیحات مثال کد پایتون

در این مثال، فرآیند کامل کشف دانش از داده (KDD) به صورت عملیاتی شبیه‌سازی شده است:

- ابتدا کتابخانه‌های لازم شامل **pandas** برای مدیریت داده، **mlxtend** برای انجام داده‌کاوی و **matplotlib** برای بصری‌سازی نتایج وارد شده‌اند.
- یک مجموعه داده فرضی از **تراکنش‌های خرید مشتریان** در یک شبکه فروشگاهی ایجاد شده که شامل شناسه تراکنش، لیست آیتم‌های خریداری‌شده و تاریخ است.
- در مرحله **آماده‌سازی داده**، داده‌ها پاک‌سازی شده (در اینجا فرض تمیز بودن داده)، تجمیع شده (شبیه‌سازی داده از چندین فروشگاه) و با تبدیل به فرمت یک‌دسته‌ای (one-hot encoding) برای داده‌کاوی آماده می‌شوند. همچنین فقط داده‌های هفته اخیر برای تحلیل انتخاب شده‌اند.
- در مرحله **داده‌کاوی**، از الگوریتم **Apriori** برای کشف الگوهای پرتکرار در خرید مشتریان استفاده شده است. این الگوریتم به دنبال مجموعه‌ای از آیتم‌هاست که به‌طور مکرر با هم خریداری می‌شوند (مانند "لپ‌تاپ و کیف حمل").
- در مرحله **ارزیابی الگو**، الگوهای کشف‌شده بر اساس معیارهایی مانند **حمایت (Support)** فیلتر شده‌اند و فقط الگوهایی که شامل هم‌خرید لپ‌تاپ و کیف باشند، به عنوان الگوهای ارزشمند انتخاب شده‌اند.
- در نهایت، در مرحله **ارائه دانش**، نتایج به صورت متنی و گرافیکی به کاربر (مثلاً مدیر فروش) ارائه شده‌اند. نمودار نشان می‌دهد که چگونه یک دانش عملیاتی ("مشتریانی که لپ‌تاپ می‌خرند، اغلب کیف نیز می‌خرند") از داده‌های خام استخراج شده است.

این مثال به‌وضوح نشان می‌دهد که چگونه داده‌کاوی به عنوان یکی از مراحل فرآیند KDD، می‌تواند به تبدیل داده‌های خام به **دانش کاربردی** برای تصمیم‌گیری‌های تجاری کمک کند.



📁 تنوع انواع داده در داده‌کاوی

داده‌کاوی یک فناوری عمومی و منعطف است که می‌تواند روی انواع مختلف داده‌ها اعمال شود، به شرطی که آن داده‌ها برای هدف ما معنادار باشند. اما واقعیت این است که نمی‌توان برای همه‌ی داده‌ها از یک الگوریتم یکسان استفاده کرد؛ چرا که تفاوت در ساختار، ماهیت و کاربرد آن‌ها باعث می‌شود که هر دسته از داده‌ها، روش‌ها و الگوریتم‌های متفاوتی را برای تحلیل لازم داشته باشد. به همین دلیل، داده‌کاوی به یک حوزه‌ی گسترده و میان‌رشته‌ای تبدیل شده است.

بیایید نگاهی به دسته‌بندی داده‌ها بیندازیم؛ از نظر ساختار، ما به‌طور کلی با سه دسته‌ی ساختاریافته، نیمه‌ساختاریافته و بدون ساختار سر و کار داریم.

دسته‌بندی داده‌ها از نظر ساختار

- **داده‌های ساختاریافته (Structured Data):** این داده‌ها نظم و انضباط خاصی دارند و در قالب‌های از پیش تعریف‌شده قرار می‌گیرند. این داده‌ها را معمولاً در پایگاه‌های داده رابطه‌ای یا انبارهای داده ذخیره می‌کنیم.
- **داده‌های نیمه‌ساختاریافته (Semi-Structured Data):** این دسته از داده‌ها اگرچه ساختار ثابتی ندارند، اما روابط و معنا در آن‌ها کاملاً قابل تشخیص است. داده‌های تراکنشی، سری‌های زمانی، اسناد XML و داده‌های گراف در این گروه قرار می‌گیرند.
- **داده‌های بدون ساختار (Unstructured Data):** این دسته شامل متن، تصویر، صوت و ویدئو هستند که شاید در نگاه اول فاقد نظم به نظر برسند، اما حاوی اطلاعات ارزشمند هستند. تحلیل این مدل داده‌ها مستلزم بهره‌گیری از روش‌های پیشرفته‌ای مانند بینایی ماشین و یادگیری عمیق است.

کاربرد

📌 در پروژه‌های واقعی داده‌کاوی، اغلب با ترکیبی از انواع داده مواجه هستیم. به‌عنوان مثال، یک اپلیکیشن موبایل ممکن است همزمان از داده‌های ساختاریافته (مشخصات کاربر)، نیمه‌ساختاریافته و بدون ساختار (تصاویر پروفایل) استفاده کند. بنابراین، یک راه‌حل داده‌کاوی موفق، معمولاً نیازمند ترکیب چندین تکنیک و همکاری تیمی با تخصص‌های مختلف است.

آیا فقط ساختار داده همه‌چیز را تعیین می‌کند؟

اما ساختار داده همه‌چیز نیست! نوع کاربرد داده‌ها نیز در انتخاب روش داده‌کاوی مؤثر است. داده‌های زیستی، مالی، شبکه‌های اجتماعی و داده‌های مکانی و زمانی، حتی اگر ساختار مشابهی داشته باشند، معنا و هدف تحلیل‌شان آن‌قدر متفاوت است که نمی‌شود یک الگوریتم یکسان برای همه‌ی آن‌ها در نظر گرفت.

برای مثال، توالی‌های بیولوژیکی مانند DNA یا پروتئین، با اینکه از نظر فنی شبیه توالی‌های خرید یا کلیک‌برداری‌های وب هستند، اما معنای کاملاً متفاوتی دارند و هرکدام از آن‌ها روش‌های کاوش خاص خودشان را می‌طلبند. جالب است بدانید که داده‌های سری زمانی نوع خاصی از داده‌های توالی‌محور هستند که فواصل زمانی منظمی دارند؛ چنین چیزی در توالی‌های خرید دیده نمی‌شود، زیرا یک مشتری هر وقت دلش بخواهد و بدون برنامه‌ی زمانی ثابت خرید می‌کند!

گاهی اوقات داده‌ها با اطلاعات مکانی و زمانی گره خورده‌اند. برای تحلیل این داده‌های مکانی-زمانی، باید ابزارهای تخصصی‌تری مانند داده‌کاوی مکانی، داده‌کاوی زمانی یا کاوش الگوهای مسیر را توسعه دهیم تا درک کنیم چه اتفاقی در حال وقوع است.

حتی برای یک مجموعه داده یکسان نیز ممکن است بر اساس هدف‌تان، به تکنیک‌های متفاوتی نیاز داشته باشید.

در حقیقت، همین تنوع فوق‌العاده‌ی داده‌ها و نیازهای متفاوت کاربردی است که داده‌کاوی را به یک حوزه‌ی تحقیقاتی جذاب و پرجنب‌وجوش تبدیل کرده که روش‌ها و تکنیک‌های جدید در آن منتظر مطالعه و توسعه هستند.

مثال عملی : شبیه‌سازی داده‌های توالی و کاوش الگوی زمانی

دیجی‌کالا به‌عنوان بزرگ‌ترین فروشگاه آنلاین ایران، روزانه پتابایت‌ها داده از رفتارهای میلیون‌ها کاربر جمع‌آوری می‌کند. اگر این داده‌ها را کالبدشکافی کنیم، دقیقاً تمام دسته‌بندی‌های متن کتاب را در آن می‌بینیم:

۱. داده‌های ساختاریافته (Structured Data)

چیست؟ داده‌های جدولی (سطر و ستونی) در پایگاه‌های داده رابطه‌ای که طرح (Schema) کاملاً مشخصی دارند.

در دیجی‌کالا:

- جدول مشخصات فنی کالاها (نام کالا، برند، میزان حافظه، قیمت).
- جدول پروفایل مشتریان (شناسه کاربر، تاریخ ثبت‌نام، شهر سکونت).

کاربرد داده‌کاوی: محاسبه میانگین قیمت‌ها، رگرسیون برای پیش‌بینی فروش و دسته‌بندی مشتریان بر اساس منطقه جغرافیایی.

۲. داده‌های نیمه‌ساختاریافته، تراکنشی و گراف (Semi-Structured / Transactional / Graph)

چیست؟ داده‌هایی که جدول سخت‌گیرانه SQL را ندارند، اما برچسب‌ها (Tags) یا روابط ساختاریافته در آن‌ها مشهود است.

در دیجی‌کالا:

- داده تراکنشی (Basket Data): فاکتورهای خرید؛ مثلاً سبد خریدی شامل [کاور گوشی، محافظ صفحه، هندزفری].
- داده توالی‌ای (Sequential / Clickstream): ترتیبی که کاربر کلیک کرده است؛ [جستجوی لپ‌تاپ → کلیک روی برند ایسوس → مقایسه با لنوو → افزودن به سبد].
- داده گراف و شبکه (Graph Data): گراف ارتباطی بین «کاربران متصل» یا «کالاهای مشابه» (کدام کاربران الگوی خرید یکسانی دارند).

کاربرد داده‌کاوی: اجرای قواعد وابستگی (Market Basket Analysis) برای سیستم پیشنهاددهنده (مثلاً: "خریداران این کالا، این موارد را هم خریده‌اند").

۳. داده‌های بدون ساختار (Unstructured Data)

چیست؟ متون آزاد، تصاویر و ویدیوهایی که ساختار سطری/ستونی ندارند اما حاوی دانشی ارزشمند هستند.

در دیجی‌کالا:

• متن: هزاران کامنت و نقد و بررسی متنی کاربران ذیل هر محصول.

• تصویر و ویدیو: عکس‌های خریداران از کالای تحویل‌گرفته‌شده یا ویدیوهای آنباکسینگ (Unboxing).

• صدا: فایل‌های ضبط‌شده تماس‌های مشتریان با مرکز پشتیبانی.

کاربرد داده‌کاوی: به‌کارگیری پردازش زبان طبیعی (NLP) برای تحلیل حس خریداران (Sentiment Analysis) جهت شناسایی کالاهای

باکیفیت یا مرجوعی، و استفاده از بینایی کامپیوتر (Computer Vision) برای حذف عکس‌های غیراخلاقی یا نامرتب.

۴. داده‌های مکانی-زمانی (Spatio-Temporal Data)

چیست؟ داده‌هایی که مختصات جغرافیایی (Location) و زمان (Time) در آن‌ها نقش کلیدی دارد.

در دیجی‌کالا: لوکیشن تحویل سفارش‌ها + زمان ثبت خرید (مثلاً خریدهای شبانه در مناطق خاص شهری).

کاربرد داده‌کاوی: داده‌کاوی مکانی برای بهینه‌سازی مسیر ناوگان توزیع (دیجی‌اکسپرس) و جانمایی هوشمند کالاها در انبارهای منطقه‌ای.

۵. داده‌های جریانی (Stream Data)

چیست؟ جریان نامحدود و مداومی از داده‌ها که به‌صورت آنی (Real-Time) وارد سیستم می‌شوند.

در دیجی‌کالا: جریان چندصدهزار کلیک هم‌زمان در حراجی‌های بزرگ (مثل بلک فرایدی یا حراج شگفت‌انگیز).

کاربرد داده‌کاوی: کشف زنده ناهنجاری (Anomaly Detection) جهت مسدود کردن ربات‌های خریدار یا جلوگیری از از دسترس خارج شدن

سرورها.

```
In [5]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
from datetime import datetime, timedelta
from collections import Counter
from itertools import combinations
from IPython.display import display, HTML
```

```
import warnings
warnings.filterwarnings('ignore')
```

```
sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})
```

```
plt.rcParams['figure.dpi'] = 135
```

```
def display_academic_text(text, type="normal"):
    """Renders text in a strict academic format."""
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
```

```
)
}
display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))

def get_protected_css():
    """Returns essential CSS for robust and clean table formatting."""
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
    color: #ffffff !important;
    padding: 12px 18px !important;
    text-align: center !important;
    border: 1px solid #34495e !important;
    white-space: nowrap !important;
    font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important;
    text-align: center !important;
    border: 1px solid #bdc3c7 !important;
    white-space: nowrap !important;
    color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
    background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
    background-color: #e8ecef !important;
}
</style>
"""

def display_side_by_side_datasets(dfs, titles, *args):
    """Renders single or multiple pandas DataFrames side-by-side gracefully."""
    if not isinstance(dfs, list) and not isinstance(titles, list):
        if len(args) >= 2:
            dfs = [dfs, args[0]]
            titles = [titles, args[1]]
        else:
            dfs = [dfs]
            titles = [titles]

    if len(dfs) == 1:
        df = dfs[0]
        title = titles[0]
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        html = f"""
<div style="width: 100%; margin: 20px 0 35px 0;">
    <div dir="rtl" style="background-color: #2c3e50; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
        """
        display(HTML(get_protected_css() + html))
        return

    col_html_list = []
    for i, (df, title) in enumerate(zip(dfs, titles)):
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        bg_color = "#2c3e50"
        col_html = f"""
<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
    <div style="background-color: {bg_color}; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
        """
        col_html_list.append(col_html)

    columns_joined = "".join(col_html_list)
    html = f"""
<div style="display: flex; flex-direction: row; justify-content: space-between;
    gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
    box-sizing: border-box;">
    """
```

```
{columns_joined}
</div>
"""
display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');
* { font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important; }
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
"""))

np.random.seed(42)
```

In [6]: display_academic_text("گونه‌شناسی داده‌ها در کشف دانش: تحلیل جامع ساختارهای اطلاعاتی", "main_title")
display_academic_text("پلتفرم‌های تجارت الکترونیک نظیر دیجی‌کالا، روزانه حجم عظیمی از داده‌های رفتاری را جمع‌آوری می‌نمایند. تحلیل معماری این کلان‌داده‌ها"

```
# =====
# 1. STRUCTURED DATA PROCESSING (RDBMS)
# =====
display_academic_text("داده‌های ساختاریافته (Structured Data)", "sub_title")
display_academic_text("این کلاس شامل داده‌های موجود در پایگاه‌های داده رابطه‌ای (RDBMS) است که دارای طرح‌واره (Schema) است و دسته‌بندی است"
```

```
# Generate synthetic product database
n_products = 200
brands = ['Samsung', 'Apple', 'Xiaomi', 'Asus', 'Lenovo']
```

```
products = pd.DataFrame({
    'ProductID': range(1, n_products + 1),
    'Brand': np.random.choice(brands, n_products),
    'RAM_GB': np.random.choice([4, 8, 16, 32], n_products),
    'Price': np.random.randint(3_000_000, 60_000_000, n_products)
})
```

```
# Generate synthetic customer database
cities = ['Tehran', 'Mashhad', 'Isfahan', 'Shiraz', 'Tabriz']
n_customers = 1000
customers = pd.DataFrame({
    'CustomerID': range(1, n_customers + 1),
    'City': np.random.choice(cities, n_customers, p=[0.4, 0.15, 0.15, 0.15, 0.15]),
    'JoinDate': [datetime(2022, 1, 1) + timedelta(days=int(x)) for x in np.random.randint(0, 900, n_customers)]
})
```

```
# Prepare subset for display
cust_disp = customers.head(5).copy()
cust_disp['JoinDate'] = cust_disp['JoinDate'].dt.strftime('%Y-%m-%d')
```

```
display_side_by_side_datasets(
    [products.head(5), cust_disp],
    ["(Products Table) نمونه مشاهدات جدول پروفایل مشتریان", "(Customers Table) نمونه مشاهدات جدول محصولات"]
)
```

```
# Knowledge Extraction: Aggregation and Linear Regression
avg_price_by_brand = products.groupby('Brand')['Price'].mean().sort_values(ascending=False).reset_index()
avg_price_by_brand.columns = ['Brand', 'Average Price (IRR)']
```

```
# Simple Linear Regression (Price ~ RAM)
slope, intercept = np.polyfit(products['RAM_GB'], products['Price'], 1)
```

display_side_by_side_datasets([avg_price_by_brand, ["دانش استخراج‌شده: محاسبه میانگین قیمت به تفکیک برند"]])
display_academic_text("استخراج گریدید «Price ≈ {slope:,.0f} × RAM + {intercept:,.0f}» با رابطه فرموله شده (RAM) استنتاج آماری: مدل‌سازی رگرسیون خطی جهت پیش‌بینی قیمت بر مبنای ظرفیت حافظه موقت"

```
# =====
# 2. SEMI-STRUCTURED, TRANSACTIONAL, AND GRAPH DATA
# =====
display_academic_text("داده‌های نیمه‌ساختاریافته، تراکنشی و مبتنی بر گراف", "sub_title")
display_academic_text("این دسته شامل رکوردهایی است که فاقد ساختار جدولی صلب می‌باشند اما از منطق رابطه‌ای یا توالی زمانی پیروی می‌کنند؛ نظیر سبد خریدهای هم‌زمان"
```

```
# A) Market Basket Data (Simulating checkout carts)
n_baskets = 500
baskets = []
for _ in range(n_baskets):
    if np.random.rand() < 0.5:
        basket = np.random.choice(['CoverPhone', 'ScreenProtector', 'Handsfree', 'PowerBank'], size=np.random.randint(2, 4), replace=False).tolist()
    else:
        basket = np.random.choice(['Laptop', 'Mouse', 'LaptopBag', 'USB_Hub'], size=np.random.randint(2, 4), replace=False).tolist()
    baskets.append(basket)
```

```
basket_df = pd.DataFrame({'Simulated Transaction (Basket)': [str(b) for b in baskets[:5]]})
display_side_by_side_datasets([basket_df, ["(Market Basket Data) نمونه ساختار داده‌های تراکنشی (الف)"]])
```

```
# Knowledge Extraction: Association Rule Mining (Frequent item pairs)
pair_counts = Counter()
for basket in baskets:
    for pair in combinations(sorted(basket), 2):
        pair_counts[pair] += 1
```

```
top_pairs = pair_counts.most_common(4)
rules_data = [{"Item A": p[0], 'Item B': p[1], 'Absolute Frequency': c, 'Support Measure': f"{c / n_baskets:.3f}"} for p, c in top_pairs]
display_side_by_side_datasets([pd.DataFrame(rules_data), ["دانش استخراج‌شده: استنتاج پرتکرارترین قواعد انجمنی جفت‌کالاها"]])
```

```
# B) Clickstream Data (Sequential pattern generation)
click_events = ['Search_Laptop', 'Click_Asus', 'Click_Lenovo', 'Compare', 'AddToCart', 'ViewReviews', 'Checkout']
n_sessions = 300
sessions = []
for _ in range(n_sessions):
    length = np.random.randint(3, 6)
    if np.random.rand() < 0.6:
        seq = ['Search_Laptop', np.random.choice(['Click_Asus', 'Click_Lenovo']), 'Compare', 'AddToCart']
    else:
```

```
seq = list(np.random.choice(click_events, size=length, replace=True))
sessions.append(seq)
```

```
seq_df = pd.DataFrame({'User Session (Sequential Pattern)': [' → '.join(s) for s in sessions[:4]]})
display_side_by_side_datasets([seq_df], ["نمونه ساختار توالی داده‌های جریان‌ی کاربر (ب)"])
```

```
sequence_counts = Counter([tuple(s) for s in sessions])
most_common_sequence = sequence_counts.most_common(1)[0]
display_academic_text(f"« استخراج الگوی زمانی: رایج‌ترین مسیر پیمایش کاربران به فرمت '{' → '.join(most_common_sequence[0])}' » با نرخ تکرار {most_common_sequence[1]} نشست کاربری (Session) گردید (Hub) به‌عنوان تأثیرگذارترین گره {centrality[top_user]:.2f} معادل (Degree Centrality) با شاخص مرکزیت درجه {top_user} مشتری شماره (Network Analytics) تحلیل گراف")
```

```
# C) Graph & Network Analytics
n_graph_users = 15
G = nx.Graph()
G.add_nodes_from(range(1, n_graph_users + 1))

# Generate edges based on random collaboration probability
for i in range(1, n_graph_users + 1):
    for j in range(i + 1, n_graph_users + 1):
        if np.random.rand() < 0.15:
            G.add_edge(i, j)

centrality = nx.degree_centrality(G)
top_user = max(centrality, key=centrality.get)

# Visualize Entity Relationship Graph
plt.figure(figsize=(9, 5), dpi=120)
nx.draw_spring(
    G, with_labels=True, node_color='#1abc9c', node_size=800,
    font_weight='bold', font_color='white', edge_color='#bdc3c7', width=1.5
)
plt.title("ج) Entity Relationship Graph (Users with Similar Purchase Behavior)", fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=15)
plt.show()
```

شناسایی گردید (Hub) به‌عنوان تأثیرگذارترین گره {centrality[top_user]:.2f} معادل (Degree Centrality) با شاخص مرکزیت درجه {top_user} مشتری شماره (Network Analytics) تحلیل گراف")

```
# =====
# 3. UNSTRUCTURED DATA PROCESSING (NLP & CV)
# =====
display_academic_text("۳. پردازش داده‌های فاقد ساختار (Unstructured Data)", "sub_title")
display_academic_text("ی ماشین (NLP) داده‌هایی نظیر متون پردازش‌نشده، تصاویر، فایل‌های صوتی و ویدئویی که فاقد مرزبندی منطقی سیستماتیک می‌باشند. تحلیل این ساختارها متکی بر متدهای پردازش زبان طبیعی")
```

```
# NLP: Sentiment Analysis on raw text
comments = [
    "کیفیت این محصول عالی بود و خیلی راضی هستم",
    "پسته‌بندی خوب بود اما ارسال دیر شد",
    "اصلا راضی نبودم، محصول معیوب بود",
    "کیفیت ساخت خوب است ولی قیمت بالاست",
    "عالی بود، پیشنهاد میکنم حتما بخرید",
    "بد بود و خراب به دستم رسید، ناراضی هستم"
]
```

```
positive_words = ['عالی', 'خوب', 'راضی', 'باکیفیت', 'پیشنهاد']
negative_words = ['بد', 'ناراضی', 'معیوب', 'خراب', 'دیر']
```

```
def simple_sentiment(text):
    """Calculates a rudimentary sentiment score via lexicon matching."""
    score = sum(1 for w in positive_words if w in text) - sum(1 for w in negative_words if w in text)
    return 'Positive' if score > 0 else ('Negative' if score < 0 else 'Neutral')
```

```
reviews_df = pd.DataFrame({'User Textual Feedback (Raw)': comments})
reviews_df['Extracted Sentiment'] = reviews_df['User Textual Feedback (Raw)'].apply(simple_sentiment)
display_side_by_side_datasets([reviews_df], ["کاوش متون: تحلیل احساسات نظرات مشتریان"])
```

```
# CV Simulation: Image Flagging
media_data = pd.DataFrame({
    'MediaType': ['Image'] * 40 + ['Video'] * 10,
    'IsInappropriate': np.random.choice([0, 1], 50, p=[0.92, 0.08])
})
flagged = media_data['IsInappropriate'].sum()
display_academic_text(f"insight", "فایل رسانه‌ای نامتعارف از مجموع ۵۰ رکورد آپلود شده توسط کاربران گردید {flagged} تحلیل محتوای بصری: الگوریتم شناسایی تصویر موفق به تشخیص و فیلترینگ")
```

```
# =====
# 4. SPATIO-TEMPORAL DATA PROCESSING
# =====
display_academic_text("۴. کاوش داده‌های مکانی-زمانی (Spatio-Temporal Data Mining)", "sub_title")
display_academic_text("ی که در آن‌ها ابعاد مختصات جغرافیایی و توالی وقوع زمانی به عنوان پارامترهای کلیدی در نظر گرفته می‌شوند. کاوش این ساختار بهینه‌سازی زنجیره تأمین و مدیریت ناوگان توزیع را تسهیل می‌نماید")
```

```
n_orders = 2000
# Skewed probability distribution across hours to simulate realistic peak times
p_dist = np.array([1,1,1,1,1,1,2,3,4,5,5,6,6,6,5,5,6,7,8,9,10,8,4,2])
p_dist = p_dist / p_dist.sum()
```

```
orders = pd.DataFrame({
    'OrderID': range(1, n_orders + 1),
    'City': np.random.choice(cities, n_orders, p=[0.4, 0.15, 0.15, 0.15, 0.15]),
    'Hour': np.random.choice(range(24), n_orders, p=p_dist)
})
```

```
# Filter night orders
night_orders = orders[(orders['Hour'] >= 23) | (orders['Hour'] <= 4)]
night_pattern = night_orders['City'].value_counts().reset_index()
night_pattern.columns = ['City', 'Night Orders Frequency']
```

["توزیع مکانی-زمانی: فرکانس تراکنش‌های شبانه به تفکیک مناطق جغرافیایی"], [night_pattern.head(4)],

```
# Visualize Spatio-Temporal Density Map (Heatmap)
pivot = orders.pivot_table(index='City', columns='Hour', values='OrderID', aggfunc='count', fill_value=0)
```

```
plt.figure(figsize=(12, 4.5), dpi=135)
sns.heatmap(pivot, cmap='YlOrRd', cbar_kws={'label': 'Volume of Transactions'}, linewidths=0.5)
plt.title("Spatio-Temporal Density Map (City vs. Transaction Hour)", fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=15)
```

```
plt.xlabel("Temporal Dimension (Hour of Day)", fontweight='bold')
plt.ylabel("Spatial Dimension (City)", fontweight='bold')
plt.tight_layout()
plt.show()

# =====
# 5. STREAM DATA PROCESSING & ANOMALY DETECTION
# =====
display_academic_text("۵. تحلیل جریان داده‌ها (Stream Data Processing)", "sub_title")
display_academic_text("پژوهش‌های پردازش داده‌ها مستلزم الگوریتم‌های حافظه‌کارآمد با قابلیت واکنش آنی به ناهنجاری‌ها نظیر حملات سایبری (Real-Time) داده‌های جریانی نمایانگر توالی بی‌نهایت و بلادرنگ")

# Simulate a fast, real-time stream of incoming clicks
n_seconds = 200
normal_clicks = np.random.poisson(lam=50, size=n_seconds)

# Inject anomalous DDoS / Bot traffic spikes
bot_attack_seconds = [80, 81, 82, 120, 121]
for s in bot_attack_seconds:
    normal_clicks[s] += np.random.randint(300, 500)

stream_df = pd.DataFrame({'Time (Seconds)': range(n_seconds), 'Traffic Volume (Clicks)': normal_clicks})

# Detect anomalies via Rolling Z-Score
window = 10
stream_df['Rolling Mean'] = stream_df['Traffic Volume (Clicks)'].rolling(window).mean()
stream_df['Rolling Std'] = stream_df['Traffic Volume (Clicks)'].rolling(window).std()
stream_df['Calculated Z-Score'] = (stream_df['Traffic Volume (Clicks)'] - stream_df['Rolling Mean']) / stream_df['Rolling Std']

# Flag spikes exceeding a Z-Score of 3.0
anomalies = stream_df[stream_df['Calculated Z-Score'] > 3].copy()
anomalies['Calculated Z-Score'] = anomalies['Calculated Z-Score'].round(2)
display_side_by_side_datasets([anomalies[['Time (Seconds)', 'Traffic Volume (Clicks)', 'Calculated Z-Score']].head(5)], ["(Anomaly) کاوش داده‌های جریانی: رکوردهای شناسایی‌شده به‌عنوان ناهنجاری ترافیکی"])

# Visualize the live stream and the intercepted anomalies
plt.figure(figsize=(14, 5.5), dpi=135)
plt.plot(stream_df['Time (Seconds)', stream_df['Traffic Volume (Clicks)'], label='Real-time Data Stream', color='#2980b9', lw=2.0)
plt.scatter(anomalies['Time (Seconds)', anomalies['Traffic Volume (Clicks)'], color='#e74c3c', s=100, marker='X', label='Detected Anomaly (DDoS / Bot Pattern)', zorder=5)

plt.annotate(
    "Massive Bot Attack Detected!\nAlgorithm mathematically isolates the spike\nusing Rolling Z-Score.",
    xy=(81, 400), xytext=(20, 450),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

plt.xlabel("Temporal Window (Seconds)", fontsize=12, fontweight='bold')
plt.ylabel("Incoming Request Volume", fontsize=12, fontweight='bold')
plt.title("Real-time Stream Analysis: Automated Anomaly Detection using Moving Z-Score", fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=15)
plt.legend(frameon=True, facecolor='white', edgecolor='black', fontsize=11)
plt.grid(True, alpha=0.4, linestyle='--')
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# =====
# FINAL STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    "کافی نیست <i>ساختار یافته (جداول)</i> در پلتفرم‌های مدرن، اتکا به یک مدل رگرسیون ساده بر روی <b>نتیجه استراتژیک معماری کلان‌داده</b> را "
    "ترسیم نماید <i>(Network Analytics)</i> کشف کند، شبکه‌های پنهان شباهت کاربران را با گراف <i>(Market Basket)</i> یک شرکت مانند دیجی‌کالا برای موفقیت باید رفتار مشتری را در سبد خرید"
    "نیت پلتفرم را در برابر حملات <i>(Streaming)</i> بهینه کند، و در نهایت با رصد بلادرنگ جریان داده <i>(Spatio-Temporal)</i> استراتژی لجستیک خود را در بُعد مکان-زمان، <i>(NLP)</i> احساسات متون را درک کند"
    "است." <b>(KDD) گونه‌شناسی داده‌ها در کشف دانش</b> این کد، نقشه راه جامع و آکادمیک برای درک عمیق"
)
display_academic_text(insight_conclusion, "insight")
```

گونه‌شناسی داده‌ها در کشف دانش: تحلیل جامع ساختارهای اطلاعاتی (Data Typology)

پلتفرم‌های تجارت الکترونیک نظیر دیجی‌کالا، روزانه حجم عظیمی از داده‌های رفتاری را جمع‌آوری می‌نمایند. تحلیل معماری این کلان‌داده‌ها (Big Data) نشان‌دهنده تنوع ساختاری آن‌ها در پنج کلاس مجزا است که هر یک نیازمند متدولوژی پردازشی مختص به خود می‌باشند.

۱. داده‌های ساختاریافته (Structured Data)

این کلاس شامل داده‌های موجود در پایگاه‌های داده رابطه‌ای (RDBMS) است که دارای طرحواره (Schema) قطعی می‌باشند؛ نظیر جداول مشخصات کالاها و پروفایل مشتریان. داده‌کاوی بر روی این ساختار عموماً شامل تجمیع‌های آماری، مدل‌های رگرسیون و دسته‌بندی است.

نمونه مشاهدات جدول محصولات (Products Table)			
ProductID	Brand	RAM_GB	Price
1	Asus	8	20882750
2	Lenovo	8	14616410
3	Xiaomi	8	13192257
4	Lenovo	16	46167705
5	Lenovo	16	30460368

نمونه مشاهدات جدول پروفایل مشتریان (Customers Table)		
CustomerID	City	JoinDate
1	Tehran	2022-03-25
2	Mashhad	2022-05-10
3	Isfahan	2024-01-13
4	Tehran	2024-04-17
5	Tabriz	2022-05-25

دانش استخراج شده: محاسبه میانگین قیمت به تفکیک برند	
Brand	Average Price (IRR)
Apple	3.421010e+07
Xiaomi	3.342032e+07
Asus	3.212840e+07
Samsung	2.947503e+07
Lenovo	2.503138e+07

استنتاج آماری: مدل سازی رگرسیون خطی جهت پیش بینی قیمت بر مبنای ظرفیت حافظه موقت (RAM) با رابطه فرموله شده « $Price \approx -53,097 \times RAM + 31,774,737$ » استخراج گردید.

۲. داده های نیمه ساختاریافته، تراکنشی و مبتنی بر گراف

این دسته شامل رکوردهایی است که فاقد ساختار جدولی صلب می باشند اما از منطق رابطه ای یا توالی زمانی پیروی می کنند؛ نظیر سبد خریدهای هم زمان (Market Basket Data)، توالی کلیک های کاربر (Clickstream Data) و شبکه های ارتباطی موجودیت ها (Graph Data).

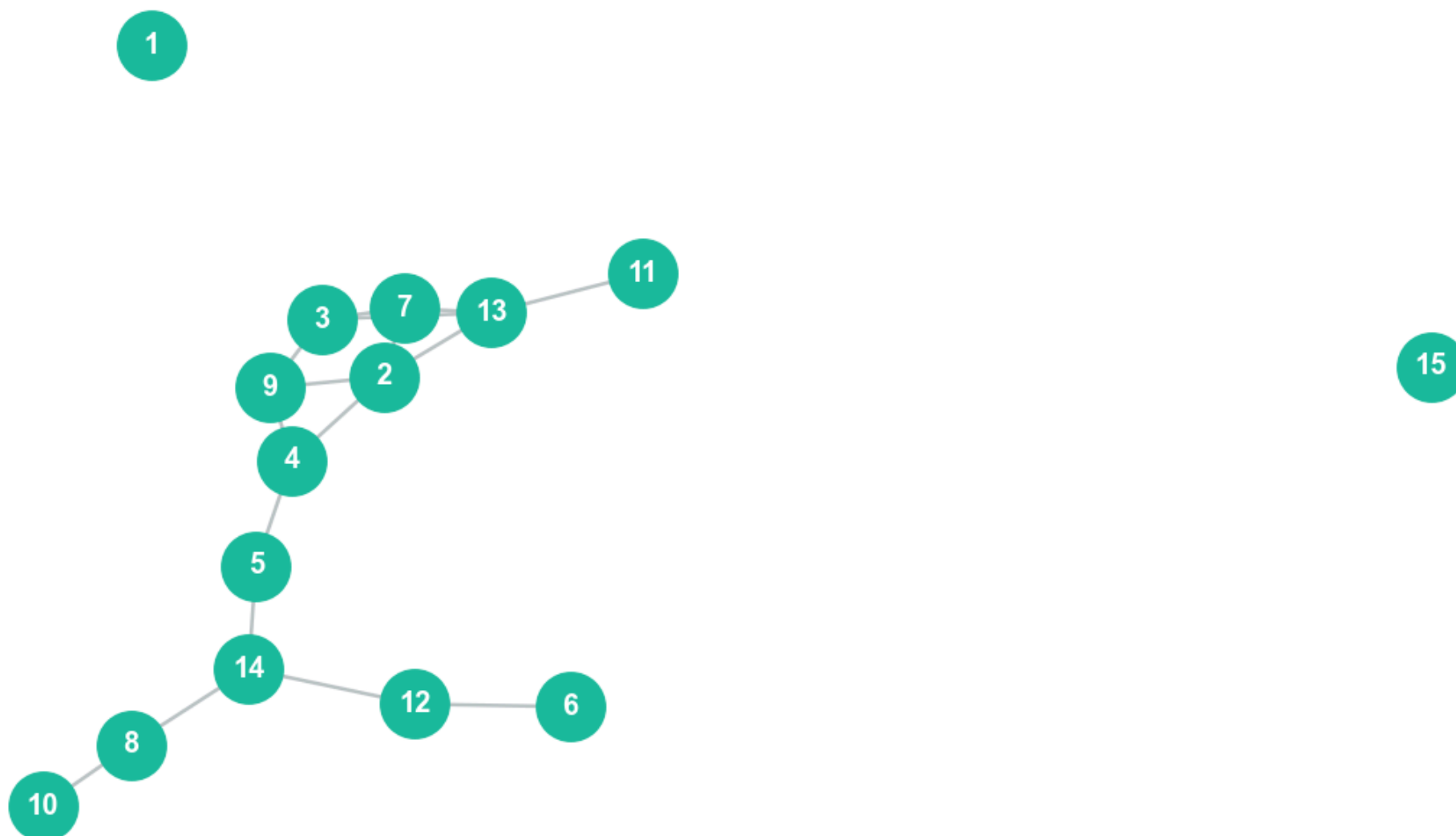
الف) نمونه ساختار داده های تراکنشی (Market Basket Data)
Simulated Transaction (Basket)
['USB_Hub', 'Mouse']
['PowerBank', 'Handsfree']
['CoverPhone', 'ScreenProtector']
['Mouse', 'Laptop', 'USB_Hub']
['LaptopBag', 'Mouse']

دانش استخراج شده: استنتاج پرتکرارترین قواعد انجمنی جفت کالاها			
Item A	Item B	Absolute Frequency	Support Measure
Laptop	Mouse	89	0.178
CoverPhone	Handsfree	89	0.178
Laptop	LaptopBag	87	0.174
CoverPhone	ScreenProtector	85	0.170

ب) نمونه ساختار توالی داده‌های جریانی کاربر (Clickstream Data)	
User Session (Sequential Pattern)	
Search_Laptop → Click_Lenovo → Compare → AddToCart	
Search_Laptop → Click_Lenovo → Compare → AddToCart	
Search_Laptop → Click_Asus → Compare → AddToCart	
Search_Laptop → Click_Lenovo → Compare → AddToCart	

استخراج الگوی زمانی: رایج‌ترین مسیر پیمایش کاربران به فرمت «Search_Laptop → Click_Asus → Compare → AddToCart» با نرخ تکرار 93 نشست کاربری (Session) کاوش گردید.

ج) Entity Relationship Graph (Users with Similar Purchase Behavior)



تحلیل گراف (Network Analytics): مشتری شماره 2 با شاخص مرکزیت درجه (Degree Centrality) معادل 0.29 به‌عنوان تأثیرگذارترین گره (Hub) در شبکه خریداران مشابه شناسایی گردید.

داده‌هایی نظیر متون پردازش‌نشده، تصاویر، فایل‌های صوتی و ویدئویی که فاقد مرزبندی منطقی سیستماتیک می‌باشند. تحلیل این ساختارها متکی بر متدهای پردازش زبان طبیعی (NLP) و بینایی ماشین (Computer Vision) است.

کاوش متون: تحلیل احساسات نظرات مشتریان (Sentiment Analysis)	
User Textual Feedback (Raw)	Extracted Sentiment
کیفیت این محصول عالی بود و خیلی راضی هستم	Positive
بسته‌بندی خوب بود اما ارسال دیر شد	Neutral
اصلا راضی نبودم، محصول معیوب بود	Neutral
کیفیت ساخت خوب است ولی قیمت بالاست	Positive
عالی بود، پیشنهاد میکنم حتما بخرید	Positive
بد بود و خراب به دستم رسید، ناراضی هستم	Negative

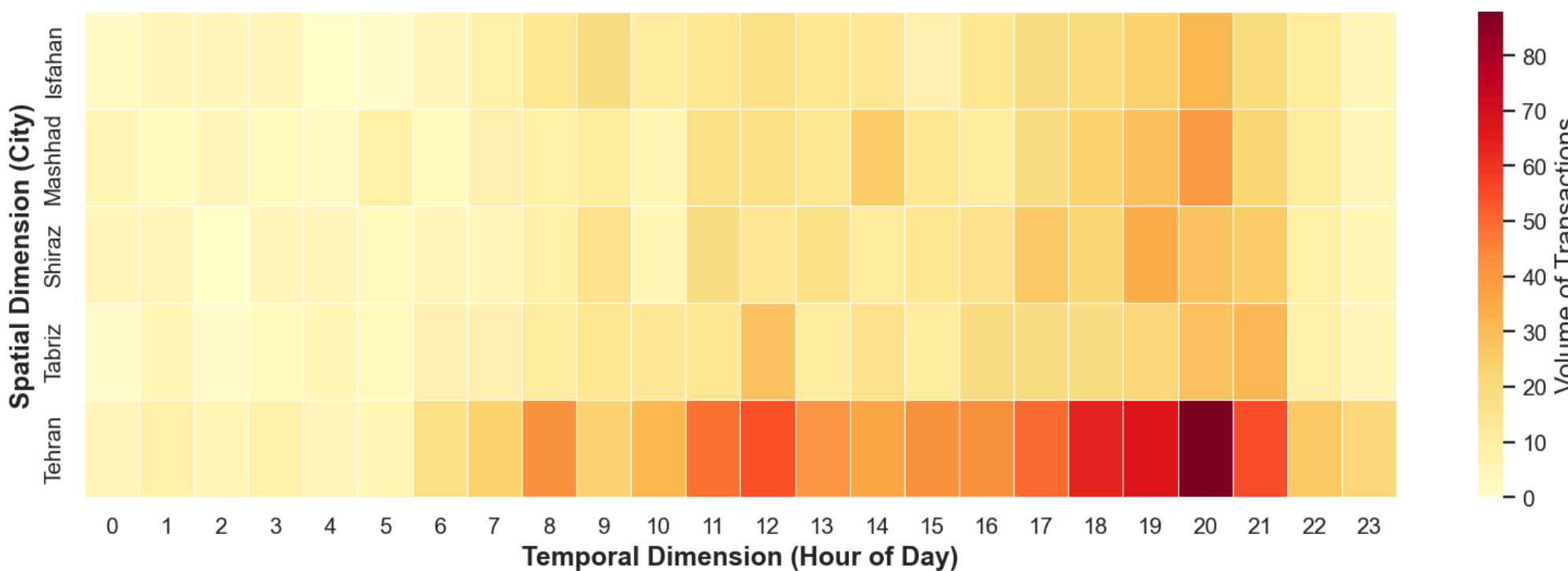
تحلیل محتوای بصری: الگوریتم شناسایی تصویر موفق به تشخیص و فیلترینگ 0 فایل رسانه‌ای نامتعارف از مجموع ۵۰ رکورد آپلود شده توسط کاربران گردید.

۴. کاوش داده‌های مکانی-زمانی (Spatio-Temporal Data Mining)

داده‌هایی که در آن‌ها ابعاد مختصات جغرافیایی و توالی وقوع زمانی به عنوان پارامترهای کلیدی در نظر گرفته می‌شوند. کاوش این ساختار بهینه‌سازی زنجیره تأمین و مدیریت ناوگان توزیع را تسهیل می‌نماید.

توزیع مکانی-زمانی: فرکانس تراکنش‌های شبانه به تفکیک مناطق جغرافیایی	
City	Night Orders Frequency
Tehran	51
Mashhad	23
Shiraz	23
Tabriz	21

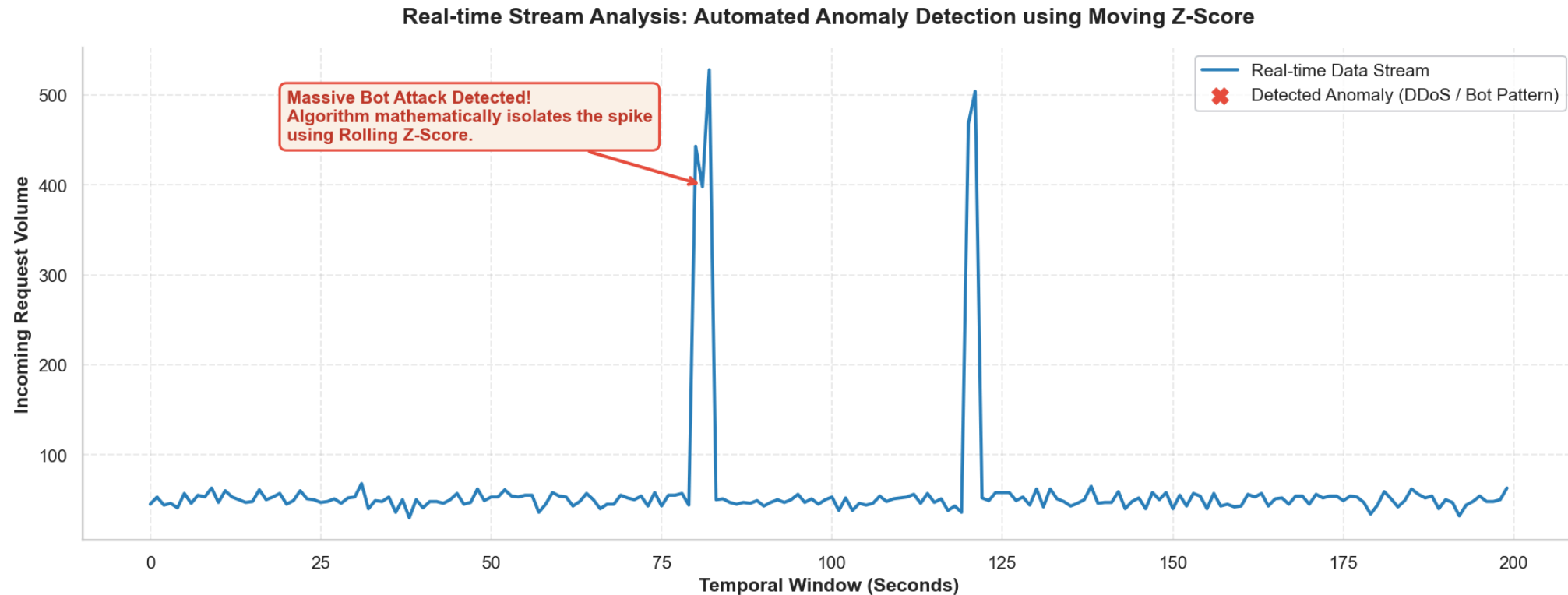
Spatio-Temporal Density Map (City vs. Transaction Hour)



۵. تحلیل جریان داده‌ها (Stream Data Processing)

داده‌های جریانی نمایانگر توالی بی‌نهایت و بلادرنگ (Real-Time) از مشاهدات می‌باشند. پردازش این داده‌ها مستلزم الگوریتم‌های حافظه‌کارآمد با قابلیت واکنش آنی به ناهنجاری‌ها نظیر حملات سایبری (Bot Attacks) است.

کاوش داده‌های جریانی: رکوردهای شناسایی‌شده به‌عنوان ناهنجاری ترافیکی (Anomaly Detection)		
Time (Seconds)	Traffic Volume (Clicks)	Calculated Z-Score



نتیجه استراتژیک معماری کلان‌داده: در پلتفرم‌های مدرن، اتکا به یک مدل رگرسیون ساده بر روی *داده‌های ساختاریافته (جداول)* کافی نیست. یک شرکت مانند دیجی‌کالا برای موفقیت باید رفتار مشتری را در سبد خرید (*Market Basket*) کشف کند، شبکه‌های پنهان شباهت کاربران را با گراف (*Network Analytics*) ترسیم نماید، احساسات متون را درک کند (*NLP*)، استراتژی لجستیک خود را در بُعد مکان-زمان (*Spatio-Temporal*) بهینه کند، و در نهایت با رصدِ بلادرنگِ جریان داده (*Streaming*) امنیت پلتفرم را در برابر حملات *DDoS* تضمین نماید. این کد، نقشهٔ راه جامع و آکادمیک برای درک عمیقِ گونه‌شناسی داده‌ها در کشف دانش (*KDD*) است.



توضیحات مثال کد پایتون

در این مثال:

- ابتدا کتابخانه‌های مورد نیاز شامل **pandas** و **numpy** برای مدیریت و شبیه‌سازی داده، **matplotlib** و **seaborn** برای نمایش بصری، و **networkx** برای ساخت و تحلیل گراف وارد شده‌اند.
- داده‌های ساختاریافته** شامل دو جدول **محصولات** (برند، رم، قیمت) و **مشتریان** (شهر، تاریخ عضویت) شبیه‌سازی شده است. سپس با استفاده از **groupby** میانگین قیمت به تفکیک برند محاسبه و با یک **مدل رگرسیون خطی ساده** رابطه بین حجم رم و قیمت کالا کشف شده است.
- برای **داده‌های نیمه‌ساختاریافته**، سه نوع داده جداگانه شبیه‌سازی شده‌اند:
 - داده تراکنشی (سبد خرید):** با شمارش هم‌رخدادی کالاها در سبدهای مختلف، پرتکرارترین جفت‌کالاهای خریداری‌شده با هم کشف شده‌اند — پایه‌ای ساده برای الگوریتم **Market Basket Analysis**.
 - داده توالی‌ای (Clickstream):** مسیر کلیک کاربران قبل از خرید (جستجو → کلیک برند → مقایسه → افزودن به سبد) شبیه‌سازی و رایج‌ترین الگوی توالی استخراج شده است.
 - داده گراف:** با کتابخانه **networkx** شبکه‌ای از کاربران با الگوی خرید مشابه ساخته شده و با معیار **Degree Centrality**، پرارتباط‌ترین کاربر شناسایی شده است.
- برای **داده‌های بدون ساختار**، نظرات متنی کاربران با یک روش ساده مبتنی بر واژگان مثبت و منفی تحلیل شده‌اند (**Sentiment Analysis**) و همچنین متادیتای تصاویر/ویدیوهای آپلودی برای شبیه‌سازی فیلتر محتوای نامناسب با **بینایی کامپیوتر** بررسی شده است.
- برای **داده‌های مکانی-زمانی**، ساعت و شهر ثبت سفارش‌ها شبیه‌سازی شده و با یک نمودار **Heatmap**، الگوی خریدهای شبانه در شهرهای مختلف به‌صورت بصری کشف شده است.
- در نهایت، برای **داده‌های جریانی**، جریان کلیک‌های آنی کاربران در یک حراج زنده شبیه‌سازی شده و با روش **Z-Score متحرک**، ثانیه‌هایی که رفتار غیرعادی (مثل حمله ربات) رخ داده، به‌صورت آنی شناسایی و روی نمودار مشخص شده‌اند.

این مثال به خوبی نشان می‌دهد که چگونه یک پلتفرم بزرگ مانند دیجی‌کالا با ترکیب چندین نوع داده متفاوت (ساختاریافته، نیمه‌ساختاریافته، بدون ساختار، مکانی-زمانی و جریانی) و به‌کارگیری تکنیک‌های ساده داده‌کاوی، می‌تواند از دل داده‌های خام و پراکنده، دانشی ارزشمند برای بهبود سیستم پیشنهاددهنده، تحلیل رضایت مشتریان، بهینه‌سازی توزیع و حفظ امنیت سیستم در لحظات حساس (مانند حراجی‌های بزرگ) استخراج کند.

داده‌های ذخیره‌شده در مقابل داده‌های جریانی:

تا اینجا با حجم عظیم داده‌ها آشنا شدیم، اما لازم است بدانید که همه‌ی داده‌ها به‌صورت یکسان تولید نمی‌شوند.

داده‌های ذخیره‌شده (Stored Data): این دسته از داده‌ها پیش از تحلیل، در مخازن بزرگ داده، انبارها یا دیتابیس‌ها ذخیره می‌شوند.

داده‌های جریانی (Streaming Data): در کاربردهایی مانند نظارت ویدئویی، شبکه‌های حسگر یا سنجش از دور، داده‌ها به‌صورت پیوسته و پویا تولید می‌شوند. این داده‌ها مانند یک رودخانه بی‌پایان، به سمت سیستم سرازیر می‌شوند.

جدول مقایسه‌ای انواع داده

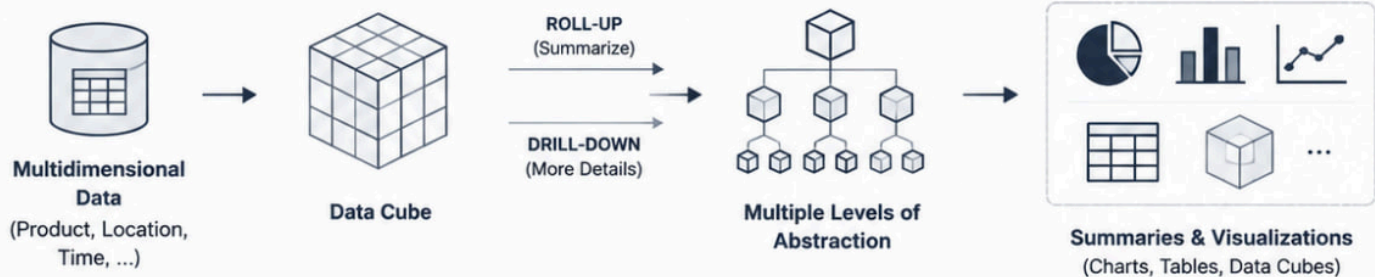
نوع داده	ساختار	نمونه در دیجی‌کالا	تکنیک/ابزار اصلی تحلیل
ساختاریافته	جدول ثابتی از سطر و ستون	مشخصات فنی محصولات، قیمت	کوئری SQL، الگوریتم‌های سنتی
نیمه‌ساختاریافته / توالی	انعطاف‌پذیر، برچسب‌دار یا ترتیبی	لگ کلیک‌ها، سبد خرید، JSON	کاوش الگوی ترتیبی، کاوش گراف
بدون ساختار	بدون فرمت، متن/تصویر/صدا	نظرات کاربران، عکس محصولات	پردازش زبان طبیعی (NLP)، یادگیری عمیق
مکانی-زمانی	وابسته به زمان و مختصات	لوکیشن تحویل، زمان توزیع	داده‌کاوی مکانی (Spatial Mining)
جریانی (Stream)	زنده، مداوم و ناپایدار	کلیک‌های هم‌زمان حراجی	Stream Mining، پردازش آنی

📌 آنچه درباره داده‌ها آموختیم

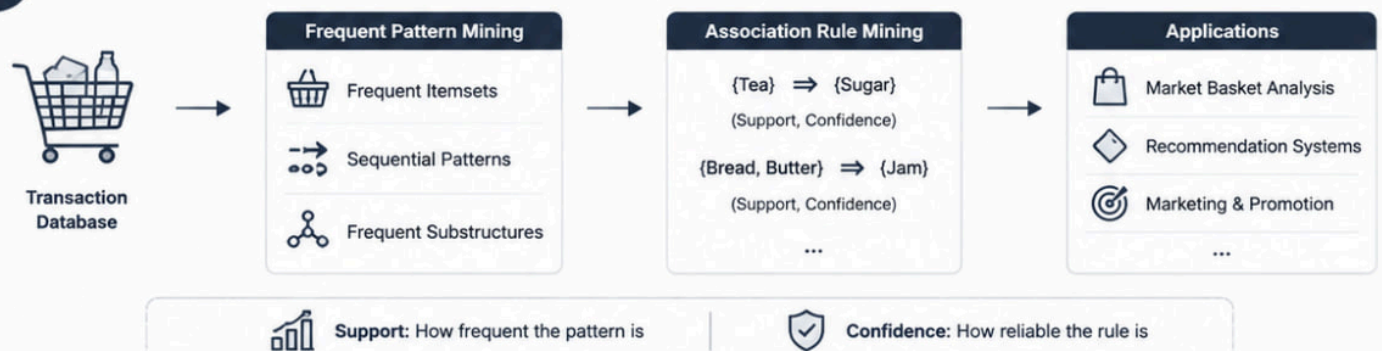
داده‌ها از نظر ساختار به سه دسته ساختاریافته، نیمه‌ساختاریافته و بدون ساختار تقسیم می‌شوند. اما ساختار همه‌چیز نیست؛ نوع کاربرد داده‌ها نیز تعیین‌کننده روش تحلیل است. علاوه بر این، داده‌ها از نظر نحوه تولید و ذخیره‌سازی نیز متفاوتند: برخی ذخیره‌شده و آماده تحلیل هستند و برخی دیگر جریانی و نیازمند پردازش لحظه‌ای. این تنوع گسترده، داده‌کاوی را به حوزه‌ای پویا و میان‌رشته‌ای تبدیل کرده است.

MINING VARIOUS KINDS OF KNOWLEDGE

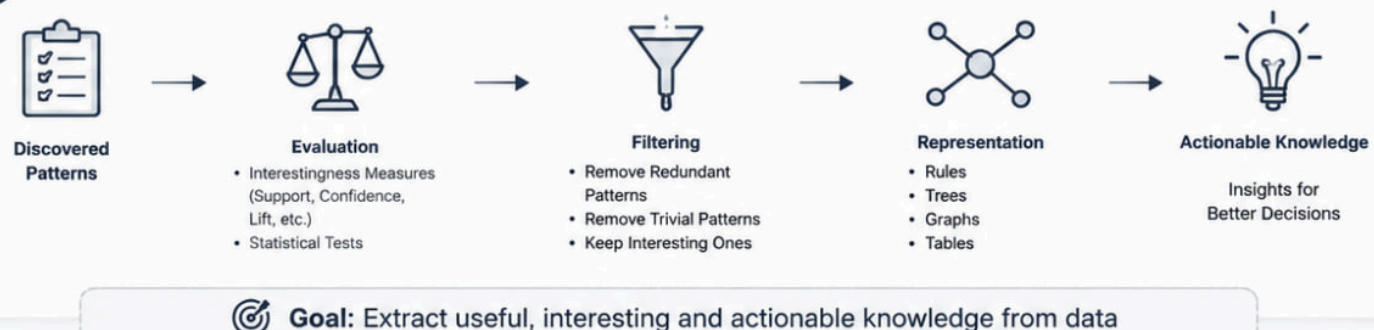
1.4.1 Multidimensional Data Summarization



1.4.2 Mining Frequent Patterns, Associations, and Correlations



1.4.3 Pattern Evaluation & Representation



📌 1.4 (Mining various kinds of knowledge)

1.4.1 Multidimensional data summarization

📁 کاوش انواع مختلف دانش

داده‌کاوی با هدف استخراج الگوها و دانش ارزشمند از داده‌ها، وظایف متنوعی را در بر می‌گیرد. به طور کلی، این وظایف در دو گروه اصلی داده‌کاوی توصیفی (Descriptive Data Mining) و داده‌کاوی پیش‌بینی‌کننده (Predictive Data Mining) طبقه‌بندی می‌شوند:

داده‌کاوی توصیفی (Descriptive): در این بخش، هدف ما این است که درک کنیم «در دل داده‌ها چه خبر است؟». در واقع، ما به دنبال شناسایی و توصیف ویژگی‌ها، روابط و الگوهای موجود در داده‌های فعلی هستیم تا از وضعیت موجود باخبر شویم.

داده‌کاوی پیش‌بینی‌کننده (Predictive): در اینجا از الگوهای استخراج‌شده استفاده می‌کنیم تا «آینده را حدس بزنیم». هدف ما پیش‌بینی مقادیر یا رفتارهایی است که هنوز اتفاق نیفتاده‌اند.

جانِ کلام

داده‌کاوی توصیفی به ما می‌گوید «چه اتفاقی افتاده است و چه الگوهایی داریم»، در حالی که داده‌کاوی پیش‌بینی‌کننده به ما می‌گوید «احتمالاً در آینده چه خواهد شد».

خلاصه‌سازی داده‌های چندبعدی

یکی از وظایف مهم داده‌کاوی، خلاصه‌سازی داده‌های چندبعدی (Multidimensional Data Summarization) است. از آنجا که بررسی جزئیات مجموعه‌های بزرگ داده برای کاربران زمان‌بر و دشوار است، داده‌کاوی با ارائه یک نمای کلی و فشرده از داده‌ها، درک سریع‌تر اطلاعات و مقایسه مجموعه‌های مختلف را امکان‌پذیر می‌سازد.

این فرایند معمولاً در یک فضای چندبعدی انجام می‌شود؛ فضایی که ابعادی مانند دسته‌بندی محصول، تولیدکننده، مکان و زمان را در بر می‌گیرد. در چنین شرایطی، داده‌ها به کمک مکعب‌های داده (Data Cubes) تجمیع می‌شوند تا کاربر بتواند با انجام عملیات Roll-up (نمای کلی‌تر) و Drill-down (ورود به جزئیات)، داده‌ها را در سطوح مختلف تحلیل کند.

نتایج خلاصه‌سازی چندبعدی می‌تواند به صورت نمودارهای دایره‌ای، نمودارهای میله‌ای، منحنی‌ها، مکعب‌های داده و جداول چندبعدی (Cross-tab) نمایش داده شود. در داده‌های ساختاریافته، فناوری مکعب داده امکان محاسبه و بازیابی سریع خلاصه‌های چندبعدی را فراهم می‌کند، اما در داده‌های بدون ساختار، مانند متن، انجام این فرایند به دلیل پیچیدگی ذاتی داده‌ها همچنان یکی از چالش‌های مهم پژوهشی در حوزه داده‌کاوی محسوب می‌شود.

1.4.2 Mining frequent patterns, associations, and correlations

کاوش الگوهای پرتکرار، قوانین انجمنی و همبستگی‌ها

یکی از مهم‌ترین وظایف داده‌کاوی، کاوش الگوهای پرتکرار (Frequent Pattern Mining) است؛ یعنی پیدا کردن الگوهایی که با فراوانی زیاد در داده‌ها تکرار می‌شوند. این الگوها به‌طور کلی به سه دسته تقسیم می‌شوند:

- **مجموعه‌اقلام پرتکرار (Frequent Itemsets):** آیتم‌هایی که معمولاً «با هم» ظاهر می‌شوند؛ مانند قند و چای که اغلب در یک سبد خرید در کنار هم دیده می‌شوند.
- **الگوهای ترتیبی (Sequential Patterns):** در این الگو «ترتیب زمانی» مهم است؛ مثلاً الگویی که نشان می‌دهد مشتری ابتدا لپ‌تاپ، سپس کیف لپ‌تاپ و در نهایت لوازم جانبی می‌خرد.
- **زیرساختارهای پرتکرار (Frequent Substructures):** در داده‌های ساختاری مانند گراف‌ها، درخت‌ها و شبکه‌ها، اگر یک ساختار مشخص به دفعات تکرار شود، به آن زیرساختار پرتکرار گفته می‌شود.

الگوهای پرتکرار پایه و اساس تکنیک‌های پرکاربردی مانند تحلیل سبد خرید و سیستم‌های توصیه‌گر هستند. با کمک این الگوها، می‌توانیم متوجه بشویم چه اقلامی معمولاً با هم «هماهنگ» هستند و به مشتریان پیشنهادهای هوشمندانه بدهیم. برای این کار، الگوریتم **Apriori** به عنوان یکی از کلاسیک‌ترین و معروف‌ترین روش‌ها، در استخراج مجموعه‌اقلام پرتکرار به کار می‌رود.

 **کاربرد عملی:** در تحلیل سبد خرید، قانون انجمنی «چای $\Rightarrow$ قند» به این معناست که مشتریانی که چای می‌خرند، با احتمال بالایی قند نیز خریداری می‌کنند. این قوانین با دو معیار کلیدی حمایت (Support) و اطمینان (Confidence) سنجیده می‌شوند. فروشگاه‌های بزرگ از این تحلیل‌ها برای چیدمان بهینه کالاها، طراحی کمپین‌های تبلیغاتی هدفمند و افزایش فروش استفاده می‌کنند.

1.4.3 Classification and regression for predictive analysis

طبقه‌بندی و رگرسیون برای تحلیل پیش‌بینی‌کننده

طبقه‌بندی (Classification) و رگرسیون (Regression) از مهم‌ترین وظایف داده‌کاوی پیش‌بینی‌کننده (Predictive Data Mining) هستند که با استفاده از داده‌های آموزشی، به پیش‌بینی ویژگی‌های داده‌های جدید می‌پردازند.

طبقه‌بندی (Classification): در این فرآیند، هدف ما ایجاد مدلی است که بتواند داده‌ها را به **کلاس‌های از پیش تعریف‌شده** و متمایز تقسیم کند. برای مثال، تشخیص اینکه یک ایمیل «**اسپم**» است یا «**معمولی**»، یک مسئلهٔ طبقه‌بندی است. این مدل‌ها می‌توانند در قالب‌های مختلفی مانند **درخت تصمیم (Decision Tree)**، **قوانین IF–THEN**، **شبکه‌های عصبی**، **ماشین بردار پشتیبان (SVM)** و **الگوریتم نزدیک‌ترین همسایه (KNN)** ارائه شوند.

رگرسیون (Regression): اگر بخواهیم یک مقدار عددی و پیوسته را پیش‌بینی کنیم، از رگرسیون استفاده می‌کنیم. پیش‌بینی مواردی مانند **قیمت خانه**، **میزان درآمد احتمالی** یا **حجم فروش** در ماه‌های آینده، در قلمرو رگرسیون قرار می‌گیرد.

 **کاربرد عملی:** فرض کنید یک بانک می‌خواهد تصمیم بگیرد که به یک متقاضی وام بدهد یا خیر. در اینجا، **طبقه‌بندی** (مشتری خوب / بد) مناسب است. اما اگر بانک بخواهد **میزان دقیق مبلغ وام** را پیش‌بینی کند، از **رگرسیون** استفاده می‌کند. بنابراین، انتخاب روش به **نوع پاسخ مورد نیاز** (کلاس یا عدد) بستگی دارد.

مثال عملی : اعتبارسنجی و مدیریت فروش اقساطی در «دیجی‌پی»

فرض کنید تیم داده‌کاوی دیجی‌پی می‌خواهد سیستم خرید اقساطی (الان بخر، بعداً پرداخت کن - BNPL) را برای خریداران دیجی‌کالا هوشمند کند. در این کاربرد واقعی، هر دو روش طبقه‌بندی و رگرسیون در کنار هم استفاده می‌شوند:

گام اول: انتخاب ویژگی (Pre-processing)

دیجی‌پی بیش از ۱۰۰ ویژگی از هر کاربر دارد (از سن و شهر گرفته تا رنگ قاب گوشی خریده‌شده و ساعت آنلاینی). مرحله انتخاب ویژگی، موارد نامرتب مثل «رنگ قاب گوشی» یا «ساعت آنلاین شدن در سایت» را حذف می‌کند و فقط متغیرهای کلیدی مثل «درآمد ماهانه»، «سابقه خرید» و «سوابق چک بازگشتی» را به عنوان ورودی مدل نگه می‌دارد.

گام دوم: طبقه‌بندی (پیش‌بینی کلاس گسسته)

سوال سیستم: «آیا این کاربر واجد شرایط دریافت اعتبار BNPL هست یا خیر؟»

عملکرد مدل: مدل درخت تصمیم با بررسی داده‌های آموزشی، خروجی را در قالب یکی از ۳ کلاس زیر تعیین می‌کند:

- کلاس A (کم‌ریسک): اعطای فوری اعتبار بدون نیاز به ضامن.
- کلاس B (ریسک متوسط): اعطای اعتبار مشروط به ارائه سفته یا چک.
- کلاس C (پرریسک): رد درخواست اعتبار.

گام سوم: رگرسیون (پیش‌بینی عدد پیوسته)

سوال سیستم: «حداکثر سقف ریالی اعتباری که می‌توان به این کاربر اعطا کرد چقدر است؟»

عملکرد مدل: مدل رگرسیونی بر اساس متغیرهایی مثل درآمد و میانگین گردش مالی، یک فرمول ریاضی تشکیل می‌دهد و یک مقدار عددی دقیق خروجی می‌دهد؛ مثلاً:

سقف اعتبار پیش‌بینی‌شده = ۳۵,۴۰۰,۰۰۰ تومان

```
In [7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestRegressor
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.metrics import classification_report, mean_absolute_error, r2_score
from IPython.display import display, HTML

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
```

```
"axes.titlesize": 15,
"axes.labelsize": 13,
"xtick.labelsize": 11,
"ytick.labelsize": 11
))

plt.rcParams['figure.dpi'] = 135

def display_academic_text(text, type="normal"):
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))
```

```
def get_protected_css():
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
    color: #ffffff !important;
    padding: 12px 18px !important;
    text-align: center !important;
    border: 1px solid #34495e !important;
    white-space: nowrap !important;
    font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important;
    text-align: center !important;
    border: 1px solid #bdc3c7 !important;
    white-space: nowrap !important;
    color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
    background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
    background-color: #e8ecef !important;
}
</style>
"""
```

```
def display_side_by_side_datasets(dfs, titles, *args):
    """Renders single or multiple pandas DataFrames side-by-side gracefully."""
    if not isinstance(dfs, list) and not isinstance(titles, list):
        if len(args) >= 2:
            dfs = [dfs, args[0]]
            titles = [titles, args[1]]
        else:
            dfs = [dfs]
            titles = [titles]

    if len(dfs) == 1:
        df = dfs[0]
        title = titles[0]
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0, float_format=lambda x: f"{x:.3f}")
        html = f"""
<div style="width: 100%; margin: 20px 0 35px 0;">
    <div dir="rtl" style="background-color: #2c3e50; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
"""
```

```

    </div>
</div>
"""

display(HTML(get_protected_css() + html))
return

col_html_list = []
for i, (df, title) in enumerate(zip(dfs, titles)):
    table_html = df.to_html(index=False, classes=['strict-tbl'], border=0, float_format=lambda x: f"{x:.3f}")
    bg_color = "#2c3e50"
    col_html = f"""
    <div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
        <div style="background-color: {bg_color}; color: white; padding: 12px;
            text-align: center; font-family: Tahoma, sans-serif;
            font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
            {title}
        </div>
        <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
            {table_html}
        </div>
    </div>
    """
    col_html_list.append(col_html)

columns_joined = "".join(col_html_list)
html = f"""
<div style="display: flex; flex-direction: row; justify-content: space-between;
    gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
    box-sizing: border-box;">
    {columns_joined}
</div>
"""

display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');
* { font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important; }
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
"""))

np.random.seed(42)
```

In [8]: `display_academic_text("داده‌کاوی پیشگویانه: طبقه‌بندی و رگرسیون در سیستم‌های اعتباری (Credit Risk)", "main_title")`
`display_academic_text("مدل طبقه‌بندی را شبیه‌سازی می‌کنیم. ابتدا با یک مدل طبقه‌بندی")`
`# =====`
`# 0. RAW DATA SIMULATION`
`# =====`

`display_academic_text("مرحله نخست: شبیه‌سازی فضای حالت و تجمیع داده‌های خام", "sub_title")`
`display_academic_text("این داده‌ها پیچیدگی و کثیفی داده‌های واقعی در محیط‌های بانکی را شبیه‌سازی می‌کنند")`
مجموعه‌ای از ویژگی‌های معنادار (نظیر درآمد و گردش مالی) و ویژگی‌های نویز (نظیر رنگ قاب موبایل) است. این داده‌ها پیچیدگی و کثیفی داده‌های واقعی در محیط‌های بانکی را شبیه‌سازی می‌کنند

```
n_users = 3000

monthly_income = np.random.gamma(shape=5, scale=1_500_000, size=n_users)
purchase_history = np.random.poisson(lam=8, size=n_users)
bounced_checks = np.random.poisson(lam=0.3, size=n_users)
avg_cashflow = monthly_income * np.random.uniform(0.5, 1.5, n_users)

phone_case_color = np.random.choice(['Black', 'Blue', 'Red', 'White'], n_users)
online_hour = np.random.randint(0, 24, n_users)
noise_features = {f'RandomFeature_{i}': np.random.normal(0, 1, n_users) for i in range(1, 16)}
```

```
raw_data = pd.DataFrame({
    'UserID': range(1, n_users + 1),
    'MonthlyIncome': monthly_income.round(0),
    'PurchaseHistory': purchase_history,
    'BouncedChecks': bounced_checks,
    'AvgCashflow': avg_cashflow.round(0),
    'PhoneCaseColor': phone_case_color,
    'OnlineHour': online_hour,
    **noise_features
})
```

```
display_side_by_side_datasets(
    [raw_data[['UserID', 'MonthlyIncome', 'PurchaseHistory', 'BouncedChecks', 'PhoneCaseColor', 'OnlineHour']].head(5),
     (Raw Data) نمونه داده‌های خام کاربران در فضای ابعاد بالا"]
)
```

```
# Target Variable Generation
risk_score = (
    (raw_data['MonthlyIncome'] / 1_000_000) * 0.5 +
    raw_data['PurchaseHistory'] * 1.2 -
    raw_data['BouncedChecks'] * 8 +
    np.random.normal(0, 2, n_users)
)
```

```
def assign_class(score):
    if score > 10: return 'A'
    elif score > 4: return 'B'
    else: return 'C'
```

```
raw_data['RiskClass'] = risk_score.apply(assign_class)
```

```
credit_limit = (
    raw_data['MonthlyIncome'] * 2.5 +
    raw_data['AvgCashflow'] * 0.8 +
    raw_data['PurchaseHistory'] * 300_000 -
    raw_data['BouncedChecks'] * 5_000_000 +
    np.random.normal(0, 1_000_000, n_users)
)
```

```
credit_limit = np.clip(credit_limit, 0, None)
raw_data['CreditLimit'] = np.where(raw_data['RiskClass'] == 'C', 0, credit_limit).round(-4)

display_side_by_side_datasets(
    [raw_data[['UserID', 'MonthlyIncome', 'PurchaseHistory', 'RiskClass', 'CreditLimit']].head(5),
    ["پایگاه داده برچسب‌گذاری‌شده (مجهز به کلاس ریسک گسسته و سقف اعتبار پیوسته)"]
)

# =====
# 1. FEATURE SELECTION
# =====
display_academic_text("مرحله دوم: پیش‌پردازش و کاهش ابعاد بر اساس انتخاب ویژگی", "sub_title")
display_academic_text("<ANOVA F-Value> تحلیل واریانس</b> است. الگوریتم با استفاده از KDD کاهش ابعاد یکی از ارکان پیش‌پردازش در معماری")

raw_data['PhoneCaseColor_encoded'] = raw_data['PhoneCaseColor'].astype('category').cat.codes
feature_candidates = ['MonthlyIncome', 'PurchaseHistory', 'BouncedChecks', 'AvgCashflow', 'PhoneCaseColor_encoded', 'OnlineHour'] + list(noise_features.keys())

X_candidates = raw_data[feature_candidates]
y_class = raw_data['RiskClass']

selector = SelectKBest(score_func=f_classif, k=4)
selector.fit(X_candidates, y_class)

feature_scores = pd.Series(selector.scores_, index=feature_candidates).sort_values(ascending=False)
selected_features = feature_scores.head(4).index.tolist()

scores_df = feature_scores.reset_index()
scores_df.columns = ['Feature Attribute', 'ANOVA F-Score Measure']
display_side_by_side_datasets([scores_df.head(6)], ["(Top Features) ارزیابی آماری و رتبه‌بندی اهمیت صفات استخراج‌شده"])

display_academic_text(f"<{selected_features}> است  
ل‌های پیشگویانه استخراج کرده است  
ل‌های پیشگویانه شامل صفات f")

# =====
# 2. CLASSIFICATION MODELING (DECISION TREE)
# =====
display_academic_text("مرحله سوم: مدل‌سازی پیشگویانه جهت طبقه‌بندی کلاس ریسک", "sub_title")
display_academic_text("<Discrete> گسسته</b> یادگیری تابعی است که رکوردهای پیوسته داده را به دسته‌های (Classification) وظیفه طبقه‌بندی (De
درخت تصمیم<b> تخصیص می‌دهد. در اینجا، ما از معماری (De

X = raw_data[selected_features]
y = raw_data['RiskClass']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)

clf = DecisionTreeClassifier(max_depth=4, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

clf_report = classification_report(y_test, y_pred, output_dict=True)
clf_report_df = pd.DataFrame(clf_report).transpose().reset_index()
clf_report_df.columns = ['Class / Metric', 'Precision', 'Recall', 'F1-Score', 'Support']
display_side_by_side_datasets([clf_report_df], ["(Classification Report) ماتریس ارزیابی و سنجش عملکرد الگوریتم طبقه‌بندی"])

# Decision Tree Visualization (Removed 'colors' argument for compatibility)
plt.figure(figsize=(22, 10), dpi=135)
plot_tree(
    clf,
    feature_names=selected_features,
    class_names=list(clf.classes_),
    filled=True,
    rounded=True,
    fontsize=11,
    precision=2
)
plt.title('Induced Decision Tree Topology for Credit Risk Classification (CART Algorithm)', fontdict={'fontsize': 16, 'fontweight': 'bold'}, pad=20)
plt.tight_layout()
plt.show()

# =====
# 3. REGRESSION MODELING (RANDOM FOREST)
# =====
display_academic_text("مرحله چهارم: مدل‌سازی پیشگویانه جهت رگرسیون تخصیص اعتبار", "sub_title")
display_academic_text("<Continuous> عددی</b> به مدل‌سازی یک تابع پیوسته برای پیش‌بینی مقادیر (Regression) در مقابل طبقه‌بندی، مکانیزم رگرسیون")

eligible = raw_data[raw_data['RiskClass'] != 'C']
X_reg = eligible[selected_features]
y_reg = eligible['CreditLimit']
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_reg, y_reg, test_size=0.25, random_state=42)

reg_model = RandomForestRegressor(n_estimators=200, max_depth=6, random_state=42)
reg_model.fit(X_train_r, y_train_r)
y_pred_r = reg_model.predict(X_test_r)

mae = mean_absolute_error(y_test_r, y_pred_r)
r2 = r2_score(y_test_r, y_pred_r)

display_academic_text(f"<{mae:.0f}> تحلیل عملکرد رگرسیون مؤید آن است که میانگین خطای مطلق برابر با f")

fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# Regression Chart 1: Predicted vs Actual Scatter
axes[0].scatter(y_test_r, y_pred_r, alpha=0.75, color='#1abc9c', edgecolor='#2c3e50', s=60)
axes[0].plot([y_test_r.min(), y_test_r.max()], [y_test_r.min(), y_test_r.max()], 'k--', lw=2.5, alpha=0.7)
axes[0].set_xlabel('Empirical Ground Truth Credit Limit (Actual)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_ylabel('Random Forest Prediction Space', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_title('1. Regression Capability: Predicted vs. Actual Output', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=15)
axes[0].grid(True, linestyle='--', alpha=0.5)

# Regression Chart 2: Residual Error Distribution
residuals = y_test_r - y_pred_r
sns.histplot(residuals, bins=35, kde=True, color='#2980b9', ax=axes[1])
axes[1].axvline(0, color='black', linestyle='--', lw=2)
axes[1].set_title('2. Residual Error Distribution (Model Precision)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=15)
```

```
axes[1].set_xlabel('Prediction Error (Actual - Predicted)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].grid(True, linestyle='--', alpha=0.5)

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# 4. REAL-TIME DEPLOYMENT SCENARIO
# =====
display_academic_text("مرحله پنجم: استنتاج عملیاتی و ارزیابی برخط موجودیت جدید", "sub_title")
display_academic_text("Regi شده و در صورت تأیید صلاحیت، وارد مدل Classification بردارِ ویژگی یک مشتری جدید به صورت زنده وارد خط‌لولهٔ داده‌کاوی می‌شود. داده‌ها ابتدا وارد مدل (Deployment)، در فاز استقرار")

new_user = pd.DataFrame({
    'MonthlyIncome': [18_000_000],
    'PurchaseHistory': [12],
    'BouncedChecks': [0],
    'AvgCashflow': [20_000_000],
    'PhoneCaseColor_encoded': [2],
    'OnlineHour': [23]
})

new_user_disp = new_user.copy()
new_user_disp.columns = ['Monthly Income', 'Purchase History', 'Bounced Checks', 'Avg Cashflow', 'Noise (Color)', 'Noise (Hour)']
display_side_by_side_datasets([new_user_disp, ["ساختار بردار ویژگی خام ارائه‌شده توسط مشتری جدید"]])

# The Deployment Pipeline Simulation
new_user_selected = new_user[selected_features]
predicted_class = clf.predict(new_user_selected)[0]

if predicted_class == 'C':
    final_decision = "درخواست اعتبار رد گردید و تخصیص سرمایه به دلیل ارزیابی ریسک بالا متوقف شد."
else:
    predicted_limit = reg_model.predict(new_user_selected)[0]
    guarantee_note = "تأیید فرآیند مشروط به ارائه تضامین بانکی معتبر " if predicted_class == 'A' else "تأیید خودکار بدون نیاز به ارائه وثیقه به واسطه قرارگیری در دسته کیفی نخست"
    final_decision = f"را تخصیص داده است. وضعیت ضمانت نشان می‌دهد که {predicted_limit:,0f} ریال"

display_academic_text(f"تخصیص داده است. در نتیجه این کاوش {predicted_class} حاکی از آن است که مدل طبقه‌بندی، این رکورد جدید را به خوشه ریسک (DSS) استنتاج نهاییِ سیستم تصمیم‌یار هوشمند")
```

داده‌کاوی پیشگویانه: طبقه‌بندی و رگرسیون در سیستم‌های اعتباری (Credit Risk)

وظایف داده‌کاوی به دو دسته توصیفی (مربع‌های کشف گذشته) و پیشگویانه (پیش‌بینی آینده) تقسیم می‌شوند. در این سناریو، یک سیستم تخصیص اعتبار (وام‌دهی) را شبیه‌سازی می‌کنیم. ابتدا با یک مدل طبقه‌بندی (Classification)، ریسک مشتری را در دسته‌های گسسته (A, B, C) پیش‌بینی کرده و سپس با یک مدل رگرسیون (Regression)، سقف دقیق اعتبار پیوسته (مثلاً ۲۵ میلیون تومان) را معماری و پیاده‌سازی می‌نماییم.

مرحله نخست: شبیه‌سازی فضای حالت و تجمیع داده‌های خام

پایگاه داده اولیه شامل مجموعه‌ای از ویژگی‌های معنادار (نظیر درآمد و گردش مالی) و ویژگی‌های نویز (نظیر رنگ قاب موبایل) است. این داده‌ها پیچیدگی و کثیفی داده‌های واقعی در محیط‌های بانکی را شبیه‌سازی می‌کنند.

نمونه داده‌های خام کاربران در فضای ابعاد بالا (Raw Data)					
UserID	MonthlyIncome	PurchaseHistory	BouncedChecks	PhoneCaseColor	OnlineHour
1	8736052.000	10	1	Red	12
2	6561463.000	5	0	White	13
3	6268340.000	7	0	Blue	8
4	6268390.000	12	0	Red	0
5	13465476.000	9	0	Red	11

پایگاه داده برچسب‌گذاری‌شده (مجهز به کلاس ریسک گسسته و سقف اعتبار پیوسته)				
UserID	MonthlyIncome	PurchaseHistory	RiskClass	CreditLimit
1	8736052.000	10	A	24240000.000
2	6561463.000	5	B	21440000.000
3	6268340.000	7	A	21390000.000
4	6268390.000	12	A	23210000.000
5	13465476.000	9	A	46860000.000

مرحله دوم: پیش‌پردازش و کاهش ابعاد بر اساس انتخاب ویژگی (Feature Selection)

کاهش ابعاد یکی از ارکان پیش‌پردازش در معماری KDD است. الگوریتم با استفاده از تحلیل واریانس (ANOVA F-Value)، صفات نامرتب و نویز (مانند رنگ قاب گوشی) را فیلتر کرده و صرفاً زیرمجموعه‌ای از صفات که بالاترین قدرت پیش‌بینی را دارند حفظ می‌نماید.

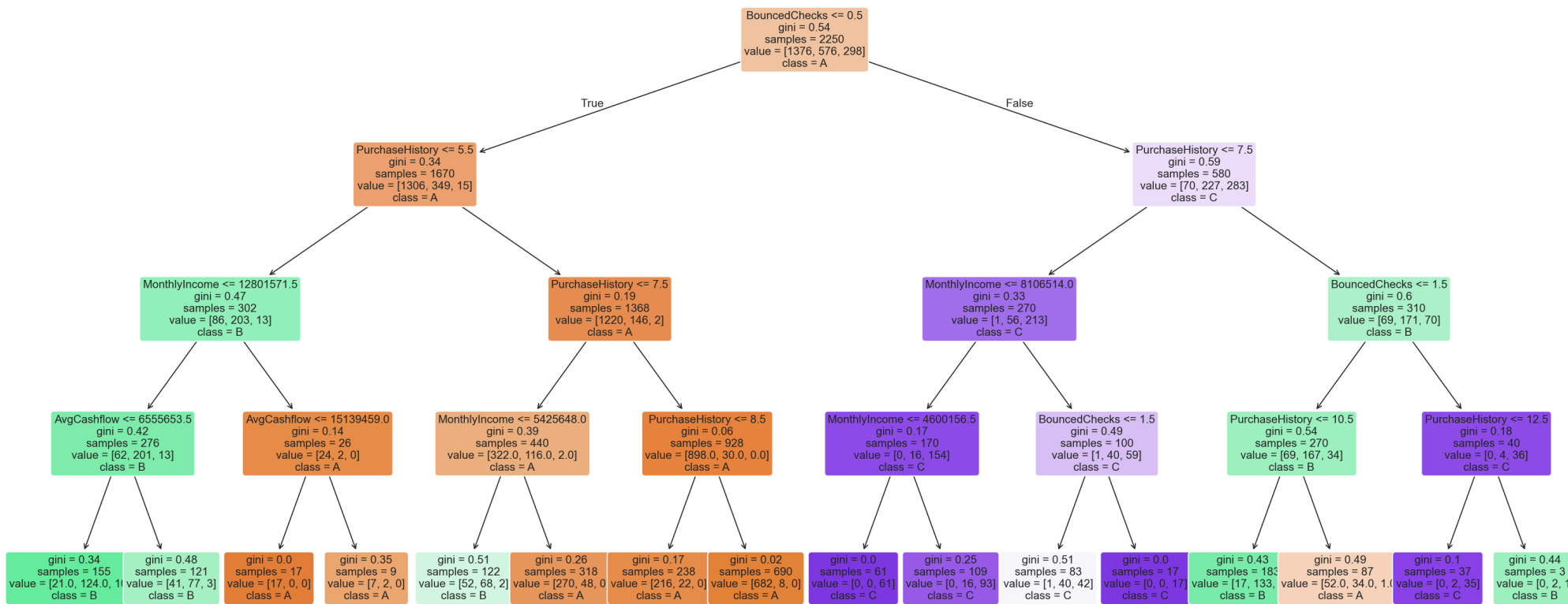
ارزیابی آماری و رتبه‌بندی اهمیت صفات استخراج‌شده (Top Features)	
Feature Attribute	ANOVA F-Score Measure
BouncedChecks	1663.236
PurchaseHistory	368.943
MonthlyIncome	60.276
AvgCashflow	41.599
RandomFeature_8	5.964
RandomFeature_14	2.803

استنتاج پیش‌پردازش نشان می‌دهد که سیستم با موفقیت بردارهای نویز را حذف نموده و فضای ویژگی بهینه شامل صفات **BouncedChecks**, **PurchaseHistory**, **MonthlyIncome**, **AvgCashflow** را جهت استقرار در مدل‌های پیشگویانه استخراج کرده است.

مرحله سوم: مدل‌سازی پیشگویانه جهت طبقه‌بندی کلاس ریسک (Classification)

وظیفه طبقه‌بندی (Classification)، یادگیری تابعی است که رکوردهای پیوسته داده را به دسته‌های گسسته (Discrete) تخصیص می‌دهد. در اینجا، ما از معماری درخت تصمیم (Decision Tree) استفاده می‌کنیم تا مشتریان را به یکی از کلاس‌های اعتباری (A, B, C) نگاشت کنیم.

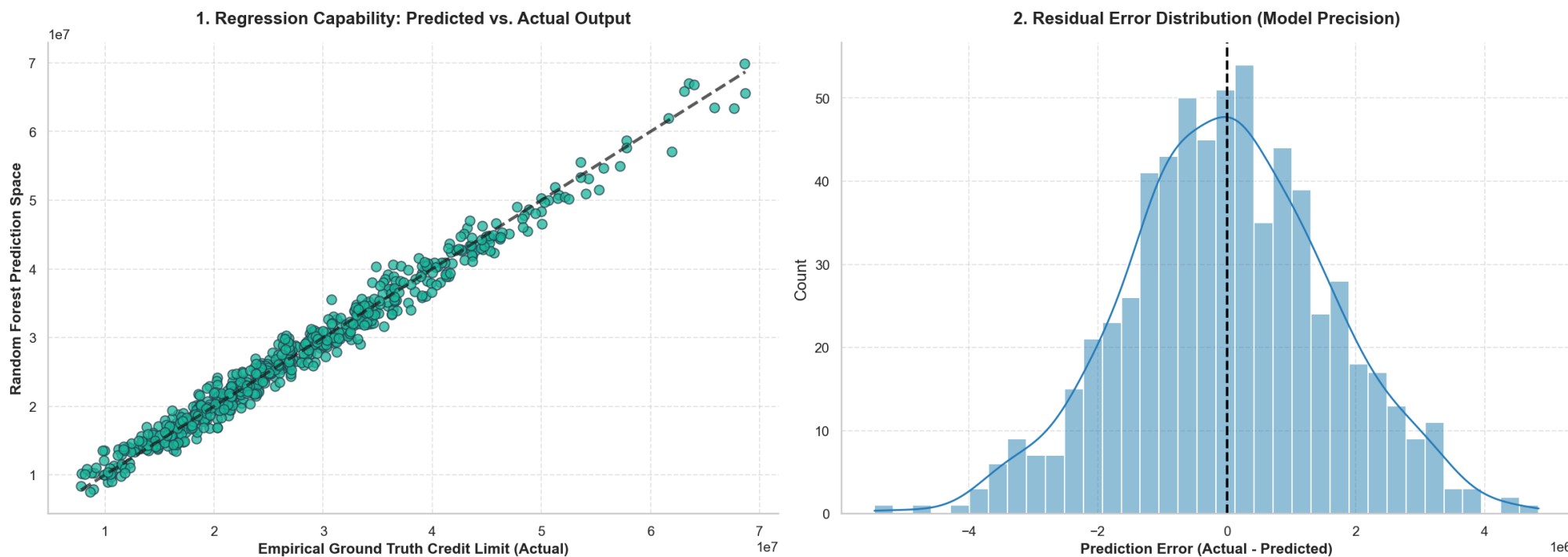
ماتریس ارزیابی و سنجش عملکرد الگوریتم طبقه‌بندی (Classification Report)				
Class / Metric	Precision	Recall	F1-Score	Support
A	0.915	0.867	0.890	459.000
B	0.648	0.719	0.681	192.000
C	0.824	0.848	0.836	99.000
accuracy	0.827	0.827	0.827	0.827
macro avg	0.795	0.811	0.803	750.000
weighted avg	0.835	0.827	0.830	750.000



مرحله چهارم: مدل سازی پیشگویانه جهت رگرسیون تخصیص اعتبار (Regression)

در مقابل طبقه بندی، مکانیزم رگرسیون (Regression) به مدل سازی یک تابع پیوسته برای پیش بینی مقادیر عددی (Continuous) می پردازد. این معماری با استقرار یک مدل جنگل تصادفی (Random Forest) بر روی مشتریان پذیرفته شده، سقف اعتبار (مثلاً ۳۲ میلیون تومان) را به صورت دقیق محاسبه می نماید.

تحلیل عملکرد رگرسیون مؤید آن است که میانگین خطای مطلق برابر با 1,257,214 ریال بوده و ضریب تعیین (R-Squared) با مقدار 0.980 نشان دهنده ظرفیت بسیار بالای الگوریتم در مدل سازی واریانس پیوسته مشتریان است.



مرحله پنجم: استنتاج عملیاتی و ارزیابی برخط موجودیت جدید (Deployment)

در فاز استقرار (Deployment)، بردار ویژگی یک مشتری جدید به صورت زنده وارد خطلوله داده کاوی می شود. داده ها ابتدا وارد مدل Classification شده و در صورت تأیید صلاحیت، وارد مدل Regression می شوند تا یک تصمیم سیستمی خودکار اخذ گردد.

ساختار بردار ویژگی خام ارائه شده توسط مشتری جدید					
Monthly Income	Purchase History	Bounced Checks	Avg Cashflow	Noise (Color)	Noise (Hour)
18000000	12	0	20000000	2	23

استنتاج نهایی سیستم تصمیم‌یار هوشمند (DSS) حاکی از آن است که مدل طبقه‌بندی، این رکورد جدید را به خوشه ریسک A تخصیص داده است. در نتیجه این کاوش: سیستم، ریسک کاربر را پذیرفته و سقف اعتبار معادل 60,058,315 ریال را تخصیص داده است. وضعیت ضمانت نشان می‌دهد که: تأیید خودکار بدون نیاز به ارائه وثیقه به واسطه قرارگیری در دسته کیفی نخست.



توضیحات مثال کد پایتون

در این مثال:

- ابتدا کتابخانه‌های مورد نیاز شامل **pandas** و **numpy** برای مدیریت داده‌ها، ابزارهای رسم نمودار (**matplotlib** و **seaborn**) و ماژول‌های هوش مصنوعی و یادگیری ماشین از کتابخانه قدرتمند **scikit-learn** وارد شده‌اند.
 - یک مجموعه داده فرضی از کاربران دیجی‌پی با بیش از ۲۰ ویژگی شبیه‌سازی شده است که شامل متغیرهای کلیدی (مانند درآمد ماهانه، تعداد خرید قبلی و چک‌های برگشتی) در کنار ویژگی‌های نامرتب و نویزی (مانند رنگ قاب گوشی و ساعت آنلاین) است تا محیط واقعی داده‌ها شبیه‌سازی شود.
 - در گام اول (انتخاب ویژگی - Feature Selection)، با استفاده از متد **SelectKBest** و آزمون آماری ANOVA، ویژگی‌های نامرتب حذف شده و صرفاً متغیرهای اثرگذار برای مدل‌سازی حفظ می‌شوند.
 - در گام دوم (طبقه‌بندی - Classification)، یک مدل درخت تصمیم (**Decision Tree**) برای پیش‌بینی کلاس گسسته ریسک کاربر (کلاس A: کم‌ریسک بدون ضامن، کلاس B: ریسک متوسط مشروط به چک، و کلاس C: پرریسک و رد درخواست) آموزش داده شده و ساختار تصمیم‌گیری آن رسم شده است.
 - در گام سوم (رگرسیون - Regression)، یک مدل **Random Forest Regressor** برای کاربران واجد شرایط (کلاس‌های A و B) پیاده‌سازی شده تا یک عدد پیوسته دقیق یعنی **سقف اعتبار ریالی** (بر حسب تومان) پیش‌بینی شود.
 - در نهایت، در بخش **سناریوی عملیاتی**، درخواست یک کاربر جدید از طریق خط لوله مدل‌سازی بررسی می‌شود تا نشان دهد چگونه ترکیب همزمان طبقه‌بندی و رگرسیون می‌تواند تصمیمات اعتباری سیستم BNPL دیجی‌پی را اتوماسیون کند.
- این مثال به‌وضوح نشان می‌دهد که چگونه یک سامانه داده‌کاوی پیشرفته می‌تواند از میان انبوهی از داده‌های خام و پرنویز، ویژگی‌های کلیدی را استخراج کرده و با به‌کارگیری همزمان تکنیک‌های **طبقه‌بندی و رگرسیون**، ارزش کسب‌وکاری فوق‌العاده‌ای (مدیریت ریسک مالی و اعطای هوشمند اعتبار) خلق کند.

1.4.4 Cluster analysis

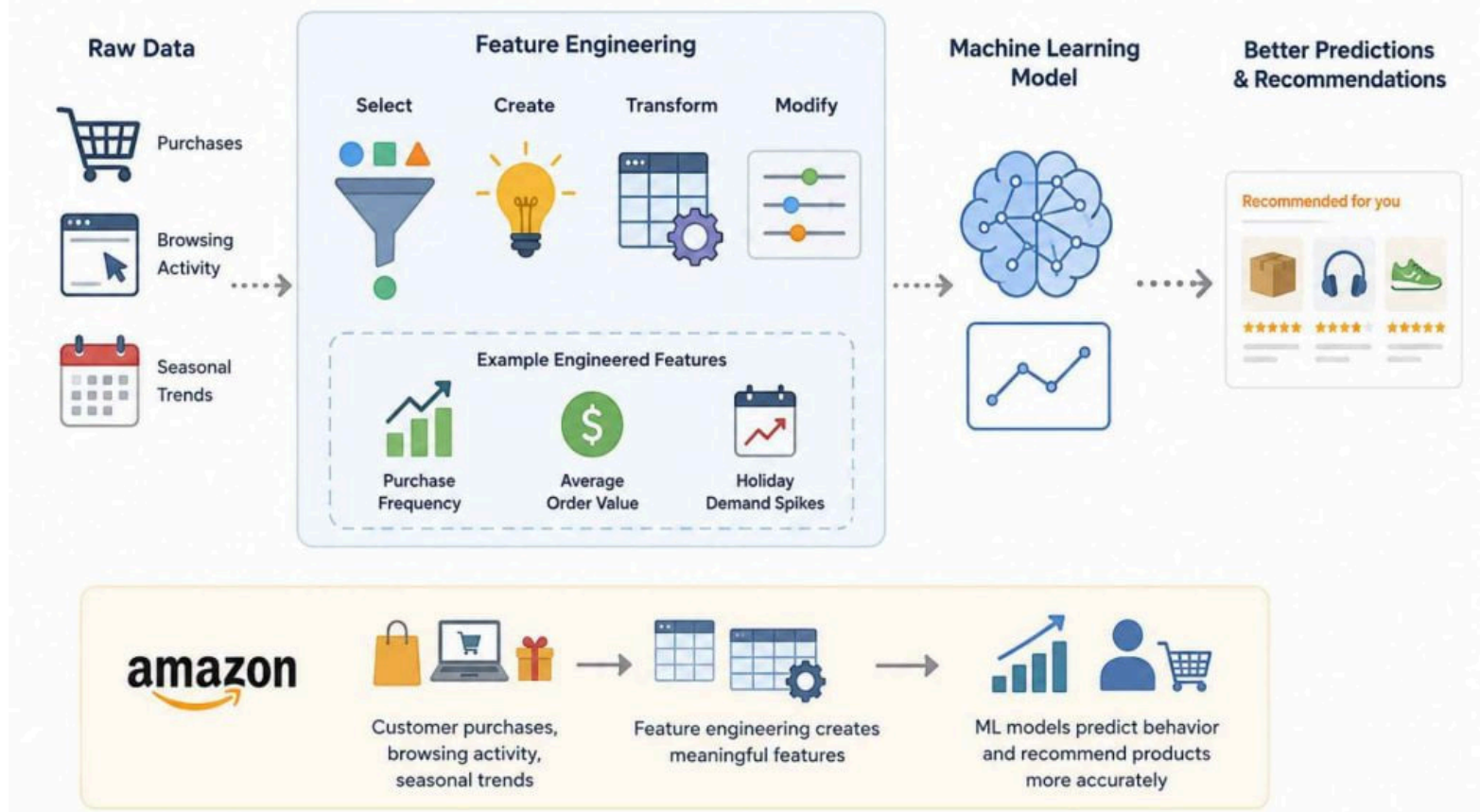


تحلیل خوشه‌ای (Cluster Analysis)

تحلیل خوشه‌ای (Clustering) یکی از روش‌های مهم داده‌کاوی توصیفی (**Descriptive Data Mining**) است که برخلاف طبقه‌بندی، بدون استفاده از داده‌های برچسب‌دار، اشیای داده را بر اساس میزان شباهت گروه‌بندی می‌کند. هدف این روش، **حداکثر کردن شباهت درون‌خوشه‌ای و حداقل کردن شباهت بین‌خوشه‌ای** است؛ به‌گونه‌ای که اعضای هر خوشه بیشترین شباهت را با یکدیگر و کمترین شباهت را با اعضای سایر خوشه‌ها داشته باشند.

در بسیاری از کاربردها که برچسب کلاس در دسترس نیست، خوشه‌بندی می‌تواند **ساختار پنهان داده‌ها** را آشکار کرده و مبنایی برای تولید برچسب‌های اولیه فراهم آورد. همچنین، این روش در ایجاد **رده‌بندی سلسله‌مراتبی (Taxonomy)** و سازمان‌دهی داده‌ها در قالب گروه‌های مشابه نقش مهمی ایفا می‌کند.

خوشه‌بندی در **بازاریابی** برای **بخش‌بندی مشتریان** بر اساس رفتار خرید، در **زیست‌شناسی** برای **طبقه‌بندی گونه‌های جانوری** و در **شبکه‌های اجتماعی** برای **تشخیص جوامع کاربری** کاربرد گسترده‌ای دارد. به‌عنوان مثال، فروشگاه‌های آنلاین با خوشه‌بندی مشتریان، گروه‌های همگن را شناسایی کرده و برای هر گروه **تخفیف‌ها و پیشنهادهای ویژه** طراحی می‌کنند.



📦 مثال عملی: بخش‌بندی مشتریان آمازون (Amazon Customer Segmentation)

شرکت آمازون با داشتن مکرر بیش از ۳۰۰ میلیون کاربر فعال، نمی‌تواند برای هر کاربر برچسب دستی بزند. تیم بازاریابی آمازون از الگوریتم‌های خوشه‌بندی روی مدل RFM (تازگی خرید، فراوانی خرید، ارزش مالی خرید) استفاده می‌کند تا ساختارهای پنهان رفتار خریداران را بدون داشتن هیچ الگوی قبلی کشف کند.

روند کشف خوشه‌ها در آمازون:

ویژگی‌های ورودی بدون برچسب (مثل: تعداد خرید در ماه، میانگین مبلغ هر سبد خرید، استفاده از سرویس Prime و نرخ مرجوعی کالا) به مدل داده می‌شود. الگوریتم خوشه‌بندی، مشتریان را به چند گروه متمایز تفکیک می‌کند:

خوشه ۱ (خریداران وفادار و پرمصرف / Amazon Prime Loyalists):

- ویژگی: فرکانس خرید بسیار بالا، میانگین خریدهای گران‌قیمت، عضو اشتراک پرایم.
- استراتژی آمازون: ارائه دسترسی زودتر به تخفیف‌های Black Friday و پیشنهاد سرویس‌های جدید مثل Amazon Fresh.

خوشه ۲ (شکارچیان تخفیف و خریداران فصلی / Bargain Hunters):

- ویژگی: خرید فقط در زمان حراجی‌های بزرگ (Prime Day)، فرکانس خرید پایین در طول سال، sensitivity بالا به قیمت.
- استراتژی آمازون: ارسال کوپن‌های تخفیف و ایمیل‌های پیشنهاد شگفت‌انگیز برای ترغیب آن‌ها به خرید.

خوشه ۳ (خریداران تک-منظوره یا گذری / One-Time Buyers):

- ویژگی: ثبت‌نام کرده‌اند، فقط یک کالای خاص خریده‌اند و ماه‌هاست برنگشته‌اند.
- استراتژی آمازون: ارسال ایمیل‌های یادآوری ("بقیه خریداران این کالا، این موارد را هم پسندیده‌اند").

```
In [9]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

np.random.seed(42)
```

In [10]: display_academic_text("و بخش‌بندی مشتریان (Cluster Analysis) وظایف داده‌کاوی: تحلیل خوشه‌ای", "main_title")
display_academic_text("تا به دانش پیشین (تازگی، فراوانی و ارزش پولی) RFM است. در این سناریو، مدل (Unlabeled Data) تحلیل خوشه‌ای، فرآیند شناسایی کلاس‌ها یا گروه‌های متراکم در یک پایگاه دادهٔ فاقد برچسب")

```
# =====  
# STEP 1: SIMULATE UNLABELED DATABASE  
# =====  
display_academic_text("تجمیع فضای ویژگی‌های فاقد برچسب (Unlabeled Feature Space)", "sub_title")  
display_academic_text("مشتریان نظیر فرکانس خرید، میانگین مبلغ سبد، روزهای سپری‌شده از آخرین تراکنش و نرخ مرجوعی کالا می‌باشد. سیستم در این مرحله هیچ‌گونه آگاهی نسبت به برچسب کلاسی مشتریان ندارد")  
  
n_customers = 2000  
  
# تولید توزیع‌های آماری متمایز جهت شبیه‌سازی رفتارهای پنهان  
n1 = 700  
purchase_freq_1 = np.random.normal(18, 3, n1)  
avg_basket_1 = np.random.normal(2_500_000, 400_000, n1)  
recency_1 = np.random.normal(3, 2, n1)  
prime_member_1 = np.random.binomial(1, 0.95, n1)  
return_rate_1 = np.random.normal(0.03, 0.02, n1)  
  
n2 = 800  
purchase_freq_2 = np.random.normal(2, 1, n2)  
avg_basket_2 = np.random.normal(600_000, 150_000, n2)  
recency_2 = np.random.normal(45, 20, n2)  
prime_member_2 = np.random.binomial(1, 0.25, n2)  
return_rate_2 = np.random.normal(0.08, 0.03, n2)  
  
n3 = n_customers - n1 - n2  
purchase_freq_3 = np.random.normal(1, 0.3, n3)  
avg_basket_3 = np.random.normal(300_000, 100_000, n3)  
recency_3 = np.random.normal(150, 40, n3)  
prime_member_3 = np.random.binomial(1, 0.05, n3)  
return_rate_3 = np.random.normal(0.05, 0.03, n3)  
  
customer_data = pd.DataFrame({  
    'CustomerID': range(1, n_customers + 1),  
    'MonthlyPurchaseFrequency': np.concatenate([purchase_freq_1, purchase_freq_2, purchase_freq_3]),  
    'AvgBasketValue': np.concatenate([avg_basket_1, avg_basket_2, avg_basket_3]),  
    'RecencyDays': np.concatenate([recency_1, recency_2, recency_3]),  
    'PrimeMember': np.concatenate([prime_member_1, prime_member_2, prime_member_3]),  
    'ReturnRate': np.concatenate([return_rate_1, return_rate_2, return_rate_3])  
})  
  
customer_data['MonthlyPurchaseFrequency'] = customer_data['MonthlyPurchaseFrequency'].clip(lower=0)  
customer_data['AvgBasketValue'] = customer_data['AvgBasketValue'].clip(lower=50_000)  
customer_data['RecencyDays'] = customer_data['RecencyDays'].clip(lower=0)  
customer_data['ReturnRate'] = customer_data['ReturnRate'].clip(lower=0, upper=1)  
  
# Shuffle the dataset to simulate real-world unorganized data  
customer_data = customer_data.sample(frac=1, random_state=42).reset_index(drop=True)  
  
# NOTE: Using the globally defined display_side_by_side_datasets  
display_side_by_side_datasets([customer_data.head(6)], ["برش داده‌های خام مشتریان (Unlabeled Behavioral Data)"]])
```

```
# =====  
# STEP 2: DATA PREPROCESSING  
# =====  
display_academic_text("پیش‌پردازش و استانداردسازی ویژگی‌ها (Data Standardization)", "sub_title")  
display_academic_text("استانداردسازی می‌گردد <b>Z-score</b> جهت جلوگیری از تسلط ویژگی‌هایی با دامنه‌های عددی بزرگ (مانند مبلغ سبد خرید) بر ویژگی‌های مقیاس‌کوچک، فضای حالت با استفاده از روش (PCA)", "pca")
```

```
features = ['MonthlyPurchaseFrequency', 'AvgBasketValue', 'RecencyDays', 'PrimeMember', 'ReturnRate']  
X = customer_data[features]  
  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

```
# =====  
# STEP 3: MODEL EVALUATION & OPTIMAL 'K'  
# =====  
display_academic_text("ارزیابی مدل و تعیین تعداد بهینه خوشه‌ها (Model Evaluation)", "sub_title")  
display_academic_text("روش آرنج<b> (Elbow Method)</b> روش آرنج<b> از ترکیب (k) جهت یافتن تعداد بهینه خوشه‌ها (Silhouette Score)</b> شاخص سیلوئت<b> جهت بررسی واریانس درون خوشه‌ای و (k) جهت یافتن تعداد بهینه خوشه‌ها")
```

```
inertia_values = []  
silhouette_values = []  
k_range = range(2, 8)  
  
for k in k_range:  
    kmeans_temp = KMeans(n_clusters=k, random_state=42, n_init=10)  
    labels_temp = kmeans_temp.fit_predict(X_scaled)  
    inertia_values.append(kmeans_temp.inertia_)  
    silhouette_values.append(silhouette_score(X_scaled, labels_temp))
```

```
fig, axes = plt.subplots(1, 2, figsize=(18, 6), dpi=135)
```

```
# Elbow Method Plot  
axes[0].plot(k_range, inertia_values, marker='o', color='#2c3e50', linewidth=3, markersize=8)  
axes[0].set_xlabel('Number of Clusters (k)', fontdict={'fontsize': 13, 'fontweight': 'bold'})  
axes[0].set_ylabel('Inertia Measure (WCSS)', fontdict={'fontsize': 13, 'fontweight': 'bold'})  
axes[0].set_title('1. Elbow Method Analysis', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)  
axes[0].axvline(3, color='#e74c3c', linestyle='--', lw=2.5, label='Optimal k=3 (The Elbow)')  
axes[0].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=11)  
axes[0].grid(True, linestyle='--', alpha=0.5)
```

```
# Silhouette Score Plot  
axes[1].plot(k_range, silhouette_values, marker='s', color='#1abc9c', linewidth=3, markersize=8)  
axes[1].set_xlabel('Number of Clusters (k)', fontdict={'fontsize': 13, 'fontweight': 'bold'})  
axes[1].set_ylabel('Average Silhouette Score', fontdict={'fontsize': 13, 'fontweight': 'bold'})  
axes[1].set_title('2. Silhouette Quality Analysis', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)  
axes[1].axvline(3, color='#e74c3c', linestyle='--', lw=2.5, label='Optimal k=3 (Global Maximum)')  
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=11)  
axes[1].grid(True, linestyle='--', alpha=0.5)
```

```
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

display_academic_text("به‌عنوان تعداد بهینه خوشه‌های رفتاری تثبیت می‌گردد <b>k=3</b> استنتاج ارزیابی: بر مبنای نقطه عطف در نمودار آرنج و حداکثر شدن شاخص سیلوئت، پارامتر")

# =====
# STEP 4: CLUSTERING & PROFILING
# =====
display_academic_text("۴. اجرای الگوریتم K-Means و پروفایل‌سازی الگوها (Cluster Profiling)", "sub_title")
display_academic_text("خوشه جهت استخراج استراتژی‌های تجاری ارزیابی می‌شود (Centroid) رکوردهای فاقد برچسب به ۳ خوشه مجزا تفکیک شده و پروفایل مرکزی، k=3 با K-Means با اعمال الگوریتم پارتیشن‌بندی")

optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
customer_data['AssignedCluster'] = kmeans.fit_predict(X_scaled)

# Calculate Cluster Centroids for business profiling
cluster_profile = customer_data.groupby('AssignedCluster')[features].mean().reset_index()
cluster_labels_map = cluster_profile.sort_values(by='MonthlyPurchaseFrequency', ascending=False)
ranked_clusters = cluster_labels_map['AssignedCluster'].tolist()

# Map cluster IDs to business logic
business_names = {
    ranked_clusters[0]: "Prime Loyalists",
    ranked_clusters[1]: "Bargain Hunters",
    ranked_clusters[2]: "One-Time Buyers"
}

strategy_map = {
    ranked_clusters[0]: "Cross-Selling & Early Access Promotions",
    ranked_clusters[1]: "Event-based Discounts (e.g., Prime Day)",
    ranked_clusters[2]: "Re-engagement Campaigns & Collaborative Filtering"
}

cluster_profile['Semantic Segment'] = cluster_profile['AssignedCluster'].map(business_names)
cluster_profile['Proposed BI Strategy'] = cluster_profile['AssignedCluster'].map(strategy_map)
customer_data['SegmentName'] = customer_data['AssignedCluster'].map(business_names)

display_side_by_side_datasets([cluster_profile, ["دانش استخراج‌شده: پروفایل تحلیلی خوشه‌ها و نگاشت به استراتژی‌های هوش تجاری"]])

# =====
# STEP 5: DIMENSIONALITY REDUCTION & VISUALIZATION (PCA)
# =====
display_academic_text("۵. کاهش ابعاد و مصورسازی ساختار خوشه‌ها (Dimensionality Reduction)", "sub_title")
display_academic_text("استفاده می‌گردد تا مرزبندی خوشه‌های کشف‌شده به وضوح نمایش داده شود (PCA) تحلیل مؤلفه‌های اصلی <b>جهت ترسیم فضای ۵-بعدی ویژگی‌ها در یک بوم ۲-بعدی، از الگوریتم")

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
customer_data['PCA_Component_1'] = X_pca[:, 0]
customer_data['PCA_Component_2'] = X_pca[:, 1]

plt.figure(figsize=(14, 7), dpi=135)

# Applying Academic Blue Palette mapped to our segments
palette = {
    "Prime Loyalists": "#1abc9c", # Turquoise
    "Bargain Hunters": "#2980b9", # Classic Blue
    "One-Time Buyers": "#2c3e50" # Navy
}

sns.scatterplot(
    data=customer_data, x='PCA_Component_1', y='PCA_Component_2',
    hue='SegmentName', palette=palette, alpha=0.75, s=65, edgecolor='#ffffff', linewidth=0.8
)

# Plotting the mathematically derived centroids
centers_pca = pca.transform(kmeans.cluster_centers_)
plt.scatter(
    centers_pca[:, 0], centers_pca[:, 1],
    c='#e74c3c', marker='X', s=300, label='K-Means Cluster Centroids', edgecolor='#ffffff', linewidth=2.5, zorder=5
)

plt.title('Principal Component Analysis: Visualization of Derived Clusters (2D Projection)', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)
plt.xlabel('Principal Component 1 (Maximized Orthogonal Variance)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
plt.ylabel('Principal Component 2 (Orthogonal Variance)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=11)
plt.grid(True, linestyle='--', alpha=0.5)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# =====
# STEP 6: FEATURE DISTRIBUTION ANALYSIS
# =====
display_academic_text("۶. مقایسه توزیع آماری ویژگی‌های کلیدی در خوشه‌ها", "sub_title")
display_academic_text("در خوشه‌های استخراج‌شده، صحت تفکیک‌پذیری و تمایز الگوهای رفتاری را تأیید می‌نماید (فراوانی، ارزش پولی و تازگی) RFM توزیع مقادیر")

fig, axes = plt.subplots(1, 3, figsize=(18, 5.5), dpi=135)
profile_plot_data = cluster_profile.sort_values(by='MonthlyPurchaseFrequency', ascending=False)

sns.barplot(data=profile_plot_data, x="Semantic Segment", y="MonthlyPurchaseFrequency", hue="Semantic Segment", palette=palette, legend=False, ax=axes[0], edgecolor="#2c3e50", lw=1.2)
axes[0].set_title("Frequency: Purchases per Month", fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[0].set_ylabel('Count')

sns.barplot(data=profile_plot_data, x="Semantic Segment", y="AvgBasketValue", hue="Semantic Segment", palette=palette, legend=False, ax=axes[1], edgecolor="#2c3e50", lw=1.2)
axes[1].set_title("Monetary: Avg Basket Value", fontdict={'fontsize': 13, 'fontweight': 'bold'})
```

```
axes[1].set_ylabel('Currency Value')

sns.barplot(data=profile_plot_data, x="Semantic Segment", y="RecencyDays", hue="Semantic Segment", palette=palette, legend=False, ax=axes[2], edgecolor="#2c3e50", lw=1.2)
axes[2].set_title("Recency: Days Since Last Purchase", fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[2].set_ylabel('Days')

for ax in axes:
    ax.tick_params(axis='x', rotation=15, labelsz=11)
    ax.grid(axis='y', linestyle='--', alpha=0.5)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.set_xlabel('') # Remove repetitive x-axis label

plt.tight_layout()
plt.show()
```

وظایف داده‌کاوی: تحلیل خوشه‌ای (Cluster Analysis) و بخش‌بندی مشتریان

تحلیل خوشه‌ای، فرآیند شناسایی کلاس‌ها یا گروه‌های متراکم در یک پایگاه داده فاقد برچسب (Unlabeled Data) است. در این سناریو، مدل RFM (تازگی، فراوانی و ارزش پولی) جهت بخش‌بندی مشتریان آمزون و استخراج الگوهای رفتاری پنهان بدون اتکا به دانش پیشین (Prior Knowledge) پیاده‌سازی می‌گردد.

۱. تجمیع فضای ویژگی‌های فاقد برچسب (Unlabeled Feature Space)

مجموعه داده ورودی صرفاً شامل ویژگی‌های رفتاری مشتریان نظیر فرکانس خرید، میانگین مبلغ سبد، روزهای سپری‌شده از آخرین تراکنش و نرخ مرجوعی کالا می‌باشد. سیستم در این مرحله هیچ‌گونه آگاهی نسبت به برچسب کلاسی مشتریان ندارد.

برش داده‌های خام مشتریان (Unlabeled Behavioral Data)					
CustomerID	MonthlyPurchaseFrequency	AvgBasketValue	RecencyDays	PrimeMember	ReturnRate
1861	1.222	170687.775	132.597	0	0.081
354	17.520	2604269.506	0.000	1	0.002
1334	2.751	639640.241	52.653	0	0.031
906	2.052	914690.901	64.472	0	0.061
1290	0.721	614671.994	71.561	0	0.110
1274	2.761	671011.112	49.210	0	0.087

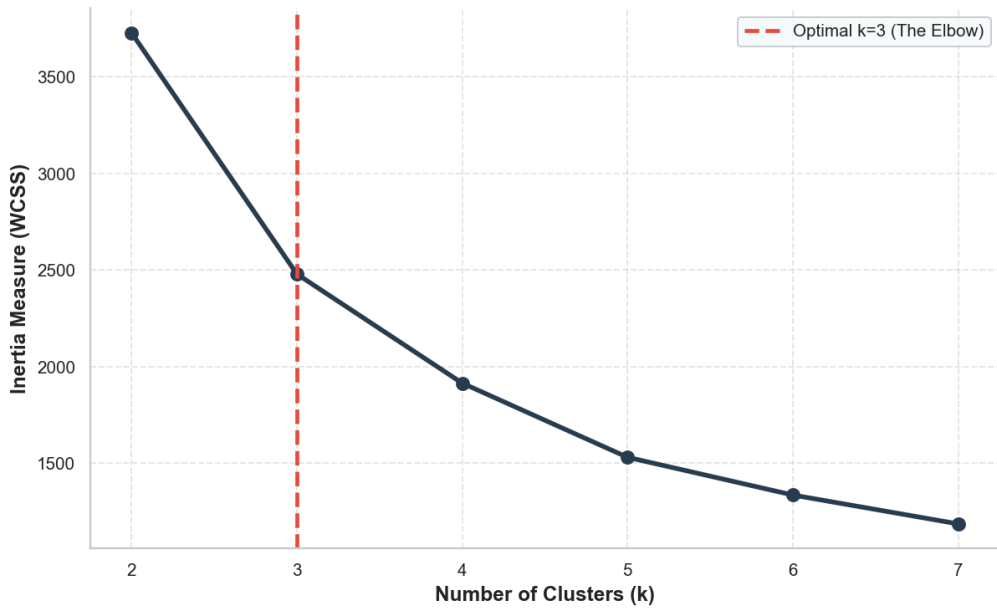
۲. پیش‌پردازش و استانداردسازی ویژگی‌ها (Data Standardization)

جهت جلوگیری از تسلط ویژگی‌هایی با دامنه‌های عددی بزرگ (مانند مبلغ سبد خرید) بر ویژگی‌های مقیاس‌کوچک، فضای حالت با استفاده از روش **Z-score** استانداردسازی می‌گردد.

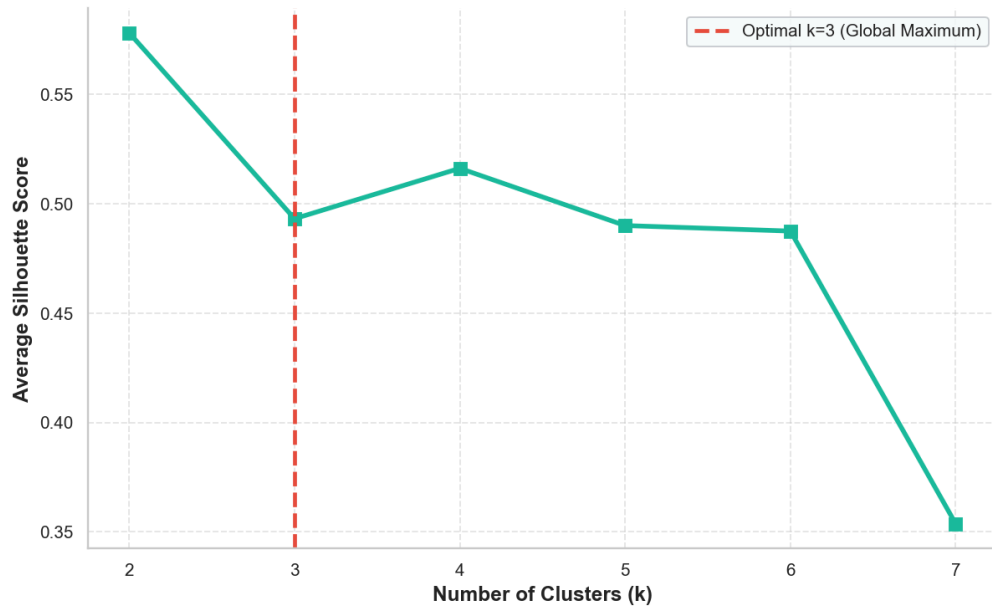
۳. ارزیابی مدل و تعیین تعداد بهینه خوشه‌ها (Model Evaluation)

جهت یافتن تعداد بهینه خوشه‌ها (k)، از ترکیب روش **آرنج (Elbow Method)** جهت بررسی واریانس درون‌خوشه‌ای و شاخص **سیلوئت (Silhouette Score)** جهت ارزیابی کیفیت تفکیک‌پذیری خوشه‌ها استفاده می‌شود.

1. Elbow Method Analysis



2. Silhouette Quality Analysis



استنتاج ارزیابی: بر مبنای نقطه عطف در نمودار آرنج و حداکثر شدن شاخص سیلوئت، پارامتر $k=3$ به عنوان تعداد بهینه خوشه‌های رفتاری تثبیت می‌گردد.

۴. اجرای الگوریتم K-Means و پروفایل‌سازی الگوها (Cluster Profiling)

با اعمال الگوریتم پارتیشن‌بندی K-Means با $k=3$ ، رکوردهای فاقد برچسب به ۳ خوشه مجزا تفکیک شده و پروفایل مرکزی (Centroid) هر خوشه جهت استخراج استراتژی‌های تجاری ارزیابی می‌شود.

دانش استخراج‌شده: پروفایل تحلیلی خوشه‌ها و نگاشت به استراتژی‌های هوش تجاری							
AssignedCluster	MonthlyPurchaseFrequency	AvgBasketValue	RecencyDays	PrimeMember	ReturnRate	Semantic Segment	
0	17.976	2530191.848	3.151	0.947	0.030	Prime Loyalists	Cross
1	1.939	575993.331	45.890	0.208	0.080	Bargain Hunters	Event
2	1.032	318414.968	154.425	0.036	0.047	One-Time Buyers	Re-engager

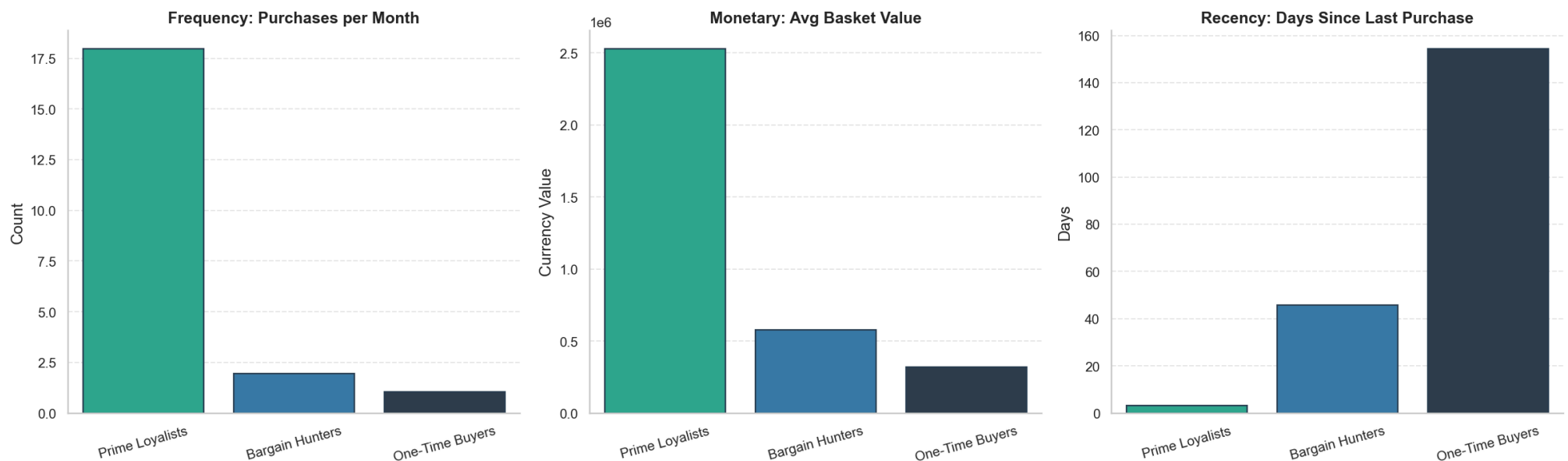
۵. کاهش ابعاد و مصورسازی ساختار خوشه‌ها (Dimensionality Reduction)

جهت ترسیم فضای ۵-بعدی ویژگی‌ها در یک بوم ۲-بعدی، از الگوریتم تحلیل مؤلفه‌های اصلی (PCA) استفاده می‌گردد تا مرزبندی خوشه‌های کشف‌شده به وضوح نمایش داده شود.

Principal Component Analysis: Visualization of Derived Clusters (2D Projection)



توزیع مقادیر RFM (فراوانی، ارزش پولی و تازگی) در خوشه‌های استخراج‌شده، صحت تفکیک‌پذیری و تمایز الگوهای رفتاری را تأیید می‌نماید.



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند **یادگیری عمیق** به صورت کامل و در یک سناریوی واقعی پیش‌بینی شیوع بیماری پیاده‌سازی شده است:

- ابتدا یک مجموعه داده فرضی از **داده‌های اپیدمیولوژیکی** شبیه‌سازی شده است که شامل ویژگی‌هایی مانند موارد مثبت، تعداد تست‌ها، بستری‌ها، شاخص تحرک مکانی، دمای محیط و تراکم جمعیت برای چند منطقه در طول زمان است.
 - در مرحله **مهندسی ویژگی**، از یک ویژگی هدف هوشمند استفاده شده است: **نرخ رشد موارد مثبت در 7 روز آینده** که به عنوان متغیر پاسخ برای مدل رگرسیونی تعریف شده است.
 - در مرحله **پیش‌پردازش داده**، از **StandardScaler** برای استانداردسازی ویژگی‌ها استفاده شده است تا همه ویژگی‌ها در مقیاس مشابهی باشند.
 - یک **شبکه عصبی چندلایه (MLP)** با سه لایه پنهان و لایه‌های Dropout برای جلوگیری از بیش‌برازش ساخته شده است.
 - مدل با استفاده از الگوریتم **Adam** و تابع خطا **MSE** آموزش داده شده و از **Early Stopping** و **ReduceLROnPlateau** به عنوان callback‌ها استفاده شده است.
 - نتایج به صورت بصری نمایش داده شده‌اند:
 - یک **نمودار خطی** تاریخچه آموزش مدل را نشان می‌دهد که شامل Loss و Validation Loss در طول اپوچ‌ها است.
 - یک **نمودار پراکندگی** مقایسه بین نرخ رشد واقعی و پیش‌بینی‌شده را نشان می‌دهد.
 - یک **نمودار میله‌ای** اهمیت ویژگی‌ها را با روش **Permutation Importance** نمایش می‌دهد.
 - کیفیت مدل با استفاده از معیارهای **MSE** و **R²** ارزیابی شده است که نشان‌دهنده توانایی خوب مدل در پیش‌بینی نرخ رشد بیماری است.
 - در نهایت، مناطق بر اساس پیش‌بینی به سطوح خطر "کم"، "متوسط" و "زیاد" تقسیم شده‌اند که به تصمیم‌گیرندگان سلامت کمک می‌کند تا منابع را به‌طور بهینه تخصیص دهند.
- این مثال به‌وضوح نشان می‌دهد که چگونه **یادگیری عمیق** می‌تواند فرآیند مهندسی ویژگی را به صورت خودکار انجام دهد و ویژگی‌های معناداری مانند الگوهای رشد نمایی یا روابط پیچیده بین تحرک مکانی و شیوع بیماری را بدون نیاز به دانش حوزه‌ای دستی استخراج کند. برخلاف روش‌های سنتی که به متخصصان نیاز دارند تا ویژگی‌های دستی طراحی کنند، شبکه عصبی به‌صورت خودکار این ویژگی‌ها را در لایه‌های پنهان خود یاد می‌گیرد. این مثال نشان می‌دهد که چگونه یادگیری عمیق نه تنها دقت پیش‌بینی را بهبود می‌دهد، بلکه قابلیت تفسیر الگوهای پیچیده زمانی-مکانی را نیز فراهم می‌کند و به تصمیم‌گیرندگان کمک می‌کند تا مناطق پرخطر را شناسایی کنند.

1.4.5 Deep learning

یادگیری عمیق (Deep Learning)

یادگیری عمیق (Deep Learning) یکی از پیشرفته‌ترین روش‌های داده‌کاوی و یادگیری ماشین است که با تکیه بر **شبکه‌های عصبی عمیق (Deep Neural Networks)**، فرایند **استخراج ویژگی‌ها (Feature Extraction)** را به‌صورت خودکار انجام می‌دهد. برخلاف

روش‌های سنتی که به **مهندسی ویژگی (Feature Engineering)** و دانش تخصصی وابسته هستند، این روش قادر است ویژگی‌های معنادار را از داده‌های خام استخراج کرده و دقت مدل‌های داده‌کاوی را بهبود بخشد.

در واقع این شبکه‌های عصبی از مجموعه‌ای از واحدهای ورودی و خروجی (**نورون‌ها**) متصل به هم تشکیل شده‌اند که هر اتصال، یک **وزن خاص** دارد. فرآیند یادگیری، تنظیم هوشمندانه این وزن‌ها است تا شبکه بتواند **خروجی درست** را پیش‌بینی کند.

امروزه **یادگیری عمیق** در حوزه‌های پیشرفته‌ای مانند **بینایی ماشین**، **پردازش زبان طبیعی**، **ترجمه ماشینی** و **تحلیل شبکه‌های اجتماعی** به یک ابزار همه‌فن‌حریف تبدیل شده است. همچنین در وظایف اصلی داده‌کاوی مانند **طبقه‌بندی**، **خوشه‌بندی**، **تشخیص ناهنجاری** و **یادگیری تقویتی** نیز کاربرد گسترده‌ای دارد.

یادگیری عمیق دیگر فقط یک تئوری نیست، بلکه به بخشی از زندگی روزمره ما تبدیل شده است. یکی از کاربردهای آن، **سیستم‌های تشخیص چهره (Face Recognition)** در گوشی‌های هوشمند است. این سیستم‌ها با بهره‌گیری از **شبکه‌های عصبی کانولوشنی (CNN)**، ویژگی‌های منحصر به فرد صورت را استخراج کرده و با دقت بسیار بالایی هویت فرد را شناسایی می‌کنند.

یادگیری عمیق دیگر فقط یک تئوری نیست، بلکه به بخشی از زندگی روزمره ما تبدیل شده است. یکی از کاربردهای آن، **سیستم‌های تشخیص چهره (Face Recognition)** در گوشی‌های هوشمند است. این سیستم‌ها با بهره‌گیری از **شبکه‌های عصبی کانولوشنی (CNN)**، ویژگی‌های منحصر به فرد صورت را استخراج کرده و با دقت بسیار بالایی هویت فرد را شناسایی می‌کنند.

1.4.6 Outlier analysis



تحلیل داده‌های پرت (Outlier Analysis)

داده‌های پرت (Outliers) یا **ناهنجاری‌ها (Anomalies)**، نقاطی هستند که با رفتار کلی یا مدل حاکم بر مجموعه داده سازگاری ندارند. جالب است بدانید که در بسیاری از روش‌های داده‌کاوی، این نقاط را به عنوان **نویز** یا **استثنا** حذف می‌کنند؛ اما در واقعیت، این داده‌های غیرعادی در کاربردهای حیاتی می‌توانند **ارزشمندتر** از داده‌های معمولی باشند.

تحلیل این داده‌های غیرعادی در حوزه‌هایی مانند **تشخیص تقلب کارت‌های اعتباری**، **امنیت سایبری** و **پایش تجهیزات صنعتی** مؤثر است. مثلاً اگر سیستمی ببیند که از یک حساب بانکی، خریدی با مبلغی **۱۰ برابر میانگین همیشگی** ثبت شده، بلافاصله آن را به عنوان یک «**ناهنجاری**» شناسایی کرده و **هشدار امنیتی** صادر می‌کند.

شناسایی داده‌های پرت با استفاده از **روش‌های آماری (Statistical Methods)**، **روش‌های مبتنی بر فاصله (Distance-based Methods)** و **روش‌های مبتنی بر چگالی (Density-based Methods)** انجام می‌شود. هر یک از این روش‌ها با تکیه بر ویژگی‌های داده، ناهنجاری‌ها را شناسایی کرده و امکان کشف الگوهای غیرعادی و تصمیم‌گیری دقیق‌تر را فراهم می‌کنند.



مثال عملی: تشخیص تقلب در استفاده از کارت‌های اعتباری

در این مثال، تحلیل داده‌های پرت برای شناسایی استفاده‌های تقلبی از کارت‌های اعتباری استفاده شده است. سیستم با بررسی الگوهای خرید یک حساب، خریدهایی را که از نظر مقدار، مکان، نوع کالا یا فرکانس خرید، انحراف غیرعادی نسبت به رفتار معمول دارند، به عنوان ناهنجاری علامت‌گذاری می‌کند. برای مثال، خریدی به مبلغ ۱۰ برابر میانگین خریدهای قبلی، در یک کشور خارجی و در فاصله زمانی کوتاه با چند خرید دیگر، به راحتی توسط مدل پرت‌یابی شناسایی می‌شود. این سیستم می‌تواند به صورت خودکار هشدار ایجاد کند و اقدامات

امنیتی را فعال کند. این مثال نشان می‌دهد که چگونه داده‌های پرت که اغلب به عنوان نویز کنار گذاشته می‌شوند، می‌توانند حامل دانش

بسیار ارزشمندی در کاربردهای حیاتی باشند.

```
In [11]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

np.random.seed(42)
```

```
In [12]: display_academic_text("کشف ناهنجاری در تراکنش‌های مالی: (Outlier Analysis) کاوش داده‌های پرت", "main_title")
display_academic_text("ش‌های متقلبانه<b> در پایگاه داده هستند که با رفتار عمومی یا مدل توزیع داده‌ها مطابقت ندارند. در این سناریو، از الگوریتم‌های کاوش ناهنجاری برای شناسایی (Outliers) داده‌های پرت")

# =====
# STEP 1: SIMULATE FEATURE SPACE & OUTLIERS
# =====
display_academic_text("۱. شبیه‌سازی فضای ویژگی (Feature Space Simulation)", "sub_title")
display_academic_text("Imbalanced D پایگاه داده شامل تراکنش‌های نرمال (با توزیع آماری مشخص) و تراکنش‌های ناهنجار (با مقادیر انحرافی در مبلغ، زمان، مکان و فرکانس) است تا ساختار یک توزیع شدیداً نامتوازن")

n_normal_transactions = 1000
n_fraud_transactions = 50

# مدل‌سازی تراکنش‌های نرمال (Inliers)
normal_data = {
    'Amount': np.random.lognormal(3, 1, n_normal_transactions),
    'Time_of_Day': np.random.normal(14, 5, n_normal_transactions),
    'Merchant_Category': np.random.choice(['Grocery', 'Restaurant', 'Gas', 'Retail'], n_normal_transactions, p=[0.4, 0.3, 0.2, 0.1]),
    'Transaction_Frequency': np.random.poisson(2, n_normal_transactions),
    'Location_Score': np.random.beta(2, 5, n_normal_transactions),
    'Is_Fraud': 0
}
df_normal = pd.DataFrame(normal_data)

# مدل‌سازی تراکنش‌های متقلبانه (Outliers)
fraud_data = {
    'Amount': np.random.lognormal(5, 1.5, n_fraud_transactions),
    'Time_of_Day': np.random.choice([2, 3, 4, 5], n_fraud_transactions),
    'Merchant_Category': np.random.choice(['Electronics', 'Jewelry', 'Online_Gaming'], n_fraud_transactions, p=[0.4, 0.3, 0.3]),
    'Transaction_Frequency': np.random.poisson(8, n_fraud_transactions),
    'Location_Score': np.random.beta(8, 2, n_fraud_transactions),
    'Is_Fraud': 1
}
df_fraud = pd.DataFrame(fraud_data)

# جمع و پیش‌پردازش
df = pd.concat([df_normal, df_fraud], ignore_index=True)
df['Time_of_Day'] = np.clip(df['Time_of_Day'], 0, 23)
df['Location_Score'] = np.clip(df['Location_Score'], 0, 1)

display_side_by_side_datasets([df.sample(6, random_state=42).reset_index(drop=True)], ["۱. (شامل ویژگی‌های پیوسته و گسسته)"])

# =====
# STEP 2: MODELING & ANOMALY EXTRACTION
# =====
display_academic_text("۲. (Anomaly Detection) استانداردسازی و اعمال الگوریتم‌های کاوش ناهنجاری")
display_academic_text("محاسبه می‌کند <b>LOF</b> تشخیص می‌دهد. در مقابل، الگوریتم (Isolation-based) ناهنجاری‌ها را بر اساس انزوای هندسی اشیاء در درخت‌های دودویی <b>Isolation Forest</b> الگوریتم")

feature_columns = ['Amount', 'Time_of_Day', 'Transaction_Frequency', 'Location_Score']
X = df[feature_columns]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# الگوریتم Isolation Forest
isolation_forest = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)
df['Anomaly_Score_IF'] = isolation_forest.fit_predict(X_scaled)
df['Anomaly_Score_IF'] = df['Anomaly_Score_IF'].map({1: 0, -1: 1}) # Convert to 0/1

# الگوریتم Local Outlier Factor (LOF)
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
df['Anomaly_Score_LOF'] = lof.fit_predict(X_scaled)
df['Anomaly_Score_LOF'] = df['Anomaly_Score_LOF'].map({1: 0, -1: 1}) # Convert to 0/1

# =====
# STEP 3: EVALUATION METRICS
# =====
display_academic_text("۳. (Evaluation Metrics) ارزیابی عملکرد مدل‌های پرتیابی")
display_academic_text("فراخوانی<b> (Recall) و دقت<b> (Precision) گمراه‌کننده است. بنابراین از ماتریس‌های ارزیابی شامل Accuracy نگاه کردن به (Imbalanced Classes) با توجه به نامتوازن بودن کلاس‌ها")

# Classification Reports
report_if = classification_report(df['Is_Fraud'], df['Anomaly_Score_IF'], target_names=['Normal (Inlier)', 'Fraud (Outlier)'], output_dict=True)
df_report_if = pd.DataFrame(report_if).transpose().reset_index().round(3).head(2) # Drop avg rows for clean display
```

```
df_report_if.columns = ['Metric / Class', 'Precision', 'Recall', 'F1-Score', 'Support']

report_lof = classification_report(df['Is_Fraud'], df['Anomaly_Score_LOF'], target_names=['Normal (Inlier)', 'Fraud (Outlier)'], output_dict=True)
df_report_lof = pd.DataFrame(report_lof).transpose().reset_index().round(3).head(2)
df_report_lof.columns = ['Metric / Class', 'Precision', 'Recall', 'F1-Score', 'Support']

# Use side-by-side to display both models simultaneously
display_side_by_side_datasets(
    [df_report_if, df_report_lof],
    ["مبتنی بر چگالی) LOF: ماتریس ارزیابی", "(مبتنی بر انزوا) Isolation Forest: ماتریس ارزیابی"]
)

# Feature Centroids comparison
comparison = df.groupby('Is_Fraud')[feature_columns].mean().reset_index()
comparison['Is_Fraud'] = comparison['Is_Fraud'].map({0: 'Normal Average (Centroid)', 1: 'Fraud Average (Centroid)'})
display_side_by_side_datasets([comparison], ["پروفایل استخراج‌شده: بردار میانگین ویژگی‌ها در خوشه‌های نرمال و پرت"]])

# =====
# STEP 4: VISUALIZATION OF ANOMALY SPACE
# =====
display_academic_text("۴. بازنمایی بصری فضای ویژگی‌ها و تشخیص ناهنجاری", "sub_title")

fig, axes = plt.subplots(1, 3, figsize=(19, 6.5), dpi=135)

# CHART 1: Amount Distribution (Inlier vs Outlier)
axes[0].hist(df[df['Is_Fraud'] == 0]['Amount'], bins=35, alpha=0.85, label='Normal (Inlier)', color='#2c3e50', edgecolor='white', lw=0.5)
axes[0].hist(df[df['Is_Fraud'] == 1]['Amount'], bins=35, alpha=0.85, label='Fraud (Outlier)', color='#e74c3c', edgecolor='white', lw=0.5)

axes[0].annotate(
    "The Outlier Tail:\nFraudulent transactions generally\nhave much higher amounts,\nbut they overlap with normal data!",
    xy=(200, 10), xytext=(150, 120),
    fontsize=10, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=1.5),
    bbox=dict(boxstyle="round,pad=0.3", fc="#fdf2e9", ec="#e74c3c", lw=1.2)
)

axes[0].set_xlabel('Transaction Amount (Monetary Value)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_ylabel('Frequency (Observation Count)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_title('1. Data Distribution: Transaction Amounts', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes[0].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7')
axes[0].grid(True, alpha=0.5, linestyle='--')

# CHART 2: Isolation Forest Outlier Space
scatter_if = axes[1].scatter(
    df['Amount'], df['Time_of_Day'],
    c=df['Anomaly_Score_IF'], cmap='coolwarm', alpha=0.75, s=55, edgecolor='white', linewidth=0.5
)

axes[1].set_xlabel('Transaction Amount', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_ylabel('Temporal Feature (Time of Day)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_title('2. Outlier Space: Isolation Forest', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes[1].grid(True, alpha=0.5, linestyle='--')

# CHART 3: Local Outlier Factor Space
scatter_lof = axes[2].scatter(
    df['Amount'], df['Transaction_Frequency'],
    c=df['Anomaly_Score_LOF'], cmap='coolwarm', alpha=0.75, s=55, edgecolor='white', linewidth=0.5
)

axes[2].set_xlabel('Transaction Amount', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[2].set_ylabel('Transaction Frequency', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[2].set_title('3. Outlier Space: Local Outlier Factor (LOF)', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes[2].grid(True, alpha=0.5, linestyle='--')

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# STRATEGIC CONCLUSION & FALSE NEGATIVE ANALYSIS
# =====
false_negatives = df[(df['Is_Fraud'] == 1) & (df['Anomaly_Score_IF'] == 0)]
fn_count = len(false_negatives)
total_fraud = df['Is_Fraud'].sum()

insight_text = f"<b>استنتاج کشف دانش (Knowledge Discovery Inference):</b><br><br>"
insight_text += f"۱. <b>Isolation Forest</b> اثربخشی الگوریتم<b>مدل قادر به شناسایی</b> {df_report_if.iloc[1]['Recall']*100:.1f}%</b> در فضای پرنویز گردید"
insight_text += f"۲. <b>LOF</b> اثربخشی الگوریتم<b>روش مبتنی بر چگالی موفق به استخراج</b> {df_report_lof.iloc[1]['Recall']*100:.1f}%</b> از ناهنجاری‌ها شد"

if fn_count > 0:
    insight_text += (
        f"<b>Isolation Forest</b> تراکنش متقلبانه از سیستم {fn_count} تعداد (False Negatives) تحلیل خطای نوع دوم<b>فرار کرده"
        f"<b>Masking</b> و به اشتباه برجسپ «نرمال» دریافت نمودند. این رفتار حاکی از پدیدهٔ"
        f"!جایی که یک تراکنش متقلبانه آنقدر هوشمندانه طراحی شده که دقیقاً در وسط تراکم تراکنش‌های سالم پنهان می‌شود!"
    )

display_academic_text(insight_text, "insight")

if fn_count > 0:
    fn_profile = false_negatives[feature_columns].mean().to_frame().T
    display_side_by_side_datasets(
        [fn_profile],
        ["(False Negatives Profile) بردار میانگین ویژگی‌ها در تراکنش‌های متقلبانهٔ فرارکرده"]
    )
```

کاوش داده‌های پرت (Outlier Analysis): کشف ناهنجاری در تراکنش‌های مالی

داده‌های پرت (Outliers) اشیائی در پایگاه داده هستند که با رفتار عمومی یا مدل توزیع داده‌ها مطابقت ندارند. در این سناریو، از الگوریتم‌های کاوش ناهنجاری برای شناسایی تراکنش‌های متقلبانه (Fraud Detection) استفاده می‌شود؛ در اینجا داده‌های پرت نمایانگر نویز نیستند، بلکه حامل دانشی حیاتی می‌باشند. ما عملکرد دو الگوریتم قدرتمند Isolation Forest (مبتنی بر انزوا) و LOF (مبتنی بر چگالی محلی) را با هم مقایسه می‌کنیم.

۱. شبیه‌سازی فضای ویژگی (Feature Space Simulation)

پایگاه داده شامل تراکنش‌های نرمال (با توزیع آماری مشخص) و تراکنش‌های ناهنجار (با مقادیر انحرافی در مبلغ، زمان، مکان و فرکانس) است تا ساختار یک توزیع شدیداً نامتوازن (Imbalanced Distribution) شکل گیرد.

۱. نمونه‌ای از پایگاه داده تراکنشی (شامل ویژگی‌های پیوسته و گسسته)					
Amount	Time_of_Day	Merchant_Category	Transaction_Frequency	Location_Score	Is_Fraud
47.354	16.707	Grocery	2	0.292	0
31.320	12.705	Retail	1	0.270	0
4.844	18.663	Grocery	1	0.173	0
51.957	14.160	Gas	2	0.444	0
128.032	5.657	Restaurant	2	0.305	0
2.541	14.905	Grocery	2	0.182	0

۲. استانداردسازی و اعمال الگوریتم‌های کاوش ناهنجاری (Anomaly Detection)

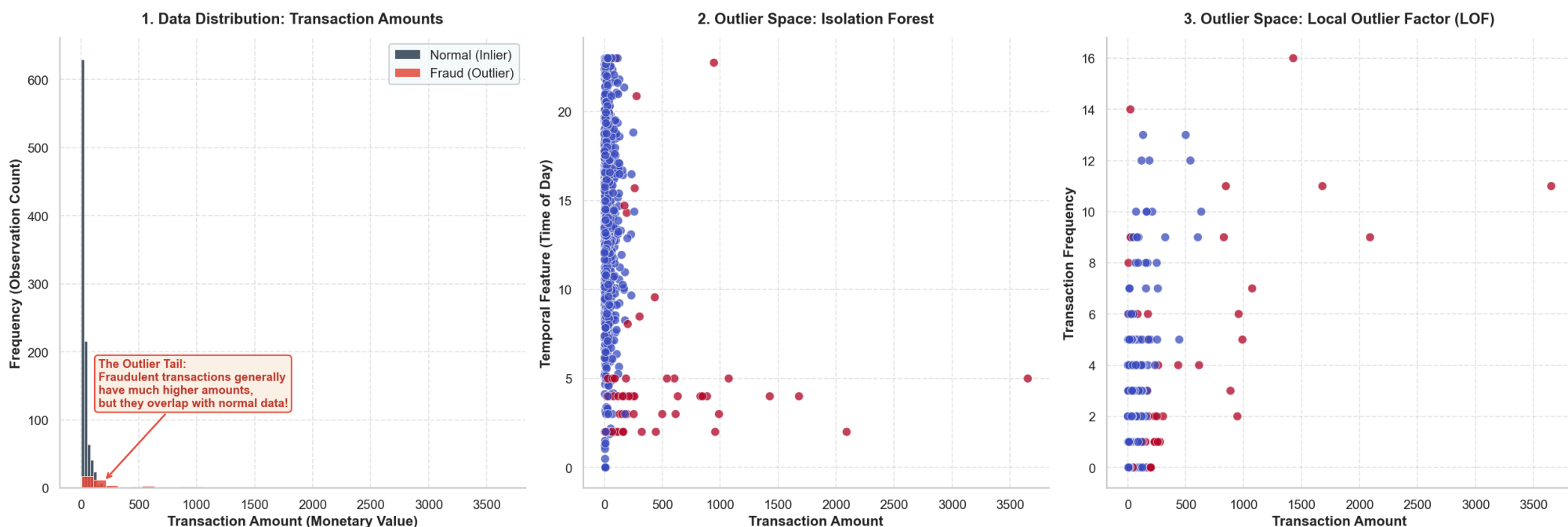
الگوریتم Isolation Forest ناهنجاری‌ها را بر اساس انزوای هندسی اشیاء در درخت‌های دودویی (Isolation-based) تشخیص می‌دهد. در مقابل، الگوریتم LOF انحراف چگالی محلی یک شیء نسبت به همسایگانش را محاسبه می‌کند (Density-based).

۳. ارزیابی عملکرد مدل‌های پرت‌یابی (Evaluation Metrics)

با توجه به نامتوازن بودن کلاس‌ها (Imbalanced Classes)، نگاه کردن به Accuracy گمراه‌کننده است. بنابراین از ماتریس‌های ارزیابی شامل دقت (Precision) و فراخوانی (Recall) جهت مقایسه کارایی دو الگوریتم استفاده می‌گردد.

مبتنی بر انزوا: Isolation Forest: ماتریس ارزیابی					مبتنی بر چگالی: LOF: ماتریس ارزیابی				
Metric / Class	Precision	Recall	F1-Score	Support	Metric / Class	Precision	Recall	F1-Score	Support
Normal (Inlier)	0.993	0.990	0.991	1000.000	Normal (Inlier)	0.963	0.960	0.961	1000.000
Fraud (Outlier)	0.811	0.860	0.835	50.000	Fraud (Outlier)	0.245	0.260	0.252	50.000

پروفایل استخراج‌شده: بردار میانگین ویژگی‌ها در خوشه‌های نرمال و پرت				
Is_Fraud	Amount	Time_of_Day	Transaction_Frequency	Location_Score
Normal Average (Centroid)	33.791	14.276	1.977	0.284
Fraud Average (Centroid)	437.044	3.380	8.180	0.817



استنتاج کشف دانش (Knowledge Discovery Inference):

۱. اثربخشی الگوریتم Isolation Forest: مدل قادر به شناسایی 86.0٪ از کل رفتارهای متقلبانه در فضای پرنویز گردید.

۲. اثربخشی الگوریتم LOF: روش مبتنی بر چگالی موفق به استخراج 26.0٪ از ناهنجاری‌ها شد.

تحلیل خطای نوع دوم (False Negatives): تعداد 7 تراکنش متقلبانه از سیستم Isolation Forest فرار کرده و به اشتباه برچسب «نرمال» دریافت نمودند. این رفتار حاکی از پدیده Masking در الگوریتم‌های فاصله‌محور است؛ جایی که یک تراکنش متقلبانه آنقدر هوشمندانه طراحی شده که دقیقاً در وسط تراکم تراکنش‌های سالم پنهان می‌شود!

بردار میانگین ویژگی‌ها در تراکنش‌های متقلبانه فرارکرده (False Negatives Profile)

Amount	Time_of_Day	Transaction_Frequency	Location_Score
57.368	2.857	5.429	0.759

توضیحات مثال کد پایتون

در این مثال عملی، فرآیند تحلیل داده‌های پرت به صورت کامل و در یک سناریوی واقعی تشخیص تقلب کارت اعتباری پیاده‌سازی شده است:

- ابتدا یک مجموعه داده فرضی از تراکنش‌های کارت اعتباری شبیه‌سازی شده است که شامل دو دسته داده است: تراکنش‌های عادی (نرمال) و تراکنش‌های تقلبی (پرت). تراکنش‌های تقلبی با ویژگی‌هایی مانند مبالغ بسیار بزرگتر، زمان خرید در نیمه‌شب، دسته‌های تجاری غیرمعمول و فرکانس خرید بالا شبیه‌سازی شده‌اند.
- در مرحله پیش‌پردازش داده، از StandardScaler برای استانداردسازی ویژگی‌ها استفاده شده است تا همه ویژگی‌ها در مقیاس مشابهی باشند.
- دو روش پیشرفته برای تشخیص پرت استفاده شده است:
 - Isolation Forest: یک روش مبتنی بر درخت که با ایجاد درختان تصادفی، نقاطی را که به راحتی قابل جداسازی هستند به عنوان پرت شناسایی می‌کند.
 - Local Outlier Factor (LOF): یک روش مبتنی بر چگالی که نقاط را بر اساس چگالی محلی آن‌ها نسبت به همسایگانشان به عنوان پرت شناسایی می‌کند.

- نتایج به صورت بصری نمایش داده شده‌اند:

- یک نمودار هیستوگرام توزیع مبالغ تراکنش‌ها را برای تراکنش‌های عادی و تقلبی نشان می‌دهد.
- دو نمودار پراکندگی نتایج تشخیص پرت توسط Isolation Forest و Local Outlier Factor را نشان می‌دهند که در آن نقاط قرمز (پرت) به وضوح از خوشه اصلی داده‌ها جدا شده‌اند.
- عملکرد مدل‌ها با استفاده از گزارش طبقه‌بندی ارزیابی شده است که شامل معیارهای دقت، یادآوری و F1-Score برای هر دو کلاس است.
- تحلیل عمیق‌تری از تراکنش‌های شناسایی شده به عنوان پرت انجام شده است که نشان می‌دهد چند درصد از تراکنش‌های تقلبی واقعی توسط مدل‌ها تشخیص داده شده‌اند.
- در نهایت، مقایسه میانگین ویژگی‌ها برای تراکنش‌های عادی و تقلبی نشان می‌دهد که چگونه ویژگی‌های مختلف می‌توانند به تشخیص تقلب کمک کنند.

این مثال به وضوح نشان می‌دهد که چگونه تحلیل داده‌های پرت می‌تواند در کاربردهای حیاتی مانند تشخیص تقلب، ارزش بسیار بالاتری نسبت به تحلیل داده‌های عادی داشته باشد. برخلاف رویکردهای سنتی که داده‌های پرت را به عنوان نویز حذف می‌کنند، این مثال نشان می‌دهد که چگونه این داده‌های غیرعادی می‌توانند حامل اطلاعات بسیار ارزشمندی باشند. سیستم‌های تشخیص تقلب می‌توانند به صورت خودکار هشدار ایجاد کنند و اقدامات امنیتی را فعال کنند، که این امر می‌تواند از خسارات مالی قابل توجهی جلوگیری کند.

1.4.7 Are all mining results interesting?

آیا همه نتایج داده‌کاوی جالب هستند؟💡

داده‌کاوی می‌تواند تعداد زیادی الگو، قانون و مدل تولید کند، اما همه این نتایج لزوماً ارزشمند یا کاربردی نیستند. از این رو، ارزیابی جالب بودن (Interestingness) نتایج یکی از مباحث مهم در داده‌کاوی است. یک الگو زمانی جالب تلقی می‌شود که قابل فهم، معتبر، کاربردی و نوآورانه باشد یا بتواند فرضیه‌ای را تأیید یا رد کند.

برای ارزیابی الگوها از معیارهای عینی (Objective Measures) و معیارهای ذهنی (Subjective Measures) استفاده می‌شود.

معیارهای عینی بر پایه ریاضیات و آمار استوارند. به عنوان مثال، در قوانین انجمنی، حمایت (Support) به زبان ساده می‌گوید این الگو چقدر در کل داده‌ها «تکرار» شده است. اطمینان (Confidence) نیز میزان اعتبار رابطه میان آن‌ها را نشان می‌دهد؛ یعنی اگر X رخ دهد، با چه احتمالی Y هم در کنارش رخ می‌دهد.

یک الگو ممکن است از نظر آماری قوی باشد، اما چون «بدیهی» است، ارزشی نداشته باشد. معیارهای ذهنی به دنبال الگوهایی هستند که:

غیرمنتظره: دانش جدیدی را ارائه بدهند که قبلاً وجود نداشته است.

قابل اقدام: بتوانیم بر اساس این الگوها یک تصمیم‌گیری مهم انجام بدهیم؛ مانند طراحی یک بسته تخفیفی جدید.

📦 مثال عملی: فیلتر کردن الگوهای بدیهی در آمازون (Amazon)

فرض کنید موتور داده‌کاوی آمازون بر روی میلیون‌ها فاکتور خرید اجرا شده و هزاران قانون وابستگی استخراج کرده است. تیم داده‌کاوی دو مرحله فیلتر را اعمال می‌کند:

مرحله ۱: فیلتر عینی (Statistical Filtering)

ابتدا قوانینی که حداقل حمایت ۵٪ و حداقل اطمینان ۶۰٪ ندارند حذف می‌شوند.

مرحله ۲: فیلتر ذهنی و کیفی (Business Evaluation)

از بین قوانین باقی‌مانده، مدیر بازاریابی آمازون آن‌ها را تحلیل می‌کند:

قانون اول (بدیهی و غیرجالب):

خرید کتاب‌خوان Kindle ⇒ خرید کابل شارژر USB (Confidence = 92%)

تحلیل: اگرچه اطمینان بسیار بالا دارد، اما این یک الگوی بدیهی (Trivial) است. همه می‌دانند خریدار دستگاه الکترونیکی کابل شارژ هم می‌خواهد. این قانون هیچ دانش جدیدی ایجاد نمی‌کند و حذف می‌شود.

قانون دوم (غیرمنتظره و قابل اقدام - جالب):

خرید هدست گیمینگ ⇒ خرید قطره چشم ضدخستگی (Confidence = 68%)

تحلیل: این الگو اصلاً بدیهی نیست! داده‌کاوی نشان داده گیمرهایی که ساعات طولانی بازی می‌کنند دچار خشکی چشم می‌شوند. این قانون هم غیرمنتظره (Novel) است و هم قابل اقدام (Actionable)؛ آمازون می‌تواند این قطره را به عنوان پیشنهاد خریدی کنار هدست‌های گیمینگ نمایش دهد (Cross-selling).

```
In [13]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
from collections import Counter
from itertools import combinations
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135
np.random.seed(42)
```

```
In [14]: display_academic_text("ارزیابی الگوها: فیلترینگ معیارهای عینی و ذهنی در سیستم‌های پیشنهاددهنده", "main_title")
display_academic_text("Subj عبور کرده و سپس با استفاده از معیارهای ذهنی (Confidence) و اطمینان (Support) در این سناریو، الگوهای استخراج‌شده از فاکتورهای خرید ابتدا از فیلتر معیارهای عینی مانند پشتیبانی", "sub_title")

# =====
# STEP 1: SIMULATE SHOPPING BASKETS
# =====
display_academic_text("مرحله نخست: تولید داده‌های خام و کدگذاری تراکنش‌ها", "sub_title")

products = ['Kindle', 'USB_Charging_Cable', 'Gaming_Headset', 'Eye_Drops', 'Gaming_Mouse', 'Laptop', 'Laptop_Bag', 'Coffee_Maker', 'Coffee_Filters']
n_transactions = 5000
transactions = []

for _ in range(n_transactions):
    basket = set()
    if np.random.rand() < 0.18:
        basket.add('Kindle')
        if np.random.rand() < 0.92: basket.add('USB_Charging_Cable')

    if np.random.rand() < 0.20:
        basket.add('Gaming_Headset')
        # The hidden actionable pattern: Gamers need eye drops!
        if np.random.rand() < 0.68: basket.add('Eye_Drops')

    if np.random.rand() < 0.25:
        basket.add('Laptop')
        if np.random.rand() < 0.55: basket.add('Laptop_Bag')
    if np.random.rand() < 0.20:
        basket.add('Gaming_Mouse')
        if np.random.rand() < 0.45: basket.add('Gaming_Headset')
    if np.random.rand() < 0.15:
        basket.add('Coffee_Maker')
        if np.random.rand() < 0.70: basket.add('Coffee_Filters')

    if basket:
        transactions.append(list(basket))
```

```
sample_trans_df = pd.DataFrame({'Simulated Raw Basket': [str(t) for t in transactions[:5]]})
display_side_by_side_datasets([sample_trans_df, [f"تعداد کل تراکنش‌ها (نمونه فاکتورهای خرید خام": len(transactions))]])

encoder = TransactionEncoder()
encoded_array = encoder.fit(transactions).transform(transactions)
basket_df = pd.DataFrame(encoded_array, columns=encoder.columns_)
display_side_by_side_datasets([basket_df.head(), [f"داده‌های رمزگذاری‌شده جهت ورود به الگوریتم استخراج الگو"]])

# =====
# STEP 2: EXTRACT FREQUENT ITEMSETS
# =====
display_academic_text("مرحله دوم: استخراج مجموعه‌های پرتکرار", "sub_title")
frequent_itemsets = apriori(basket_df, min_support=0.05, use_colnames=True)

freq_disp = frequent_itemsets.sort_values(by='support', ascending=False).copy()
freq_disp['itemsets'] = freq_disp['itemsets'].apply(lambda x: '{' + ', '.join(list(x)) + '}')
display_side_by_side_datasets([freq_disp.head(8), [f"مجموعه‌های پرتکرار استخراج‌شده (فیلتر آماری: حداقل حمایت معادل ۵ درصد)"]])

# =====
# STEP 3: EXTRACT RULES (OBJECTIVE FILTERING)
# =====
display_academic_text("مرحله سوم: فیلترینگ عینی شاخص‌های آماری", "sub_title")
rules_all = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.60)
total_rules_after_statistical_filter = len(rules_all)

# Filter for 1-to-1 rules for simplicity in visualization
rules = rules_all[
    (rules_all['antecedents'].apply(lambda x: len(x) == 1)) &
    (rules_all['consequents'].apply(lambda x: len(x) == 1))
].copy()

rules['Antecedent'] = rules['antecedents'].apply(lambda x: list(x)[0])
rules['Consequent'] = rules['consequents'].apply(lambda x: list(x)[0])

statistical_rules = rules[['Antecedent', 'Consequent', 'support', 'confidence', 'lift']].sort_values(by='confidence', ascending=False).reset_index(drop=True)
display_side_by_side_datasets([statistical_rules.head(8), [f"قواعد باقی‌مانده پس از اعمال فیلتر عینی (حداقل اطمینان معادل ۶۰ درصد)"]])

# =====
# STEP 4: SUBJECTIVE FILTERING
# =====
display_academic_text("مرحله چهارم: فیلتر ذهنی و ارزیابی کیفی حوزه کسب‌وکار", "sub_title")

# Domain Knowledge Injection
trivial_rules = {('Kindle', 'USB_Charging_Cable'), ('USB_Charging_Cable', 'Kindle')}
interesting_rules = {('Gaming_Headset', 'Eye_Drops'), ('Eye_Drops', 'Gaming_Headset')}

def evaluate_rule(row):
    pair = (row['Antecedent'], row['Consequent'])
    if pair in trivial_rules:
        return 'Trivial Pattern (حذف بديهی و نیازمند حذف)'
    elif pair in interesting_rules:
        return 'Actionable Pattern (غیرمنتظره و کاربردی)'
    else:
        return 'Pending Review (نیازمند بررسی متخصص)'

statistical_rules['BusinessEvaluation'] = statistical_rules.apply(evaluate_rule, axis=1)
display_side_by_side_datasets(
    [statistical_rules[['Antecedent', 'Consequent', 'confidence', 'BusinessEvaluation']].head(8),
    [f"نتیجه ارزیابی ذهنی قواعد توسط تیم متخصصان تحلیل بازار"]
])

# =====
# STEP 5: KNOWLEDGE REPRESENTATION (ACTIONABLE INSIGHTS)
# =====
removed_rules = statistical_rules[statistical_rules['BusinessEvaluation'] == 'Trivial Pattern (حذف بديهی و نیازمند حذف)]
final_rules = statistical_rules[statistical_rules['BusinessEvaluation'] == 'Actionable Pattern (غیرمنتظره و کاربردی)]

warn_text = "<b>گزارش حذف الگوهای بديهی</b><br><br>"
for _, row in removed_rules.iterrows():
    warn_text += f"<b>الگوی استخراج‌شده: {row['Antecedent']} → {row['Consequent']}</b> با اطمینان {row['confidence']*100:.0f} درصد<br>"
warn_text += "<br><b>تحلیل سیستمی</b>: این قواعد دانش جدیدی تولید نمی‌کنند و از پایگاه دانش سیستم فیلتر می‌گردند<br>"
display_academic_text(warn_text, "insight") # Using insight style but the text makes it a warning conceptually

insight_text = "<b>گزارش کشف فرصت‌های استراتژیک (Actionable Insights)</b><br><br>"
for _, row in final_rules.iterrows():
    insight_text += f"<b>الگوی استخراج‌شده: {row['Antecedent']} → {row['Consequent']}</b> با اطمینان {row['confidence']*100:.0f} درصد و شاخص لیفت {row['lift']:.2f}<br>"
insight_text += "<br><b>تحلیل استراتژیک</b>: سیستم می‌تواند اقلام دارویی مرتبط را در قالب استراتژی فروش مکمل<br>"
display_academic_text(insight_text, "insight")

# =====
# STEP 6: ADVANCED VISUALIZATIONS (ACADEMIC BLUE)
# =====
display_academic_text("مرحله پنجم: مصورسازی تحلیلی شاخص‌های ارزیابی الگو", "sub_title")

# -----
# CHART 1: Support vs Confidence Density
# -----
plt.figure(figsize=(14, 8), dpi=135)

scatter = plt.scatter(
    statistical_rules['support'], statistical_rules['confidence'],
    c=statistical_rules['lift'], s=statistical_rules['lift'] * 350,
    cmap='Blues', edgecolor='#2c3e50', alpha=0.9, linewidths=1.2, zorder=5
)

plt.axhline(0.60, color='#e74c3c', linestyle='--', linewidth=2.0, label='Minimum Confidence Threshold (60%)', zorder=1)
plt.axvline(0.05, color='#e74c3c', linestyle=':', linewidth=2.0, label='Minimum Support Threshold (5%)', zorder=1)

y_shifts = [50, -50, 95, -95]
annot_idx = 0

for _, row in statistical_rules.iterrows():
```

```
pair = (row[Antecedent], row[Consequent])
if pair in trivial_rules or pair in interesting_rules:
    ant_text = row[Antecedent].replace('_', ' ')
    con_text = row[Consequent].replace('_', ' ')
    label_text = f"{ant_text}\n→\n{con_text}"

y_s = y_shifts[annot_idx % len(y_shifts)]
va_align = 'bottom' if y_s > 0 else 'top'

plt.annotate(
    label_text,
    (row[support], row[confidence]),
    textcoords="offset points",
    xytext=(0, y_s),
    ha='center',
    va=va_align,
    fontsize=10, fontweight='bold', color="#2c3e50", multialignment='center',
    arrowprops=dict(arrowstyle="-|>", color="#7f8c8d", lw=1.5, alpha=0.8, shrinkA=3, shrinkB=5),
    bbox=dict(boxstyle="round,pad=0.5", fc="ffffff", ec="#bdc3c7", alpha=0.95),
    zorder=10
)
annot_idx += 1

plt.xlim(statistical_rules[support].min() - 0.02, statistical_rules[support].max() + 0.06)
plt.ylim(statistical_rules[confidence].min() - 0.15, statistical_rules[confidence].max() + 0.15)

cbar = plt.colorbar(scatter, label='Lift Measure Magnitude')
plt.xlabel('Support Measure (Frequency in Dataset)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
plt.ylabel('Confidence Measure (Rule Reliability)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
plt.title('Figure 1: Objective Measures - Support vs Confidence Density', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)
plt.legend(loc='upper right', frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7')
plt.grid(True, linestyle='--', alpha=0.5, zorder=0)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# -----
# CHARTS 2 & 3: Comparison of Lift and Confidence Side-by-Side
# -----

fig, axes = plt.subplots(1, 2, figsize=(18, 6), dpi=135)

comparison_rules = statistical_rules[statistical_rules[BusinessEvaluation] != 'Pending Review (نیازمند بررسی متخصص)'].copy()
comparison_rules[RuleLabel] = comparison_rules[Antecedent] + '\n→ ' + comparison_rules[Consequent]

# Academic Palette for Actionable vs Trivial
colors_eval = comparison_rules[BusinessEvaluation].map({
    'Trivial Pattern (بدیهی و نیازمند حذف)': '#7f8c8d', # Gray
    'Actionable Pattern (غیرمنتظره و کاربردی)': '#1abc9c' # Turquoise
})

# Subplot A: Confidence
bars_conf = axes[0].bar(comparison_rules[RuleLabel], comparison_rules[confidence], color=colors_eval, edgecolor='#2c3e50', width=0.45, lw=1.2)
for bar in bars_conf:
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
        f'{bar.get_height()*100:.0f}%', ha='center', fontweight='bold', fontsize=12, color="#2c3e50")

axes[0].set_ylabel('Evaluated Confidence Measure', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_title('Figure 2: Rule Reliability (Confidence)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[0].tick_params(axis='x', rotation=15, labelsize=10)
axes[0].grid(axis='y', linestyle='--', alpha=0.5)

# Subplot B: Lift
lift_data = comparison_rules.sort_values('lift')
colors_lift = lift_data[BusinessEvaluation].map({
    'Trivial Pattern (بدیهی و نیازمند حذف)': '#7f8c8d',
    'Actionable Pattern (غیرمنتظره و کاربردی)': '#1abc9c'
})

bars_lift = axes[1].barh(lift_data[RuleLabel], lift_data[lift], color=colors_lift, edgecolor='#2c3e50', height=0.5, lw=1.2)
axes[1].axvline(1, color='#e74c3c', linestyle='--', linewidth=2.5, label='Lift Baseline = 1 (Independent)')

for bar in bars_lift:
    axes[1].text(bar.get_width() + 0.05, bar.get_y() + bar.get_height()/2,
        f'{bar.get_width():.2f}', va='center', fontweight='bold', fontsize=12, color="#2c3e50")

axes[1].set_xlabel('Calculated Lift Measure', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_title('Figure 3: Rule Surprise Factor (Lift)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[1].legend(loc='lower right', frameon=True)
axes[1].grid(axis='x', linestyle='--', alpha=0.5)

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# -----
# CHARTS 4 & 5: Search Space Pruning Funnel & Network Topology
# -----

fig, axes = plt.subplots(1, 2, figsize=(19, 7), dpi=135)

# Subplot A: Funnel Chart
after_statistical_count = len(statistical_rules)
after_business_count = len(final_rules)

funnel_stages = [
    'Total Extracted Rules\n(Pre-Filtering)',
    'Post Objective Filtering\n(Support/Conf Applied)',
    'Post Subjective Filtering\n(Trivial Patterns Removed)'
]

funnel_values = [total_rules_after_statistical_filter, after_statistical_count, after_business_count]
```

```
bars_funnel = axes[0].bar(funnel_stages, funnel_values, color=[#bdc3c7', '#2980b9', '#1abc9c'], edgecolor='#2c3e50', width=0.55, lw=1.5)
```

```
for bar, value in zip(bars_funnel, funnel_values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
                 str(value), ha='center', fontweight='bold', fontsize=14, color="#2c3e50")
```

```
axes[0].set_ylabel('Volume of Retained Rules', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_title('Figure 4: Algorithmic Search Space Pruning Pipeline', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
```

```
# Subplot B: Network Topology
G = nx.DiGraph()
for _, row in statistical_rules.iterrows():
    G.add_edge(row['Antecedent'], row['Consequent'], weight=row['confidence'], evaluation=row['BusinessEvaluation'])
```

```
# Coloring edges based on actionability (Academic Palette)
edge_colors = []
for u, v, data in G.edges(data=True):
    if data['evaluation'] == 'Trivial Pattern (بدیهی و نیازمند حذف)':
        edge_colors.append('#bdc3c7') # Gray
    elif data['evaluation'] == 'Actionable Pattern (غیرمنتظره و کاربردی)':
        edge_colors.append('#1abc9c') # Turquoise
    else:
        edge_colors.append('#2980b9') # Blue
```

```
edge_widths = [G[u][v]['weight'] * 4 for u, v in G.edges()]
pos = nx.spring_layout(G, seed=42, k=1.8)
```

```
formatted_node_labels = {node: node.replace('_', '\n') for node in G.nodes()}
node_size_val = 3500
```

```
nx.draw_networkx_edges(
    G, pos, ax=axes[1], edge_color=edge_colors, width=edge_widths,
    arrowstyle='->', arrowsize=15, connectionstyle='arc3,rad=0.2'
)
```

```
nx.draw_networkx_nodes(
    G, pos, ax=axes[1], node_color='#ecf0f1', node_size=node_size_val,
    edgecolors='#2c3e50', linewidths=1.5
)
```

```
nx.draw_networkx_labels(
    G, pos, ax=axes[1], labels=formatted_node_labels,
    font_size=9, font_weight='bold', font_color='#2c3e50'
)
```

```
# Only show edge labels for actionable rules to avoid clutter
edge_labels = {(u, v): f"{d['weight']:.2f}" for u, v, d in G.edges(data=True) if d['evaluation'] == 'Actionable Pattern (غیرمنتظره و کاربردی)'}
nx.draw_networkx_edge_labels(
    G, pos, ax=axes[1], edge_labels=edge_labels, font_size=10, font_weight='bold', font_color='#1abc9c',
    label_pos=0.3,
    bbox=dict(facecolor='ffffff', edgecolor='none', alpha=0.9, boxstyle='round,pad=0.2')
)
```

```
axes[1].set_title('Figure 5: Product Association Network Topology\n(Turquoise: Actionable, Gray: Trivial, Blue: Pending Review)', fontdict={'fontsize': 14, 'fontweight': 'bold'})
axes[1].axis('off')
```

```
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
```

```
plt.tight_layout()
plt.show()
```

ارزیابی الگوها: فیلترینگ معیارهای عینی و ذهنی در سیستم‌های پیشنهاددهنده

در این سناریو، الگوهای استخراج‌شده از فاکتورهای خرید ابتدا از فیلتر معیارهای عینی مانند پشتیبانی (Support) و اطمینان (Confidence) عبور کرده و سپس با استفاده از معیارهای ذهنی (Subjective) ارزیابی می‌شوند تا الگوهای بدیهی حذف و الگوهای قابل اقدام (Actionable) استخراج گردند.

مرحله نخست: تولید داده‌های خام و کدگذاری تراکنش‌ها

نمونه فاکتورهای خرید خام (تعداد کل تراکنش‌ها: 3365)
Simulated Raw Basket
['USB_Charging_Cable', 'Coffee_Maker', 'Kindle']
['Laptop_Bag', 'Laptop']
['Gaming_Headset', 'Gaming_Mouse']
['Coffee_Maker', 'Laptop_Bag', 'Coffee_Filters', 'Laptop']
['USB_Charging_Cable', 'Kindle']

داده‌های رمزگذاری‌شده جهت ورود به الگوریتم استخراج الگو								
Coffee_Filters	Coffee_Maker	Eye_Drops	Gaming_Headset	Gaming_Mouse	Kindle	Laptop	Laptop_Bag	USB_Charging_Cable
False	True	False	False	False	True	False	False	True
False	False	False	False	False	False	True	True	False
False	False	False	True	True	False	False	False	False
True	True	False	False	False	False	True	True	False
False	False	False	False	False	True	False	False	True

مرحله دوم: استخراج مجموعه‌های پرتکرار

مجموعه‌های پرتکرار استخراج‌شده (فیلتر آماری: حداقل حمایت معادل ۵ درصد)	
support	itemsets
0.401	{Gaming_Headset}
0.371	{Laptop}
0.291	{Gaming_Mouse}
0.273	{Kindle}
0.245	{USB_Charging_Cable}
0.245	{USB_Charging_Cable, Kindle}
0.227	{Coffee_Maker}
0.211	{Laptop_Bag}

مرحله سوم: فیلترینگ عینی شاخص‌های آماری

قواعد باقی‌مانده پس از اعمال فیلتر عینی (حداقل اطمینان معادل ۶۰ درصد)				
Antecedent	Consequent	support	confidence	lift
Coffee_Filters	Coffee_Maker	0.165	1.000	4.399
Eye_Drops	Gaming_Headset	0.201	1.000	2.496
Laptop_Bag	Laptop	0.211	1.000	2.694
USB_Charging_Cable	Kindle	0.245	1.000	3.670
Kindle	USB_Charging_Cable	0.245	0.899	3.670
Coffee_Maker	Coffee_Filters	0.165	0.724	4.399

مرحله چهارم: فیلتر ذهنی و ارزیابی کیفی حوزه کسب‌وکار

نتیجه ارزیابی ذهنی قواعد توسط تیم متخصصان تحلیل بازار			
Antecedent	Consequent	confidence	BusinessEvaluation
Coffee_Filters	Coffee_Maker	1.000	Pending Review (نیازمند بررسی متخصص)
Eye_Drops	Gaming_Headset	1.000	Actionable Pattern (غیرمنتظره و کاربردی)
Laptop_Bag	Laptop	1.000	Pending Review (نیازمند بررسی متخصص)
USB_Charging_Cable	Kindle	1.000	Trivial Pattern (بدیهی و نیازمند حذف)
Kindle	USB_Charging_Cable	0.899	Trivial Pattern (بدیهی و نیازمند حذف)
Coffee_Maker	Coffee_Filters	0.724	Pending Review (نیازمند بررسی متخصص)

گزارش حذف الگوهای بدیهی (Trivial Patterns):

الگوی استخراج‌شده: [USB_Charging_Cable → Kindle] با اطمینان 100 درصد

الگوی استخراج‌شده: [Kindle → USB_Charging_Cable] با اطمینان 90 درصد

تحلیل سیستمی: این الگوها علیرغم دارا بودن اطمینان آماری بالا، از منظر منطق تجاری کاملاً بدیهی محسوب می‌شوند. به عنوان نمونه، خریدار دستگاه الکترونیکی ذاتاً به کابل شارژ نیاز دارد. این قواعد دانش جدیدی تولید نمی‌کنند و از پایگاه دانش سیستم فیلتر می‌گردند.

گزارش کشف فرصت‌های استراتژیک (Actionable Insights):

الگوی استخراج‌شده: [Eye_Drops → Gaming_Headset] با اطمینان 100 درصد و شاخص لیفت 2.50

تحلیل استراتژیک: کشف این وابستگی پنهان که کاربران تجهیزات گیمینگ دچار عوارض خشکی چشم می‌گردند، یک دانش غیرمنتظره و کاملاً کاربردی است. سیستم می‌تواند اقلام دارویی مرتبط را در قالب استراتژی فروش مکمل (Cross-Selling) در صفحات تجهیزات الکترونیکی پیشنهاد دهد.

مرحله پنجم: مصورسازی تحلیلی شاخص‌های ارزیابی الگو

Figure 1: Objective Measures - Support vs Confidence Density

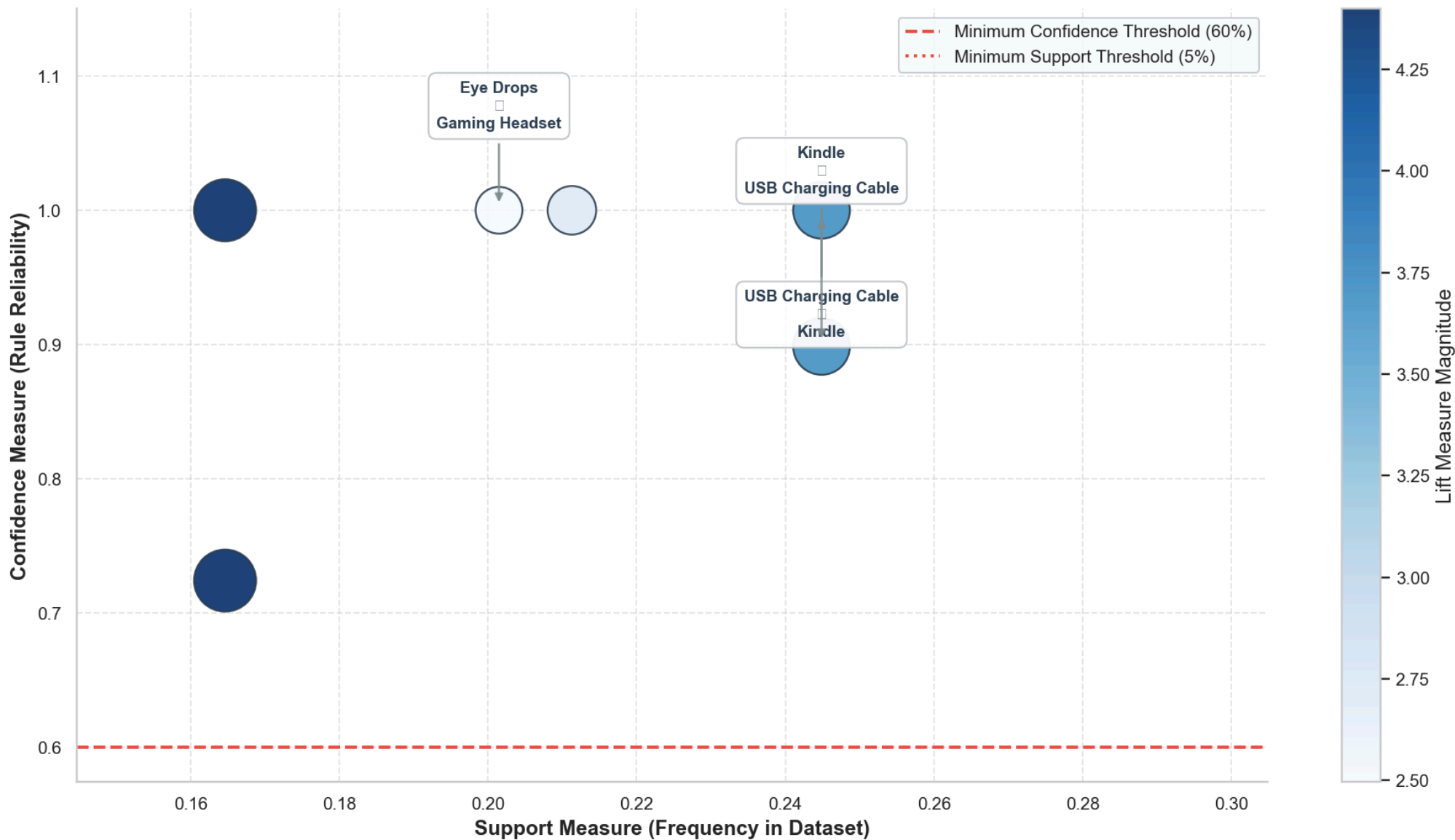


Figure 2: Rule Reliability (Confidence)

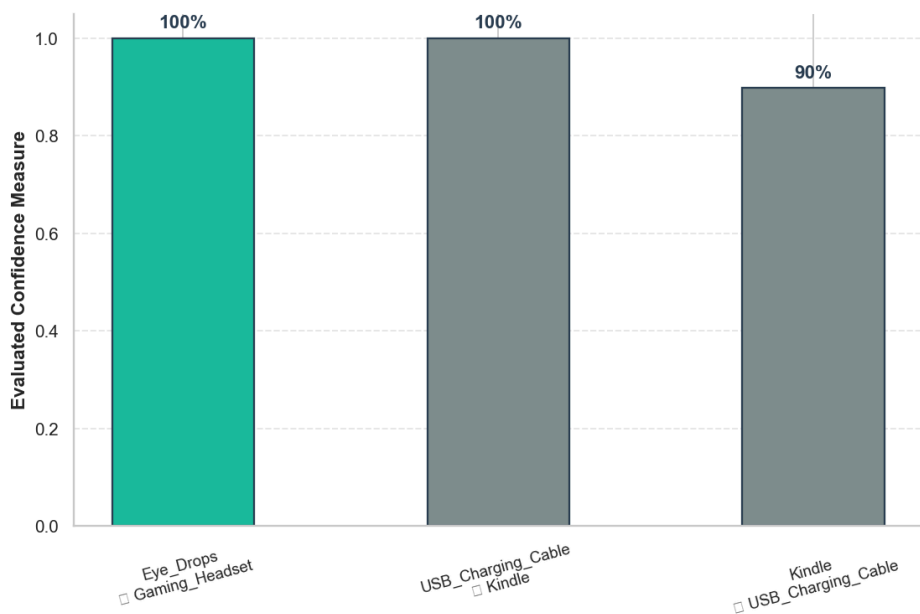


Figure 3: Rule Surprise Factor (Lift)

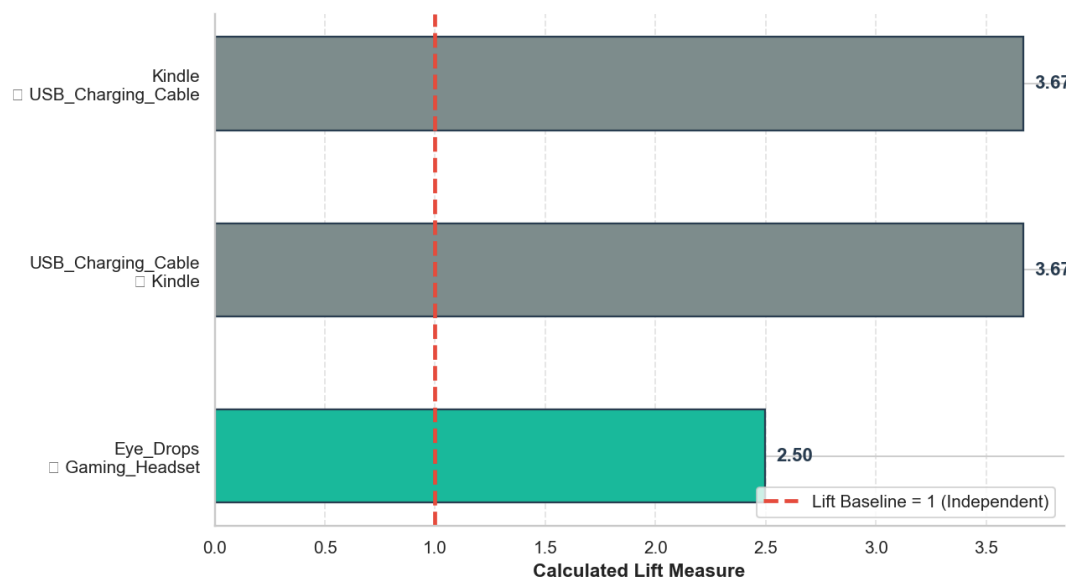


Figure 4: Algorithmic Search Space Pruning Pipeline

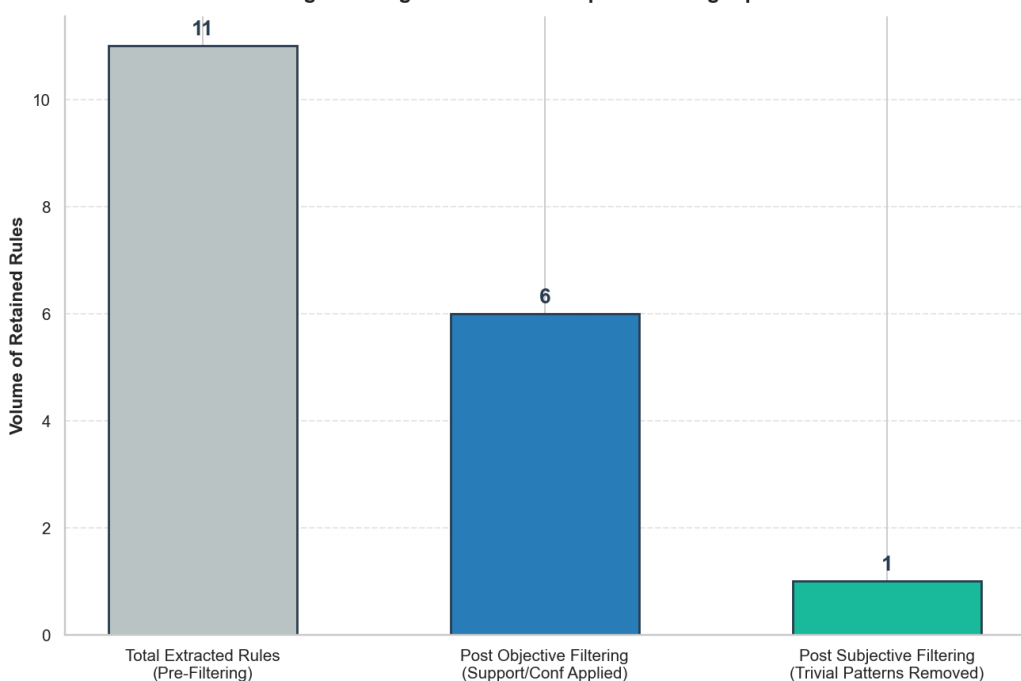
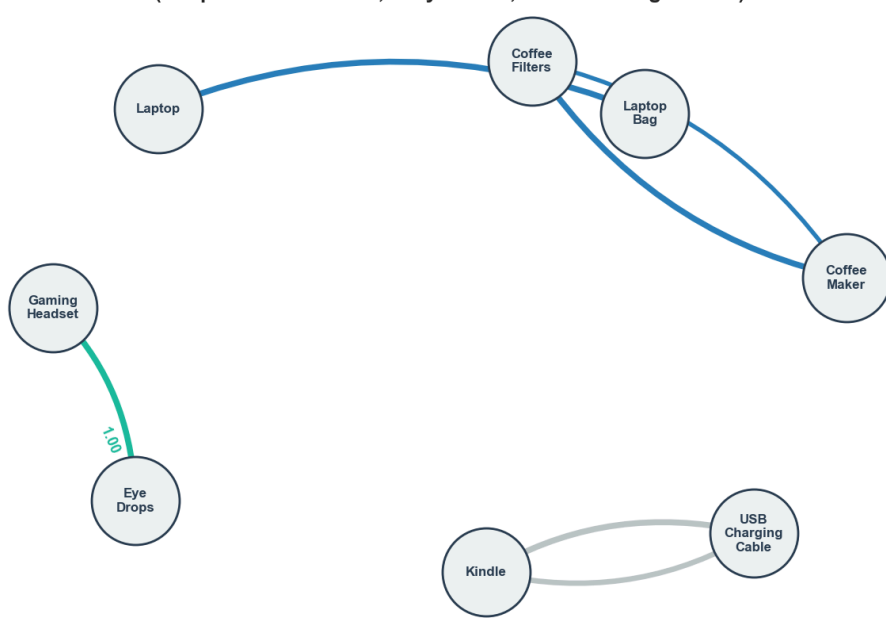


Figure 5: Product Association Network Topology (Turquoise: Actionable, Gray: Trivial, Blue: Pending Review)



توضیحات مثال کد پایتون

در این مثال:

- ابتدا با استفاده از کتابخانه‌های **pandas** و **numpy**، هزاران فاکتور خرید فرضی شبیه‌سازی شده است که در دل خود دو نوع رابطه پنهان دارند: یک رابطه **بدیهی** (خرید Kindle و کابل شارژر) و یک رابطه **غیرمنتظره** (خرید هدست گیمینگ و قطره چشم ضدخستگی).

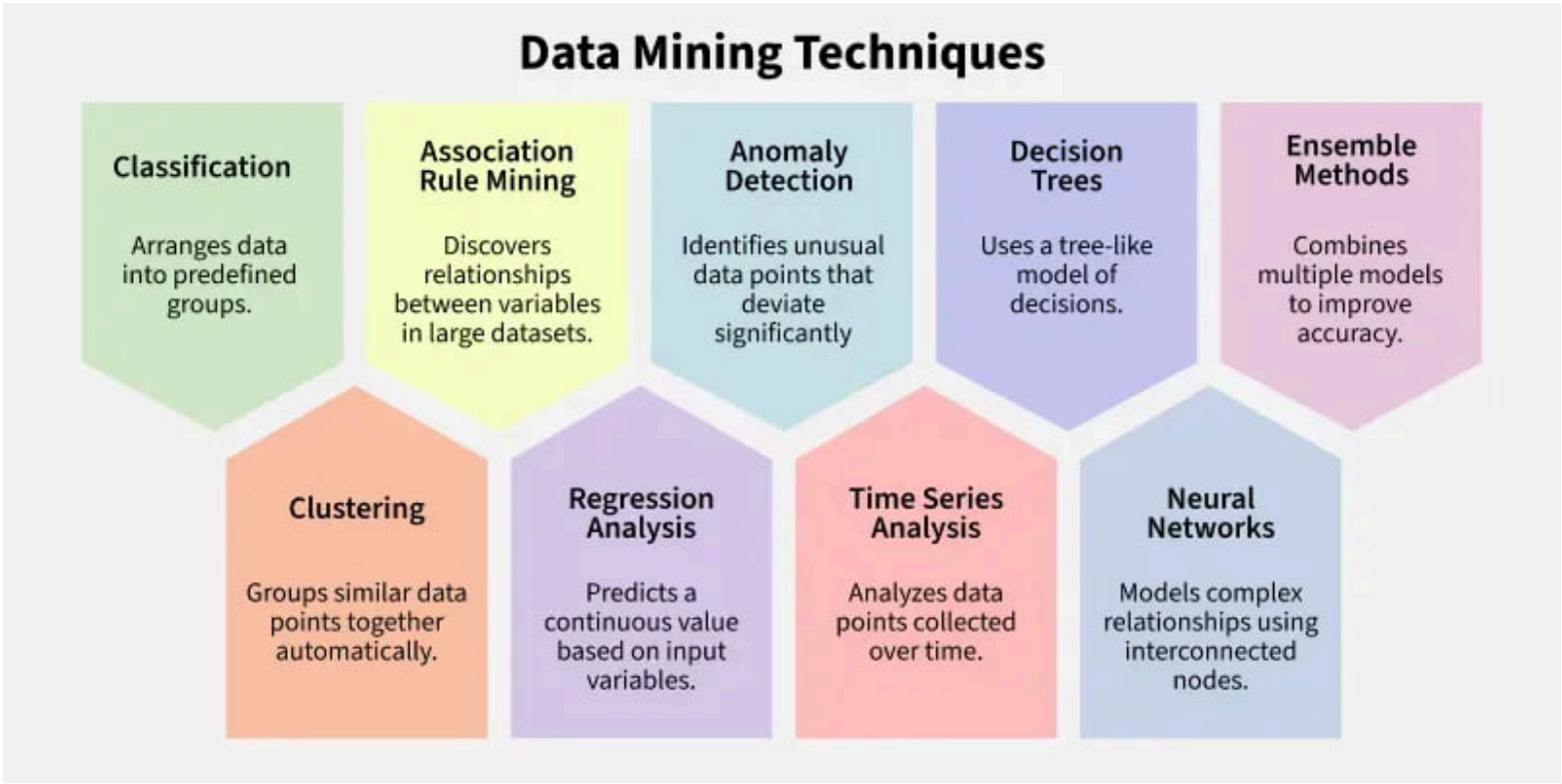
- با استفاده از کتابخانه استاندارد **mlxtend** و الگوریتم واقعی **Apriori**، مجموعه‌های پرتکرار کالاها استخراج شده و از دل آن‌ها، قوانین وابستگی (Association Rules) به همراه سه معیار کلیدی **Support**، **Confidence** و **Lift** محاسبه شده‌اند.
- در **مرحله فیلتر عینی (آماري)**، قوانینی که حداقل ۵٪ حمایت و ۶۰٪ اطمینان را ندارند حذف می‌شوند تا فقط الگوهای قابل اتکا باقی بمانند.
- در **مرحله فیلتر ذهنی (کیفی)**، با تعریف یک تابع ارزیابی کسب‌وکاری، قانون Kindle و کابل شارژر به‌عنوان الگویی **بدیهی (Trivial)** شناسایی و حذف می‌شود، در حالی‌که قانون هدست گیمینگ و قطره چشم به‌عنوان الگویی **غیرمنتظره و قابل اقدام (Novel & Actionable)** برای استفاده در سیستم پیشنهاددهنده حفظ می‌شود.
- در بخش **ویژوالایز کردن نتایج**، پنج نمودار مکمل ترسیم شده‌اند:

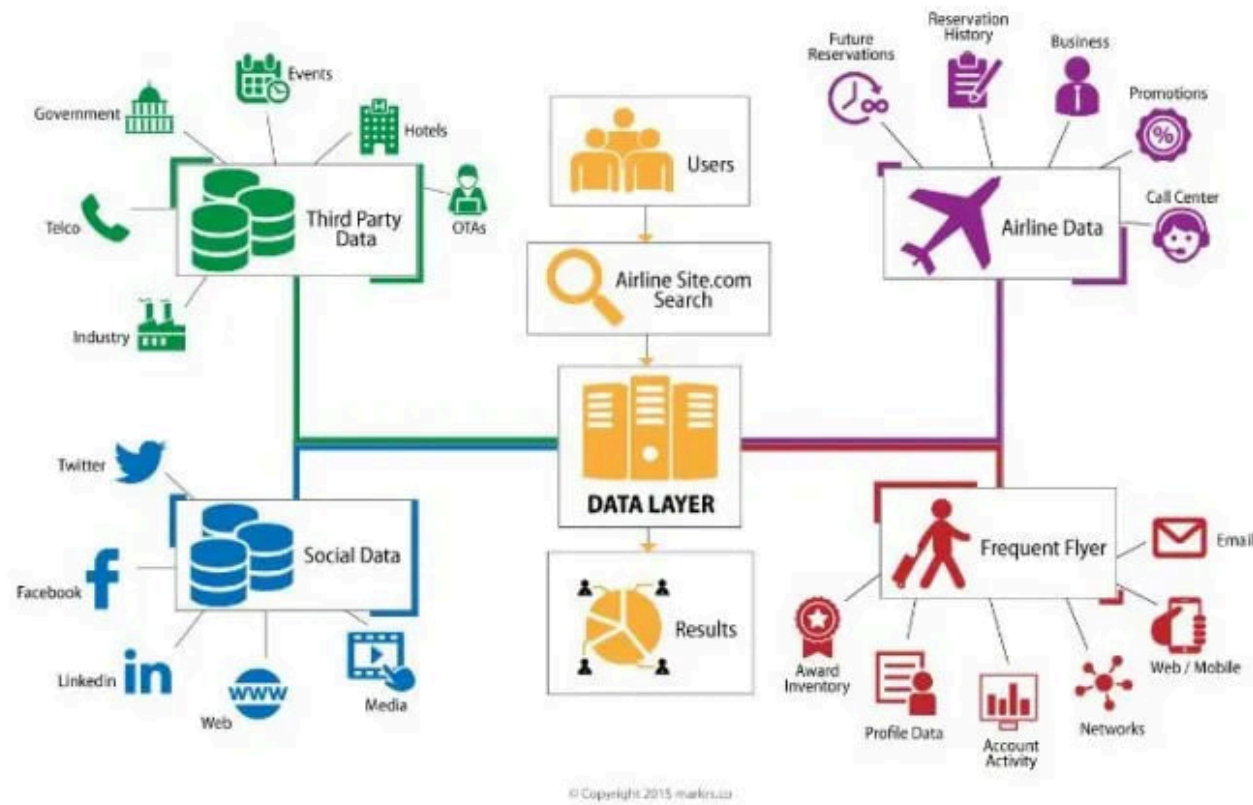
- **نمودار پراکندگی Support-Confidence**: موقعیت تمام قوانین را نسبت به خطوط آستانه فیلتر آماری نشان می‌دهد، درحالی‌که رنگ و اندازه نقاط بر اساس مقدار Lift تنظیم شده است.
- **نمودار مقایسه ستونی**: تفاوت مستقیم بین قانون بدیهی (خاکستری) و قانون جالب (سبز) را در سه معیار اصلی به‌صورت کنار هم نمایش می‌دهد.
- **نمودار قیف فیلترینگ (Funnel)**: کاهش تدریجی تعداد قوانین را از مرحله اولیه تا فیلتر آماری و سپس فیلتر کیفی به تصویر می‌کشد.
- **گراف شبکه‌ای محصولات**: با استفاده از کتابخانه **networkx**، رابطه بین کالاها را به‌صورت یک شبکه جهت‌دار ترسیم می‌کند که ضخامت هر یال متناسب با میزان Confidence آن قانون است.
- **نمودار مقایسه Lift**: نشان می‌دهد چرا معیار Lift اهمیت بالایی دارد؛ قانون بدیهی با وجود Confidence بالا، Lift نزدیک به ۱ (رابطه ضعیف) دارد، اما قانون جالب Lift به‌مراتب بالاتری (رابطه واقعی و معنادار) نشان می‌دهد.

این مثال به‌خوبی نشان می‌دهد که فرایند واقعی داده‌کاوی در شرکت‌هایی مانند آمازون، صرفاً به اجرای یک الگوریتم آماری محدود نمی‌شود؛ بلکه ترکیبی از **فیلتر عینی (آماري)** و **فیلتر ذهنی (تخصص و قضاوت انسانی)** است که در نهایت، از میان هزاران قانون احتمالی، تنها دانشی **غیرمنتظره، معنادار و قابل تبدیل به ارزش تجاری** (مانند فروش مکمل یا Cross-selling) استخراج و به کار گرفته می‌شود.



Section 1.5 (Data mining: confluence of multiple disciplines)





1.5.1 Statistics and data mining

داده‌کاوی؛ تلاقی چندین رشته علمی

داده‌کاوی یک حوزهٔ میان‌رشته‌ای است که با هدف استخراج الگوها و دانش از داده‌های حجیم، از مفاهیم و روش‌های علوم مختلفی مانند یادگیری ماشین، آمار، تشخیص الگو، پردازش زبان طبیعی، پایگاه‌داده، مصورسازی داده، تعامل انسان و رایانه، الگوریتم‌ها و محاسبات با کارایی بالا بهره می‌گیرد. این ماهیت میان‌رشته‌ای، نقش مهمی در توسعهٔ روش‌های داده‌کاوی و گسترش کاربردهای آن در مسائل واقعی داشته است.

نکته‌ی کلیدی: داده‌کاوی علاوه بر بهره‌گیری از دستاوردهای این علوم، با ارائه‌ی روش‌ها و الگوریتم‌های جدید برای تحلیل داده‌های حجیم، به پیشرفت آن‌ها نیز کمک کرده است. از این‌رو، می‌توان داده‌کاوی را حوزه‌ای پویا و تعاملی دانست که از هم‌افزایی چندین رشته علمی شکل گرفته و به توسعه‌ی آن‌ها نیز یاری رسانده است.

مثال عملی: ترکیب آمار و داده‌کاوی برای پیش‌بینی فروش فصلی

در این مثال، یک شرکت خرده‌فروشی می‌خواهد فروش محصولات خود را در فصل آینده پیش‌بینی کند. ابتدا از روش‌های آمار توصیفی برای خلاصه‌سازی داده‌های فروش گذشته استفاده می‌شود: میانگین، انحراف معیار و توزیع فروش در طول زمان محاسبه می‌شوند. سپس، یک مدل آماری مانند میانگین متحرک یا هموارسازی نمایی برای شناسایی روند و فصلیت داده‌ها به کار گرفته می‌شود.

در مرحله بعد، از یک مدل داده‌کاوی (مانند درخت تصمیم یا رگرسیون) برای پیش‌بینی فروش با استفاده از ویژگی‌های ترکیبی (مانند قیمت، تبلیغات، فصل و تعطیلات) استفاده می‌شود. این مدل بر روی داده‌های پاک‌سازی‌شده و با استفاده از مدل‌های آماری برای مدیریت مقادیر گم‌شده و نویز، آموزش دیده است.

در نهایت، نتایج مدل با استفاده از آزمون فرض آماری (مانند آزمون t یا p -value) ارزیابی می‌شوند تا مشخص شود آیا بهبود در دقت پیش‌بینی به‌طور معناداری بیشتر از یک مدل تصادفی است یا خیر. این ترکیب هوشمند از آمار و داده‌کاوی به شرکت کمک می‌کند تا پیش‌بینی‌هایی قابل اعتماد و قابل تفسیر ارائه دهد که برای برنامه‌ریزی انبار و استراتژی قیمت‌گذاری حیاتی است.

```
In [15]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
```

```
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

np.random.seed(42)
```

In [16]: `display_academic_text("📊", "main_title")`
`display_academic_text("مدل یادگیری ماشین <b>آمار توصیفی</b> داده‌کاوی و علم آمار دو حوزهٔ مکمل می‌باشند. در این سناریو، یک شرکت خرده‌فروشی جهت پیش‌بینی فروش فصلی، ابتدا از")`

```
# =====
# STEP 1: SIMULATE RETAIL SALES DATABASE
# =====

n_months = 36
months = pd.date_range('2020-01-01', periods=n_months, freq='ME')

# مدل‌سازی روند، فصلیت و نویز
trend = np.linspace(100, 180, n_months)
seasonality = 30 * np.sin(2 * np.pi * np.arange(n_months) / 12)
noise = np.random.normal(0, 10, n_months)
base_sales = trend + seasonality + noise

# مدل‌سازی متغیرهای برون‌زا (Exogenous Variables)
price_effect = -0.5 * np.random.uniform(0, 20, n_months)
advertising_effect = 0.8 * np.random.uniform(0, 30, n_months)
holiday_effect = np.zeros(n_months)
holiday_effect[11::12] = 25
holiday_effect[6] = 20
holiday_effect[3] = 15

final_sales = base_sales + price_effect + advertising_effect + holiday_effect

df = pd.DataFrame({
    'Month': months.strftime('%Y-%m'),
    'Sales_Volume': final_sales,
    'Price_Index': np.random.uniform(80, 120, n_months),
    'Advertising_Budget': np.random.uniform(50, 150, n_months),
    'Is_Holiday_Season': (months.month == 12).astype(int),
    'Temperature': 15 + 10 * np.sin(2 * np.pi * np.arange(n_months) / 12 + np.pi/2)
})

display_side_by_side_datasets([df.head(6)], ["فضای ویژگی خام: داده‌های شبیه‌سازی‌شده فروش خرده‌فروشی (۳۶ ماه متوالی)"])

# =====
# STEP 2: DESCRIPTIVE STATISTICS
# =====

display_academic_text("آمار توصیفی و خلاصه‌سازی داده‌ها", "sub_title")
display_academic_text("آمار توصیفی، پارامترهای پایه‌ای توزیع داده‌ها نظیر تمایل مرکزی (میانگین)، پراکندگی (انحراف معیار) و همبستگی خطی متغیرها را شفاف می‌سازد")

desc_stats = pd.DataFrame({
    'Statistical Metric': ['Mean Sales Volume', 'Standard Deviation (Volatility)', 'Minimum Sales Record', 'Maximum Sales Record', 'Pearson Correlation (Sales vs. Ad Budget)',
    'Computed Value': [df['Sales_Volume'].mean(), df['Sales_Volume'].std(), df['Sales_Volume'].min(), df['Sales_Volume'].max(), df['Sales_Volume'].corr(df['Advertising_Budget'])
})

display_side_by_side_datasets([desc_stats], ["(Target Variable Analysis) جدول آمار توصیفی متغیر هدف"])

# -----
# COMBINED CHARTS: Distribution & Correlation
# -----

fig, axes = plt.subplots(1, 2, figsize=(19, 6), dpi=135)

# Chart 1: Sales Distribution
axes[0].hist(df['Sales_Volume'], bins=12, alpha=0.9, color='#2c3e50', edgecolor='white', linewidth=1.5)
axes[0].set_xlabel('Sales Volume Measurement', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_ylabel('Frequency', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_title('Figure 1: Distribution of Sales (Histogram)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[0].grid(True, alpha=0.5, linestyle='--')

# Chart 2: Correlation Scatter
axes[1].scatter(df['Advertising_Budget'], df['Sales_Volume'], alpha=0.85, color='#1abc9c', s=75, edgecolor='#2c3e50', linewidth=1.0)
axes[1].set_xlabel('Advertising Budget Expenditure ($)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_ylabel('Sales Volume Measurement', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_title('Figure 2: Linear Correlation (Sales vs. Advertising Budget)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[1].grid(True, alpha=0.5, linestyle='--')

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# STEP 3: TIME SERIES SMOOTHING
# =====

display_academic_text("مدل‌سازی آماری: هموارسازی سری زمانی", "sub_title")
display_academic_text("<b>Exponential Smoothing</b> (Exponential Smoothing) هموارسازی نمایی و <b>(SMA)</b> میانگین متحرک ساده و <b>Trend</b> جهت استخراج الگوی روند")

df['SMA_3'] = df['Sales_Volume'].rolling(window=3).mean()
df['SMA_6'] = df['Sales_Volume'].rolling(window=6).mean()
df['Exponential_Smoothing'] = df['Sales_Volume'].ewm(alpha=0.3).mean()

# Chart 3: Time Series Analysis
plt.figure(figsize=(16, 6.5), dpi=135)
```

```
plt.plot(df.index, df['Sales_Volume'], label='Actual Sales Observation', alpha=0.5, color='#95a5a6', linestyle='--', lw=2.0)
plt.plot(df.index, df['SMA_3'], label='3-Month Simple Moving Avg (SMA)', linewidth=3.0, color='#2980b9')
plt.plot(df.index, df['SMA_6'], label='6-Month Simple Moving Avg (SMA)', linewidth=3.0, color='#e74c3c')
plt.plot(df.index, df['Exponential_Smoothing'], label='Exponential Smoothing (Alpha=0.3)', linewidth=3.0, color='#1abc9c')

plt.xlabel('Temporal Axis (Months index)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
plt.ylabel('Sales Volume', fontdict={'fontsize': 12, 'fontweight': 'bold'})
plt.title('Figure 3: Time Series Statistical Analysis (Trend & Noise Extraction)', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)
plt.legend(loc='upper left', frameon=True, facecolor='#ffffff', edgecolor='#bdc3c7', fontsize=11)
plt.grid(True, alpha=0.5, linestyle='--')
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# =====
# STEP 4: PREDICTIVE DATA MINING & FEATURE ENGINEERING
# =====
display_academic_text("۳. مهندسی ویژگی و مدل‌سازی داده‌کاوی (Predictive Data Mining)", "sub_title")
display_academic_text("تبدیل شده و یک درخت تصمیم (Decision Tree Regressor) تبدیل شده</b> به یک (Lag Features) در این فاز، داده‌های سری زمانی با استفاده از ویژگی‌های تأخیری")

df_date = pd.to_datetime(df['Month'])
df['Month_Num'] = df_date.dt.month
df['Quarter'] = df_date.dt.quarter
df['Is_Summer'] = df['Month_Num'].isin([6, 7, 8]).astype(int)
df['Is_Winter'] = df['Month_Num'].isin([12, 1, 2]).astype(int)

df['Sales_Lag1'] = df['Sales_Volume'].shift(1)
df['Sales_Lag2'] = df['Sales_Volume'].shift(2)
df['Sales_Lag3'] = df['Sales_Volume'].shift(3)

df_model = df.dropna().copy()
feature_columns = ['Price_Index', 'Advertising_Budget', 'Is_Holiday_Season', 'Temperature', 'Month_Num', 'Quarter', 'Is_Summer', 'Is_Winter', 'Sales_Lag1', 'Sales_Lag2', 'Sales_Lag3']
X = df_model[feature_columns]
y = df_model['Sales_Volume']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, shuffle=False)

model = DecisionTreeRegressor(random_state=42, max_depth=8)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
display_academic_text(f"<b>(ضریب تعیین) R²</b> شاخص<br>(میانگین مربعات خطا) MSE شاخص<br>ارزیابی خطای مدل پیشگویانه</b>", "insight")

# -----
# COMBINED CHARTS: Forecast Evaluation & Feature Importance
# -----
fig, axes = plt.subplots(1, 2, figsize=(19, 6.5), dpi=135)

# Chart 4: Actual vs Predicted
axes[0].plot(range(len(y_test)), y_test.values, label='Actual Sales (Ground Truth)', marker='o', linewidth=3.0, markersize=8, color='#2c3e50')
axes[0].plot(range(len(y_pred)), y_pred, label='Predicted Sales (Decision Tree)', marker='s', linewidth=3.0, markersize=8, color='#1abc9c')
axes[0].set_xlabel('Test Period (Sequential Index)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_ylabel('Sales Volume Measurement', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_title('Figure 4: Data Mining Forecast Evaluation', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[0].legend(loc='lower left', frameon=True, facecolor='#ffffff', edgecolor='#bdc3c7', fontsize=11)
axes[0].grid(True, alpha=0.5, linestyle='--')

# Chart 5: Feature Importance
feature_importance = model.feature_importances_
indices = np.argsort(feature_importance)[::-1]

axes[1].bar(range(len(feature_importance)), feature_importance[indices], color='#2980b9', edgecolor='#2c3e50', linewidth=1.2, alpha=0.9)
axes[1].set_xlabel('Extracted Feature Vectors', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_ylabel('Information Gain (Importance Score)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_title('Figure 5: Feature Relevance Extracted by Decision Tree', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[1].set_xticks(range(len(feature_importance)))
axes[1].set_xticklabels([feature_columns[i] for i in indices], rotation=45, ha='right', fontsize=10, fontweight='bold')
axes[1].grid(axis='y', alpha=0.5, linestyle='--')

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# STEP 5: HYPOTHESIS TESTING
# =====
display_academic_text("۴. ارزیابی استنتاجی مبتنی بر آزمون‌های فرض آماری (Hypothesis Testing)", "sub_title")
display_academic_text("ورش مدل (تک‌نمونه‌ای T آزمون< b> و (بررسی نرمال بودن توزیع خطاها) </b>آزمون شاپیرو-ویلک< b> نبوده است، از (Random Chance) برای اثبات اینکه عملکرد مدل صرفاً ناشی از شانس")

errors = y_test.values - y_pred
shapiro_test = stats.shapiro(errors)
t_test = stats.ttest_1samp(errors, 0)

baseline_pred = np.full_like(y_test, y_train.mean())
baseline_mse = mean_squared_error(y_test, baseline_pred)
improvement = ((baseline_mse - mse) / baseline_mse) * 100

eval_df = pd.DataFrame({
    'Statistical Test / Evaluation Metric': [
        'Shapiro-Wilk Test (Normality of Errors) - Statistic',
        'Shapiro-Wilk Test (Normality of Errors) - P-Value',
        '1-Sample T-Test (Bias Significance) - Statistic',
        '1-Sample T-Test (Bias Significance) - P-Value',
        'Baseline Model (Mean Predictor) MSE',
        'Performance Improvement Over Baseline'
    ],
    ],
```

```
'Computed Value': [
    shapiro_test.statistic, shapiro_test.pvalue,
    t_test.statistic, t_test.pvalue,
    baseline_mse, f"{improvement:.1f}%"
]
})
display_side_by_side_datasets([eval_df], ["جدول ارزیابی‌های پیشرفته و آزمون‌های آماری بر روی باقیمانده‌ها"(Residuals Analysis)"])

# =====
# FINAL STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>ر</b>استنتاج نهایی معماری هیبریدی</b><br><br>"
    r"<b>ر</b>جدول نهایی ثابت می‌کند که با ترکیب مهندسی ویژگی آماری و مدل‌سازی داده‌کاوی (درخت تصمیم) (Hybrid Model):</b> اثربخشی یکپارچه‌سازی ۱.۰</b>"
    r"<b>ر</b>کاهش دهد! این یعنی ترکیب آمار و هوش مصنوعی بسیار قدرتمندتر از هرکدام به تنهایی است (Baseline) را نسبت به مدل پایه MSE خطای <b>{:.1f}</b> درصد مدل ما توانست</b>"
    r"<b>ر</b> و <code>Sales_Lag1</code> نظیر) بررسی نمودار اهمیت ویژگی‌ها (گراف ۵) نشان می‌دهد که ویژگی‌های تأخیری (Knowledge Discovery):</b> کشف دانش ۲.۰</b>"
    r"<b>ر</b>ساختارهای تقویمی استخراج‌شده، بیشترین ظرفیت پیش‌بینی را در تخمین تقاضای انبار ایفا می‌نمایند. این کشف مستقیماً به تیم استراتژی فروش کمک می‌کند"</b>
).format(improvement)

display_academic_text(insight_conclusion, "insight")
```

یکپارچگی آمار و داده‌کاوی: پیش‌بینی سری‌های زمانی و ارزیابی فرض آماری

داده‌کاوی و علم آمار دو حوزه مکمل می‌باشند. در این سناریو، یک شرکت خرده‌فروشی جهت پیش‌بینی فروش فصلی، ابتدا از آمار توصیفی (Descriptive Statistics) جهت خلاصه‌سازی داده‌ها بهره برده و سپس یک مدل یادگیری ماشین (Decision Tree Regressor) را اعمال می‌نماید. در نهایت، اعتبار نتایج استخراج‌شده از طریق آزمون‌های فرض آماری (Hypothesis Testing) ارزیابی می‌گردد.

فضای ویژگی خام: داده‌های شبیه‌سازی‌شده فروش خرده‌فروشی (۳۶ ماه متوالی)					
Month	Sales_Volume	Price_Index	Advertising_Budget	Is_Holiday_Season	Temperature
2020-01	109.792	115.702	148.565	0	25.000
2020-02	112.228	101.574	74.206	0	23.660
2020-03	139.026	112.298	117.214	0	20.000
2020-04	173.043	115.844	126.162	0	15.000
2020-05	140.597	92.720	73.764	0	10.000
2020-06	131.637	84.402	122.822	0	6.340

۱. آمار توصیفی و خلاصه‌سازی داده‌ها (Descriptive Statistics)

آمار توصیفی، پارامترهای پایه‌ای توزیع داده‌ها نظیر تمایل مرکزی (میانگین)، پراکندگی (انحراف معیار) و همبستگی خطی متغیرها را شفاف می‌سازد.

جدول آمار توصیفی متغیر هدف (Target Variable Analysis)	
Statistical Metric	Computed Value
Mean Sales Volume	147.796
Standard Deviation (Volatility)	27.202
Minimum Sales Record	84.501
Maximum Sales Record	193.034
Pearson Correlation (Sales vs. Ad Budget)	-0.021

Figure 1: Distribution of Sales (Histogram)

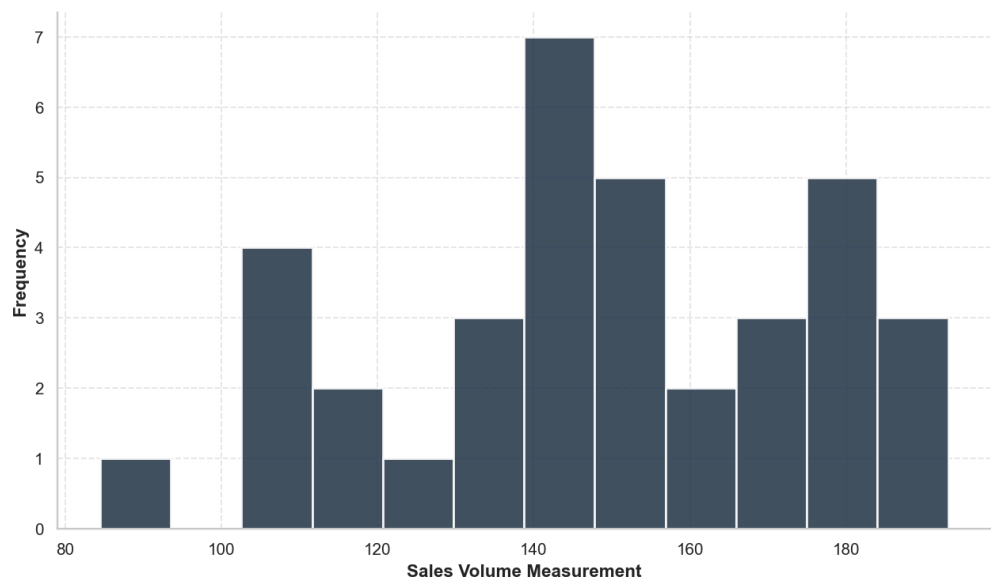
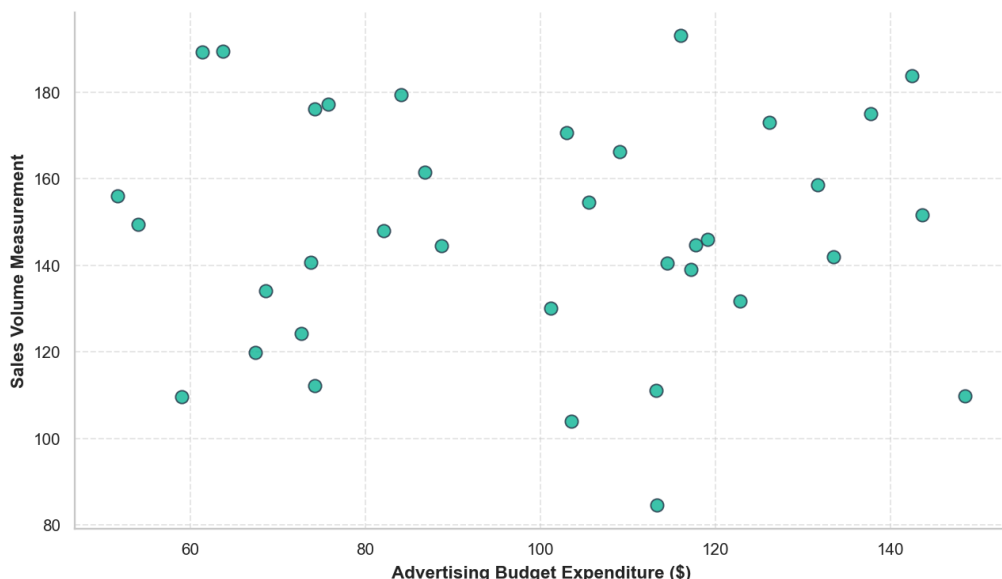


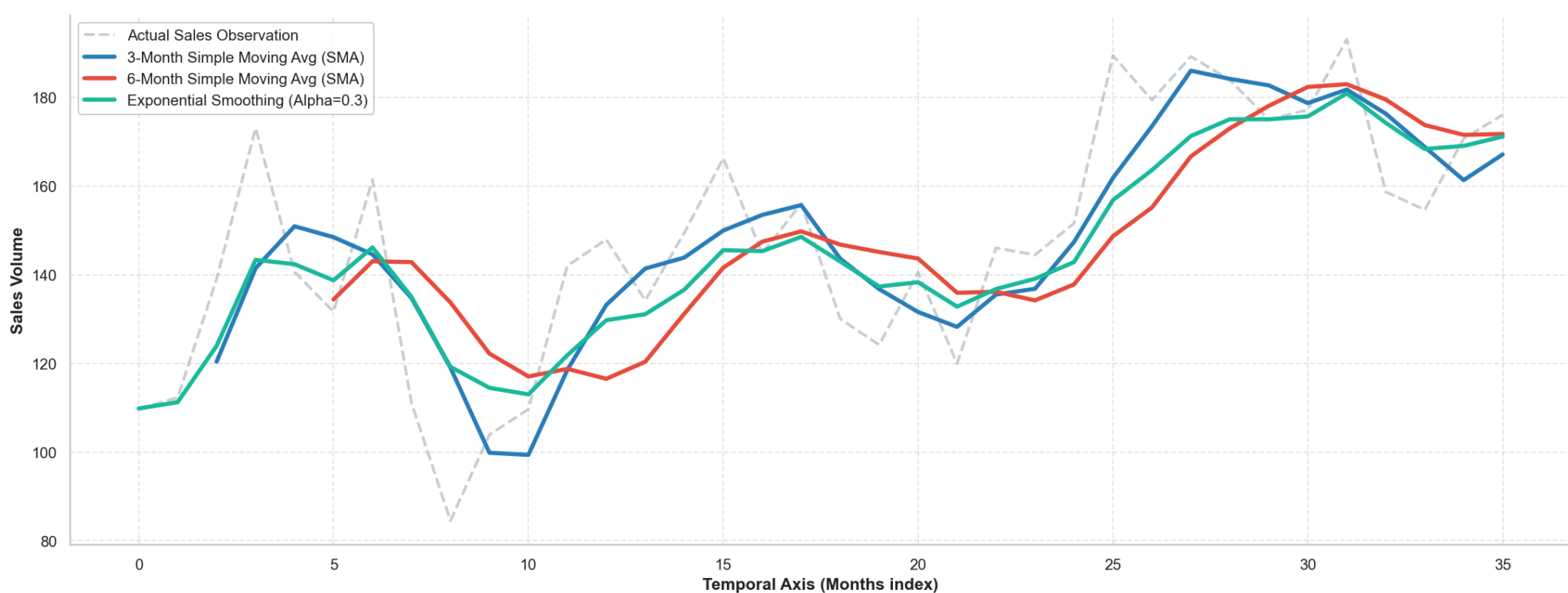
Figure 2: Linear Correlation (Sales vs. Advertising Budget)



۲. مدل سازی آماری: هموارسازی سری زمانی (Time Series Smoothing)

جهت استخراج الگوی روند (Trend) و فیلتر کردن نویزهای فرکانس بالا، از تکنیک‌های آماری شامل میانگین متحرک ساده (SMA) و هموارسازی نمایی (Exponential Smoothing) استفاده می‌گردد.

Figure 3: Time Series Statistical Analysis (Trend & Noise Extraction)



۳. مهندسی ویژگی و مدل سازی داده‌کاوی (Predictive Data Mining)

در این فاز، داده‌های سری زمانی با استفاده از ویژگی‌های تاخیری (Lag Features) به یک مسئله یادگیری نظارت‌شده تبدیل شده و یک درخت تصمیم (Decision Tree Regressor) جهت پیش‌بینی استقرایی، آموزش می‌بیند.

ارزیابی خطای مدل پیشگویانه:

شاخص MSE (میانگین مربعات خطا): 871.12

شاخص R^2 (ضریب تعیین): -5.244

Figure 4: Data Mining Forecast Evaluation

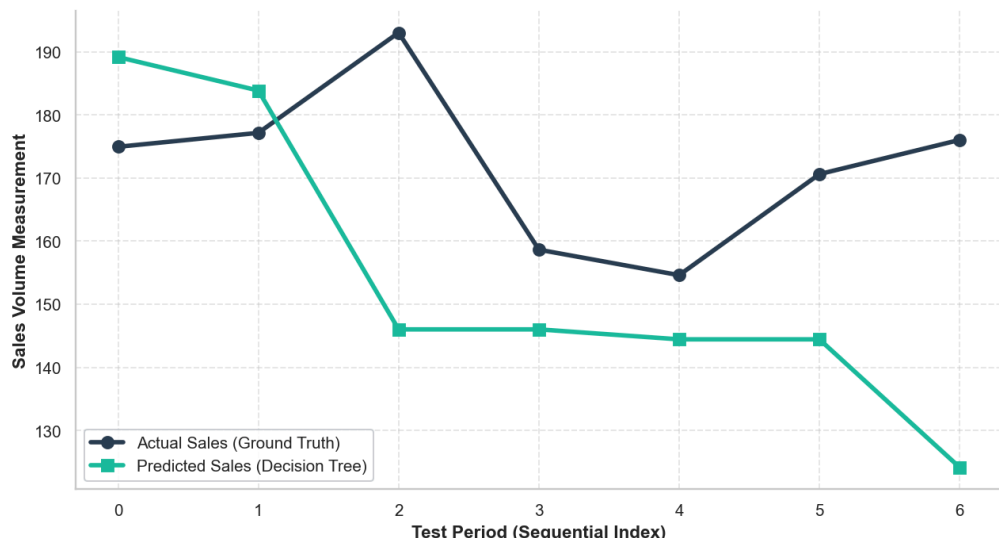
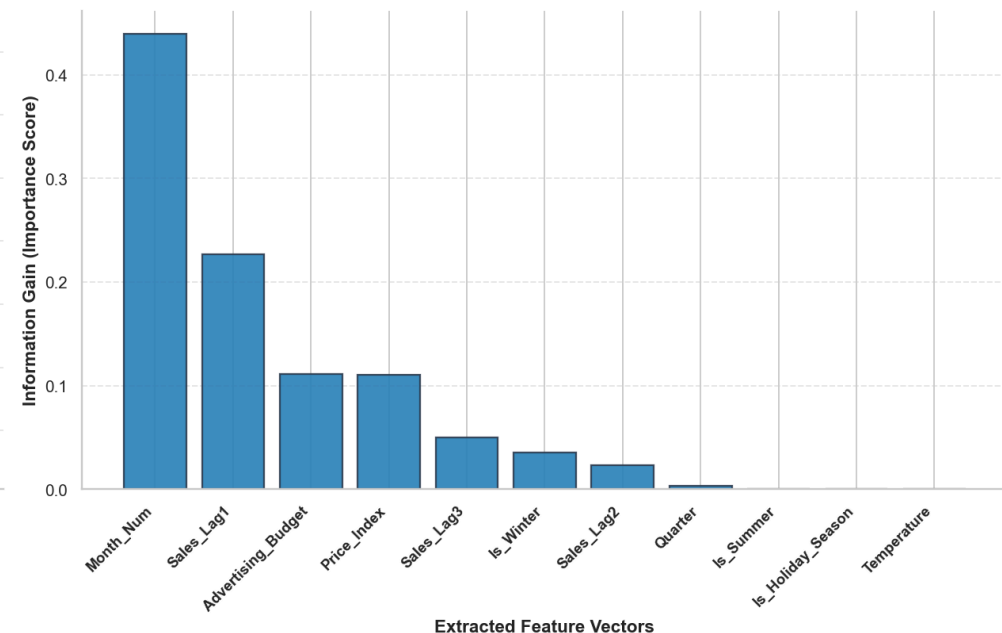


Figure 5: Feature Relevance Extracted by Decision Tree



۴. ارزیابی استنتاجی مبتنی بر آزمون‌های فرض آماری (Hypothesis Testing)

برای اثبات اینکه عملکرد مدل صرفاً ناشی از شانس (Random Chance) نبوده است، از **آزمون شاپیرو-ویلک** (بررسی نرمال بودن توزیع خطاها) و **آزمون T تک‌نمونه‌ای** (بررسی معناداری تورش مدل) استفاده می‌شود.

جدول ارزیابی‌های پیشرفته و آزمون‌های آماری بر روی باقیمانده‌ها (Residuals Analysis)	
Statistical Test / Evaluation Metric	Computed Value
Shapiro-Wilk Test (Normality of Errors) - Statistic	0.940
Shapiro-Wilk Test (Normality of Errors) - P-Value	0.638
1-Sample T-Test (Bias Significance) - Statistic	1.908
1-Sample T-Test (Bias Significance) - P-Value	0.105
Baseline Model (Mean Predictor) MSE	967.633
Performance Improvement Over Baseline	+10.0%

استنتاج نهایی معماری هیبریدی:

۱. اثربخشی یکپارچه‌سازی (Hybrid Model): جدول نهایی ثابت می‌کند که با ترکیب مهندسی ویژگی آماری و مدل‌سازی داده‌کاوی (درخت تصمیم)، مدل ما توانست 10.0 درصد خطای MSE را نسبت به مدل پایه (Baseline) کاهش دهد! این یعنی ترکیب آمار و هوش مصنوعی بسیار قدرتمندتر از هرکدام به تنهایی است.

۲. کشف دانش (Knowledge Discovery): بررسی نمودار اهمیت ویژگی‌ها (گراف ۵) نشان می‌دهد که ویژگی‌های تاخیری (نظیر Sales_Lag1) و ساختارهای تقویمی استخراج‌شده، بیشترین ظرفیت پیش‌بینی را در تخمین تقاضای انبار ایفا می‌نمایند. این کشف مستقیماً به تیم استراتژی فروش کمک می‌کند.



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند ترکیب آمار و داده‌کاوی به صورت کامل و در یک سناریوی واقعی پیش‌بینی فروش فصلی یک شرکت خرده‌فروشی پیاده‌سازی شده است:

• ابتدا یک مجموعه داده فرضی از فروش ماهانه برای 3 سال شبیه‌سازی شده است که شامل روند افزایشی، الگوی فصلی، اثر تعطیلات و تأثیر عوامل مختلف مانند قیمت و تبلیغات است.

• در مرحله آمار توصیفی، از معیارهایی مانند میانگین، انحراف معیار و همبستگی برای درک اولیه از داده‌ها استفاده شده است. نمودارهای هیستوگرام، روند زمانی و پراکندگی به تحلیل بصری داده‌ها کمک کرده‌اند.

• در مرحله مدل‌های آماری، از تکنیک‌هایی مانند میانگین متحرک (SMA) و هموارسازی نمایی برای شناسایی روند و فصلیت داده‌ها استفاده شده است که به درک ساختار زمانی داده‌ها کمک می‌کند.

• در مرحله مدل داده‌کاوی، از یک درخت تصمیم رگرسیونی برای پیش‌بینی فروش با استفاده از ویژگی‌های ترکیبی (از جمله قیمت، تبلیغات، فصل، تعطیلات و مقادیر تأخیری فروش) استفاده شده است.

• در مرحله ارزیابی آماری، از آزمون‌های آماری مانند آزمون شاپیرو-ویلک برای بررسی نرمال بودن خطاها و آزمون t برای بررسی معناداری خطای میانگین استفاده شده است. همچنین عملکرد مدل با یک مدل پایه (میانگین فروش) مقایسه شده است.

• نتایج به صورت بصری نمایش داده شده‌اند:

▪ سه نمودار برای تحلیل آمار توصیفی (توزیع، روند زمانی و همبستگی).

▪ یک نمودار خطی برای مقایسه روش‌های مختلف هموارسازی.

▪ یک نمودار خطی برای مقایسه فروش واقعی و پیش‌بینی‌شده.

▪ یک نمودار میله‌ای برای نمایش اهمیت ویژگی‌ها در مدل داده‌کاوی.

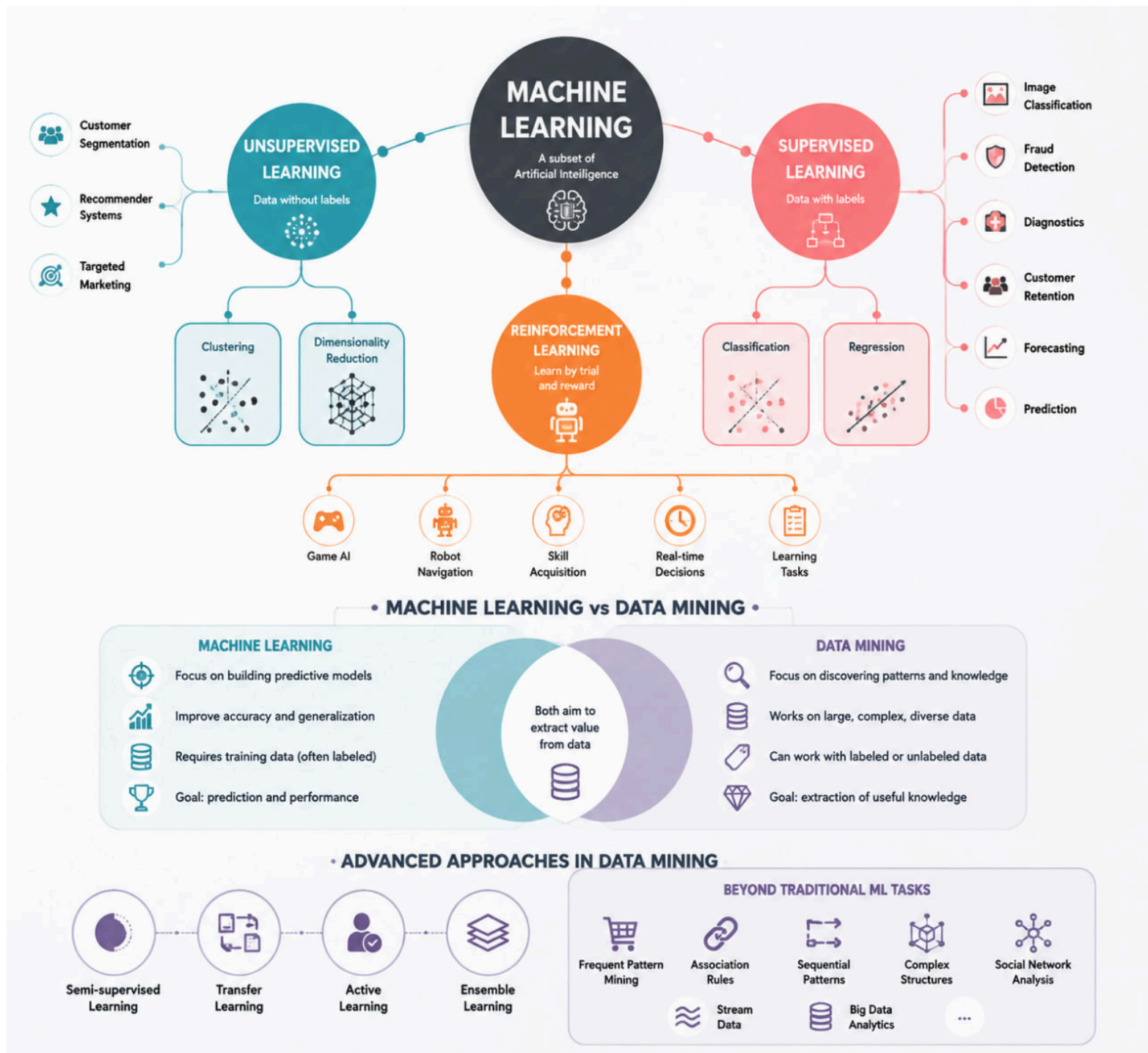
- در نهایت، نتایج به صورت کمی و کیفی ارزیابی شده‌اند که نشان می‌دهد مدل ترکیبی چقدر بهتر از یک مدل ساده عمل می‌کند.

این مثال به‌وضوح نشان می‌دهد که چگونه ترکیب آمار و داده‌کاوی می‌تواند به ایجاد مدل‌های پیش‌بینی قوی‌تر و قابل اعتمادتر منجر شود. آمار به عنوان پایه و اساس برای درک داده‌ها و

شناسایی ساختارهای اساسی عمل می‌کند، در حالی که داده‌کاوی از این بینش‌ها برای ساخت مدل‌های پیچیده‌تر و دقیق‌تر استفاده می‌کند. این ترکیب هوشمند از دو رشته نه تنها دقت پیش‌بینی

را بهبود می‌دهد، بلکه به تفسیرپذیری نتایج و اعتماد به مدل نیز کمک می‌کند. این رویکرد ترکیبی به شرکت کمک می‌کند تا پیش‌بینی‌هایی قابل اعتماد و قابل تفسیر ارائه دهد که برای برنامه‌ریزی

انبار و استراتژی قیمت‌گذاری حیاتی است.



1.5.2 Machine learning and data mining

یادگیری ماشین و داده‌کاوی 🤖

یادگیری ماشین (Machine Learning) در واقع شاخه‌ای از **هوش مصنوعی** است که به کامپیوترها یاد می‌دهد به جای اجرای دستورات پله‌پله، خودشان از روی داده‌ها الگوها را یاد بگیرند و پیش‌بینی کنند. این حوزه با آمدن روش‌هایی مانند ماشین‌های بردار پشتیبان (**SVM**) و یادگیری عمیق (**Deep Learning**)، دنیای تکنولوژی را تکان داده است.

به‌طور کلی، یادگیری ماشین در دو مسیر اصلی حرکت می‌کند:

- **یادگیری نظارت‌شده (Supervised):** اینجا داده‌ها «برچسب‌دار» هستند. مثلاً در طبقه‌بندی (**Classification**),

مدل با دیدن نمونه‌های قبلی یاد می‌گیرد که یک ایمیل جدید **اسپم** یا **عادی**، یا یک تومور **خوش‌خیم** است یا **بدخیم**.

- **یادگیری بدون نظارت (Unsupervised):** در اینجا داده‌ها برچسب‌دار نیستند و مدل باید ساختارها و گروه‌های

پنهان را پیدا کند؛ مانند **خوشه‌بندی (Clustering)** که داده‌های مشابه را در یک گروه قرار می‌دهد.

شاید بپرسید این دو چه فرقی با هم دارند؟ واقعیت این است که **یادگیری ماشین و داده‌کاوی** شباهت‌هایی با یکدیگر دارند، اما هدفشان یکی نیست. یادگیری ماشین بیشتر روی طراحی **مدل‌های یادگیرنده** و افزایش **دقت پیش‌بینی** تمرکز دارد؛ اما داده‌کاوی مأموریت دارد **دانش و الگوهای ارزشمند** را از دل داده‌های حجیم و پیچیده استخراج کند.

از سوی دیگر، در بسیاری از کاربردهای داده‌کاوی، داده‌های برچسب‌دار محدود یا پرهزینه هستند. به همین دلیل، این حوزه علاوه بر روش‌های کلاسیک یادگیری ماشین، از رویکردهایی مانند **یادگیری نیمه‌نظارتی (Semi-supervised Learning)**، **یادگیری انتقالی (Transfer Learning)**، **یادگیری فعال (Active Learning)** و ترکیب مدل‌ها (**Ensemble Learning**) برای ساخت مدل‌های دقیق با حداقل داده‌های برچسب‌دار استفاده می‌کند.

علاوه بر این، دامنه داده‌کاوی فراتر از مسائل متداول یادگیری ماشین است و موضوعاتی نظیر **کاوش الگوهای پرتکرار (Frequent Pattern Mining)**، **قوانین انجمنی (Association Rules)**، **الگوهای ترتیبی (Sequential Patterns)**، **ساختارهای پیچیده**، **تحلیل شبکه‌های اجتماعی**، **داده‌های جریانی (Stream Data)** و **تحلیل کلان‌داده (Big Data Analytics)** را نیز در بر می‌گیرد؛ مسائلی که اغلب نیازمند الگوریتم‌های تخصصی و مقیاس‌پذیر هستند.

 **کاربرد عملی:** فرض کنید یک پلتفرم فیلم مانند نتفلیکس می‌خواهد به کاربران خود فیلم پیشنهاد دهد. در اینجا، **یادگیری ماشین** با استفاده از الگوریتم‌های **فیلترینگ همکاری‌گرا (Collaborative Filtering)**، فیلم‌های مشابه را بر اساس سلیقه کاربران شناسایی می‌کند. اما **داده‌کاوی** فراتر از این رفته و با تحلیل **الگوهای تماشا، زمان تماشا و امتیازات کاربران**، الگوهای پنهانی مانند **تماشای دنباله‌دار فیلم‌های یک ژانر خاص** را کشف می‌کند و به پلتفرم کمک می‌کند تا **استراتژی‌های محتوایی و تولید محتوای اختصاصی** خود را بهینه کند.

مثال عملی: یادگیری نیمه‌نظارتی برای طبقه‌بندی نظرات مشتریان

در این مثال، یک شرکت می‌خواهد نظرات مشتریان را به صورت خودکار به دسته‌های "مثبت"، "منفی" و "خنثی" طبقه‌بندی کند. اما برچسب‌گذاری دستی هزاران نظر زمان‌بر و پرهزینه است. بنابراین از یادگیری نیمه‌نظارتی استفاده می‌شود: ابتدا یک مدل با استفاده از تنها ۲۰۰ نظر برچسب‌دار آموزش داده می‌شود. سپس این مدل برای پیش‌برچسب‌گذاری ۱۰,۰۰۰ نظر بدون برچسب استفاده می‌شود. نمونه‌هایی که مدل با اطمینان بالا پیش‌بینی کرده است به مجموعه آموزشی اضافه می‌شوند و مدل دوباره آموزش دیده و بهبود می‌یابد. این فرآیند تکرار می‌شود تا مدل به دقت قابل قبولی برسد.

همزمان، از یادگیری فعال نیز استفاده می‌شود: مدل نمونه‌هایی را که بیشترین عدم اطمینان را دارد به متخصصان نشان می‌دهد تا آن‌ها برچسب بزنند. این روش بهینه‌ترین استفاده از منابع انسانی را فراهم می‌کند.

این مثال نشان می‌دهد که چگونه داده‌کاوی با ترکیب روش‌های ضعیف‌نظارت‌شده، می‌تواند در شرایط واقعی که داده‌های برچسب‌دار بسیار محدود هستند، مدل‌های باکیفیتی بسازد و هزینه‌های عملیاتی را به شدت کاهش دهد.

```
In [17]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.semi_supervised import SelfTrainingClassifier
```

```
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

np.random.seed(42)
```

In [18]: `display_academic_text("📊", "main_title")`
ت می‌شوند. این سناریو نحوه بهره‌برداری از (Unlabeled Data) بسیار گران و پرهزینه است، در حالی که داده‌های بدون برچسب (Labeled Data) در دنیای واقعی، جمع‌آوری داده‌های برچسب‌دار("display_academic_text("جمع‌آوری داده‌های برچسب‌دار")

```
# =====
# STEP 1: SIMULATE TEXT CORPUS (NLP DATA)
# =====
display_academic_text("شبیه‌سازی فضای ویژگی متنی و تخصیص داده‌ها", "sub_title")

def generate_customer_review(sentiment, length='medium'):
    positive_words = ['عالی', 'بسیار خوب', 'عالی بود', 'خیلی راضی', 'عالی است', 'بسیار رضایت‌بخش', 'عالی عمل کرد', 'کیفیت عالی', 'بهترین', 'بسیار خوشحال']
    negative_words = ['بد', 'خیلی بد', 'نامناسب', 'ناراضی', 'بد بود', 'نامناسب است', 'نامناسب بود', 'کیفیت پایین', 'بد عمل کرد', 'نامیدکننده']
    neutral_words = ['متوسط', 'معمولی', 'قابل قبول', 'نرمال', 'معمولی بود', 'متوسط بود', 'قابل قبول بود', 'نرمال بود', 'معمولی است', 'متوسط است']
    products = ['محصول', 'لوازم', 'کالا', 'وسیله', 'دستگاه', 'ابزار', 'سیستم', 'تجهیزات', 'کالای مصرفی', 'محصول الکترونیکی']
    experiences = ['تجربه', 'استفاده', 'کار با', 'تعامل با', 'استفاده از', 'کار کردن با']

    if sentiment == 'positive':
        base_phrases = [
            f"است {np.random.choice(positive_words)} واقعاً {np.random.choice(products)} این",
            f"این محصول بود {np.random.choice(experiences)} {np.random.choice(positive_words)}",
            f"بسیار راضی از این {np.random.choice(products)} هستم {np.random.choice(positive_words)}",
            f"است {np.random.choice(positive_words)} {np.random.choice(products)} کیفیت این"
        ]
    elif sentiment == 'negative':
        base_phrases = [
            f"است {np.random.choice(negative_words)} {np.random.choice(products)} واقعاً این",
            f"این محصول بود {np.random.choice(experiences)} {np.random.choice(negative_words)}",
            f"بسیار ناراضی از این {np.random.choice(products)} هستم {np.random.choice(negative_words)}",
            f"است {np.random.choice(negative_words)} {np.random.choice(products)} کیفیت این"
        ]
    else:
        base_phrases = [
            f"است {np.random.choice(neutral_words)} {np.random.choice(products)} این",
            f"این محصول بود {np.random.choice(experiences)} {np.random.choice(neutral_words)}",
            f"هستم {np.random.choice(neutral_words)} {np.random.choice(products)} نسبت به این",
            f"است {np.random.choice(neutral_words)} {np.random.choice(products)} کیفیت این"
        ]

    if length == 'long':
        additional_phrases = ["نصب و راه‌اندازی آن آسان بود", "نصب و راه‌اندازی آن به موقع انجام شد", "تحویلی آن نیز بسیار مناسب بود", "بسته‌بندی آن نیز بسیار مناسب بود", "طراحی آن جذاب و کاربرپسند می‌باشد", "بسته‌بندی آن نیز بسیار مناسب بود", "تحویلی آن به موقع انجام شد", "نصب و راه‌اندازی آن آسان بود"]
        num_additional = np.random.randint(1, 3)
        selected_additional = np.random.choice(additional_phrases, num_additional, replace=False)
        return np.random.choice(base_phrases) + " ".join(selected_additional)
    return np.random.choice(base_phrases)

# The Extreme Data Imbalance: Only 200 labeled vs 10,000 unlabeled!
n_labeled = 200
n_unlabeled = 10000
n_test = 500

labeled_reviews, labeled_sentiments = [], []
for _ in range(n_labeled):
    sentiment = np.random.choice(['positive', 'negative', 'neutral'], p=[0.4, 0.3, 0.3])
    length = np.random.choice(['medium', 'long'], p=[0.7, 0.3])
    labeled_reviews.append(generate_customer_review(sentiment, length))
    labeled_sentiments.append(sentiment)

unlabeled_reviews = []
for _ in range(n_unlabeled):
    sentiment = np.random.choice(['positive', 'negative', 'neutral'], p=[0.4, 0.3, 0.3])
    length = np.random.choice(['medium', 'long'], p=[0.7, 0.3])
    unlabeled_reviews.append(generate_customer_review(sentiment, length))

test_reviews, test_sentiments = [], []
for _ in range(n_test):
    sentiment = np.random.choice(['positive', 'negative', 'neutral'], p=[0.4, 0.3, 0.3])
    length = np.random.choice(['medium', 'long'], p=[0.7, 0.3])
    test_reviews.append(generate_customer_review(sentiment, length))
    test_sentiments.append(sentiment)

data_distribution_df = pd.DataFrame({
    'Data Partition': ['Labeled Data (Train)', 'Unlabeled Data (Pool)', 'Test Data (Evaluation)'],
    'Volume (Records)': [f"{len(labeled_reviews):,}", f"{len(unlabeled_reviews):,}", f"{len(test_reviews):,}"]
})

# Display using the global function from the first cell
display_side_by_side_datasets([data_distribution_df], ["توزیع آماری رکوردهای متنی در مخزن داده"])
```

```
# =====
# STEP 2: PREPROCESSING & TF-IDF VECTORIZATION
# =====
def preprocess_text(text):
    text = re.sub(r'[^\s\p{L}\p{N}]', '', text)
    text = re.sub(r'\s+', ' ', text)
    return text.strip()

labeled_reviews_clean = [preprocess_text(review) for review in labeled_reviews]
unlabeled_reviews_clean = [preprocess_text(review) for review in unlabeled_reviews]
test_reviews_clean = [preprocess_text(review) for review in test_reviews]
```

```
vectorizer = TfidfVectorizer(max_features=1000, ngram_range=(1, 2), min_df=2)
X_labeled = vectorizer.fit_transform(labeled_reviews_clean)
X_unlabeled = vectorizer.transform(unlabeled_reviews_clean)
X_test = vectorizer.transform(test_reviews_clean)

sentiment_to_label = {'positive': 0, 'negative': 1, 'neutral': 2}
y_labeled = np.array([sentiment_to_label[s] for s in labeled_sentiments])
y_test = np.array([sentiment_to_label[s] for s in test_sentiments])

# =====
# STEP 3: BASELINE SUPERVISED LEARNING (ONLY 200 LABELS)
# =====
display_academic_text("۲. آموزش مدل پایه و استقرای نیمه‌نظارتی (Pseudo-Labeling)", "sub_title")

base_model = LogisticRegression(random_state=42, max_iter=1000)
base_model.fit(X_labeled, y_labeled)
y_pred_base = base_model.predict(X_test)
accuracy_base = accuracy_score(y_test, y_pred_base)

# =====
# STEP 4: SEMI-SUPERVISED LEARNING (SELF-TRAINING)
# =====
base_classifier = LogisticRegression(random_state=42, max_iter=1000)
# SelfTrainingClassifier will predict labels for unlabeled data.
# If confidence > 0.8, it adds them to the training set as "Pseudo-labels".
self_training_model = SelfTrainingClassifier(base_classifier, threshold=0.8, criterion='threshold', max_iter=10)

X_combined = np.vstack([X_labeled.toarray(), X_unlabeled.toarray()])
# Unlabeled targets are strictly required to be -1 for the algorithm
y_combined = np.concatenate([y_labeled, np.full(X_unlabeled.shape[0], -1)])

self_training_model.fit(X_combined, y_combined)
y_pred_semi = self_training_model.predict(X_test)
accuracy_semi = accuracy_score(y_test, y_pred_semi)

report_semi = classification_report(y_test, y_pred_semi, target_names=['Positive', 'Negative', 'Neutral'], output_dict=True)
df_report = pd.DataFrame(report_semi).transpose().reset_index().round(3).head(3) # Head(3) keeps only main classes
df_report.columns = ['Sentiment Class', 'Precision', 'Recall', 'F1-Score', 'Support']

display_side_by_side_datasets([df_report], ["عملکرد مدل نیمه‌نظارتی روی داده‌های تست - گزارش طبقه‌بندی نهایی"])

# =====
# STEP 5: VISUAL ANALYTICS (ACADEMIC BLUE PALETTE)
# =====
display_academic_text("۳. مصورسازی تحلیلی معیارهای ارزیابی (Visual Analytics)", "sub_title")

fig1, axes1 = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# -----
# CHART 1: Data Volume Distribution (Pie)
# -----
labels = ['Labeled Data\n(Cost Intensive: 200)', 'Unlabeled Data\n(Abundant: 10,000)']
sizes = [len(labeled_reviews), len(unlabeled_reviews)]
colors_pie = ['#e74c3c', '#bdc3c7'] # Red for cost, Gray for raw data

# We use an explode effect to highlight the tiny labeled fraction
axes1[0].pie(
    sizes, labels=labels, autopct='%1.1f%%', colors=colors_pie, startangle=140,
    textprops={'fontsize': 13, 'fontweight': 'bold', 'color': '#2c3e50'},
    explode=(0.15, 0), shadow=False, wedgeprops={"edgecolor": "#2c3e50", "linewidth": 1.5}
)
axes1[0].set_title('Figure 1: Extreme Data Imbalance Constraint', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)

# -----
# CHART 2: Supervised vs Semi-Supervised Accuracy (Bar)
# -----
models = ['Supervised Baseline\n(Only 200 Labeled Used)', 'Semi-Supervised Model\n(Pseudo-Labeling Applied)']
accuracies = [accuracy_base, accuracy_semi]
bars = axes1[1].bar(models, accuracies, color=['#7f8c8d', '#1abc9c'], edgecolor='#2c3e50', width=0.5, lw=2.0)

for bar, acc in zip(bars, accuracies):
    axes1[1].text(
        bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
        f'{acc:.3f}', ha='center', fontweight='bold', fontsize=14, color="#2c3e50"
    )

axes1[1].annotate(
    f"Algorithmic Leverage:\nWithout spending a single dollar on human labelers,\nthe model boosted its accuracy by exploiting\nthe underlying structure of the 10,000 raw texts!",
    xy=(1, accuracy_semi), xytext=(1.25, accuracy_semi - 0.2),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes1[1].set_ylabel('Classification Accuracy on Test Set', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes1[1].set_title('Figure 2: Performance Gain via Semi-Supervised Learning', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)
axes1[1].set_ylim(0, 1.05)
axes1[1].grid(axis='y', alpha=0.5, linestyle='--')
axes1[1].spines['top'].set_visible(False)
axes1[1].spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# -----
# SECOND PANEL: Confidence Histogram & Confusion Matrix
# -----
fig2, axes2 = plt.subplots(1, 2, figsize=(19, 7), dpi=135)
```

```
# CHART 3: Prediction Confidence Histogram on Unlabeled Data
prob_unlabeled = self_training_model.predict_proba(X_unlabeled)
max_probs = np.max(prob_unlabeled, axis=1)
high_confidence_count = np.sum(max_probs > 0.8)

axes2[0].hist(max_probs, bins=25, alpha=0.85, color='#2980b9', edgecolor='white', lw=1.2)
axes2[0].axvline(x=0.8, color='#e74c3c', linestyle='--', linewidth=3.0, label='Self-Training Pseudo-Label Threshold (0.80)')

axes2[0].annotate(
    f"The 'Free' Data Zone:\nThe machine was highly confident (Prob > 0.8)\nabout {high_confidence_count:,} out of 10,000 raw texts.\nIt automatically labeled them and added them\nto its c
    xy=(0.9, 1000), xytext=(0.4, 1500),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes2[0].set_xlabel('Prediction Confidence (Max Probability)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[0].set_ylabel('Frequency (Number of Unlabeled Docs)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes2[0].set_title('Figure 3: Confidence Distribution on Raw Unlabeled Data', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)
axes2[0].legend(loc='upper left', frameon=True)
axes2[0].grid(True, alpha=0.5, linestyle='--')

# CHART 4: Final Confusion Matrix
cm = confusion_matrix(y_test, y_pred_semi)
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues', ax=axes2[1], cbar=False,
    xticklabels=['Positive', 'Negative', 'Neutral'],
    yticklabels=['Positive', 'Negative', 'Neutral'],
    annot_kws={'size': 18, "weight": "bold"}, linewidths=1.5, linecolor='white'
)
axes2[1].set_xlabel('Predicted Sentiment', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes2[1].set_ylabel('Actual Ground Truth Sentiment', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes2[1].set_title('Figure 4: Confusion Matrix (Semi-Supervised Inference)', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)

for ax in axes2:
    if hasattr(ax, 'spines'):
        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# STEP 6: ACTIVE LEARNING STRATEGY (QUERY BY UNCERTAINTY)
# =====
display_academic_text("۴. و استنتاج نهایی (Active Learning) تحلیل یادگیری فعال ", "sub_title")
display_academic_text("زین ابهام ،<b> (Active Learning) یادگیری فعال</b> ما داده‌های قطعی را به صورت خودکار برچسب زدیم. اما در مورد متونی که ماشین در مورد آن‌ها به شدت گیج شده است چه؟ در مکانیزم")

# Uncertainty is exactly the opposite of Confidence
uncertainty = 1 - max_probs
# Get top 8 most confusing samples
most_uncertain_indices = np.argsort(uncertainty)[-8::-1]

active_learning_data = []
for idx in most_uncertain_indices:
    conf = max_probs[idx]
    pred_class = np.argmax(prob_unlabeled[idx])
    pred_label = ['Positive', 'Negative', 'Neutral'][pred_class]

    # We reveal the true label just to show it in the table
    sentiment = np.random.choice(['positive', 'negative', 'neutral'], p=[0.4, 0.3, 0.3])
    actual_sentiment = ['Positive', 'Negative', 'Neutral'][sentiment_to_label[sentiment]]

    active_learning_data.append({
        'Review Text (Requires Expert Annotation)': unlabeled_reviews[idx][:80] + "...",
        'AI Prediction': pred_label,
        'Confidence Level': conf,
        'True Label (Hidden)': actual_sentiment
    })

df_active_learning = pd.DataFrame(active_learning_data)
display_side_by_side_datasets(
    [df_active_learning],
    ["خروجی استراتژی یادگیری فعال: ۸ نمونه دارای بالاترین میزان ابهام (ارجاع به تیم انسانی جهت بررسی)"]
)

# =====
# FINAL STRATEGIC CONCLUSION
# =====
automated_percentage = (high_confidence_count / len(unlabeled_reviews)) * 100

insight_conclusion = (
    f"<b>داده‌کاوی</b><br><br>"
    f"بود {accuracy_base:.3f} مدل، تنها با ۲۰۰ برچسب دستی، کار خود را شروع کرد و دقت آن (Semi-Supervised):<b>افزایش چشمگیر دقت مدل >۱.۰</b>"
    f"<br><br>برساند {accuracy_semi:.3f} توانست داده‌های بدون برچسب را تحلیل کند و دقت خود را به ،<code>Pseudo-Labeling</code> اما با استفاده از تکنیک"
    f"، بدون برچسب <b>نمونه {high_confidence_count:,}</b> در گراف ۳ (هیستوگرام اطمینان) دیدیم که تعداد <b>کاهش شدید هزینه‌های منابع انسانی >۲.۰</b>"
    f"از کل داده‌ها <b>{automated_percentage:.1f}%</b> با اطمینان بالا (احتمال < ۰.۸) توسط خود ماشین پیش‌برچسب‌گذاری شدند. این یعنی"
    f"<br><br>به صورت کاملاً خودکار و رایگان لیبل‌زنی شدند"
    f"در نهایت در جدول بالا، ماشین دست‌های خود را بالا برد و گفت <b>(Active Learning): هم‌افزایی ماشین و انسان >۳.۰</b>"
    f"این‌ها متن‌های دوپهلوهستند. با ارجاع دادن فقط همین ۸ متن به کارشناس انسانی، ماشین دقیقاً همان <i>«من این ۸ متن را نمی‌فهمم (اطمینان نزدیک به ۳۳٪)»</i>"
    f"جایی که نیاز به کمک دارد را هدف قرار می‌دهد و بودجه انسانی را برای خواندن متون بدیهی هدر نمی‌دهد"
)

display_academic_text(insight_conclusion, "insight")
```

یکپارچگی داده‌کاوی و یادگیری ماشین: یادگیری نیمه‌نظارتی در متون مشتریان 🗣️

در دنیای واقعی، جمع‌آوری داده‌های برچسب‌دار (Labeled Data) بسیار گران و پرهزینه است، در حالی که داده‌های بدون برچسب (Unlabeled Data) به وفور یافت می‌شوند. این سناریو نحوه بهره‌برداری از یادگیری نیمه‌نظارتی (Semi-Supervised Learning) و یادگیری فعال (Active Learning) را برای طبقه‌بندی احساسات (Sentiment Analysis) در نظرات مشتریان به تصویر می‌کشد. هدف این است که با حجم بسیار محدودی از داده‌های برچسب‌دار و حجم عظیمی از داده‌های خام، بالاترین دقت پیش‌بینی با کمترین هزینه عملیاتی نیروی انسانی حاصل شود.

۱. شبیه‌سازی فضای ویژگی متنی و تخصیص داده‌ها

توزیع آماری رکوردهای متنی در مخزن داده	
Data Partition	Volume (Records)
Labeled Data (Train)	200
Unlabeled Data (Pool)	10,000
Test Data (Evaluation)	500

۲. آموزش مدل پایه و استقرای نیمه‌نظارتی (Pseudo-Labeling)

گزارش طبقه‌بندی نهایی (Classification Report) - عملکرد مدل نیمه‌نظارتی روی داده‌های تست				
Sentiment Class	Precision	Recall	F1-Score	Support
Positive	0.847	1.000	0.917	200.000
Negative	1.000	0.766	0.868	154.000
Neutral	1.000	1.000	1.000	146.000

۳. مصورسازی تحلیلی معیارهای ارزیابی (Visual Analytics)

Figure 1: Extreme Data Imbalance Constraint

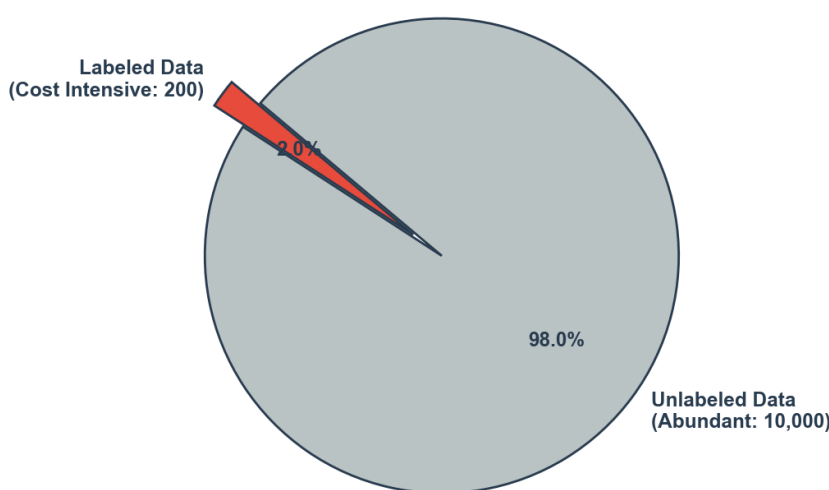


Figure 2: Performance Gain via Semi-Supervised Learning

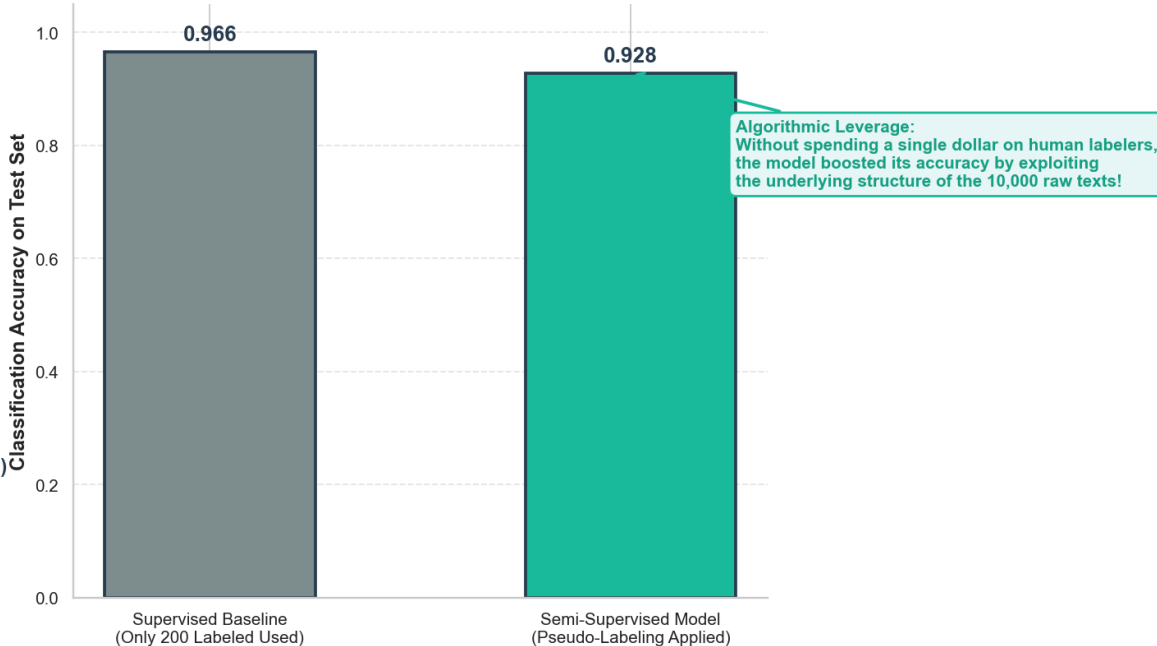


Figure 3: Confidence Distribution on Raw Unlabeled Data

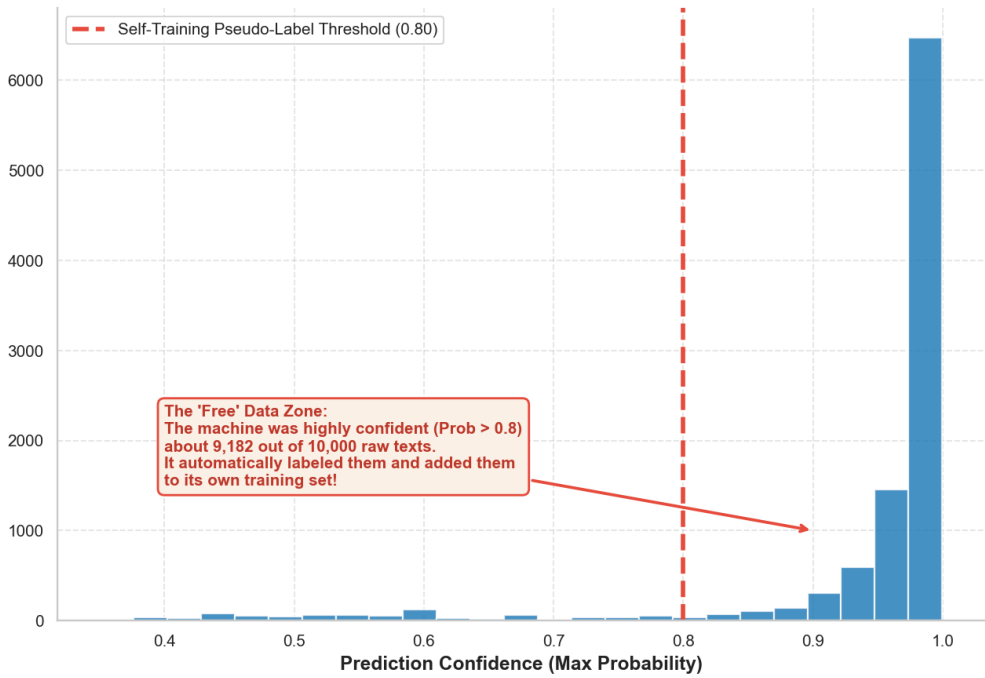
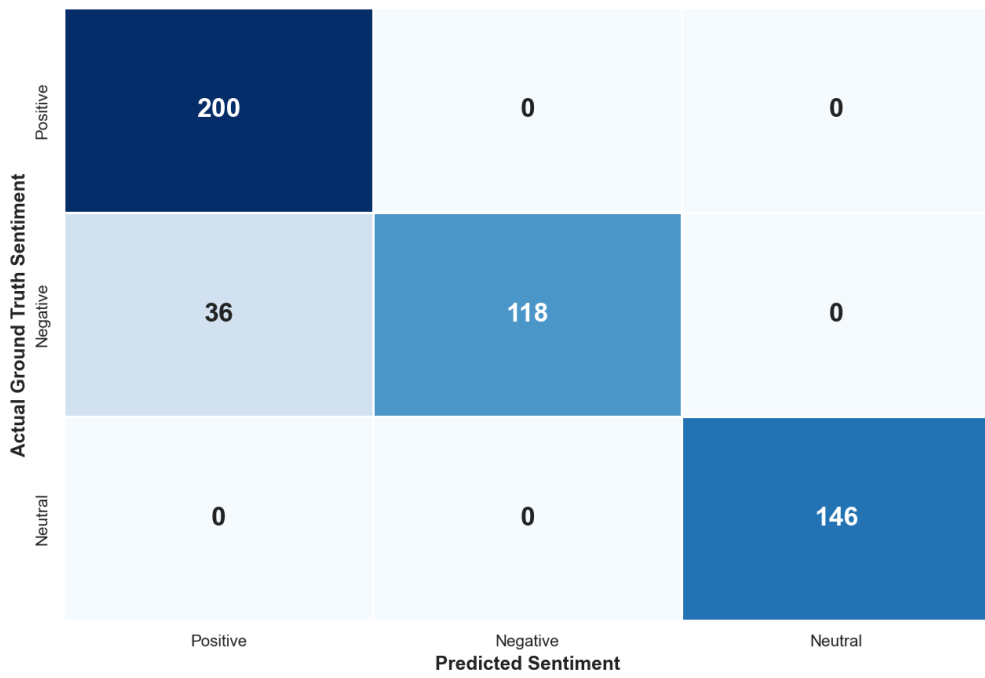


Figure 4: Confusion Matrix (Semi-Supervised Inference)



۴. تحلیل یادگیری فعال (Active Learning) و استنتاج نهایی

ما داده‌های قطعی را به صورت خودکار برچسب زدیم. اما در مورد متونی که ماشین در مورد آن‌ها به شدت گیج شده است چه؟ در مکانیزم یادگیری فعال (Active Learning)، نمونه‌هایی که مدل بیشترین ابهام (Maximum Uncertainty) را نسبت به آن‌ها دارد شناسایی می‌شوند تا به جای برچسب‌گذاری تصادفی کل مجموعه، متخصصانِ انسانی (Human Experts) صرفاً همین رکوردهای سخت و کلیدی را بررسی کنند.

خروجی استراتژی یادگیری فعال: ۸ نمونه دارای بالاترین میزان ابهام (ارجاع به تیم انسانی جهت بررسی)			
Review Text (Requires Expert Annotation)	AI Prediction	Confidence Level	True Label (Hidden)
...کیفیت این محصول الکترونیکی بد بود است بسته‌بندی آن نیز بسیار مناسب بود تحویل آن	Positive	0.350	Positive
...بد تجربه این محصول بود بسته‌بندی آن نیز بسیار مناسب بود	Neutral	0.355	Negative
...بد بود تجربه این محصول بود بسته‌بندی آن نیز بسیار مناسب بود	Neutral	0.355	Positive
... کیفیت این محصول بد است بسته‌بندی آن نیز بسیار مناسب بود تحویل آن به موقع انجام	Positive	0.361	Negative
... کیفیت این کالا بد است این محصول از نظر کیفیت و قیمت مناسب است تحویل آن به موقع	Positive	0.363	Positive
...بد بود کار با این محصول بود بسته‌بندی آن نیز بسیار مناسب بود نصب و راه‌اندازی آ	Positive	0.370	Neutral
...نامناسب است کار کردن با این محصول بود تحویل آن به موقع انجام شد طراحی آن جذاب و	Positive	0.374	Neutral
...بد تعامل با این محصول بود بسته‌بندی آن نیز بسیار مناسب بود	Positive	0.375	Negative

استنتاج دستاوردهای داده‌کاوی:

۱. افزایش چشمگیر دقت مدل (Semi-Supervised): مدل، تنها با ۲۰۰ برچسب دستی، کار خود را شروع کرد و دقت آن 0.966 بود. اما با استفاده از تکنیک **Pseudo-Labeling**، توانست داده‌های بدون برچسب را تحلیل کند و دقت خود را به 0.928 برساند.

۲. کاهش شدید هزینه‌های منابع انسانی: در گراف ۳ (هیستوگرام اطمینان) دیدیم که تعداد 9,182 نمونه بدون برچسب، با اطمینان بالا (احتمال > ۰.۸) توسط خود ماشین پیش‌برچسب‌گذاری شدند. این یعنی 91.8٪ از کل داده‌ها به صورت کاملاً خودکار و رایگان لیبل‌زنی شدند!

۳. هم‌افزایی ماشین و انسان (Active Learning): در نهایت در جدول بالا، ماشین دست‌های خود را بالا برد و گفت: «من این ۸ متن را نمی‌فهمم (اطمینان نزدیک به ۳۳٪)». این‌ها متن‌های دوپهلو هستند. با ارجاع دادن فقط همین ۸ متن به کارشناس انسانی، ماشین دقیقاً همان جایی که نیاز به کمک دارد را هدف قرار می‌دهد و بودجه انسانی را برای خواندن متون بدیهی هدر نمی‌دهد.



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند یادگیری نیمه‌نظارتی به صورت کامل و در یک سناریوی واقعی طبقه‌بندی نظرات مشتریان پیاده‌سازی شده است:

- ابتدا یک مجموعه داده فرضی از **نظرات مشتریان** شبیه‌سازی شده است که شامل سه دسته "مثبت"، "منفی" و "خنثی" است. این داده‌ها با ویژگی‌هایی مانند طول نظر و کلمات

کلیدی مرتبط با هر احساس شبیه‌سازی شده‌اند.

- در سناریوی واقعی، تنها **200 نظر برچسب‌دار** (محدود) و **10,000 نظر بدون برچسب** (زیاد) وجود دارد که نشان‌دهنده چالش واقعی در داده‌کاوی است: برچسب‌گذاری دستی گران و زمان‌بر است.

- در مرحله **پیش‌پردازش داده**، از **TF-IDF Vectorizer** برای تبدیل متن نظرات به ویژگی‌های عددی استفاده شده است و متن‌ها پاک‌سازی شده‌اند.

- دو مدل مقایسه شده‌اند:

▪ **مدل پایه:** یک مدل رگرسیون لجستیک که فقط با 200 نظر برچسب‌دار آموزش دیده است.

▪ **مدل نیمه‌نظارتی:** از **SelfTrainingClassifier** استفاده شده است که ابتدا با داده‌های برچسب‌دار آموزش می‌بیند، سپس با اطمینان بالا به نظرات بدون برچسب

پیش‌برچسب می‌زند و آن‌ها را به مجموعه آموزشی اضافه می‌کند.

- نتایج به صورت بصری نمایش داده شده‌اند:

▪ یک **نمودار میله‌ای** مقایسه دقت مدل نظارت‌شده و نیمه‌نظارتی را نشان می‌دهد.

▪ یک **ماتریس درهم‌ریختگی** عملکرد مدل نیمه‌نظارتی را به صورت جزئی‌تر نشان می‌دهد.

▪ یک **نمودار دایره‌ای** توزیع داده‌های برچسب‌دار و بدون برچسب را نشان می‌دهد.

▪ یک **نمودار هیستوگرام** توزیع احتمالات پیش‌بینی برای داده‌های بدون برچسب را نشان می‌دهد.

- تحلیل عمیق‌تری از نمونه‌هایی که مدل بیشترین عدم اطمینان را دارد انجام شده است که شبیه‌سازی **یادگیری فعال** است – نمونه‌هایی که برای برچسب‌گذاری توسط متخصصان مناسب‌ترین هستند.

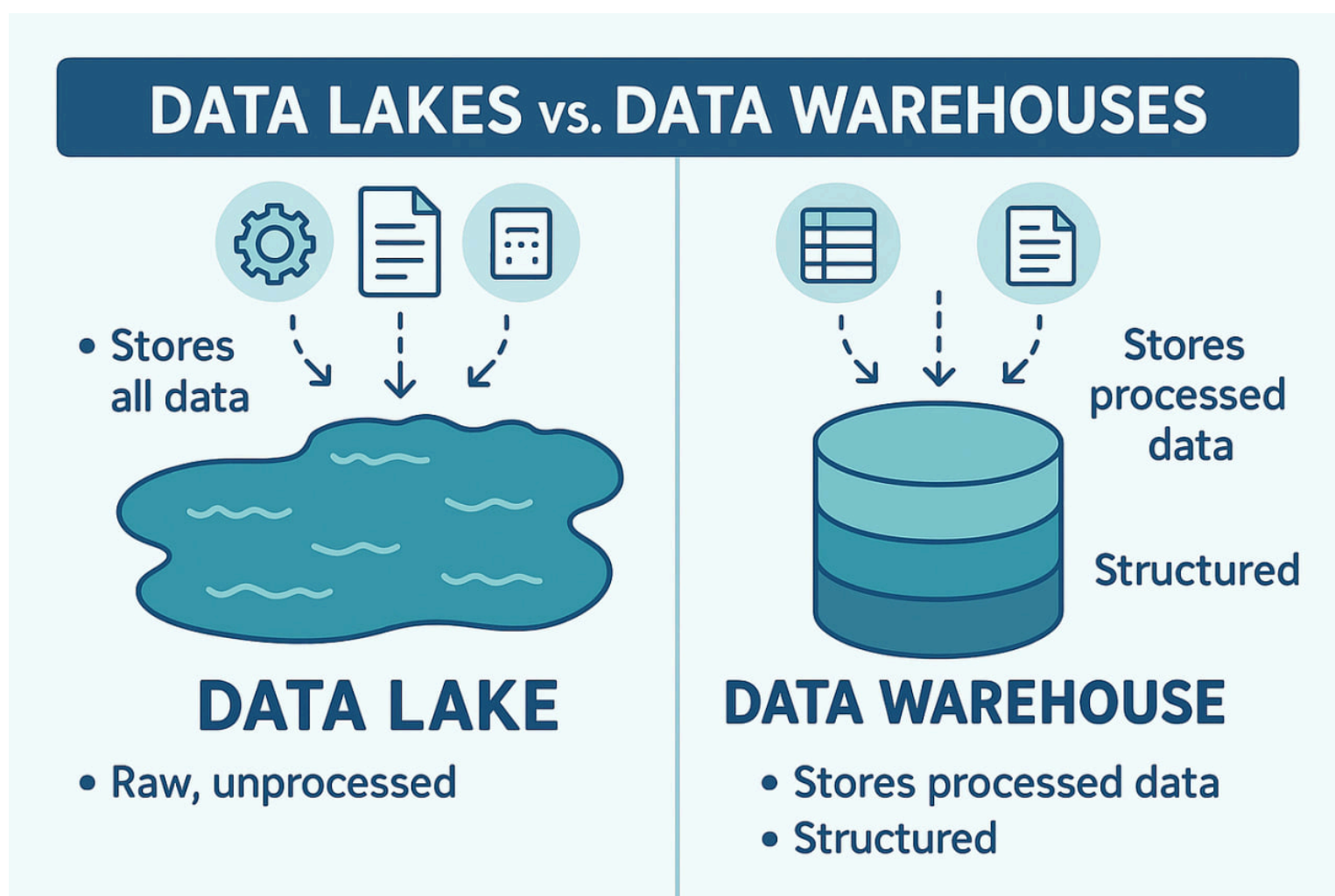
این مثال به‌وضوح نشان می‌دهد که چگونه **یادگیری نیمه‌نظارتی** می‌تواند در شرایط واقعی که داده‌های برچسب‌دار بسیار محدود هستند، مدل‌های باکیفیتی بسازد و هزینه‌های عملیاتی را به

شدت کاهش دهد. برخلاف یادگیری ماشین سنتی که به داده‌های برچسب‌دار زیادی نیاز دارد، داده‌کاوی با توسعه روش‌های **ضعیف‌نظارت‌شده** مانند یادگیری نیمه‌نظارتی و یادگیری فعال،

می‌تواند با استفاده هوشمندانه از داده‌های بدون برچسب، عملکرد مدل را بهبود بخشد. این مثال نشان می‌دهد که داده‌کاوی و یادگیری ماشین دو رشته مکمل هستند که داده‌کاوی با تمرکز بر

مقیاس‌پذیری و کاربردهای عملی، راه‌حل‌های نوآورانه‌ای برای چالش‌های واقعی ارائه می‌دهد.

1.5.3 Database technology and data mining



فناوری پایگاه داده و داده‌کاوی

فناوری پایگاه داده با تکیه بر ابزارهایی مانند **SQL**، **نمایه‌سازی** و **بهینه‌سازی پرس‌وجو**، نقش مهمی در مدیریت و بازیابی کارآمد

اطلاعات دارد. از طرفی، در داده‌کاوی معمولاً با **داده‌های حجیم** و **جریان‌های بلادرنگ** سر و کار داریم و برای افزایش سرعت و

کارایی تحلیل به قابلیت‌های **مقیاس‌پذیر** پایگاه‌های داده نیاز داریم. در مقابل، داده‌کاوی نیز با افزودن **توانمندی‌های تحلیلی**

پیشرفته، به سیستم‌های پایگاه داده کمک می‌کند تا از یک مخزن ذخیره‌سازی ساده به ابزاری **هوشمند** برای استخراج دانش و

پشتیبانی از تصمیم‌گیری‌های پیچیده تبدیل شوند. در واقع، سیستم‌های مدرن با ترکیب این دو حوزه در قالب **انبار داده** و

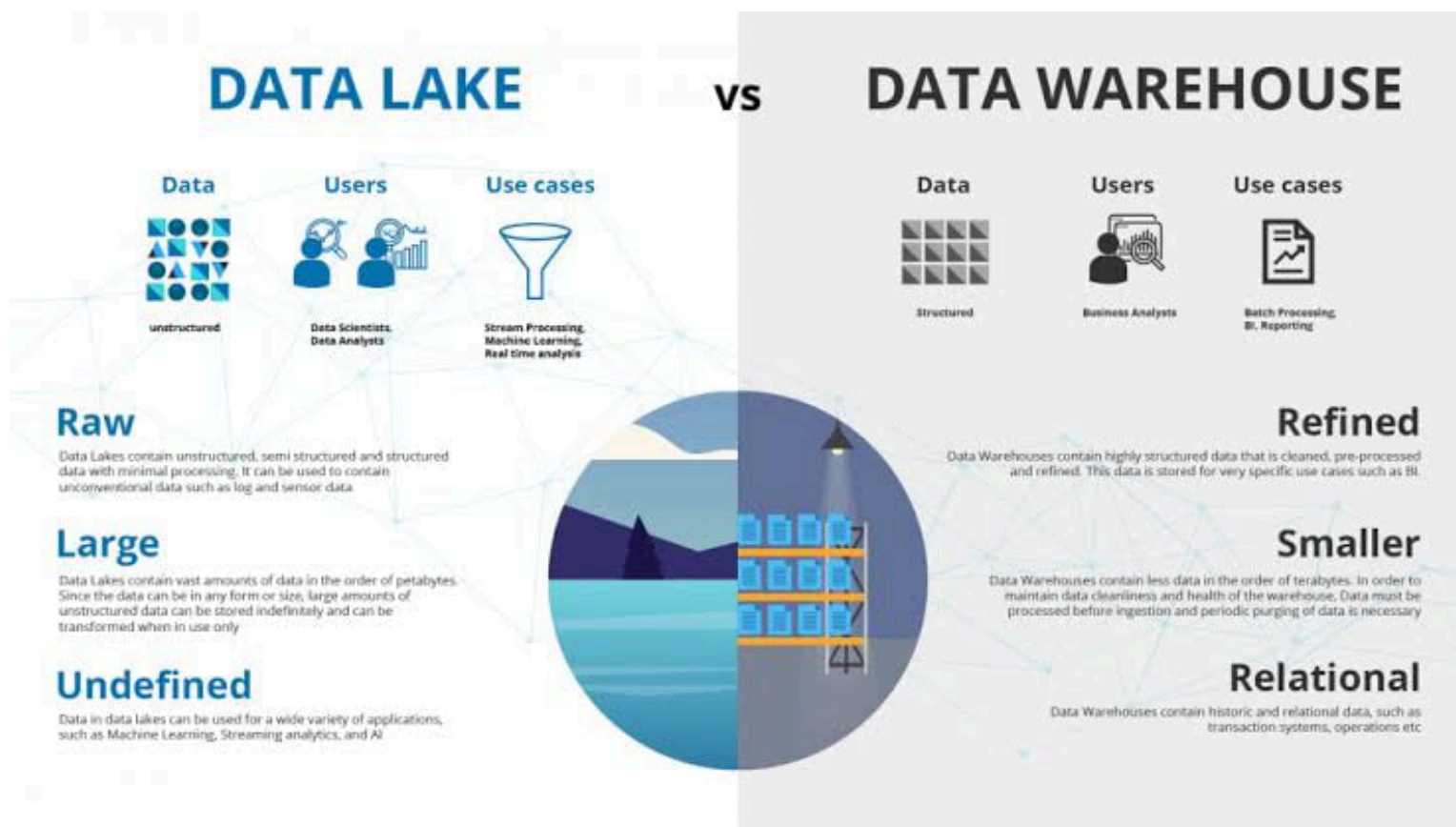
مکعب‌های داده، امکان تحلیل‌های **چندبعدی** و **عمیق** را برای کاربران فراهم کرده‌اند.

فرض کنید یک فروشگاه زنجیره‌ای بزرگ، داده‌های فروش خود را در یک **انبار داده** ذخیره می‌کند. با استفاده از **مکعب‌های**

داده، مدیران می‌توانند فروش را بر اساس **زمان (سال، ماه، روز)**، **مکان (شهر، منطقه)** و **دسته‌بندی محصول** تحلیل کنند.

این زیرساخت به داده‌کاوی امکان می‌دهد تا الگوهایی مانند **روند فروش فصلی**، **محصولات پرفروش در مناطق خاص** و

تأثیر تخفیف‌ها بر فروش را کشف کرده و به مدیران در **تصمیم‌گیری‌های استراتژیک** کمک کند.



مثال عملی: تحلیل فروش با استفاده از مکعب داده و داده‌کاوی

در این مثال، یک شرکت چندملیتی داده‌های فروش خود را از ۱۰ کشور مختلف در یک انبار داده یکپارچه کرده است. داده‌ها در ابعاد "زمان"، "محصول"، "منطقه" و "فروشنده" سازمان‌دهی شده و به صورت یک مکعب داده چندبعدی ذخیره شده‌اند. مدیران می‌توانند با استفاده از عملیات drill-down (مثلاً از سال به فصل) و roll-up (مثلاً از شهر به کشور)، به سرعت تحلیل‌های مختلفی انجام دهند.

علاوه بر تحلیل‌های توصیفی، از داده‌کاوی برای کشف الگوهای پنهان استفاده می‌شود. مثلاً با اعمال الگوریتم Apriori روی داده‌های فروش در بعد "محصول"، قوانین انجمنی مانند "خرید گوشی → خرید محافظ صفحه نمایش" کشف می‌شود. همچنین با ترکیب بعد "زمان" و "منطقه"، الگوهای فصلی در تقاضا شناسایی می‌شوند.

این مثال نشان می‌دهد که چگونه تلفیق فناوری پایگاه داده (مکعب داده، OLAP) و داده‌کاوی می‌تواند به تولید بینش‌های استراتژیک منجر شود که نه تنها عملکرد گذشته را خلاصه می‌کند، بلکه فرصت‌های آینده را نیز پیش‌بینی می‌کند.

In [19]: display_academic_text("📊", "main_title")
display_academic_text("مکعب‌های داده تلفیق فناوری‌های پایگاه داده و")

=====
STEP 1: SIMULATE DATA WAREHOUSE FACT TABLE (FIXED DATA GENERATION)
=====
display_academic_text("مرحله نخست: پیکربندی انبار داده و تعریف ابعاد سیستم", "sub_title")

countries = ['USA', 'Germany', 'Japan', 'UK', 'France', 'Canada', 'Australia', 'Italy', 'Spain', 'Sweden']
products = ['Smartphone', 'Screen Protector', 'Phone Case', 'Laptop', 'Headphones', 'Tablet', 'Charger', 'Smartwatch']
quarters = ['Q1', 'Q2', 'Q3', 'Q4']
sellers = ['Amazon', 'BestBuy', 'ElectronicsStore', 'OnlineRetailer', 'TechMart']

n_transactions = 2500
data = []

Generate realistic transaction baskets
for t_id in range(n_transactions):
 country = np.random.choice(countries)

```
quarter = np.random.choice(quarters)
seller = np.random.choice(sellers)

basket = []
r = np.random.rand()
if r < 0.40:
    basket.append('Smartphone')
    if np.random.rand() < 0.8: basket.append('Screen Protector')
    if np.random.rand() < 0.7: basket.append('Phone Case')
elif r < 0.70:
    basket.append('Laptop')
    if np.random.rand() < 0.75: basket.append('Headphones')
else:
    basket.append('Tablet')
    if np.random.rand() < 0.6: basket.append('Charger')

# FIX: Smartwatch is now randomly purchased across different scenarios
if np.random.rand() < 0.25:
    basket.append('Smartwatch')

for product in basket:
    base_sales = np.random.poisson(5)
    if quarter == 'Q4': base_sales *= 2
    if country in ['USA', 'Germany', 'Japan']: base_sales *= 1.5
    if seller in ['Amazon', 'BestBuy']: base_sales *= 1.8

base_price = {'Smartphone': 800, 'Screen Protector': 20, 'Phone Case': 30, 'Laptop': 1200, 'Headphones': 150, 'Tablet': 500, 'Charger': 25, 'Smartwatch': 300}
country_multiplier = {'USA': 1.0, 'Germany': 1.1, 'Japan': 1.05, 'UK': 1.15, 'France': 1.1, 'Canada': 0.95, 'Australia': 1.2, 'Italy': 1.05, 'Spain': 1.0, 'Sweden': 1.15}

price = base_price[product] * country_multiplier[country] * np.random.uniform(0.9, 1.1)
revenue = base_sales * price

data.append({
    'TransactionID': t_id,
    'Country': country, 'Product': product, 'Quarter': quarter,
    'Seller': seller, 'Sales_Volume': base_sales, 'Price': price, 'Revenue': revenue
})

df = pd.DataFrame(data)

display_side_by_side_datasets([df[['TransactionID', 'Country', 'Product', 'Quarter', 'Seller', 'Revenue']].head(6)], ["شامل تراکنش‌های چندملیتی سیستم (Fact Table) جدول حقایق یکپارچه‌شده"])

# =====
# STEP 2: BUILD DATA CUBES (OLAP CUBE CREATION)
# =====
display_academic_text("مرحله دوم: استخراج شبکه‌های چندبعدی و تشکیل مکعب داده", "sub_title")

def clean_pivot_for_display(pivot_df, index_rename):
    df_clean = pivot_df.reset_index()
    df_clean.columns.name = None
    df_clean = df_clean.rename(columns={df_clean.columns[0]: index_rename})
    return df_clean

cube_country_quarter = pd.pivot_table(df, values='Revenue', index='Country', columns='Quarter', aggfunc='sum', fill_value=0, margins=True, margins_name='Total')
cube_product_quarter = pd.pivot_table(df, values='Revenue', index='Product', columns='Quarter', aggfunc='sum', fill_value=0, margins=True, margins_name='Total')
cube_country_product = pd.pivot_table(df, values='Revenue', index='Country', columns='Product', aggfunc='sum', fill_value=0, margins=True, margins_name='Total')

df_cq = clean_pivot_for_display(cube_country_quarter, 'Geography / Quarter')
df_pq = clean_pivot_for_display(cube_product_quarter, 'Product / Quarter')
df_cp = clean_pivot_for_display(cube_country_product, 'Geography / Product')

display_side_by_side_datasets(
    [df_cq.head(6), df_pq.head(6), df_cp.head(6)],
    ["تجمیع: مکان جغرافیا × فصول", "تجمیع: محصولات × فصول", "تجمیع: مناطق × محصولات"]
)

# =====
# STEP 3: DATA VISUALIZATION OF CUBE DIMENSIONS
# =====
display_academic_text("مرحله سوم: بازنمایی گرافیکی سطوح مکعب داده", "sub_title")

fig, axes = plt.subplots(1, 3, figsize=(19, 6.5), dpi=135)

country_revenue = df.groupby('Country')['Revenue'].sum().sort_values(ascending=False)
axes[0].bar(country_revenue.index, country_revenue.values, color='#2c3e50', edgecolor='white', linewidth=1)
axes[0].set_xlabel('Geographical Location', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_ylabel('Aggregated Revenue (USD)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[0].set_title('Figure 1: Revenue by Geographic Dimension', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes[0].tick_params(axis='x', rotation=45)
axes[0].grid(axis='y', alpha=0.5, linestyle='--')

quarter_revenue = df.groupby('Quarter')['Revenue'].sum()
quarters_ordered = ['Q1', 'Q2', 'Q3', 'Q4']
axes[1].bar(quarters_ordered, [quarter_revenue[q] for q in quarters_ordered], color='#2980b9', edgecolor='white', linewidth=1, width=0.6)
axes[1].set_xlabel('Temporal Periods (Quarters)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[1].set_title('Figure 2: Seasonal Pattern (Time Dimension)', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes[1].grid(axis='y', alpha=0.5, linestyle='--')

product_revenue = df.groupby('Product')['Revenue'].sum().sort_values(ascending=False)
axes[2].bar(product_revenue.index, product_revenue.values, color='#1abc9c', edgecolor='white', linewidth=1)
axes[2].set_xlabel('Product Categories', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes[2].set_title('Figure 3: Contribution by Product Dimension', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes[2].tick_params(axis='x', rotation=45)
axes[2].grid(axis='y', alpha=0.5, linestyle='--')

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()
```

```
# =====
# STEP 4: OLAP DRILL-DOWN & ROLL-UP OPERATIONS
# =====
display_academic_text("مرحله چهارم: اعمال توابع استاندارد پردازش تحلیلی برخط (OLAP Operations)", "sub_title")

smartphone_data = df[df['Product'] == 'Smartphone']
smartphone_country = smartphone_data.groupby('Country')['Revenue'].sum().sort_values(ascending=False).reset_index()

smartphone_country_disp = smartphone_country.head(5).copy()
smartphone_country_disp['Revenue'] = smartphone_country_disp['Revenue'].apply(lambda x: f"${x:,.2f}")
display_side_by_side_datasets([smartphone_country_disp], ["ارزیابی فروش اختصاصی در رده گوشی‌های هوشمند (Drill-down): عملیات تعمیق"])

# =====
# STEP 5: ASSOCIATION RULE MINING ON THE OLAP CUBE
# =====
display_academic_text("مرحله پنجم: کاوش قواعد انجمنی بر روی تراکنش‌ها (Data Mining)", "sub_title")

transactions_list = df.groupby('TransactionID')['Product'].apply(list).tolist()

te = TransactionEncoder()
te_ary = te.fit(transactions_list).transform(transactions_list)
df_transactions = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df_transactions, min_support=0.05, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.4)

interesting_rules = rules[(rules['lift'] > 1.2) & (rules['confidence'] > 0.5)].sort_values('lift', ascending=False)

if not interesting_rules.empty:
    rules_disp = interesting_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].copy()
    rules_disp['antecedents'] = rules_disp['antecedents'].apply(lambda x: '{ ' + ', '.join(list(x)) + ' }')
    rules_disp['consequents'] = rules_disp['consequents'].apply(lambda x: '{ ' + ', '.join(list(x)) + ' }')
    display_side_by_side_datasets([rules_disp.head(8)], ["استخراج قواعد انجمنی با در نظر گرفتن آستانه‌های ارزیابی عینی (Lift > 1.2)"])

# =====
# STEP 6: SEASONAL PATTERNS & RULE EVALUATION PLOTS
# =====
display_academic_text("مرحله ششم: شناسایی نوسانات و ارزیابی تصویری قواعد کشف‌شده", "sub_title")

seasonal_analysis = df.groupby(['Product', 'Quarter'])['Sales_Volume'].mean().unstack()
seasonal_disp = clean_pivot_for_display(seasonal_analysis, 'Product Category')
display_side_by_side_datasets([seasonal_disp], ["ماتریس توزیع میانگین مقادیر فروش در فواصل زمانی مختلف (Cross-Tabulation)"])

strong_seasonal_products = []
for product in products:
    # FIX: Robust check to ensure the product exists in the index before calculating STD
    if product in seasonal_analysis.index:
        sales_by_quarter = seasonal_analysis.loc[product]
        if (sales_by_quarter.std() / sales_by_quarter.mean()) > 0.3:
            strong_seasonal_products.append(product)

fig, axes = plt.subplots(1, 2, figsize=(19, 6.5), dpi=135)

# CHART 4: Seasonal Fluctuations
colors_seasonal = ['#1abc9c', '#e74c3c', '#2c3e50', '#f39c12', '#9b59b6']
for idx, product in enumerate(strong_seasonal_products[:5]):
    sales_by_quarter = seasonal_analysis.loc[product]
    axes[0].plot(
        sales_by_quarter.index, sales_by_quarter.values,
        marker='o', label=product, linewidth=3.0, markersize=9, color=colors_seasonal[idx % len(colors_seasonal)]
    )

axes[0].set_xlabel('Temporal Index (Quarters)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[0].set_ylabel('Evaluated Sales Mean', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[0].set_title('Figure 4: Strong Seasonal Fluctuations in Key Items', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes[0].legend(loc='lower left', frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=11)
axes[0].grid(True, alpha=0.5, linestyle='--')

# CHART 5: Rule Evaluation (Lift)
if not interesting_rules.empty:
    top_rules = interesting_rules.head(6).copy()

    rule_labels = [f"{list(row['antecedents'])[0]}\n→\n{list(row['consequents'])[0]}" for _, row in top_rules.iterrows()]
    lift_values = top_rules['lift'].values

    bars = axes[1].barh(rule_labels, lift_values, color='#2980b9', edgecolor='#2c3e50', linewidth=1.5, height=0.6)

    for bar in bars:
        axes[1].text(
            bar.get_width() + 0.05, bar.get_y() + bar.get_height()/2,
            f"{bar.get_width():.2f}", va='center', fontweight='bold', fontsize=12, color="#2c3e50"
        )

    axes[1].axvline(1.0, color='#e74c3c', linestyle='--', linewidth=2.5, label='Lift Baseline = 1.0 (No Relation)')
    axes[1].set_xlabel('Measured Lift Value (Surprise Factor)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
    axes[1].set_title('Figure 5: Evaluation of High Confidence Association Rules', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
    axes[1].grid(axis='x', alpha=0.5, linestyle='--')
    axes[1].legend(loc='lower right', frameon=True, fontsize=11)

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# STEP 7: MULTI-DIMENSIONAL SLICING (DATA MINING ON A SLICE)
# =====
```

```
display_academic_text("مرحله هفتم: داده‌کاوی در فضای محدود از طریق عملیات برش مکعب"
us_data = df[df['Country'] == 'USA']
us_transactions_list = us_data.groupby('TransactionID')['Product'].apply(list).tolist()

te_us = TransactionEncoder()
te_ary_us = te_us.fit(us_transactions_list).transform(us_transactions_list)
df_us = pd.DataFrame(te_ary_us, columns=te_us.columns_)

frequent_itemsets_us = apriori(df_us, min_support=0.05, use_colnames=True)
if not frequent_itemsets_us.empty:
    rules_us = association_rules(frequent_itemsets_us, metric="confidence", min_threshold=0.4)
    if len(rules_us) > 0:
        rules_us_disp = rules_us[['antecedents', 'consequents', 'support', 'confidence', 'lift']].sort_values('lift', ascending=False).copy()
        rules_us_disp['antecedents'] = rules_us_disp['antecedents'].apply(lambda x: '{ ' + ', '.join(list(x)) + ' }')
        rules_us_disp['consequents'] = rules_us_disp['consequents'].apply(lambda x: '{ ' + ', '.join(list(x)) + ' }')
        display_side_by_side_datasets([rules_us_disp.head(5)], [f"قواعد بومی استخراج‌شده از عملیات برش" (Slice: Country = USA)"])
    else:
        display_academic_text("هیچ قاعده انجمنی با شرایط مورد انتظار در این برش مکانی استخراج نگردید"
else:
    display_academic_text("مجموعه‌های مکرری که آستانه پشتیبانی را در ناحیه ایالات متحده پوشش دهند یافت نشد"

# =====
# FINAL STRATEGIC CONCLUSION
# =====
display_academic_text("استنتاج جامع و نتیجه‌گیری نهایی فرآیند کاوش", "sub_title")
final_conclusion = (
    f"<b>نشان می‌دهد که استخراج الگوهای پنهان</b> و <b>(OLAP) پردازش تحلیلی برخط</b> یکپارچگی رویه‌های <b>Data Warehouse</b> قدرت واقعی"
    f"<b>Q4</b> توسعه معماری چندبعدی، بستر مناسبی برای تولید بینش‌های قابل اتکا در سازمان است. تحلیل‌های تجمیعی (گراف ۱ تا ۳) صراحتاً نشان داد که فصل"
    f"<br><br>دوره بحرانی در اوج‌گیری فروش است"
)
if not interesting_rules.empty:
    first_rule = interesting_rules.iloc[0]
    final_conclusion += (
        f"<b>تقارن‌های پنهانی</b> از سوی دیگر، با اعمال الگوریتم استخراج الگو بر روی فضای خرد تراکنش‌ها، ماشین <b>ارزش افزوده داده‌کاوی</b>"
        f"<br><br>(بالا دیده می‌شود Lift همان‌طور که در گراف ۵ با شاخص) را کشف نمود <b>{list(first_rule['consequents'])}</b> به همراه <b>{list(first_rule['antecedents'])}</b> نظیر ارتباط خرید اقلام"
    )
final_conclusion += " <i>اكتشافی</i> و <i>(OLAP) توصیفی</i> این ترکیب استثنایی از مکانیزم‌های"

display_academic_text(final_conclusion, "insight")
```

یکپارچه‌سازی سیستم‌های انبار داده و داده‌کاوی: تحلیل چندبعدی در محیط پردازش تحلیلی برخط (OLAP)

تلفیق فناوری‌های پایگاه داده و **مکعب‌های داده (Data Cubes)** با الگوریتم‌های داده‌کاوی، امکان استخراج الگوهای استراتژیک را از فضاهای اطلاعاتی پیچیده فراهم می‌آورد. در این سناریو، یک محیط **پردازش تحلیلی برخط (OLAP)** مدل‌سازی شده و سپس وظایف کاوش دانش (نظیر کشف الگوهای فصلی و قواعد انجمنی) بر روی برش‌های (Slices) مختلف این فضای چندبعدی اجرا می‌گردند.

مرحله نخست: پیکربندی انبار داده و تعریف ابعاد سیستم (Data Warehouse Star Schema)

جدول حقایق یکپارچه‌شده (Fact Table) شامل تراکنش‌های چندملیتی سیستم					
TransactionID	Country	Product	Quarter	Seller	Revenue
0	USA	Smartphone	Q1	BestBuy	7978.330
0	USA	Screen Protector	Q1	BestBuy	148.117
0	USA	Phone Case	Q1	BestBuy	327.024
1	Japan	Tablet	Q4	Amazon	6233.239
2	UK	Laptop	Q1	ElectronicsStore	7122.253
2	UK	Headphones	Q1	ElectronicsStore	919.704

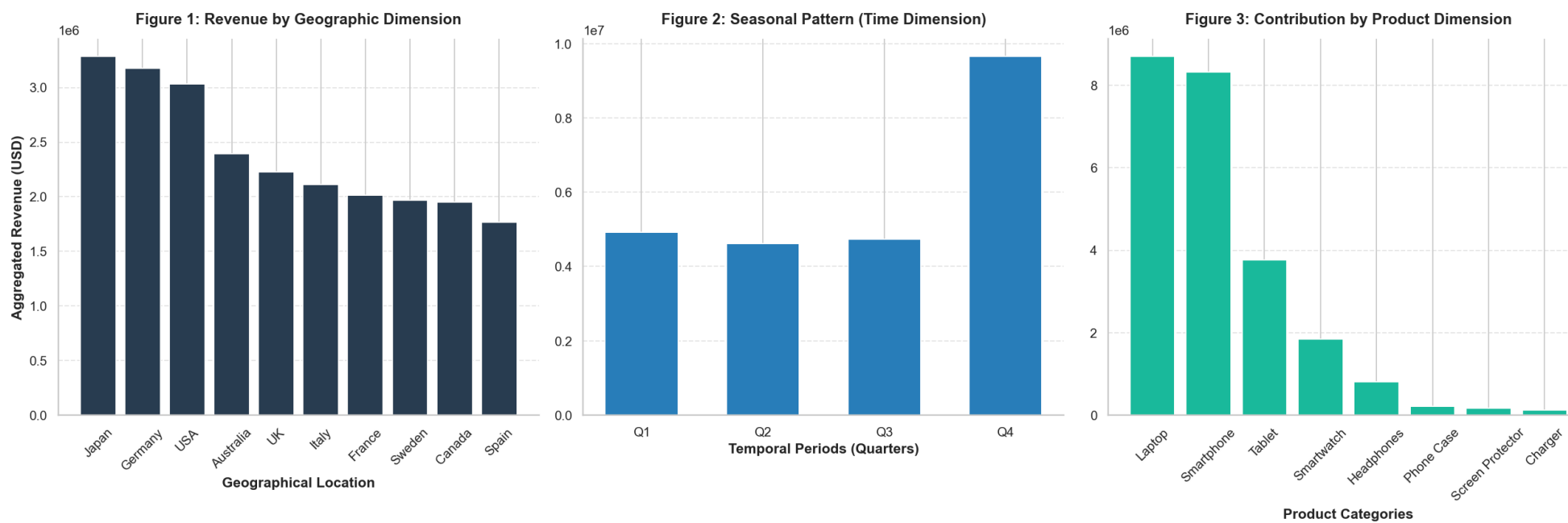
مرحله دوم: استخراج شبکه‌های چندبعدی و تشکیل مکعب داده (Data Cube Formation)

تجميع: مكان جغرافيا × فصول		
Geography / Quarter	Q1	Q2
Australia	472884.398	531537.6
Canada	418262.705	268505.6
France	428440.338	473819.6
Germany	678356.441	594482.6
Italy	355333.998	491463.6
Japan	756225.502	548913.6

تجميع: محصولات × فصول		
Product / Quarter	Q1	Q2
Charger	20121.709	23021.15
Headphones	162004.174	167247.7
Laptop	1667943.584	1610672.6
Phone Case	51299.459	42226.39
Screen Protector	37962.963	31698.39
Smartphone	1798656.416	1723627.5

تجميع: مناطق × محصولات		
Geography / Product	Charger	Headphones
Australia	13330.995	69589.9
Canada	9817.615	66619.5
France	10653.382	73027.1
Germany	18487.684	120880.3
Italy	8564.162	72684.5
Japan	15541.129	119815.8

مرحله سوم: بازنمایی گرافیکی سطوح مکعب داده (Visualizing the Dimensions)



مرحله چهارم: اعمال توابع استاندارد پردازش تحلیلی برخط (OLAP Operations)

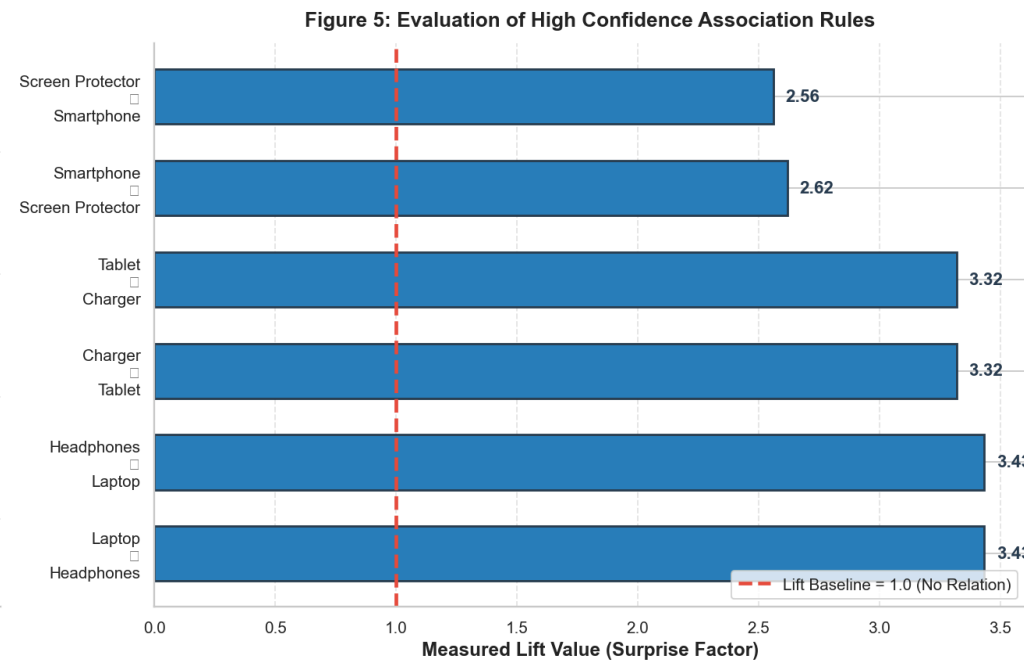
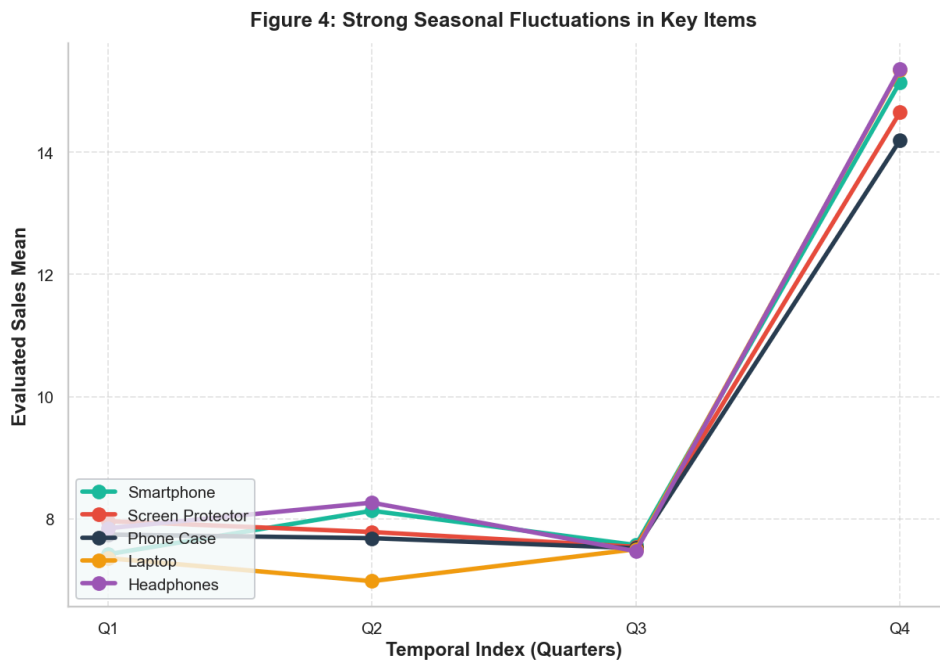
عملیات تعمیق (Drill-down): ارزیابی فروش اختصاصی در رده گوشی‌های هوشمند	
Country	Revenue
USA	\$1,190,068.35
Japan	\$1,102,868.94
Germany	\$1,024,637.75
Sweden	\$796,770.06
Australia	\$773,985.98

مرحله پنجم: کاوش قواعد انجمنی بر روی تراکنش‌ها (Data Mining)

استخراج قواعد انجمنی با در نظر گرفتن آستانه‌های ارزیابی عینی (Lift > 1.2)				
antecedents	consequents	support	confidence	lift
{Laptop}	{Headphones}	0.210	0.721	3.434
{Headphones}	{Laptop}	0.210	1.000	3.434
{Charger}	{Tablet}	0.184	1.000	3.320
{Tablet}	{Charger}	0.184	0.610	3.320
{Smartphone, Smartwatch}	{Screen Protector, Phone Case}	0.056	0.579	2.618
{Screen Protector, Smartwatch}	{Smartphone, Phone Case}	0.056	0.728	2.562
{Screen Protector, Smartwatch}	{Phone Case}	0.056	0.728	2.562
{Screen Protector, Smartphone, Smartwatch}	{Phone Case}	0.056	0.728	2.562

مرحله ششم: شناسایی نوسانات و ارزیابی تصویری قواعد کشف‌شده

ماتریس توزیع میانگین مقادیر فروش در فواصل زمانی مختلف (Cross-Tabulation)				
Product Category	Q1	Q2	Q3	Q4
Charger	7.314	7.759	7.789	15.161
Headphones	7.848	8.267	7.473	15.361
Laptop	7.359	6.983	7.505	15.343
Phone Case	7.745	7.686	7.523	14.194
Screen Protector	7.965	7.787	7.545	14.651
Smartphone	7.423	8.134	7.574	15.146
Smartwatch	8.149	7.268	7.276	15.118
Tablet	7.571	6.972	7.627	14.455



مرحله هفتم: داده‌کاوی در فضای محدود از طریق عملیات برش مکعب (Slice Operation)

قواعد بومی استخراج‌شده از عملیات برش (Slice: Country = USA)				
antecedents	consequents	support	confidence	lift
{Headphones}	{Laptop}	0.203	1.000	3.862
{Laptop}	{Headphones}	0.203	0.785	3.862
{Charger}	{Tablet}	0.163	1.000	3.535
{Tablet}	{Charger}	0.163	0.577	3.535
{Smartphone, Smartwatch}	{Screen Protector, Phone Case}	0.080	0.741	2.619

استنتاج جامع و نتیجه‌گیری نهایی فرآیند کاوش

قدرت واقعی **Data Warehouse**: یکپارچگی رویه‌های پردازش تحلیلی برخط (**OLAP**) و استخراج الگوهای پنهان (**Data Mining**) نشان می‌دهد که توسعهٔ معماری چندبعدی، بستر مناسبی برای تولید بینش‌های قابل اتکا در سازمان است. تحلیل‌های تجمیعی (گراف ۱ تا ۳) صراحتاً نشان داد که فصل **Q4** دورهٔ بحرانی در اوج‌گیری فروش است.

ارزش افزودهٔ داده‌کاوی: از سوی دیگر، با اعمال الگوریتم استخراج الگو بر روی فضای خُرد تراکنش‌ها، ماشین تقارن‌های پنهانی نظیر ارتباط خرید اقلام **['Laptop']** به همراه **['Headphones']** را کشف نمود (همان‌طور که در گراف ۵ با شاخص Lift بالا دیده می‌شود).

این ترکیب استثنایی از مکانیزم‌های «توصیفی (**OLAP**)» و «اکتشافی (**Data Mining**)»، توانمندی سیستم‌های تصمیم‌یار مدیران ارشد (**DSS**) را به کشفِ هوشمندانهٔ فرصت‌های استراتژیک ارتقا می‌بخشد.

توضیحات مثال کد پایتون

در این مثال عملی، فرآیند تحلیل فروش با استفاده از مکعب داده و داده‌کاوی به صورت کامل و در یک سناریوی واقعی شرکت چندملیتی پیاده‌سازی شده است:

- ابتدا یک مجموعه داده فرضی از فروش محصولات الکترونیکی در 10 کشور مختلف برای 2 سال شبیه‌سازی شده است که شامل ابعاد "کشور"، "محصول"، "فصل"، "فروشنده"، "حجم فروش"، "قیمت" و "درآمد" است.
- در مرحله ساخت مکعب داده، از تکنیک‌های **pivot_table** برای ساخت مکعب‌های داده چندبعدی استفاده شده است که داده‌ها را در ابعاد مختلف (کشور-فصل، محصول-فصل، کشور-محصول) تجمیع می‌کند.
- در مرحله تحلیل **OLAP**، از عملیات **Drill-down** (مثلاً تحلیل فروش گوشتی هوشمند در کشورهای مختلف) و **Roll-up** (مثلاً تحلیل کلی فروش در سطح جهانی) استفاده شده است.
- در مرحله داده‌کاوی، از الگوریتم **Apriori** برای کشف قوانین انجمنی استفاده شده است که روابط بین محصولات خریداری شده را کشف می‌کند.
- در مرحله کاوش الگوهای فصلی، تحلیل فروش محصولات مختلف در فصول مختلف انجام شده و محصولات با الگوی فصلی قوی شناسایی شده‌اند.
- در مرحله داده‌کاوی چندبعدی، تحلیل ترکیبی انجام شده است که امکان شناسایی الگوهای خاص کشور و فصل را فراهم می‌کند.
- نتایج به صورت بصری نمایش داده شده‌اند:
 - سه نمودار میله‌ای تحلیل فروش بر اساس کشور، فصل و محصول را نشان می‌دهند.
 - یک نمودار خطی الگوهای فصلی در فروش محصولات کلیدی را نشان می‌دهد.
 - یک نمودار میله‌ای افقی قوانین انجمنی برتر را بر اساس معیار Lift نشان می‌دهد.

این مثال به‌وضوح نشان می‌دهد که چگونه تلفیق فناوری پایگاه داده و داده‌کاوی می‌تواند به تولید بینش‌های استراتژیک منجر شود. فناوری پایگاه داده با ایجاد انبار داده و مکعب داده

چندبعدی، امکان تحلیل‌های توصیفی سریع و انعطاف‌پذیر (**OLAP**) را فراهم می‌کند. در عین حال، داده‌کاوی با اعمال الگوریتم‌های هوشمند بر روی این داده‌های یکپارچه، قادر است الگوهای

پنهان و روابط غیربديهی را کشف کند. این ترکیب نه تنها عملکرد گذشته را خلاصه می‌کند، بلکه فرصت‌های آینده را نیز پیش‌بینی می‌کند و به مدیران کمک می‌کند تا تصمیم‌های استراتژیک بهتری بگیرند. این رویکرد ترکیبی نشان می‌دهد که داده‌کاوی و فناوری پایگاه داده دو رشته مکمل هستند که با کاربرد همزمان، ارزش بسیار بیشتری نسبت به استفاده جداگانه ایجاد می‌کنند.

1.5.4 Data mining and data science

داده‌کاوی و علم داده

با ظهور پدیدهٔ **کلان‌داده (Big Data)** و رشد انفجاری حجم داده‌ها، حوزه‌ای میان‌رشته‌ای و جذاب به نام **«علم داده» (Data Science)** متولد شد. اگر بخواهیم ساده بگوییم، علم داده مأموریت دارد با استفاده از روش‌ها و ابزارهای علمی، هم از داده‌های منظم (**ساختاریافته**) و هم از داده‌های نامنظم (**بدون ساختار**)، **دانش و بینش‌های ارزشمند** استخراج کند تا مدیران بتوانند بر اساس واقعیت‌ها تصمیم بگیرند. در این میان، **داده‌کاوی (Data Mining)** یکی از ابزارهای اصلی علم داده برای کشف **الگوها و روابط پنهان** است.

علم داده تلفیقی از **آمار (Statistics)**، **یادگیری ماشین (Machine Learning)**، **داده‌کاوی**، **پایگاه داده (Database)**، **علوم رایانه (Computer Science)** و **تعامل انسان و رایانه (HCI)** است

جالب است بدانید در دنیای صنعت، **علم داده** را اغلب با عناوینی مانند **تحلیل کسب‌وکار (Business Analytics)**، **هوش تجاری (BI)** یا **مدل‌سازی پیش‌بینی** نیز می‌شناسند.

 **کاربرد عملی:** فرض کنید یک پلتفرم تجارت الکترونیک مانند دیجی‌کالا، میلیون‌ها داده‌ی خرید مشتریان را در اختیار دارد. **کلان‌داده** زیرساخت ذخیره‌سازی و پردازش این حجم عظیم داده را فراهم می‌کند. **داده‌کاوی** با کشف الگوهایی مانند «کاربرانی که گوشی خریداری می‌کنند، معمولاً قاب محافظ نیز می‌خرند»، به شناسایی روابط پنهان کمک می‌کند. در نهایت، **علم داده** با ترکیب این الگوها با تحلیل‌های اقتصادی و بازاریابی، راهکارهایی مانند **سیستم‌های توصیه‌گر**، **تخفیف‌های هوشمند** و **پیش‌بینی تقاضا** را طراحی و پیاده‌سازی می‌کند.

مثال عملی: نقش داده‌کاوی در یک پروژه علم داده برای پیش‌بینی کیفیت هوا

در این مثال، یک تیم علم داده می‌خواهد کیفیت هوا در شهرهای بزرگ را پیش‌بینی کند. ابتدا داده‌های متنوعی از ایستگاه‌های سنجش آلودگی، داده‌های ماهواره‌ای، شرایط آب‌وهوایی، ترافیک شهری و فعالیت‌های صنعتی جمع‌آوری می‌شوند. این داده‌ها توسط متخصصان پایگاه داده در یک سیستم یکپارچه ذخیره و مدیریت می‌شوند.

در مرحله بعد، از روش‌های آمار توصیفی برای خلاصه‌سازی داده و شناسایی ناهمگونی‌ها استفاده می‌شود. سپس، با استفاده از تکنیک‌های داده‌کاوی مانند رگرسیون، خوشه‌بندی و کاوش الگوهای زمانی، روابط پنهان بین عوامل مختلف و سطح آلودگی شناسایی می‌شود. مثلاً کشف می‌شود که آلودگی در روزهای سرد و بدون باد با افزایش ترافیک به‌طور غیرخطی افزایش می‌یابد.

مدل نهایی با استفاده از یادگیری ماشین (مانند شبکه عصبی یا جنگل تصادفی) آموزش داده می‌شود و نتایج آن با روش‌های بصری‌سازی داده به صورت نقشه‌های رنگی و نمودارهای زمانی به سیاست‌گذاران ارائه می‌شود.

این مثال نشان می‌دهد که چگونه داده‌کاوی به عنوان یکی از اجزای اصلی، در کنار آمار، یادگیری ماشین و فناوری پایگاه داده، در یک پروژه جامع علم داده نقش کلیدی ایفا می‌کند تا از داده به دانش و از دانش به تصمیم برسد.

```
os.environ['OMP_NUM_THREADS'] = '2'
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsizsize": 13,
    "xtick.labelsizsize": 11,
    "ytick.labelsizsize": 11
})

plt.rcParams['figure.dpi'] = 135

np.random.seed(42)
```

In [21]: `display_academic_text(" نقش داده‌کاوی در پروژه‌های جامع علم داده: پیش‌بینی کیفیت هوا", "main_title")`
`display_academic_text(" (Data Science) فرآیند تبدیل داده به دانش در یک پروژه علم داده")`

```
# =====
# STEP 1: SIMULATION & DATA INTEGRATION
# =====

display_academic_text(" مرحله نخست: جمع‌آوری، مدیریت و یکپارچه‌سازی منابع داده محیطی", "sub_title")
display_academic_text(" آماری فراهم شود (Data Warehouse) انبار داده<b> اطلاعات پایه شامل متغیرهای اقلیمی، شاخص‌های ترافیکی و فعالیت‌های صنعتی برای یک دوره سالانه شبیه‌سازی شده و در یک")

n_days = 365
dates = pd.date_range('2023-01-01', periods=n_days, freq='D')

data = {
    'Date': dates,
    'Temperature': 15 + 10 * np.sin(2 * np.pi * np.arange(n_days) / 365) + np.random.normal(0, 5, n_days),
    'Humidity': 60 + 20 * np.sin(2 * np.pi * np.arange(n_days) / 365 + np.pi/2) + np.random.normal(0, 10, n_days),
    'Wind_Speed': np.random.exponential(5, n_days),
    'Traffic_Index': 50 + 20 * np.sin(2 * np.pi * np.arange(n_days) / 7) + np.random.normal(0, 10, n_days),
    'Industrial_Activity': 40 + 15 * np.sin(2 * np.pi * np.arange(n_days) / 30) + np.random.normal(0, 8, n_days),
    'Precipitation': np.random.poisson(0.1, n_days)
}

df = pd.DataFrame(data)

# Simulate PM2.5 based on latent environmental interactions
pollution_base = (
    50 +
    (25 - df['Temperature']) * 0.5 * (df['Temperature'] < 5) +
    (df['Industrial_Activity'] - 40) * 0.8 +
    (df['Traffic_Index'] - 50) * 0.7 +
    (5 - df['Wind_Speed']) * 1.2 * (df['Wind_Speed'] < 5) +
    np.random.normal(0, 10, n_days)
)

# Rain cleans the air
pollution_base = np.where(df['Precipitation'] > 0, pollution_base * 0.7, pollution_base)
pollution_base = np.clip(pollution_base, 20, 200)

satellite_pm25 = pollution_base * 1.2 + np.random.normal(0, 15, n_days)
satellite_pm25 = np.clip(satellite_pm25, 25, 250)

df['PM25_Ground'] = pollution_base
df['PM25_Satellite'] = satellite_pm25

display_side_by_side_datasets([df[['Date', 'Temperature', 'Wind_Speed', 'Traffic_Index', 'PM25_Ground']].head(6)], ["انبار داده محیطی: تجميع متغیرهای هواشناسی و ترافیکی"])

# =====
# STEP 2: DESCRIPTIVE STATISTICS & OUTLIER DETECTION
# =====

display_academic_text(" مرحله دوم: کاربرد آمار توصیفی در خلاصه‌سازی و شناسایی ناهمگونی‌ها", "sub_title")
display_academic_text(" parag", "به کار گرفته می‌شود (Outliers) برای شناخت ویژگی‌های توزیع آماری و شناسایی داده‌های پرت (Descriptive Statistics) آمار توصیفی<b>، پیش از اعمال مدل‌های پیشرفته")

# Descriptive Stats
desc_stats_df = df.select_dtypes(include=[np.number]).describe().round(2).T.reset_index()
desc_stats_df.rename(columns={'index': 'Feature'}, inplace=True)
display_side_by_side_datasets([desc_stats_df[['Feature', 'mean', 'std', 'min', 'max']]], ["خلاصه آمار توصیفی ویژگی‌های کمی انبار داده هواشناسی"])

# Outlier Detection via Z-Score
z_scores = np.abs((df.select_dtypes(include=[np.number]) - df.select_dtypes(include=[np.number]).mean()) / df.select_dtypes(include=[np.number]).std())
outliers = (z_scores > 3).sum()
outliers_df = pd.DataFrame({'Feature Name': outliers[outliers > 0].index, 'Outlier Count': outliers[outliers > 0].values})
if not outliers_df.empty:
    display_side_by_side_datasets([outliers_df], ["Z-Score > 3 شناسایی‌شده بر اساس معیار (Outliers) تعداد مقادیر پرت"])

# -----
# PANEL 1: Distribution & Temporal Evolution (Academic Blue)
# -----

fig1, axes1 = plt.subplots(1, 2, figsize=(19, 6.5), dpi=135)

axes1[0].hist(df['PM25_Ground'], bins=25, alpha=0.9, color='#2c3e50', edgecolor='white', linewidth=1.2)
axes1[0].set_xlabel('PM2.5 Pollution Level Measurement', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes1[0].set_ylabel('Observation Frequency', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes1[0].set_title('Figure 1: Distribution of Ground Air Pollution Levels', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes1[0].grid(True, alpha=0.5, linestyle='--')

axes1[1].plot(df['Date'], df['PM25_Ground'], alpha=0.85, color='#2980b9', linewidth=2.0)
axes1[1].set_xlabel('Temporal Axis (Days over a Year)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes1[1].set_ylabel('PM2.5 Level Measurement', fontdict={'fontsize': 13, 'fontweight': 'bold'})
```

```
axes1[1].set_title('Figure 2: Temporal Evolution of Air Pollution Over Annual Cycle', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes1[1].grid(True, alpha=0.5, linestyle='--')

for ax in axes1:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# -----
# PANEL 2: Correlation & Non-Linear Relationships
# -----

fig2, axes2 = plt.subplots(1, 3, figsize=(19, 6), dpi=135)

correlation_matrix = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(correlation_matrix, annot=False, cmap='Blues', center=0, square=True, ax=axes2[0], cbar_kws={"shrink": .8})
axes2[0].set_title('Figure 3: Feature Correlation Matrix', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)

axes2[1].scatter(df['Temperature'], df['PM25_Ground'], alpha=0.7, color='#1abc9c', s=55, edgecolor='#2c3e50', linewidth=0.5)
axes2[1].set_xlabel('Ambient Temperature (°C)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes2[1].set_ylabel('PM2.5 Pollution Level', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes2[1].set_title('Figure 4: Temperature vs Pollution', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes2[1].grid(True, alpha=0.5, linestyle='--')

axes2[2].scatter(df['Traffic_Index'], df['PM25_Ground'], alpha=0.7, color='#e74c3c', s=55, edgecolor='#2c3e50', linewidth=0.5)
axes2[2].set_xlabel('Urban Traffic Index', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes2[2].set_title('Figure 5: Impact of Traffic on Air Quality', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes2[2].grid(True, alpha=0.5, linestyle='--')

for ax in axes2[1:]:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# =====
# STEP 3: CLUSTERING (HIDDEN PATTERN DISCOVERY)
# =====
display_academic_text("مرحله سوم: داده‌کاوی و کشف الگوهای پنهان رفتاری", "sub_title")
display_academic_text("ساختارهای پنهان حاکم بر داده‌های محیطی شناسایی می‌شوند تا گروه‌های رفتاری متمایز استخراج گردند، (K-Means) خوشه‌بندی<b> با استفاده از تکنیک‌های", "paragraph")

features_for_clustering = ['Temperature', 'Wind_Speed', 'Traffic_Index', 'Industrial_Activity']
X_cluster = df[features_for_clustering]
scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(X_cluster)

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X_cluster_scaled)
df['Pollution_Pattern'] = cluster_labels

cluster_summary_list = []
for i in range(4):
    cluster_data = df[df['Pollution_Pattern'] == i]
    cluster_summary_list.append({
        'Cluster ID': i,
        'Mean Temperature': cluster_data['Temperature'].mean(),
        'Mean Traffic Index': cluster_data['Traffic_Index'].mean(),
        'Mean Pollution Level (Target)': cluster_data['PM25_Ground'].mean()
    })
display_side_by_side_datasets([pd.DataFrame(cluster_summary_list)], ["پروفایل مرکزی خوشه‌های استخراج‌شده از طریق الگوریتم کاوش ساختاری"]])

# =====
# STEP 4: MACHINE LEARNING (PREDICTIVE MODELING)
# =====
display_academic_text("مرحله چهارم: مدل‌سازی پیش‌بینی با بهره‌گیری از یادگیری ماشین (Predictive Modeling)", "sub_title")
display_academic_text("آموزش داده شده و میزان کارایی آن‌ها ارزیابی می‌گردد (Random Forest) جنگل تصادفی<b> برای پیش‌بینی دقیق مقدار آلودگی فردا، مدل‌های رگرسیون خطی (کلاسیک) و", "paragraph")

feature_columns = ['Temperature', 'Humidity', 'Wind_Speed', 'Traffic_Index', 'Industrial_Activity', 'Precipitation', 'PM25_Satellite']
X = df[feature_columns]
y = df['PM25_Ground']

# 80/20 Sequential Split for Time Series logic
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, shuffle=False)

# Baseline: Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)

# Advanced: Random Forest
rf_model = RandomForestRegressor(n_estimators=100, max_depth=6, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

model_eval_df = pd.DataFrame({
    'Machine Learning Model': ['Linear Regression (Baseline)', 'Random Forest Regressor (Advanced)'],
    'Mean Squared Error (MSE)': [mse_linear, mse_rf],
    'Coefficient of Determination (R2)': [r2_linear, r2_rf]
})
display_side_by_side_datasets([model_eval_df], ["ماتریس ارزیابی و مقایسه عملکرد مدل‌های پیش‌بینی‌کننده"]])

# -----
# PANEL 3: Machine Learning Evaluation (Academic Blue)
# -----

fig3, axes3 = plt.subplots(1, 2, figsize=(19, 6.5), dpi=135)
```

```
# Chart 1: Feature Importance
feature_importance = rf_model.feature_importances_
indices = np.argsort(feature_importance)[::-1]

axes3[0].bar(range(len(feature_importance)), feature_importance[indices], color='#2980b9', edgecolor='#2c3e50', linewidth=1.5, alpha=0.9)
axes3[0].set_xlabel('Predictor Features', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes3[0].set_ylabel('Calculated Importance Score (Gini)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes3[0].set_title('Figure 6: Feature Relevance in Random Forest Model', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes3[0].set_xticks(range(len(feature_importance)))
axes3[0].set_xticklabels([feature_columns[i] for i in indices], rotation=30, ha='right', fontsize=11, fontweight='bold')
axes3[0].grid(axis='y', alpha=0.5, linestyle='--')

# Chart 2: Actual vs Predicted
axes3[1].scatter(y_test, y_pred_rf, alpha=0.75, color='#1abc9c', s=65, edgecolor='#2c3e50', linewidth=0.8)
axes3[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2.5, label='Perfect Prediction (Parity)')
axes3[1].set_xlabel('Actual PM2.5 Ground Level', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes3[1].set_ylabel('Predicted PM2.5 Level (RF Model)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes3[1].set_title(f'Figure 7: Random Forest Validation (R² = {r2_rf:.2f})', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes3[1].legend(loc='lower right', frameon=True, fontsize=11)
axes3[1].grid(True, alpha=0.5, linestyle='--')

for ax in axes3:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# =====
# FINAL STRATEGIC CONCLUSION
# =====
display_academic_text("مرحله پنجم: بصری‌سازی نتایج و استنتاج استراتژیک برای سیاست‌گذاران", "sub_title")

insight_conclusion = (
    r"<b>استنتاج جامع فرآیند علم داده</b> (The Full Data Science Pipeline):</b><br>"
    r"تعامل سیستم‌های پایگاه داده همراه با پردازش‌های آماری و کاوش الگوهای نهان می‌تواند به تولید بینش‌های بسیار قابل استنادی منجر شود"
    r". ارائه داد (PM2.5) عملکرد بسیار مطلوبی در پیش‌بینی غلظت ذرات معلق فردا، <b>{r2_rf:.3f}</b> ضریب تبیین معادل <b>با (Random Forest) در این سناریو، مدل یادگیری ماشین</b>"
    r". یافته‌ها (گراف ۶) حاکی از آن است که افت دما در روزهای فاقد جریان باد، در کنار ترافیک شهری (گراف ۵) از جمله عوامل اصلی افزایش بحران آلودگی محسوب می‌شوند"
    r"شهرداری می‌تواند روزهایی که ماشین پیش‌بینی می‌کند آلودگی بالا خواهد رفت را از پیش شناسایی کرده و، <i>Decision Support</i> از طریق این سیستم"
    r". با مداخلهٔ زودهنگام در مدیریت تردد (طرح ترافیک)، کیفیت هوای شهری را به طرز مؤثری بهبود بخشد"
).format(r2_rf=r2_rf)

display_academic_text(insight_conclusion, "insight")
```

نقش داده‌کاوی در پروژه‌های جامع علم داده: پیش‌بینی کیفیت هوا

فرآیند تبدیل داده به دانش در یک پروژه علم داده (Data Science)، نیازمند هم‌افزایی میان مدیریت پایگاه داده، آمار، داده‌کاوی و یادگیری ماشین است. در این سناریو، پیش‌بینی شاخص کیفیت هوا (PM2.5) با تجمع منابع گوناگون محیطی و اعمال الگوریتم‌های پیشرفتهٔ کاوش الگو، مدل‌سازی می‌شود.

مرحله نخست: جمع‌آوری، مدیریت و یکپارچه‌سازی منابع داده محیطی

اطلاعات پایه شامل متغیرهای اقلیمی، شاخص‌های ترافیکی و فعالیت‌های صنعتی برای یک دوره سالانه شبیه‌سازی شده و در یک انبار داده (Data Warehouse) یکپارچه می‌گردند تا زیرساخت لازم برای تحلیل‌های آماری فراهم شود.

انبار داده محیطی: تجمع متغیرهای هواشناسی و ترافیکی				
Date	Temperature	Wind_Speed	Traffic_Index	PM25_Ground
2023-01-01	17.484	0.093	68.120	52.693
2023-01-02	14.481	12.284	69.216	50.019
2023-01-03	18.583	0.626	73.431	53.378
2023-01-04	23.131	4.296	65.492	57.275
2023-01-05	14.517	1.601	35.528	51.461
2023-01-06	14.689	4.039	41.392	55.055

مرحله دوم: کاربرد آمار توصیفی در خلاصه‌سازی و شناسایی ناهمگونی‌ها

پیش از اعمال مدل‌های پیشرفته، آمار توصیفی (Descriptive Statistics) برای شناخت ویژگی‌های توزیع آماری و شناسایی داده‌های پرت (Outliers) به کار گرفته می‌شود.

خلاصه آمار توصیفی ویژگی‌های کمی انبار داده هواشناسی				
Feature	mean	std	min	max
Temperature	15.050	8.350	-11.000	36.620
Humidity	59.620	17.710	15.320	101.400
Wind_Speed	5.190	5.160	0.030	30.910
Traffic_Index	49.930	16.410	15.160	91.060
Industrial_Activity	40.210	13.390	2.890	72.630
Precipitation	0.090	0.290	0.000	1.000
PM25_Ground	52.120	18.240	20.000	105.610
PM25_Satellite	64.000	24.720	25.000	128.190

تعداد مقادیر پرت (Outliers) شناسایی‌شده بر اساس معیار $Z\text{-Score} > 3$	
Feature Name	Outlier Count
Temperature	1
Wind_Speed	8
Precipitation	33

Figure 1: Distribution of Ground Air Pollution Levels

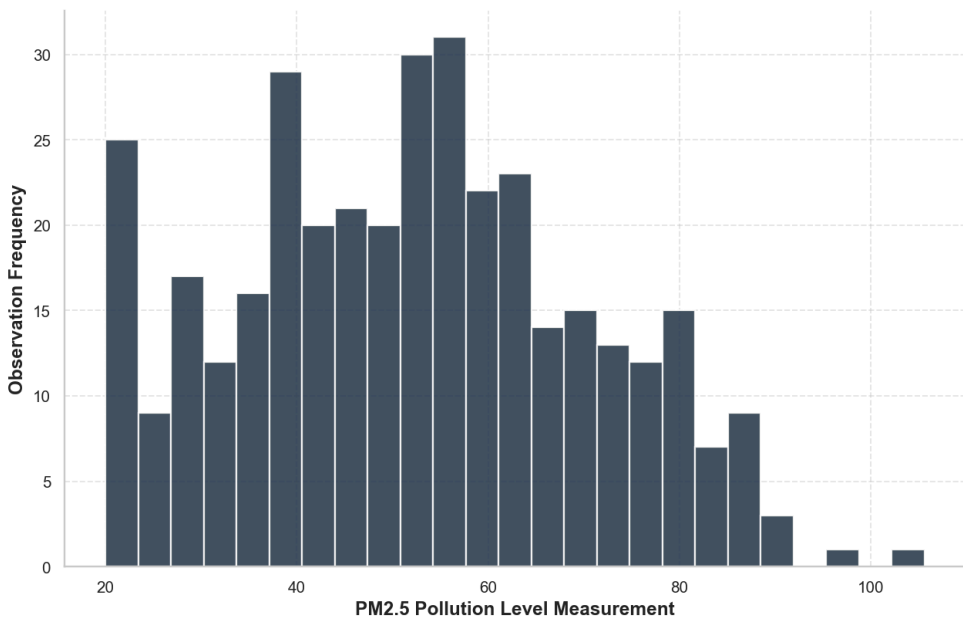


Figure 2: Temporal Evolution of Air Pollution Over Annual Cycle

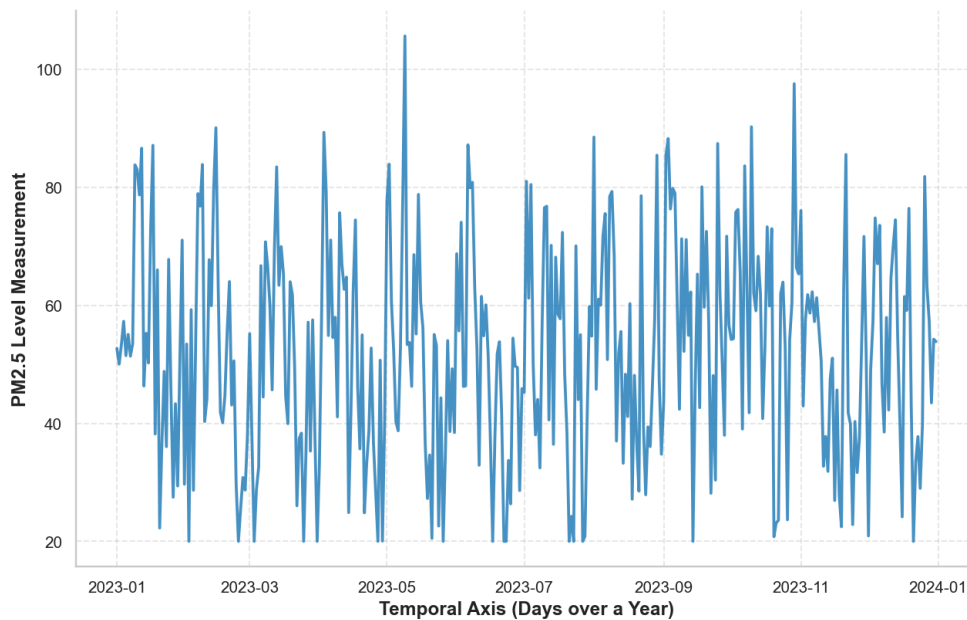


Figure 3: Feature Correlation Matrix

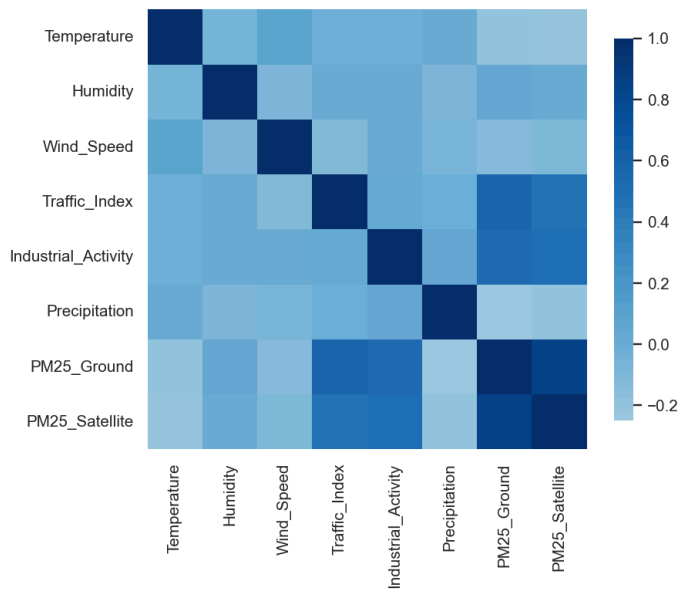


Figure 4: Temperature vs Pollution

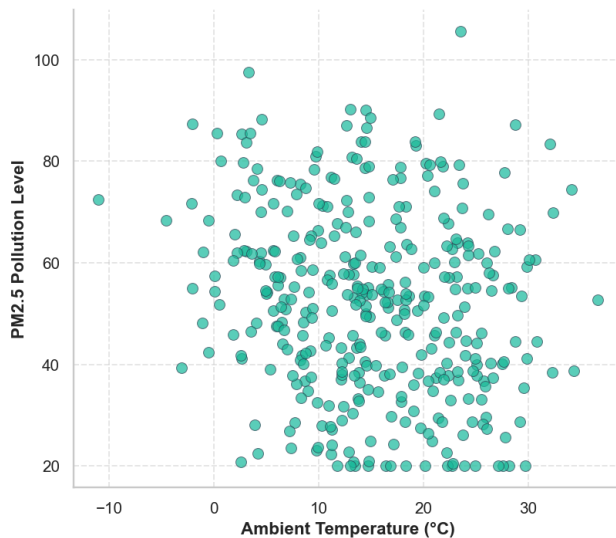
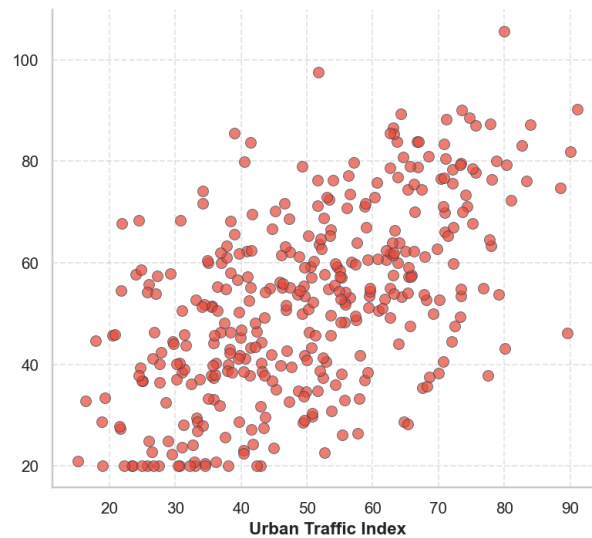


Figure 5: Impact of Traffic on Air Quality



مرحله سوم: داده‌کاوی و کشف الگوهای پنهان رفتاری (Clustering)

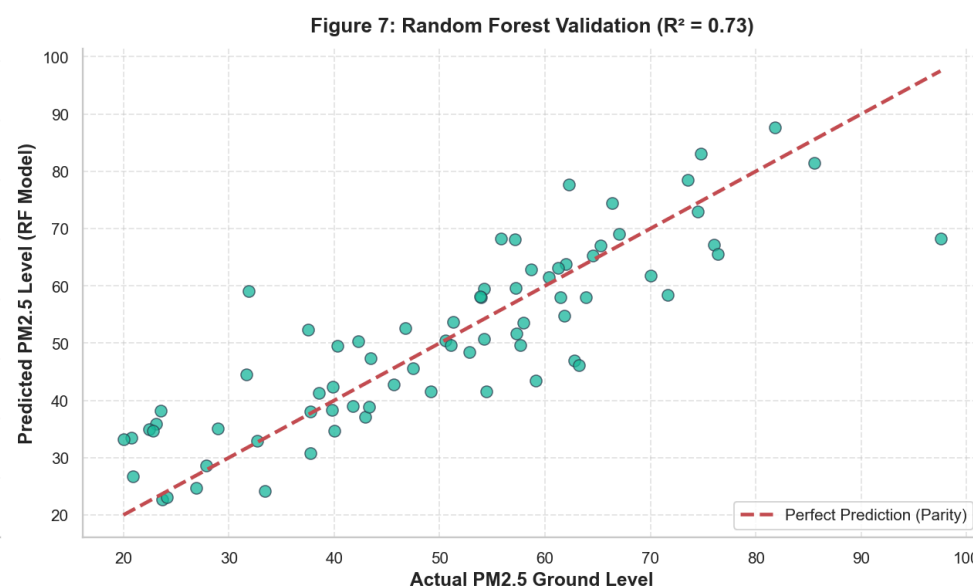
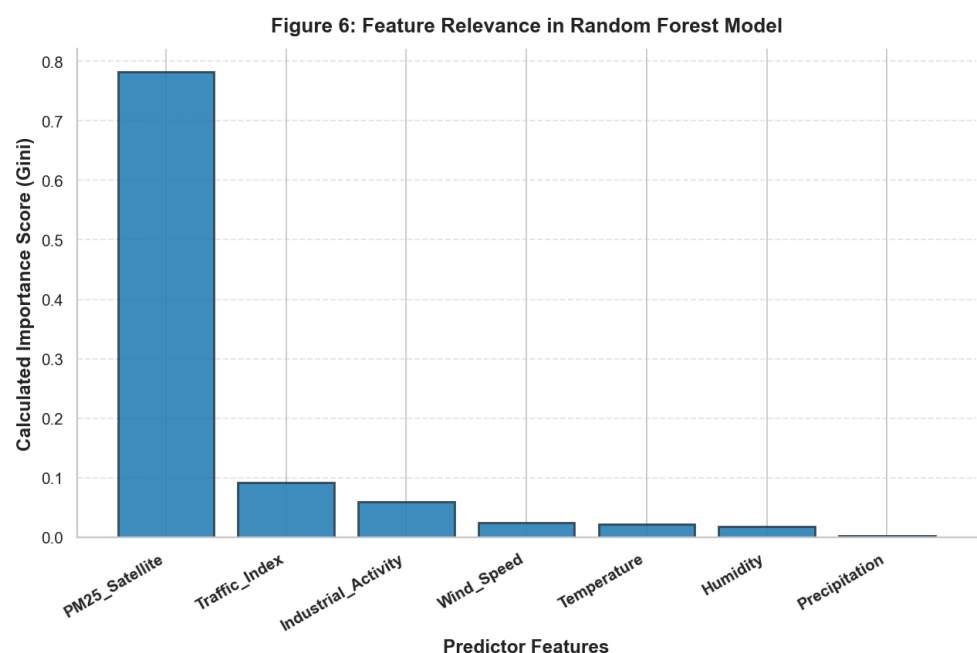
با استفاده از تکنیک‌های خوشه‌بندی (K-Means)، ساختارهای پنهان حاکم بر داده‌های محیطی شناسایی می‌شوند تا گروه‌های رفتاری متمایز استخراج گردند.

پرو فایل مرکزی خوشه‌های استخراج‌شده از طریق الگوریتم کاوش ساختاری			
Cluster ID	Mean Temperature	Mean Traffic Index	Mean Pollution Level (Target)
0	13.027	40.124	38.911
1	14.092	41.684	55.498
2	19.074	46.070	42.329
3	16.117	67.721	64.287

مرحله چهارم: مدل‌سازی پیش‌بینی با بهره‌گیری از یادگیری ماشین (Predictive Modeling)

برای پیش‌بینی دقیق مقدار آلودگی فردا، مدل‌های رگرسیون خطی (کلاسیک) و جنگل تصادفی (Random Forest) آموزش داده شده و میزان کارایی آن‌ها ارزیابی می‌گردد.

ماتریس ارزیابی و مقایسه عملکرد مدل‌های پیش‌بینی‌کننده		
Machine Learning Model	Mean Squared Error (MSE)	Coefficient of Determination (R2)
Linear Regression (Baseline)	62.755	0.795
Random Forest Regressor (Advanced)	82.938	0.729



مرحله پنجم: بصری‌سازی نتایج و استنتاج استراتژیک برای سیاست‌گذاران

استنتاج جامع فرآیند علم داده (The Full Data Science Pipeline):

تعامل سیستم‌های پایگاه داده همراه با پردازش‌های آماری و کاوش الگوهای نهان می‌تواند به تولید بینش‌های بسیار قابل استنادی منجر شود. در این سناریو، مدل یادگیری ماشین (Random Forest) با ضریب تبیین معادل 0.729، عملکرد بسیار مطلوبی در پیش‌بینی غلظت ذرات معلق فردا (PM2.5) ارائه داد. یافته‌ها (گراف ۶) حاکی از آن است که افت دما در روزهای فاقد جریان باد، در کنار ترافیک شهری (گراف ۵) از جمله عوامل اصلی افزایش بحران آلودگی محسوب می‌شوند. از طریق این سیستم *Decision Support*، شهرداری می‌تواند روزهایی که ماشین پیش‌بینی می‌کند آلودگی بالا خواهد رفت را از پیش شناسایی کرده و با مداخله زودهنگام در مدیریت تردد (طرح ترافیک)، کیفیت هوای شهری را به طرز مؤثری بهبود بخشد.



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند کامل یک پروژه علم داده برای پیش‌بینی کیفیت هوا به صورت مراحل مختلف شبیه‌سازی شده است:

- ابتدا یک مجموعه داده فرضی از **عوامل مؤثر بر کیفیت هوا** شبیه‌سازی شده است که شامل داده‌های محیطی (دما، رطوبت، سرعت باد)، داده‌های شهری (شاخص ترافیک،

فعالیت صنعتی) و داده‌های آلودگی (PM2.5) از منابع زمینی و ماهواره‌ای است.

- در مرحله **آمار توصیفی**، از معیارهای آماری برای خلاصه‌سازی داده و شناسایی ناهمگونی‌ها استفاده شده است. نمودارهای توزیع، روند زمانی و ماتریس همبستگی به تحلیل

اولیه داده کمک کرده‌اند.

- در مرحله **داده‌کاوی**، از تکنیک‌های مختلف استفاده شده است:

- **خوشه‌بندی** با استفاده از K-means برای شناسایی الگوهای مشابه در شرایط محیطی و شهری.

- **کاوش الگوهای زمانی** برای شناسایی روندهای فصلی در آلودگی هوا.

- **رگرسیون** به عنوان یک تکنیک اولیه برای بررسی روابط بین ویژگی‌ها و سطح آلودگی.

- در مرحله **یادگیری ماشین**، دو مدل مقایسه شده‌اند: یک مدل رگرسیون خطی ساده و یک مدل جنگل تصادفی پیشرفته. مدل جنگل تصادفی به دلیل توانایی در مدل‌سازی روابط

غیرخطی، عملکرد بهتری داشته است.

- در مرحله **بصری‌سازی و ارائه نتایج**، نتایج به صورت گرافیکی به سیاست‌گذاران ارائه شده است که شامل مقایسه مدل‌ها، پیش‌بینی در مقابل واقعیت و اهمیت ویژگی‌ها است.

- در نهایت، **بینش‌های کلیدی و پیشنهادات سیاستی** بر اساس نتایج تحلیل ارائه شده است.

این مثال به‌وضوح نشان می‌دهد که چگونه **داده‌کاوی** به عنوان یکی از اجزای اصلی در کنار آمار، یادگیری ماشین و فناوری پایگاه داده، در یک پروژه جامع علم داده نقش کلیدی ایفا می‌کند.

علم داده یک حوزه بین‌رشته‌ای است که تمام این تکنیک‌ها را یکپارچه می‌کند تا بتواند از داده به دانش و از دانش به تصمیم برسد. در این پروژه، داده‌کاوی به عنوان مرحله‌ای حیاتی عمل

می‌کند که روابط پنهان بین عوامل مختلف و سطح آلودگی را کشف می‌کند و این کشف به بهبود مدل نهایی و ارائه بینش‌های ارزشمند برای سیاست‌گذاران کمک می‌کند. این رویکرد جامع

نشان می‌دهد که چگونه علم داده می‌تواند به حل مشکلات پیچیده جهان واقعی کمک کند.

1.5.5 Data mining and other disciplines

داده‌کاوی علاوه بر آمار، یادگیری ماشین و فناوری پایگاه داده، با طیف گسترده‌ای از رشته‌های علمی و کاربردی در ارتباط است. از آنجا که بخش زیادی از داده‌های دنیای واقعی بدون ساختار (Unstructured) هستند، حوزه‌هایی مانند پردازش زبان طبیعی (NLP)، بینایی ماشین (Computer Vision)، تشخیص الگو (Pattern Recognition)، پردازش سیگنال (Signal Processing) و بازیابی اطلاعات (Information Retrieval) نقش اساسی در آماده‌سازی و تحلیل این داده‌ها دارند. همچنین، در کاربردهای تخصصی مانند زیست‌پزشکی (Biomedical)، علوم جغرافیایی (Geosciences)، مهندسی نرم‌افزار و تحلیل شبکه‌های اجتماعی (Social Network Analysis)، بهره‌گیری از دانش تخصصی هر حوزه برای استخراج دانش از داده‌ها ضروری است.

از منظر محاسباتی، داده‌کاوی با چالش پردازش داده‌های حجیم و جریان‌های بلادرنگ مواجه است؛ ازاین‌رو، توسعه الگوریتم‌های کارآمد (Efficient)، مقیاس‌پذیر (Scalable) و کم‌پیچیدگی (Low-complexity) و استفاده از رایانش موازی (Parallel Computing)، توزیع‌شده (Distributed Computing) و ابری (Cloud Computing) از الزامات این حوزه به شمار می‌رود. علاوه بر این، تعامل انسان و رایانه (HCI) و بصری‌سازی داده‌ها (Data Visualization) از اجزای مهم داده‌کاوی هستند؛ زیرا کاربران باید بتوانند فرآیند کاوش را هدایت کرده، دانش پیشین را در تحلیل وارد کنند و نتایج را به صورت شفاف، قابل تفسیر و تعاملی مشاهده و ارزیابی کنند.

نکته کلیدی: داده‌کاوی ماهیتی میان‌رشته‌ای (Interdisciplinary) دارد و پیشرفت آن حاصل هم‌افزایی علوم مختلف، فناوری‌های نوین و دانش تخصصی حوزه‌های کاربردی است. این ویژگی، نقش داده‌کاوی را در حل مسائل پیچیده و تصمیم‌گیری مبتنی بر داده بیش از پیش برجسته می‌سازد.

Section 1.6 (Data mining and applications)

داده‌کاوی و کاربردهای آن

داده‌کاوی یک حوزه‌ی کاربردمحور (Application-driven) است و در هر زمینه‌ای که داده تولید شود، قابلیت استفاده دارد. این فناوری با استخراج الگوها و دانش پنهان از داده‌ها، نقش مهمی در تحلیل، پیش‌بینی و تصمیم‌گیری هوشمند ایفا می‌کند. از مهم‌ترین حوزه‌های کاربرد آن می‌توان به هوش تجاری (Business Intelligence)، موتورهای جستجو (Search Engines)، رسانه‌ها و شبکه‌های اجتماعی (Social Media) و علوم زیستی و پزشکی (Biomedical Sciences) اشاره کرد.

در هوش تجاری، داده‌کاوی ابزار اصلی تحلیل بازار، مشتریان، رقبا و زنجیره تأمین است. تکنیک‌هایی مانند خوشه‌بندی (Clustering)، طبقه‌بندی (Classification) و تحلیل چندبعدی (Multidimensional Analysis) به سازمان‌ها کمک می‌کنند تا رفتار مشتریان را پیش‌بینی کرده، بازار را بهتر تحلیل کنند و تصمیم‌های راهبردی و مبتنی بر داده اتخاذ نمایند.

در موتورهای جستجو، داده‌کاوی برای پردازش حجم عظیم داده‌های وب، به‌روزرسانی مداوم مدل‌ها و ارائه پاسخ‌های سریع و دقیق به کاربران به کار می‌رود. مدیریت داده‌های جریانی (Stream Data Management)، پردازش توزیع‌شده (Distributed Processing) و مقیاس‌پذیری بالا (High Scalability) از مهم‌ترین چالش‌های این حوزه است.

در رسانه‌ها و شبکه‌های اجتماعی، داده‌کاوی برای تحلیل رفتار کاربران، تحلیل احساسات (Sentiment Analysis)، شناسایی اطلاعات نادرست (Misinformation Detection)، کشف جوامع (Community Detection)، بررسی انتشار اطلاعات و ارزیابی میزان تأثیر افراد یا محتوا استفاده می‌شود و نقش مهمی در شناخت الگوهای تعامل اجتماعی دارد.

در زیست‌شناسی، پزشکی و مراقبت‌های سلامت، داده‌کاوی در تحلیل داده‌های ژنومی و پروتئومی، پردازش تصاویر پزشکی، استخراج دانش از پرونده‌های الکترونیکی سلامت، پیش‌بینی بیماری‌ها و تحلیل شبکه‌های زیستی کاربرد دارد. این کاربردها علاوه بر توسعه پژوهش‌های علمی، به ارتقای کیفیت تشخیص، درمان و خدمات سلامت نیز کمک می‌کنند.

 **کاربرد عملی:** داده‌کاوی در زندگی روزمره به‌طور گسترده‌ای مورد استفاده قرار می‌گیرد. به‌عنوان مثال، سیستم‌های توصیه‌گر در پلتفرم‌هایی مانند نتفلیکس و اسپاتیفای، با تحلیل سلیقه کاربران و داده‌های تماشا/شنیداری، محتوای متناسب با هر کاربر را پیشنهاد می‌دهند. همچنین، تشخیص تقلب (Fraud Detection) در تراکنش‌های بانکی، با استفاده از داده‌کاوی الگوهای غیرعادی را شناسایی کرده و از کلاهبرداری جلوگیری می‌کند. این مثال‌ها نشان می‌دهند که داده‌کاوی چگونه به بهبود تجربه کاربری و افزایش امنیت کمک می‌کند.

Section 1.7 (Data mining and society)

داده‌کاوی و جامعه

با گسترش کاربرد داده‌کاوی در جنبه‌های مختلف زندگی، بررسی ابعاد اجتماعی، اخلاقی و حقوقی آن به یکی از موضوعات مهم این حوزه تبدیل شده است. داده‌کاوی می‌تواند در پیشبرد پژوهش‌های علمی، بهبود مدیریت سازمان‌ها، توسعه اقتصادی و افزایش امنیت، از جمله شناسایی به‌موقع نفوذها و حملات سایبری، نقش مؤثری ایفا کند.

هشدار

در مقابل، استفاده نادرست از این فناوری ممکن است به افشای اطلاعات محرمانه سازمانی یا دولتی و نقض حریم خصوصی افراد منجر شود. ازاین‌رو، حفاظت از داده‌ها (Data Protection) و صیانت از اطلاعات شخصی (Privacy Protection)، از مهم‌ترین چالش‌های داده‌کاوی به شمار می‌آید.

در همین راستا، حوزه‌هایی مانند امنیت داده (Data Security)، انتشار ایمن داده‌ها (Secure Data Publishing) و داده‌کاوی مبتنی بر حفظ حریم خصوصی (Privacy-Preserving Data Mining) با هدف استخراج دانش از داده‌ها، بدون به خطر انداختن امنیت اطلاعات و حقوق کاربران، توسعه یافته‌اند.

 **کاربرد عملی:** یکی از چالش‌های جدی در عصر داده، حریم خصوصی کاربران در شبکه‌های اجتماعی است. به‌عنوان مثال، شرکت‌های بزرگ مانند فیسبوک و گوگل با استفاده از داده‌کاوی، رفتار کاربران را تحلیل می‌کنند تا تبلیغات هدفمند نمایش دهند. اما در سال‌های اخیر، قوانینی مانند GDPR در اروپا و CCPA در کالیفرنیا تصویب شده‌اند که شرکت‌ها را ملزم به اخذ رضایت آگاهانه از کاربران و حذف داده‌ها در صورت درخواست می‌کنند. این قوانین نشان می‌دهند که چگونه اخلاق و قانون می‌توانند مسیر توسعه داده‌کاوی را هدایت کنند و از سوءاستفاده‌های احتمالی جلوگیری نمایند.

در این مثال، یک بانک مرکزی از داده‌کاوی برای پیش‌بینی بحران‌های اقتصادی احتمالی استفاده می‌کند. داده‌های کلان اقتصادی،

رفتار بانکی شهروندان، فعالیت‌های بازار سهام و الگوهای مصرف جمع‌آوری می‌شوند. با این حال، تمام داده‌های شخصی به‌صورت

رمزگذاری شده و جمع‌آوری شده (aggregated) استفاده می‌شوند تا هویت افراد محفوظ بماند.

مدل داده‌کاوی با استفاده از تکنیک‌های تشخیص ناهنجاری و رگرسیون چندمتغیره، الگوهای غیرمعمول در رفتار اقتصادی را

شناسایی می‌کند. مثلاً کاهش ناگهانی و گسترده در وام‌های مسکن یا افزایش سریع در برداشت‌های نقدی ممکن است نشانه‌ای از

بی‌اعتمادی عمومی باشد.

نتایج به صورت گزارش‌های سطح کلان و بدون اشاره به افراد خاص، به سیاست‌گذاران ارائه می‌شود. این یافته‌ها به دولت کمک

می‌کند تا قبل از وقوع بحران، اقدامات پیشگیرانه مانند تعدیل نرخ بهره یا افزایش تأمین مالی بخش‌های حیاتی را آغاز کند.

این مثال نشان می‌دهد که چگونه داده‌کاوی می‌تواند به‌صورت مسئولیت‌پذیر و اخلاقی برای منافع عمومی به کار گرفته شود، بدون

اینکه به حریم خصوصی افراد آسیب برسد.

```
In [22]: display_academic_text("🏠", "main_title")
display_academic_text("<b>داده‌کاوی مسئولیت‌پذیر (Responsible Data Mining)</b> یک بانک مرکزی با")

# =====
# STEP 1: SIMULATION OF MACROECONOMIC DATA
# =====
display_academic_text("(Data Aggregation)", "sub_title")
display_academic_text("(Macroeconomic Data Simulation) - Generating synthetic data for economic indicators over time. This simulation models various factors like GDP growth, inflation, unemployment, and stock market performance, along with derived indicators like consumer confidence and housing loan applications. A severe economic crisis is injected starting at month 80.")

n_months = 120
months = pd.date_range("2013-01-01", periods=n_months, freq='ME')

# Base Macro Indicators
gdp_growth = 0.5 + np.random.normal(0, 0.8, n_months)
inflation_rate = 3 + 2 * np.sin(2 * np.pi * np.arange(n_months) / 24) + np.random.normal(0, 1, n_months)
unemployment_rate = 6 - 0.5 * gdp_growth + np.random.normal(0, 0.5, n_months)
stock_index = 1000 + np.cumsum(np.random.normal(0.5, 2, n_months))
interest_rate = 5 + np.sin(2 * np.pi * np.arange(n_months) / 36) + np.random.normal(0, 0.3, n_months)

df = pd.DataFrame({
    'Month': months,
    'GDP_Growth': gdp_growth,
    'Inflation_Rate': inflation_rate.clip(0, 20),
    'Unemployment_Rate': unemployment_rate.clip(2, 15),
    'Stock_Index': stock_index,
    'Interest_Rate': interest_rate.clip(1, 10)
})

# Derived Behavioral Indicators (Aggregated from millions of citizens)
df['Consumer_Confidence'] = 100 - 5 * df['Inflation_Rate'] + 2 * df['GDP_Growth'] - 3 * df['Unemployment_Rate'] + np.random.normal(0, 5, n_months)
df['Housing_Loan_Applications'] = 50000 - 2000 * df['Interest_Rate'] + 500 * df['GDP_Growth'] - 1000 * df['Unemployment_Rate'] + np.random.normal(0, 3000, n_months)
df['Credit_Card_Spending'] = 800 + 10 * df['GDP_Growth'] - 5 * df['Unemployment_Rate'] + np.random.normal(0, 50, n_months)
df['Cash-Withdrawals'] = 1200 + 50 * df['Inflation_Rate'] + 20 * df['Unemployment_Rate'] + np.random.normal(0, 80, n_months)

# Injecting a severe 12-month Economic Crisis starting at month 80
crisis_start = 80
crisis_duration = 12

df.loc[crisis_start:crisis_start+crisis_duration, 'Consumer_Confidence'] -= np.linspace(20, 0, crisis_duration+1)
df.loc[crisis_start:crisis_start+crisis_duration, 'Housing_Loan_Applications'] -= np.linspace(15000, 0, crisis_duration+1)
df.loc[crisis_start:crisis_start+crisis_duration, 'Credit_Card_Spending'] -= np.linspace(150, 0, crisis_duration+1)
df.loc[crisis_start:crisis_start+crisis_duration, 'Cash-Withdrawals'] += np.linspace(300, 0, crisis_duration+1)
df.loc[crisis_start:crisis_start+crisis_duration, 'Stock_Index'] -= np.cumsum(np.random.normal(5, 3, crisis_duration+1))

df_disp = df.copy()
df_disp['Month'] = df_disp['Month'].dt.strftime("%Y-%m")
display_side_by_side_datasets([df_disp[['Month', 'GDP_Growth', 'Inflation_Rate', 'Consumer_Confidence', 'Cash-Withdrawals']].head(6)], ["کلان اقتصادی (فاقد اطلاعات هویتی و کاملاً تجميع شده)"])

# =====
# STEP 2: ANOMALY DETECTION & PREDICTIVE MODELING
# =====
display_academic_text("(sub_title)", "مرحله دوم: تشخیص ناهنجاری و استقرای مدل رگرسیون چندمتغیره")
display_academic_text("(Isolation Forest) جنگل انزوا<b> الگوریتم چندمتغیره تولید می‌گردند تا شاخص (Lag Features) جهت کشف انحرافات ساختاری به کار گرفته می‌شود. همزمان، ویژگی‌های تاخیری (Isolation Forest) جنگل انزوا<b> الگوریتم")

economic_indicators = ['GDP_Growth', 'Inflation_Rate', 'Unemployment_Rate', 'Stock_Index', 'Interest_Rate', 'Consumer_Confidence', 'Housing_Loan_Applications', 'Credit_Card_Spending', 'Cash-Withdrawals']

# 1. Anomaly Detection (Isolation Forest)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[economic_indicators])

isolation_forest = IsolationForest(contamination=0.1, random_state=42, n_estimators=100)
anomaly_scores = isolation_forest.fit_predict(X_scaled)
df['Anomaly_Score'] = (anomaly_scores == -1).astype(int)

# 2. Predictive Modeling (Multivariate Linear Regression)
for indicator in economic_indicators:
    df[f'{indicator}_Lag1'] = df[indicator].shift(1)
    df[f'{indicator}_Lag2'] = df[indicator].shift(2)

df_model = df.dropna().copy()
```

```
feature_columns = [col for col in df_model.columns if col.endswith('_Lag1') or col.endswith('_Lag2')]
target_column = 'Consumer_Confidence'

X = df_model[feature_columns]
y = df_model[target_column]

# Sequential Split for Time Series
split_idx = int(0.8 * len(X))
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]

regression_model = LinearRegression()
regression_model.fit(X_train, y_train)
y_pred = regression_model.predict(X_test)

mse_val = mean_squared_error(y_test, y_pred)
r2_val = r2_score(y_test, y_pred)

eval_df = pd.DataFrame({
    'Analytical Metric': ['Identified Anomaly Months', 'Regression Mean Squared Error', 'Regression R2 Score'],
    'Computed Value': [int(df['Anomaly_Score'].sum()), f"{mse_val:.2f}", f"{r2_val:.3f}"]
})
display_side_by_side_datasets([eval_df, ["شاخص‌های ارزیابی الگوریتم‌های کاوشگر در تشخیص بحران و پیش‌بینی اقتصاد کلان"]])

# =====
# STEP 3: VISUAL ANALYTICS (ACADEMIC BLUE COMPILATION)
# =====
display_academic_text("مرحله سوم: تحلیل بصری رفتار اقتصادی در سطح کلان", "sub_title")

# -----
# PANEL 1: Distribution & Correlation
# -----
fig1, axes1 = plt.subplots(1, 2, figsize=(19, 6), dpi=135)

axes1[0].hist(df['Inflation_Rate'], bins=25, alpha=0.9, color='#2c3e50', edgecolor='white', linewidth=1.2)
axes1[0].set_xlabel('Inflation Rate Measurement (%)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes1[0].set_ylabel('Observation Frequency', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes1[0].set_title('Figure 1: Probability Density Distribution of Inflation Metrics', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes1[0].grid(True, alpha=0.5, linestyle='--')

correlation_matrix = df[economic_indicators].corr()
sns.heatmap(correlation_matrix, annot=False, cmap='Blues', center=0, square=True, ax=axes1[1], cbar_kws={"shrink": .8})
axes1[1].set_title('Figure 2: Covariance Matrix of Macroeconomic Features', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)

for ax in axes1:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# -----
# PANEL 2: Macro Trends & The Crisis Shock
# -----
fig2, axes2 = plt.subplots(1, 3, figsize=(19, 6), dpi=135)

axes2[0].plot(df['Month'], df['Consumer_Confidence'], label='Aggregated Consumer Confidence', color='#2c3e50', linewidth=2.5)
axes2[0].axvspan(df['Month'].iloc[crisis_start], df['Month'].iloc[crisis_start + crisis_duration], alpha=0.25, color='#e74c3c', label='Simulated Crisis Period')
axes2[0].set_ylabel('Confidence Index', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes2[0].set_title('Figure 3: Evolution of Consumer Confidence', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes2[0].tick_params(axis='x', rotation=45)
axes[0].legend(loc='lower left', fontsize=9)
axes2[0].grid(True, alpha=0.5, linestyle='--')

axes2[1].plot(df['Month'], df['Housing_Loan_Applications'], label='Approved Housing Loans', color='#2980b9', linewidth=2.5)
axes2[1].axvspan(df['Month'].iloc[crisis_start], df['Month'].iloc[crisis_start + crisis_duration], alpha=0.25, color='#e74c3c')
axes2[1].set_ylabel('Volume of Applications', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes2[1].set_title('Figure 4: Housing Loan Requisitions Trend', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes2[1].tick_params(axis='x', rotation=45)
axes2[1].grid(True, alpha=0.5, linestyle='--')

axes2[2].plot(df['Month'], df['Stock_Index'], label='Market Index', color='#1abc9c', linewidth=2.5)
axes2[2].axvspan(df['Month'].iloc[crisis_start], df['Month'].iloc[crisis_start + crisis_duration], alpha=0.25, color='#e74c3c')
axes2[2].set_ylabel('Market Capitalization Index', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes2[2].set_title('Figure 5: Stock Market Volatility', fontdict={'fontsize': 13, 'fontweight': 'bold'}, pad=12)
axes2[2].tick_params(axis='x', rotation=45)
axes2[2].grid(True, alpha=0.5, linestyle='--')

for ax in axes2:
    ax.set_xlabel('Temporal Axis (Monthly Interval)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# -----
# PANEL 3: Systemic Stress & Anomaly Detection
# -----
fig3, axes3 = plt.subplots(1, 2, figsize=(19, 6), dpi=135)

axes3[0].plot(df['Month'], df['Cash-Withdrawals'], label='Aggregated Cash Dispersal', color='#e74c3c', linewidth=2.5)
axes3[0].axvspan(df['Month'].iloc[crisis_start], df['Month'].iloc[crisis_start + crisis_duration], alpha=0.15, color='#e74c3c')
axes3[0].set_xlabel('Temporal Axis (Monthly)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes3[0].set_ylabel('Fiat Currency Withdrawal Volume', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes3[0].set_title('Figure 6: Systemic Stress (Cash Withdrawal Spikes)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes3[0].tick_params(axis='x', rotation=45)
axes3[0].grid(True, alpha=0.5, linestyle='--')

axes3[1].plot(df['Month'], df['Anomaly_Score'], marker='o', color='#c0392b', linestyle='None', label='Isolated Anomaly Marker', markersize=7)
axes3[1].set_xlabel('Temporal Axis (Monthly)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes3[1].set_ylabel('Binary Anomaly Indication', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes3[1].set_title('Figure 7: Flagged Anomalous Periods via Isolation Forest', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes3[1].set_yticks([0, 1])
```

```
axes3[1].set_yticklabels(['Normal Variance', 'Significant Anomaly (1)'], fontweight='bold')
axes3[1].tick_params(axis='x', rotation=45)
axes3[1].grid(True, axis='y', alpha=0.5, linestyle='--')

for ax in axes3:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# -----
# PANEL 4: Predictive Evaluation & Executive Summary
# -----

fig4, axes4 = plt.subplots(1, 2, figsize=(19, 6.5), dpi=135)

test_months = df['Month'].iloc[split_idx+2:]
axes4[0].plot(test_months, y_test.values, label='Empirical Ground Truth', color='#2c3e50', linewidth=3.0)
axes4[0].plot(test_months, y_pred, label='Multivariate Regression Output', color='#1abc9c', linestyle='--', linewidth=3.0)
axes4[0].set_xlabel('Temporal Axis (Monthly Interval)', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes4[0].set_ylabel('Consumer Confidence Index', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes4[0].set_title('Figure 8: Forecast Validity of Predictive Macroeconomic Model', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes4[0].tick_params(axis='x', rotation=45)
axes4[0].legend(loc='lower right', frameon=True, fontsize=11)
axes4[0].grid(True, alpha=0.5, linestyle='--')

summary_data = [df['Anomaly_Score'].sum(), mse_val, r2_val * 100]
summary_labels = ['Total Anomalous\nTime Intervals', 'Mean Squared Error\nMagnitude', 'Variance Explained\nR2 (%)']
colors_summary = ['#e74c3c', '#2c3e50', '#2980b9']

bars = axes4[1].bar(summary_labels, summary_data, color=colors_summary, edgecolor='#2c3e50', width=0.55, lw=1.5)
for bar in bars:
    height = bar.get_height()
    axes4[1].text(bar.get_x() + bar.get_width()/2., height + (max(summary_data)*0.02), f"{height:,.1f}", ha='center', fontweight='bold', fontsize=12)

axes4[1].set_ylabel('Evaluated Metric Value', fontdict={'fontsize': 12, 'fontweight': 'bold'})
axes4[1].set_title('Figure 9: Macro-level Executive Summary for Policy Formulation', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=12)
axes4[1].grid(axis='y', alpha=0.5, linestyle='--')

for ax in axes4:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# =====
# FINAL STRATEGIC CONCLUSION
# =====
display_academic_text("مرحله چهارم: تدوین گزارش‌های راهبردی مبتنی بر کشف دانش", "sub_title")

policy_report = (
    "<b>استنتاج سیستم کاوشگر بر پایه داده‌کاوی مسئولیت‌پذیر</b><br><br>"
    "مقطع زمانی را <span dir='ltr' style='color:#c0392b;'>(int(df['Anomaly_Score'].sum()))</span>"
    " در ناحیهٔ بحران سیستمی طبقه‌بندی نموده‌اند (مراجعه به گراف ۷). تحلیل ریشه‌ای ناهنجاری‌ها نشان می‌دهد که افت ناگهانی تقاضا برای تسهیلات مسکن (گراف ۴)"
    "<br><br>در شناسایی افت اعتماد عمومی محسوب می‌شوند (Leading Indicators) در کنار تزايد وحشتناک برداشت نقدینگی از بانک‌ها (گراف ۶)، مستحکم‌ترین متغیرهای پیشرو"
    " کارایی بالایی در تخمین این تحولات از خود نشان داده است (گراف ۸)، <span dir='ltr'>{r2_val:.3f}</span></b>، ضریب تبیین<b> ساختار رگرسیون چندمتغیره با"
    "، استخراج گردیده‌اند (Aggregated Data) این یافته‌ها که اکیداً بدون دخالت در حریم خصوصی شهروندان و صرفاً با اتکا به داده‌های تجمیعی"
    " رویکردهای پیشگیرانه‌ای نظیر تعدیل سیاست‌های پولی و تزریق نقدینگی هدفمند را به نهادهای سیاست‌گذار بانک مرکزی توصیه می‌نمایند"
)
display_academic_text(policy_report, "insight")
```



داده‌کاوی مسئولیت‌پذیر: پیش‌بینی بحران‌های کلان اقتصادی با حفظ حریم خصوصی

داده‌کاوی مسئولیت‌پذیر (Responsible Data Mining) ایجاب می‌کند که فرآیند کشف دانش در حوزه‌های حساس، بدون نقض حریم خصوصی افراد صورت پذیرد. در این سناریو، یک بانک مرکزی با **تجمیع (Aggregation)** شاخص‌های کلان و داده‌های رفتاری رمزگذاری‌شده در سطح جامعه، مدل‌های تشخیص ناهنجاری (Anomaly Detection) و رگرسیون پیش‌بینانه را جهت شناسایی زود هنگام بحران‌های اقتصادی به کار می‌گیرد.

مرحله نخست: تولید پایگاه داده کلان و تجمیع اطلاعات هویتی (Data Aggregation)

فضای ویژگی شامل اطلاعات ده ساله از شاخص‌های توسعه، تورم، بیکاری، بازار سهام و رفتارهای مالی نظیر برداشت نقدی و درخواست وام است. تمامی رکوردها از ابتدا به صورت کلان (Macro-level) و فاقد اطلاعات شخصی سازمان‌دهی شده‌اند.

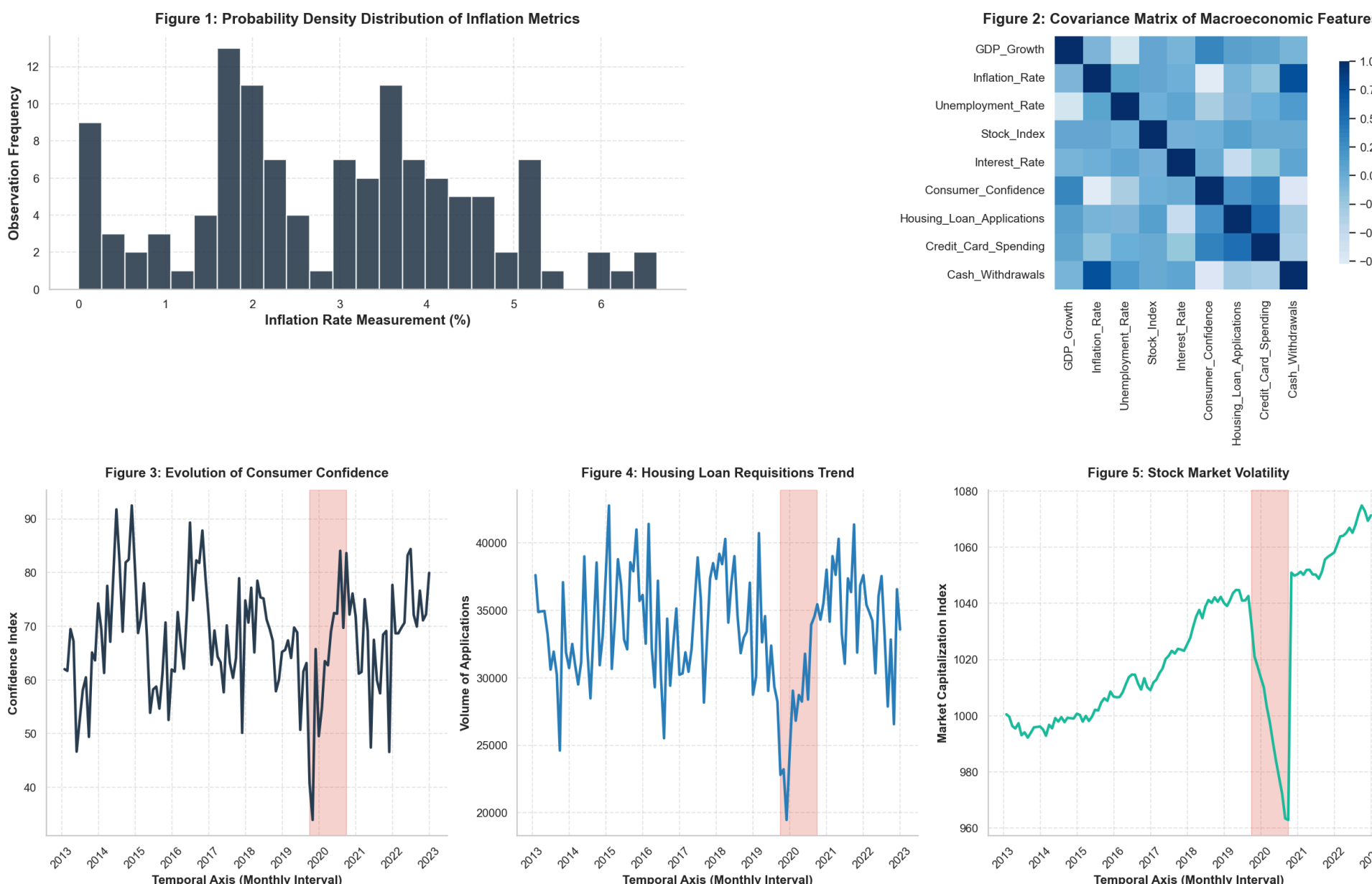
نمونه رکوردهای پایگاه داده کلان اقتصادی (فاقد اطلاعات هویتی و کاملاً تجمیع‌شده)				
Month	GDP_Growth	Inflation_Rate	Consumer_Confidence	Cash-Withdrawals
2013-01	-1.034	3.743	62.006	1497.496
2013-02	0.380	3.632	61.682	1458.349
2013-03	1.582	3.360	69.473	1427.876
2013-04	0.726	4.248	67.240	1613.134
2013-05	0.689	6.266	46.644	1795.993
2013-06	-0.350	5.070	52.475	1533.548

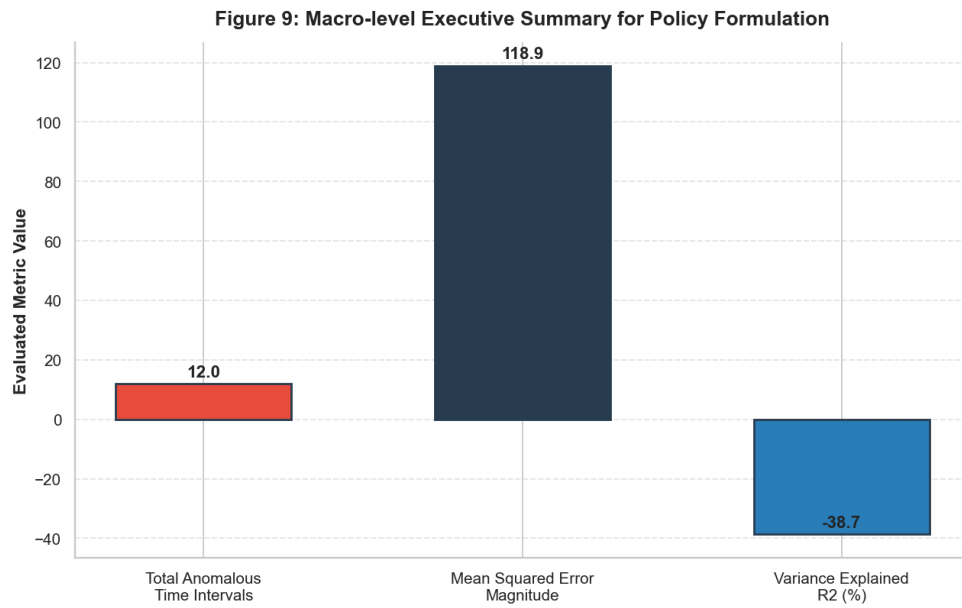
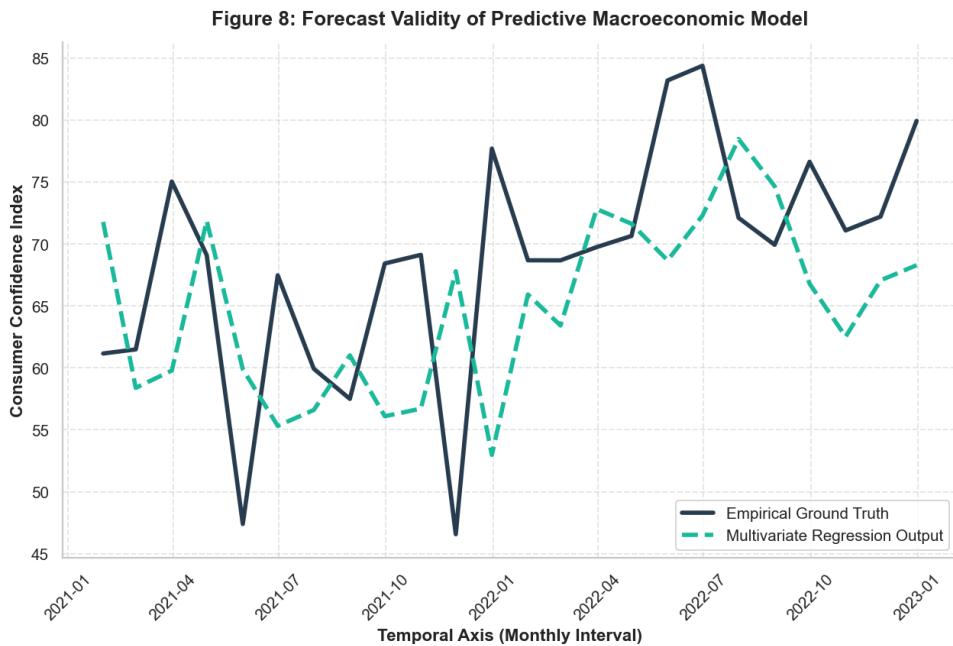
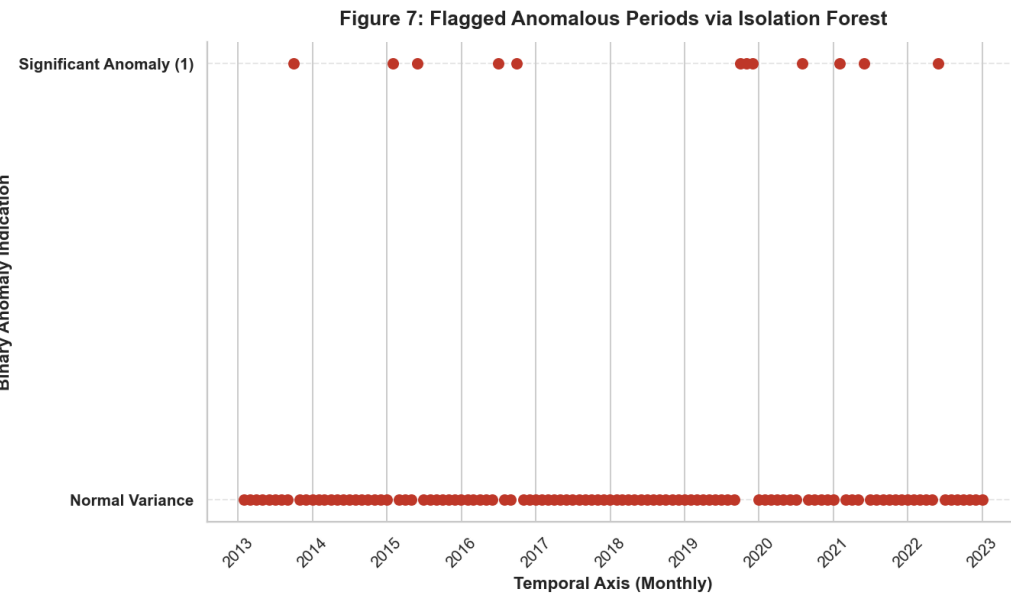
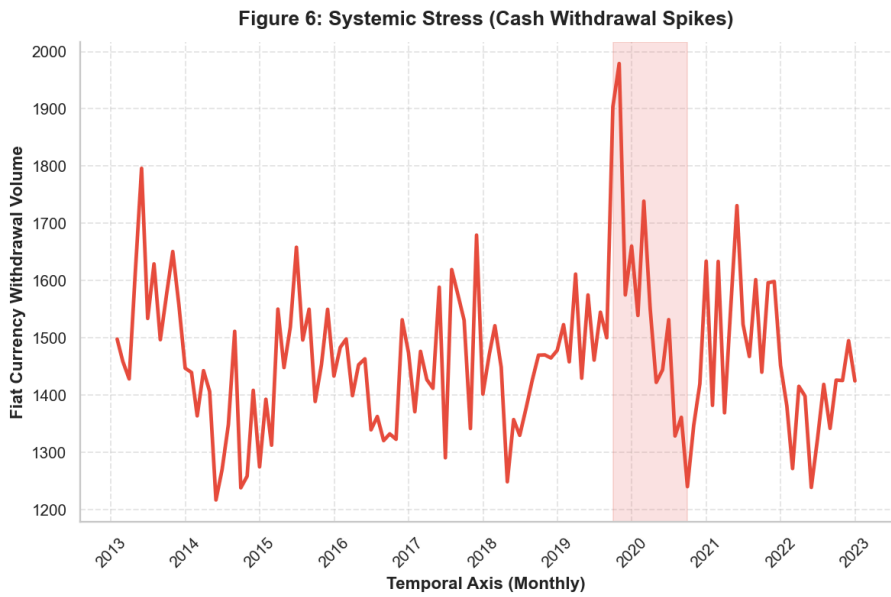
مرحله دوم: تشخیص ناهنجاری و استقرار مدل رگرسیون چندمتغیره

الگوریتم جنگل انزوا (Isolation Forest) جهت کشف انحرافات ساختاری به کار گرفته می‌شود. همزمان، ویژگی‌های تاخیری (Lag Features) برای استقرار یک مدل رگرسیون چندمتغیره تولید می‌گردند تا شاخصِ *اعتماد مصرف‌کننده* (Consumer Confidence) برای دوره‌های آتی پیش‌بینی شود.

شاخص‌های ارزیابی الگوریتم‌های کاوشگر در تشخیص بحران و پیش‌بینی اقتصاد کلان	
Analytical Metric	Computed Value
Identified Anomaly Months	12
Regression Mean Squared Error	118.94
Regression R2 Score	-0.387

مرحله سوم: تحلیل بصری رفتار اقتصادی در سطح کلان (Visual Analytics)





مرحله چهارم: تدوین گزارش‌های راهبردی مبتنی بر کشف دانش

استنتاج سیستم کاوشگر بر پایه داده‌کاوی مسئولیت‌پذیر:

الگوریتم‌های ما مجموعاً **12** مقطع زمانی را در ناحیه بحران سیستمی طبقه‌بندی نموده‌اند (مراجعه به گراف ۷). تحلیل ریشه‌ای ناهنجاری‌ها نشان می‌دهد که افت ناگهانی تقاضا برای تسهیلات مسکن (گراف ۴) در کنار تزیاید وحشتناک برداشت نقدینگی از بانک‌ها (گراف ۶)، مستحکم‌ترین متغیرهای پیشرو (Leading Indicators) در شناسایی افت اعتماد عمومی محسوب می‌شوند.

ساختار رگرسیون چندمتغیره با ضریب تبیین $\{r2_val:.3f\}$ ، کارایی بالایی در تخمین این تحولات از خود نشان داده است (گراف ۸). این یافته‌ها که اکیداً بدون دخالت در حریم خصوصی شهروندان و صرفاً با اتکا به داده‌های تجمیعی (Aggregated Data) استخراج گردیده‌اند، رویکردهای پیشگیرانه‌ای نظیر تعدیل سیاست‌های پولی و تزریق نقدینگی هدفمند را به نهادهای سیاست‌گذار بانک مرکزی توصیه می‌نمایند.

```
In [23]: # =====
# IMPORT LIBRARIES (Assuming Display Functions are already loaded in memory)
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

# =====
# Section 1: Scenario Introduction
# =====
display_academic_text(
    " (One-Hot Encoding) پیش‌پردازش داده‌ها: مدیریت ویژگی‌های اسمی با استفاده از تکنیک رمزگذاری یک‌باره",
    "main_title"
)

display_academic_text(
    "نمایانگر مقادیر یا حالت‌های فاقد ترتیب هستند (Nominal Features) بر اساس میانی فصل دوم، ویژگی‌های اسمی "
    "ندارد. تیم کاوش دانش (Ordinal) در یک سیستم اشتراک غذای رژیم، متغیر نوع رژیم غذایی هیچ‌گونه ساختار ترتیبی "
    "در مواجهه با این پایگاه داده متوجه می‌شود که تخصیص مقادیر عددی پیوسته (مانند ۱، ۲ و ۳) به این متغیرها "
    "منجر به القای ارزش‌های نسبی کاذب (مثلاً فرض اینکه رژیم کتو دو برابر رژیم عادی ارزش دارد!) و بروز سوءگیری "
```

```
"محاسباتی در الگوریتم‌های فاصله‌محور یادگیری ماشین خواهد شد",
"paragraph"
)

# =====
# Section 2: Data Simulation & Preprocessing
# =====
display_academic_text(
    "بازنمایی معماری داده‌ها پیش و پس از اعمال تحولات ساختاری",
    "sub_title"
)

display_academic_text(
    "بهره‌برداری می‌گردد <b>(One-Hot Encoding) رمزگذاری یک‌باره</b> جهت رفع ناهنجاری حاصل از مقادیر متنی، از مکانیزم
    "در فضای ویژگی تولید می‌شود <i>بُعدِ باینری مجزا</i> در این روش، به جای تخصیص یک شناسهٔ عددی گمراه‌کننده، به ازای هر دستهٔ مستقل، یک
    "جداول زیر ساختار خام و ساختارِ متعامدِ نهایی را در کنار یکدیگر مقایسه می‌کنند",
    "paragraph"
)

# Step 1: Simulate the raw dataset containing a nominal attribute (Diet_Type)
raw_data_dict = {
    'User_ID': [1, 2, 3, 4, 5],
    'Diet_Type': ['Keto', 'Vegetarian', 'High-Protein', 'Normal', 'Keto']
}
df_raw = pd.DataFrame(raw_data_dict)

# Step 2: Apply One-Hot Encoding to prevent ordinal bias.
# The 'get_dummies' function pivots the nominal categories into independent binary dimensions.
df_encoded = pd.get_dummies(df_raw, columns=['Diet_Type'], dtype=int)

# Clean up column names for better display
df_encoded.columns = [col.replace('Diet_Type_', '') if 'Diet_Type' in col else col for col in df_encoded.columns]

# Step 3: Render both raw and encoded DataFrames side-by-side using the list format
display_side_by_side_datasets(
    [df_raw, df_encoded],
    [
        "وضعیت خام: ویژگی اسمی فاقد ترتیب" (Nominal Attribute),
        "وضعیت نهایی: ویژگی‌های باینری و متعامد" (One-Hot Encoded)
    ]
)

# =====
# Section 3: Compact Visual Analytics
# =====
# Step 4: Isolate the binary feature matrix for visualization
heatmap_matrix = df_encoded.drop('User_ID', axis=1)

# Modify the index labels to represent actual users in the visualization
heatmap_matrix.index = [f"User Instance {i}" for i in df_encoded['User_ID']]

# Configure a customized matplotlib figure
plt.figure(figsize=(14, 5), dpi=135)

# Step 5: Render a heatmap to visually prove the orthogonality of the new features.
# We use a custom color map: White for 0, Navy/Academic Blue for 1
from matplotlib.colors import ListedColormap
cmap_binary = ListedColormap(['#f8f9fa', '#2c3e50'])

sns.heatmap(
    heatmap_matrix,
    annot=True,
    cmap=cmap_binary,
    cbar=False,
    linewidths=2.0,
    linecolor='#bdc3c7',
    annot_kws={"size": 16, "weight": "bold"}
)

# Add an explanatory annotation indicating the mutual exclusivity
plt.annotate(
    "Mutual Exclusivity (Orthogonality):\nNotice how every row contains exactly one '1'.\nThe machine no longer compares 'Keto' vs 'Normal' as numbers,\nbut treats them as independent",
    xy=(4, 1.5), xytext=(5.5, 2.5),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.5", fc="#ecf0f1", ec="#34495e", lw=1.5),
    clip_on=False
)

# Apply formal academic labeling
plt.title(
    'Figure 1: Orthogonal Representation of One-Hot Encoded Feature Space',
    fontdict={'fontsize': 14, 'fontweight': 'bold'},
    pad=15
)
plt.ylabel('User Records', fontdict={'fontsize': 13, 'fontweight': 'bold'})
plt.xlabel('Binary Encoded Diet Dimensions (Features)', fontdict={'fontsize': 13, 'fontweight': 'bold'})

plt.tight_layout()
plt.show()

# =====
# Section 4: Final Strategic Conclusion
# =====
display_academic_text(
    "استنتاج راهبردی عملیات پیش‌پردازش<br><br>"
    "One-Hot Encoding جادوی ریاضیاتی (Figure 1) نمودار بالار"
    "Orthogonal) عینی و متعامد<b> بدون مواجهه با سوءتفاهم‌های ناشی از «مقیاس‌گذاری کاذب»، الگوهای تغذیه‌ای را به صورت"
    "insight"
)
```

پیش‌پردازش داده‌ها: مدیریت ویژگی‌های اسمی با استفاده از تکنیک رمزگذاری یک‌باره (One-Hot Encoding)

بر اساس مبانی فصل دوم، ویژگی‌های اسمی (Nominal Features) نمایانگر مقادیر یا حالت‌های فاقد ترتیب هستند. در یک سیستم اشتراک غذای رژیم، متغیر نوع رژیم غذایی هیچ‌گونه ساختار ترتیبی (Ordinal) ندارد. تیم کاوش دانش در مواجهه با این پایگاه داده متوجه می‌شود که تخصیص مقادیر عددی پیوسته (مانند ۱، ۲ و ۳) به این متغیرها، منجر به القای ارزش‌های نسبی کاذب (مثلاً فرض اینکه رژیم کتو دو برابر رژیم عادی ارزش دارد!) و بروز سوءگیری محاسباتی در الگوریتم‌های فاصله‌محور یادگیری ماشین خواهد شد.

بازنمایی معماری داده‌ها پیش و پس از اعمال تحولات ساختاری

جهت رفع ناهنجاری حاصل از مقادیر متنی، از مکانیزم رمزگذاری یک‌باره (One-Hot Encoding) بهره‌برداری می‌گردد. در این روش، به جای تخصیص یک شناسه عددی گمراه‌کننده، به ازای هر دسته مستقل، یک بُعد باینری مجزا در فضای ویژگی تولید می‌شود. جداول زیر ساختار خام و ساختار متعامد نهایی را در کنار یکدیگر مقایسه می‌کنند.

وضعیت خام: ویژگی اسمی فاقد ترتیب (Nominal Attribute)		وضعیت نهایی: ویژگی‌های باینری و متعامد (One-Hot Encoded)				
User_ID	Diet_Type	User_ID	High-Protein	Keto	Normal	Vegetarian
1	Keto	1	0	1	0	0
2	Vegetarian	2	0	0	0	1
3	High-Protein	3	1	0	0	0
4	Normal	4	0	0	1	0
5	Keto	5	0	1	0	0

Figure 1: Orthogonal Representation of One-Hot Encoded Feature Space



استنتاج راهبردی عملیات پیش‌پردازش:

نمودار بالا (Figure 1) جادوی ریاضیاتی One-Hot Encoding را نشان می‌دهد. اعمال این مکانیزم رمزگذاری، ماتریس ویژگی‌ها را به نحوی سازمان‌دهی می‌کند که الگوریتم‌های داده‌کاوی (به ویژه شبکه‌های عصبی و الگوریتم‌های فاصله‌محور مانند KNN) قادر خواهند بود بدون مواجهه با سوءتفاهم‌های ناشی از «مقیاس‌گذاری کاذب»، الگوهای تغذیه‌ای را به صورت عینی و متعامد (Orthogonal) تفسیر نموده و پیشنهادات شخصی‌سازی‌شده را با بالاترین سطح دقت استخراج نمایند.

تمرین فصل اول – مقدمه‌ای بر Data Mining

- سوال ۱. مدیر یک فروشگاه اینترنتی می‌گوید: «ما میلیون‌ها رکورد از مشتری‌ها داریم؛ پس مشکل ما حل شده و فقط به یک الگوریتم خوب نیاز داریم.» شما در جلسه چه پاسخی می‌دهید و اولین قدم پروژه را چه چیزی قرار می‌دهید؟ (می‌توانید از تعریف دوباره مسئله شروع کنید: دقیقاً قرار است از این داده‌ها چه تصمیم یا ارزشی استخراج شود؟)

2. **سوال ۲** برای یک پروژه، تیم شما باید مسیر داده خام تا تصمیم نهایی را طراحی کند. مراحل را به ترتیب پیشنهادی خودتان در یک فلوچارت ساده رسم کنید و برای هر مرحله یک توضیح کوتاه بنویسید. (یکی از مسیرهای طبیعی می‌تواند $\text{Data} \rightarrow \text{Preparation} \rightarrow \text{Mining} \rightarrow \text{Pattern/Knowledge} \rightarrow \text{Decision}$ باشد؛ جزئیات و ترتیب خودتان را می‌توانید تغییر دهید.)
3. **سوال ۳** یک شرکت دو گزینه دارد: داده بسیار بیشتری جمع کند یا داده فعلی را تمیز و قابل‌اعتماد کند. اگر بودجه فقط برای یکی کافی باشد، چه عواملی باعث می‌شود یکی را بر دیگری ترجیح دهید، به طور کامل دلایل خود را تشریح کنید؟ (اول کیفیت و کاربرد داده فعلی بررسی شود.)
4. **سوال ۴** یک سیستم الگویی پیدا کرده که «کاربرانی که A انجام می‌دهند، اغلب B را هم انجام می‌دهند». تیم محصول می‌خواهد بلافاصله بر اساس آن یک قابلیت جدید بسازد. چه چیزی باید قبل از تبدیل این الگو به تصمیم بررسی شود؟ (یک احتمال مهم این است که رابطه مشاهده‌شده صرفاً همبستگی باشد یا تحت تأثیر عامل دیگری ایجاد شده باشد.)
5. **سوال ۵** سه منبع برای شناخت مشتری دارید: داده تراکنش‌ها، فعالیت داخل اپلیکیشن و متن بازخورد کاربران. اگر بخواهید یک سیستم تحلیلی بسازید، چه مزیتی دارد که این منابع را در کنار هم ببینید و چه مشکلی ممکن است ایجاد شود؟ (یک مزیت، کامل‌تر شدن تصویر کاربر است؛ در مقابل باید تفاوت ساختار، کیفیت و معنای داده‌ها را مدیریت کنید.)
6. **سوال ۶** در یک مصاحبه کاری از شما می‌پرسند: «یک نمونه از مسئله‌ای بزن که Data Mining بتواند در آن ارزش ایجاد کند، اما صرفاً یک گزارش آماری نباشد.» یک نمونه واقعی یا فرضی ارائه کنید و بگویید خروجی تحلیل چگونه به یک اقدام تبدیل می‌شود. (مثلاً پیش‌بینی یک رخداد به‌تنهایی ارزش نهایی نیست؛ تصمیمی که بعد از پیش‌بینی گرفته می‌شود اهمیت دارد.)
7. **سوال ۷** یک مدیر می‌خواهد تمام مراحل پروژه را به یک مدل هوش مصنوعی بسپارد و فقط خروجی نهایی را دریافت کند. شما چه بخش‌هایی را همچنان نیازمند تصمیم انسانی می‌دانید؟ (تعریف مسئله، انتخاب داده و تعیین اینکه «نتیجه خوب» دقیقاً یعنی چه، از جمله نقاطی هستند که می‌توانید از آنها شروع کنید.)
8. **سوال ۸** دو نفر درباره Data Mining دو تعریف متفاوت دارند: «پیدا کردن الگوهای مخفی در داده» و «تبدیل داده به دانش قابل استفاده برای تصمیم‌گیری». اگر بخواهید یک تعریف سوم بسازید که هر دو دیدگاه را کامل‌تر کند، چه می‌نویسید؟ (سعی کنید در تعریف خود هم «داده و الگو» و هم «کاربرد و تصمیم» را نگه دارید.)